

Lecture 1

*Lecturer: Scott Aaronson**Scribe: Yinmeng Zhang*

1 Administrivia

Welcome to Great Ideas in Theoretical Computer Science. Please refer to the syllabus for course information.

The only prerequisite for the class is “mathematical maturity,” which means that you know your way around a proof. What is a proof? There’s a formal definition of proof where each statement must follow from previous statements according to specified rules. This is a definition we will study in this course, but it’s not the relevant definition for when you’re doing your homework. For this class, a proof is an argument that can withstand all criticism from a highly caffeinated adversary.

Please interrupt if anything is ever unclear; the simplest questions are often the best. If you are not excited and engaged then complain.

2 What is computer science?

Computer science is not glorified programming. Edsger Dijkstra, Turing Award winner and extremely opinionated man, famously said that computer science has as much to do with computers as astronomy has to do with telescopes. We claim that computer science is a mathematical set of tools, or body of ideas, for understanding just about any system—brain, universe, living organism, or, yes, computer. Scott got into computer science as a kid because he wanted to understand video games. It was clear to him that if you could really understand video games then you could understand the entire universe. After all, what is the universe if not a video game with really, really realistic special effects?

OK, but isn’t physics the accepted academic path to understanding the universe? Well, physicists have what you might call a top-down approach: you look for regularities and try to encapsulate them as general laws, and explain those laws as deeper laws. The Large Hadron Collider is scheduled to start digging a little deeper in less than a year.

Computer science you can think of as working in the opposite direction. (Maybe we’ll eventually meet the physicists half-way.) We start with the simplest possible systems, and sets of rules that we haven’t necessarily confirmed by experiment, but which we just suppose are true, and then ask what sort of complex systems we can and cannot build.

3 Student Calibration Questions

A *quine* is a program that prints itself out. Have you seen such a program before? Could you write one?

Here’s a quine in English:

Print the following twice, the second time in quotes.

“Print the following twice, the second time in quotes.”

Perhaps the most exciting self-replicating programs are living organisms. DNA is slightly different from a quine because there are mutations and also sex. Perhaps more later.

Do you know that there are different kinds of infinities? In particular, there are more real numbers than there are integers, though there are the same number of integers as even integers. We'll talk about this later in the course, seeing as it is one of the crowning achievements of human thought.

4 How do you run an online gambling site?

This is one example of a “great idea in theoretical computer science,” just to whet your appetite.

Let's see what happens when we try to play a simple kind of roulette over the Internet.

We have a wheel cut into some even number of equal slices: half are red and half are black. A player bets n dollars on either red or black. A ball is spun on the wheel and lands in any slice with equal probability. If it lands on the player's color he wins n dollars; otherwise he loses n dollars, and there is some commission for the house. Notice that the player wins with probability $1/2$ —an alternate formulation of this game is “flipping a coin over the telephone.”

What could go wrong implementing this game? Imagine the following.

Player: I bet on red.

Casino: The ball landed on black. You lose.

Player: I bet on black.

Casino: The ball landed on red. You lose.

Player: I bet on black.

Casino: The ball landed on red. You lose.

Player: I bet on red.

Casino: The ball landed on black. You lose.

Player: This #\$%ing game is rigged!

So actually the player could probably figure out if the casino gives him odds significantly different from 50-50 if he plays often enough. But what if we wanted to guarantee the odds, even if the player only plays one game?

We could try making the casino commit to a throw before the player bets, but we have to be careful.

Casino: The ball landed on black.

Player: That's funny, I bet black!

We also need to make the player commit to a bet before the casino throws. At physical casinos it's possible to time it so that the throw starts before the Player bets and lands after. But what with packet-dropping and all the other things that can go wrong on them intertubes, it's not clear at all that we can implement such delicate timing over the Internet.

One way to fix this would be to call in a trusted third party, which could play man-in-the-middle for the player and casino. It would receive bet and throw information from the two parties, and

only forward them after it had received both. But who can be trusted?

Another approach, one that has been extremely fruitful for computer scientists, is to assume that one or both parties have limited computational power.

5 Factoring is Hard

Multiplying two numbers is pretty easy. In grade school we learned an algorithm for multiplication that takes something like N^2 steps to multiply two N -digit numbers. Today there are some very clever algorithms that multiply in very close to $N \log N$ time. That's fast enough even for thousand-digit numbers.

In grade school we also learned the reverse operation, factoring. However, “just try all possible factors” does not scale well at all. If a number X is N bits long, then there are something like 2^N factors to try. If we are clever and only try factors up to the square root of X , there are still $2^{N/2}$ factors to try. So what do we do instead? If you had a quantum computer you'd be set, but you probably don't have one. Your best bet, after centuries of research (Gauss was very explicitly interested in this question), is the so-called *number field sieve* which improves the exponent to roughly a constant times $N^{1/3}$.

Multiplication seems to be an operation that goes forward easily, but is really hard to reverse. In physics this is a common phenomenon. On a microscopic level every process can go either forwards or backwards: if an electron can emit a photon then it can also absorb one, etc. But on a macroscopic level, you see eggs being scrambled all the time, but never eggs being unscrambled. Computer scientists make the assumption that multiplying two large prime numbers is an operation of the latter kind: easy to perform but incredibly hard to reverse (on a classical computer).

For our purposes, let's make a slightly stronger assumption: not only is factoring numbers hard, it's even hard to determine if the last digit of one of the factors is a 7. (This assumption is true, so far as anyone knows.) Now, to bet on red, the player picks two primes that don't end in 7 and multiplies them together. To bet on black, the player picks two primes, at least one of which ends in 7, and multiplies them together to get X .

Player: sends X to the casino.

Casino: announces red or black.

Player: reveals factors to casino.

Casino: checks that factors multiply to X .

Is this a good protocol? Can the casino cheat? Can the player? The player might try to cheat by sending over a number which is the product of three primes. For example, suppose the factors were A , B , and C , and they ended in 1, 3, and 7 respectively. Then if the casino announces red, the player could send the numbers AB and C ; if the Casino announces black, the player sends A and BC – the player wins both ways. But all is not lost. It turns out that checking if a number has non-trivial factors is a very different problem from actually producing those factors. If you just want to know whether a number is prime or composite, there are efficient algorithms for that—so we just need to modify the last step to say “Casino checks that the factors are primes which multiply to X .”

This is a taste of the weird things that turn out to be possible. Later, we'll see how to convince someone of a statement without giving them any idea why it's true, or the ability to convince other people that the statement is true. We'll see how to “compile” any proof into a special format such that anyone who wants to check the proof only has to check a few random bits—regardless of the size of the proof!—to be extremely confident that it's correct. That these counterintuitive things are possible is a discovery about how the world works. Of course, not everyone has the interest to go into these ideas in technical detail, just as not everyone is going to seriously study quantum mechanics. But in Scott's opinion, any scientifically educated person should at least be aware that these great ideas exist.

6 Compass and Straightedge

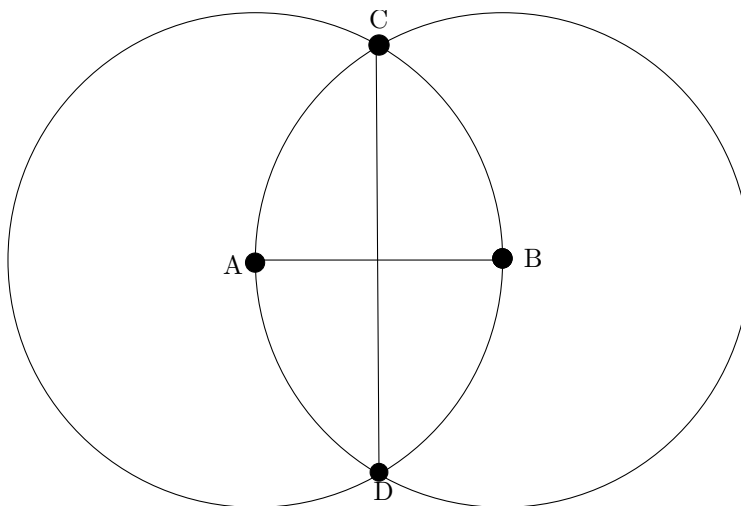
Now let's go back to the prehistory of computer science – the time of the Ancient Greeks. The Greeks were very interested in a formal model of computation called *compass-straightedge constructions*: what kind of figures can you draw in the plane using a compass and straightedge? The rules are as follows.

We start with two points. The distance between them defines the unit length.

We can draw a line between any two points.

We can draw a circle given its center and a point on its circumference.

We can draw a point at the intersection of any two previously constructed objects.



By applying these rules over and over, we can construct all kinds of things. Above is the construction of the perpendicular bisector of a line segment. [Can you prove that it works?] In 1796, Gauss constructed a regular 17-gon, which he was so proud of that he asked to have it inscribed on his tombstone. (The carver apparently refused, saying it would look just like a circle.) In principle you could construct a regular 65535-sided polygon, though presumably no one has actually done this. Instead of actually drawing figures, we can reason about them instead. The key to fantastically complicated constructions is *modularity*. For example, once we have an algorithm for constructing perpendicular lines, we encapsulate it into the perpendicular lines subroutine. The next time we need a perpendicular line in a line of reasoning, we don't have to build it from scratch, we get to assume it.

By building on previous work in this way over the course of centuries, people erected a veritable cathedral of geometry. And for centuries, this manipulation of production rules was the canonical example of what it meant to think precisely about something. But the game pointed its way to its own limitations. Some constructions eluded geometers—among them, famously, squaring the circle, trisecting an angle, and doubling the cube.

Today we'll talk about doubling the cube. In this problem, you're given the side length of a cube, and asked to construct the side length of a new cube that would have twice the volume of the old one. In other words, given a unit length line segment, construct a line segment of length $\sqrt[3]{2}$. You can do this if you assume you have some extra production rules, and you can approximate it arbitrarily well, but no one managed to give an exact construction with just a straightedge and compass.

In the 1800's geometers stepped back and started asking meta-questions about the fundamental limitations of the rules, a very computer science-y thing to do. They were able to do so due to a couple of revolutionary ideas that had occurred in the years since Rome annexed the Grecian provinces.

The first of these ideas was Cartesian coordinates, named for Descartes in the 1600's. This moves the game to the Cartesian plane. The initial points are $(0, 0)$ and $(1, 0)$. A nonvertical line through (a, b) and (c, d) is described by the function $y = \frac{d-b}{c-a}x + \frac{ad-bc}{a-c}$. A circle centered at (a, b) through (c, d) has the function $(x-a)^2 + (y-b)^2 = (a-c)^2 + (b-d)^2$. Intersection points are the solutions to systems of equations. For the intersection of lines, this is simply a linear system, which is easy to solve. For a line and circle or circle and circle, we get a quadratic system, and the quadratic formula leads us to the solution.

The upshot is that no matter how many systems of equations we solve and new points we draw, all we're doing is taking the original coordinates and applying $+$, $-$, $\times$, $\div$, and taking square roots. In fact, it would be sufficient for our purposes to reinterpret the problem as follows. We start with the numbers 0 and 1, and apply the above operations as often as we'd like. (Note that we can't divide by 0, but square roots of negative numbers are fine.) Can we construct the number $\sqrt[3]{2}$ with these operations?

It seems like we shouldn't be able to muck around with square roots and produce a cube root, and in fact we can't. There's a really nifty proof using Galois theory which we won't talk about because it requires Galois theory.

Even though this example was historically part of pure math, it illustrates many of the themes that today typify theoretical computer science. You have some well-defined set of allowed operations. Using those operations, you build all sorts of beautiful and complex structures—often reusing structures you previously built to build even more complex ones. But then certain kinds of structures, it seems you *can't* build. And at that point, you have to engage in metareasoning. You have to step back from the rules themselves, and ask yourself, what are these rules *really* doing? Is there some fundamental reason why these rules are never going to get us what we want?

As computer scientists we're particularly interested in building things out of ANDs and ORs and NOTs or jump-if-not-equal's and other such digital operations. We're still trying to understand the fundamental limitations of these rules. Note when the rules are applied *arbitrarily many times*, we actually understand pretty well by now what is and isn't possible: that's a subject called

computability theory, which we'll get to soon. But if we limit ourselves to a “reasonable” number of applications of the rules (as in the famous P vs. NP problem), then to this day we haven't been able to step back and engage in the sort of metareasoning that would tell us what's possible.

7 Euclid's GCD Algorithm

Another example of ancient computational thinking, a really wonderful non-obvious efficient algorithm is Euclid's GCD algorithm. It starts with this question.

How do you reduce a fraction like 510/646 to lowest terms?

Well, we know we need to take out the greatest common divisor (GCD). How do you do that? The grade school method is to factor both numbers; the product of all the common prime factors is the GCD, and we simply cancel them out. But we said before that factoring is believed to be hard. The brute force method is OK for grade school problems, but it won't do for thousand-digit numbers. But just like testing whether a number is prime or composite, it turns out that the GCD problem can be solved in a different, more clever way—one that *doesn't* require factoring.

Euclid's clever observation was that if a number divides two numbers, say 510 and 646, it also divides any integer linear combination of them, say $646 - 510$. [Do you see why this is so?] In general, when we divide B by A , we get a quotient q and a remainder r connected by the equation $B = qA + r$, which means that $r = B - qA$, which means that r is a linear combination of A and B !

So finding the GCD of 510 and 646 is the same as finding the GCD of 510 and the remainder when we divide 646 and 510. This is great because the remainder is a smaller number. We've made progress!

$$\text{GCD}(510, 646) = \text{GCD}(136, 510)$$

And we can keep doing the same thing.

$$\text{GCD}(136, 510) = \text{GCD}(102, 136) = \text{GCD}(34, 102) = 34$$

Here we stopped because 34 divides 102, and we know this means that 34 is the GCD of 34 and 102. We could also take it a step further and appeal to the fact that the GCD of any number and 0 is that number: $\text{GCD}(34, 102) = \text{GCD}(0, 34) = 34$.

GIVEN: natural numbers A,B
Assume B is the larger (otherwise swap them)
If A is 0 return B
Else find the GCD of $(B \% A)$ and A

The numbers get smaller every time, and we're working with natural numbers, so we know that we'll arrive at 0. So Euclid's algorithm will eventually wrap up and return an answer. But exactly how many remainders are we going to have to take? Well, exactly how much smaller do these numbers get each time? We claim that $(B \bmod A) < B/2$. Can you see why? (Hint: case by whether A is bigger, smaller, or equal to $B/2$.) So the numbers are getting *exponentially* smaller. Every

other equal sign, the numbers are half as big as they were before: $102 < 510/2$ and $136 < 646/2$, and so on. This means Euclid's algorithm is pretty great.

QUESTION: Could you speed up Euclid's algorithm if you had lots of parallel processors?

We can speed up the work within each step—say with a clever parallel division algorithm, but it seems impossible to do something more clever because each step depends on the output of the previous step. If you find a way to parallelize this algorithm, you will have discovered the first really striking improvement to it in 2300 years.

8 For further reference

Check out the Wikipedia articles on “Compass and straightedge”, “General number field sieve”, “Edsger Dijkstra”, etc.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 3

*Lecturer: Scott Aaronson**Scribe: Adam Rogal*

1 Administrivia

1.1 Scribe notes

The purpose of scribe notes is to transcribe our lectures. Although I have formal notes of my own, these notes are intended to incorporate other information we may mention during class - a record for future reference.

1.2 Problem sets

A few comments on the problem sets. Firstly, you are welcome to collaborate, but please mark on your problem sets the names of whom you worked with. Our hope is to try all the problems. Some are harder than others; there are those marked challenge problems as well. If you can't solve the given problem, be sure to state what methods you tried and your process up to the point you could not continue. This is partial credit and much better than writing a complete, but incorrect solution. After all, according to Socrates *the key to knowledge is to know what you don't know*.

1.3 Office hours

We will have office hours once a week.

2 Recap

2.1 Computer science as a set of rules

We can view computer science as the study of simple set of rules and what you can and can't build with them. Maybe the first example of that could be considered Euclidian geometry. And the key to discovering what processes we can build is that these rules are well-defined.

2.2 Logic

The field of logic focuses on automating or systematizing not just any mechanical processes, but rational thought itself. If we could represent our thoughts by manipulations of sequences of symbols, then in principle we could program a computer to do our reasoning for us.

We talked the simplest logical systems which were the only ones for thousands of years. Syllogisms and propositional logic, the logic of Boolean variables that can be either true or false, and related to each other through operators like and, or, and not. We finally discussed first order logic.

2.2.1 First order logic

The system of first order logic is built up of sentences. Each of these sentences contain variables, such as x, y , and z . Furthermore we can define functions which take these variables as input.

For example: let's define a function $Prime(x)$. Given an integer, it will return true if the number is prime, false if it is composite. Just like functions in any programming languages, we can build functions out of other functions by calling them as subroutines. In fact, many programming languages themselves were modeled after first order logic.

Furthermore, as in propositional logic, symbols such as $\wedge$ (and), $\vee$ (or), $\neg$ (not), and $\rightarrow$ (implies) allow us to relate objects to each other.

Quantifiers are a crucial part of first order logic. Quantifiers allow us to state propositions such as "Every positive integer x is either prime or composite."

$$\forall x. Prime(x) \vee Composite(x)$$

There's a counterexample, of course, namely 1. We can also say: "There exists an x , such that something is true."

$$\exists x. Something(x)$$

When people talk about first-order logic, they also normally assume that the equals sign is available.

2.2.2 Inference rules

We want a set of rules that will allow us to form true statements from other true statements. Propositional tautologies:

$$A \vee \neg A$$

Modus ponens:

$$A \wedge (A \rightarrow B) \rightarrow B$$

Equals:

$$Equals(X, X)$$

$$Equals(X, Y) \iff Equals(Y, X)$$

Transitivity property:

$$Equals(X, Y) \wedge Equals(Y, Z) \rightarrow Equals(X, Z)$$

Furthermore, we have the rule of change of variables. If you have a valid sentence, that sentence will remain valid if we change variables.

2.2.3 Quantifier rules

If $A(x)$ is a valid sentence for any choice of x , then for all x , $A(x)$ is a valid sentence. Conversely, if $A(x)$ is a valid sentence for all x , then any $A(x)$ for a fixed x is a valid sentence.

$$A(X) \iff \forall x. A(x)$$

We also have rules for dealing with quantifiers. For example, it is false, that for all x , $A(x)$ iff there exists an x , $\neg A(x)$.

$$\neg \forall x. A(x) \iff \exists \neg A(x)$$

2.2.4 Completeness theorem

Kurt Gödel proved that the rules thus stated were all the rules we need. He proved that if you could not derive a logical contradiction by using this set of rules, there must be a way of assigning variables, such that all the sentences are satisfied.

3 Circuits

Electrical engineers views circuits to be complete loops typically represented in figure 1. However, in computer science, circuits have no loops and are built with logic gates.

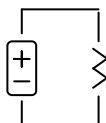


Figure 1: A simple EE circuit.

3.1 Logic gates

The three best-known logic gates are the *NOT*, *AND*, and *OR* gates shown in figure 2.

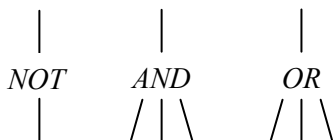


Figure 2: The logical gates *NOT*, *AND*, and *OR*.

Though primitive on their own, these logic gates can be strung together to form complex logical operations. For example, we can design a circuit, shown in figure 3, that takes the majority of 3 variables: x , y , and z . We can also use De Morgan's law to form a *AND* gate from an *OR* gate and vice versa as shown figure 4.

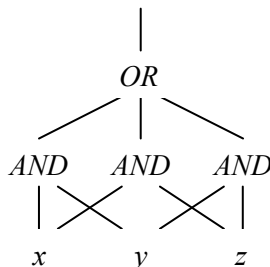


Figure 3: The majority circuit.

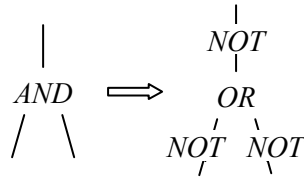


Figure 4: An *AND* gate can be constructed from an *OR* and three *NOT* gates by using De Morgan's law.

These logic gates can also be combined to form other gates such as the *XOR* and *NAND* gates shown in figure 5. Conversely, by starting with the *NAND* gate, we can build any other gate we want.

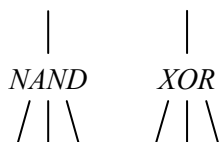


Figure 5: *NAND* and *XOR* gates.

On the other hand, no matter how we construct a circuit with *AND* and *OR* gates, if the input is all 1's we can never get an output of 0. We call a Boolean function that can be built solely out of *AND* and *OR* gates a *monotone* Boolean function.

Are there any other interesting sets of gates that *don't* let us express all Boolean functions? Yes: the *XOR* and *NOT* gates. Because of their linearity, no matter how we compose these gates we can never get functions like *AND* and *OR*.

4 Puzzle

Here's an amusing puzzle: can you compute the NOT's of 3 input variables, using as many AND/OR gates as you like but only 2 NOT gates?

4.0.1 Limitations

Although we have discovered that circuits can be a powerful tool, as a model of computation they have some clear limitations. Firstly, circuits offer no form of storage or memory. They also have no feedback; the output of a gate never gets fed as the input. But from a modern standpoint, the *biggest* limitation of circuits is that (much like computers from the 1930s) they can only be designed for a fixed-size task. For instance, one might design a circuit to sort 100 numbers. But to sort 1000 numbers, one would need to design a completely new circuit. There's no *general-purpose* circuit for the sorting task, one able to process inputs of arbitrary sizes.

5 Finite automata

We'll now consider a model of computation that *can* handle inputs of arbitrary length, unlike circuits – though as we'll see, this model has complementary limitations of its own.

5.1 Description

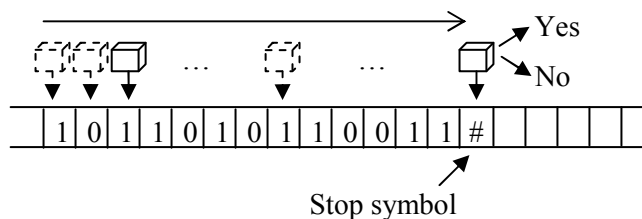


Figure 6: At any given time, the machine has some unique state. The machine reads the tape in one motion (in this case left to right) and the state changes depending on the value of the current square. When it reaches the stop state (signaled by the # sign, the machine returns a yes or no answer - an accept or reject state respectively.)

The simple way of thinking of a finite automaton is that it's a crippled computer that can only move along memory in one direction. As shown in figure 6, a computer with some information written along a tape, in some sort of encoding, will scan this tape one square at a time, until it reaches the stop symbol. The output of this machine will be a yes or no - accept or reject. This will be the machine's answer to some question that it was posed about the input.

5.2 State and internal configuration

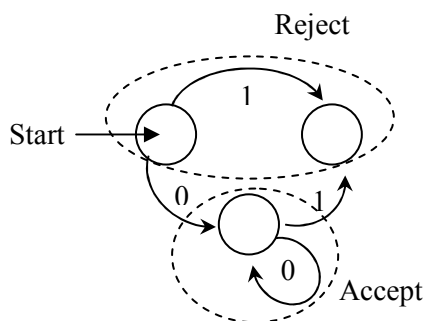


Figure 7: This simple machine has 3 states. Given an input of 0 or 1, the state will transition to a new state. The final state will determine its output - accept or reject.

It is unnecessary to determine what the internal configuration of this machine is. We can abstract this notion into the statement that this machine will have some state and the ability to transition between states given a certain input. The machine will begin with a start state, before it has read any input. When the machine reads the stop symbol, the correct state will determine if the machine should output an accept or reject.

It is crucial that we define the machine as having a finite number of states. If the machine had an infinite number of states, then it could compute absolutely *anything*, but such an assumption is physically unrealistic.

5.3 Some examples

Let us design a machine that determines if any 1's exist in a stream given the alphabet of 0 or 1. We define two states of the machine - 0 and 1. The 0 represents the state that the machine has not seen a 1 yet. The 1 state represents the state that the machine has seen a 1. When the machine has transitioned to the 1 state, neither a 1 or 0 will ever change the state back to 0. That is, regardless of input or length of input, our question, "Are there any 1's in the stream?" has been answered. Therefore, the 1 state should produce an accept, while the 0 state should produce a reject when a stop symbol has been reached.

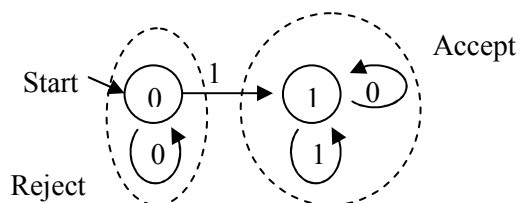


Figure 8: This FA determines if any 1's exist in our data stream.

Let us now design a machine that determines if the number of 1's is even or odd in the stream. We define two states again - 0 and 1. The 0 state represents a machine that has seen an even number of 1's and the 1 state describes a machine that has seen an odd number of 1's. An input of 0 will only transition the state to itself. That is, we are only concerned about the number of 1's in this stream. At each input of a 1, the machine will alternate state between 0 and 1. The final state will determine if the data stream has seen an even or odd number of 1's, with 1 being set as the acceptance state.

It should be noted that regardless of input size, this machine will determine the correct answer to the question we posed. Unlike with circuits, our machine size was not dictated by the size of the input.

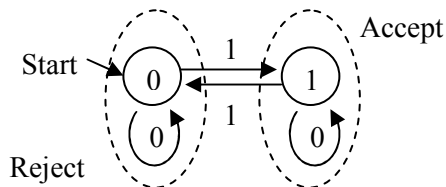


Figure 9: This FA determines if there are an even or odd number of 1's in our data stream.

5.4 Palindromes

Let us now explore if we could create a finite machine that can determine if an input string is a palindrome, a string that reads the same backwards and forwards. The input will be finite, and there will be a terminator at the end. We begin by defining the possible states of the machine. If we let our machine contain 2^N states, then as shown in figure 10, we could just label each final leaf as an accept or reject for every possible sequence of 1's and 0's.

The question still remains, can we create a machine with a finite number of states that can

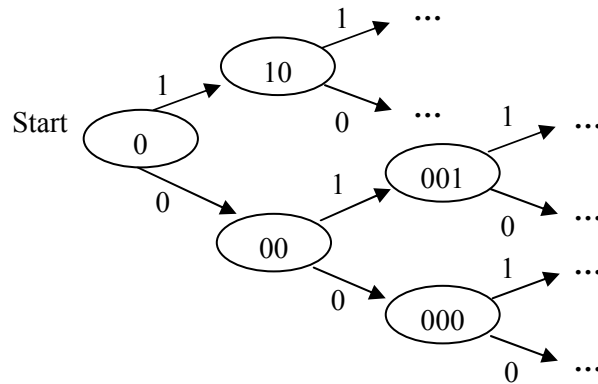


Figure 10: For a stream of N bits, a finite automaton, intended to determine if the stream is a palindrome, grows exponentially. For N bits, 2^N states are required.

act as a palindrome detector. The answer lies in using the Pigeonhole Principle to analyze the limitations of finite automata.

5.5 The Pigeonhole Principle

The Pigeonhole Principle states that if we have N pigeons and we want put them into $N - 1$ holes, at least one hole will have two or more pigeons. Although very simple, this principle allows us to prove that no finite automaton can act as a palindrome detector.

5.5.1 A digression: proving the pigeonhole principle

Even though the pigeonhole principle is simple, it is non-trivial to prove in simple systems of logic. We can express the requirements that every pigeon goes into some hole, and that no two pigeons go into the same hole, using propositional logic. The challenge is then to prove that not all the statements can be true, using only mechanical logical manipulation of the statements (and not higher-order reasoning about what they "mean").

In other words, the pigeonhole principle seems obvious to us because we can stand back and see the larger picture. But a propositional proof system like the ones we saw in the last lecture can't do this; it can only reason locally. ("Let's see: if I put this pigeon here and that one there ... darn, *still* doesn't work!") A famous theorem of Haken states that any proof of the Pigeonhole Principle based on "resolution" of logical statements, requires a number of steps that increases exponentially with N (the number of pigeons). This is an example of something studied by a field called *proof complexity*, which deals with questions like, "does any proof have to have a size that is exponentially larger than the theorem we are trying to prove?"

5.6 Using the Pigeonhole Principle for palindromes

We use the Pigeonhole Principle to prove that no finite automaton that can be constructed such that we can detect if any string is a palindrome.

To begin this proof, let us split a palindrome down the middle. We will ignore everything about the finite automaton except its state at the middle point; any information that the automaton will carry over to the second half of the string, must be encoded in that state.

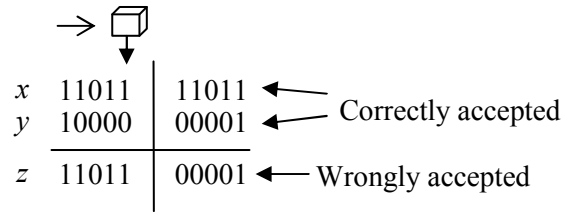


Figure 11: By using the Pigeonhole principle, we can show that we can split two strings at their reflection points such that a finite automaton will be in at the same state for both sub strings. We can then cross the two strings to form a new string that “tricks” the machine into thinking that it has correctly accepted a string as a palindrome.

A finite automaton must have a fixed number of states. On the other hand, there are infinitely many possibilities for the first half of the string. Certainly, you can’t put infinitely many pigeons into a finite number of holes without having at least one hole with at least two pigeons. This means that there is at least one state that does “double duty,” in that two different first halves of the string lead to the same state.

As shown in figure 11, we consider two palindromes x and y . If the machine works correctly, then it has to accept both of them. On the other hand, for some x, y pair, the machine will lie in the same state for both x and y when it’s at the halfway point. Then by crossing the remaining halves of x and y , we can create a new string, z , which is accepted by the machine even though it’s not a palindrome. This proves that no finite automaton exists that recognizes all and only the palindromes.

5.7 Regular expressions

Regular expressions allow us to search for a keyword in a large string. Yet, they are more powerful than simply searching for the keyword 110 in the string 001100. We can use regular expressions to locate patterns as well.

For example, we can create an expression like $(0110)|(0001)$ which will either match the keyword 0110 or 0001. We can also create expressions that will find any 3 bit string with a 1 in the middle: $(0|1)1(0|1)$.

We can also use more advanced characters such as the asterisk to represent repetition. $(0|1)1(0|1)0^*$ searches for any 3 bit string with a 1 in the middle followed by any number of 0’s. We can also repeat larger patterns such as $[(0|1)1(0|1)]^*$. This states that we would like to match any number of 3 bit strings with 1’s in the middle. It should be noted that each time the pattern repeats, the 0 or 1’s can be chosen differently.

We can now state (without proof) a very interesting theorem: any language is expressible by a regular expression, if and only if it’s recognized by a finite automaton. Regular expressions and finite automaton are different ways of looking at the same thing.

To give an example: earlier we created a finite automaton that was able to recognize all strings with an even number of 1’s. According to the theorem, there must be regular expression that generates this same set of strings. And indeed there is: $0^*(0^*10^*1)^*$.

6 Nondeterministic finite automata

Nondeterministic finite automata represent machines that can not only transition between states, but between sets of states. As before, we have a machine that reads a tape from left to right with a finite number of states. When the machine reads an input, each state that the machine is now on, is allowed to transition to any other states emanating from the previous states based on the input. The machine is in acceptance if any final state is an accept state.

You might guess that NDFA's (nondeterministic finite automata) would be much more powerful than DFA's (deterministic finite automata). This is not the case, however: given an NDFA with N states, we can always simulate it by a DFA with 2^N states, by creating a single state in the DFA to represent each *set* of states in the NDFA.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 3
Nancy Lynch

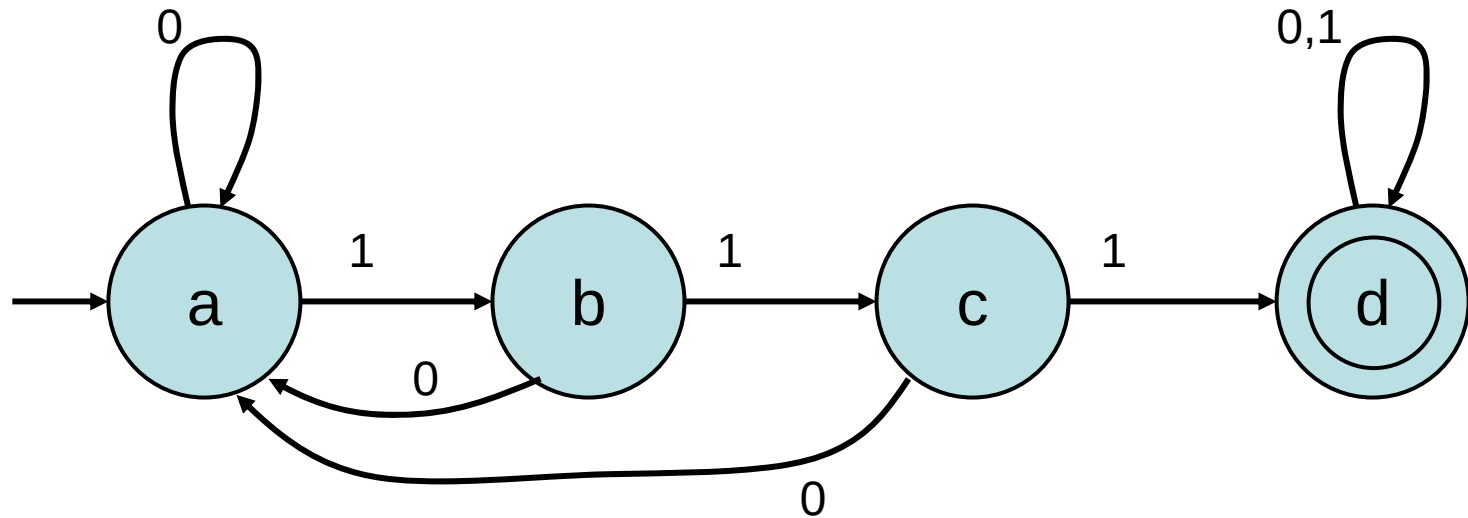
Today

- **Finite Automata (FAs)**
 - Our third machine model, after circuits and decision trees.
- **Designed to:**
 - **Accept** some strings of symbols.
 - **Recognize** a language, which is the set of strings it accepts.
- **FA takes as its input a string of any length.**
 - One machine for all lengths.
 - Circuits and decision trees use a different machine for each length.
- **Today's topics:**
 - Finite Automata and the languages they recognize
 - Examples
 - Operations on languages
 - Closure of FA languages under various operations
 - Nondeterministic FAs
- **Reading:** Sipser, Section 1.1.
- **Next:** Sections 1.2, 1.3.

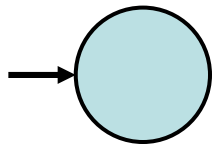
Finite Automata and the languages they recognize

Example 1

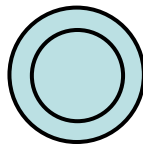
- An FA diagram, machine M



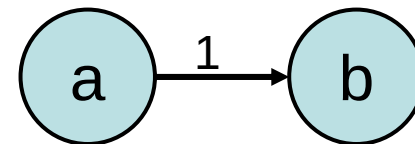
- Conventions:



Start state

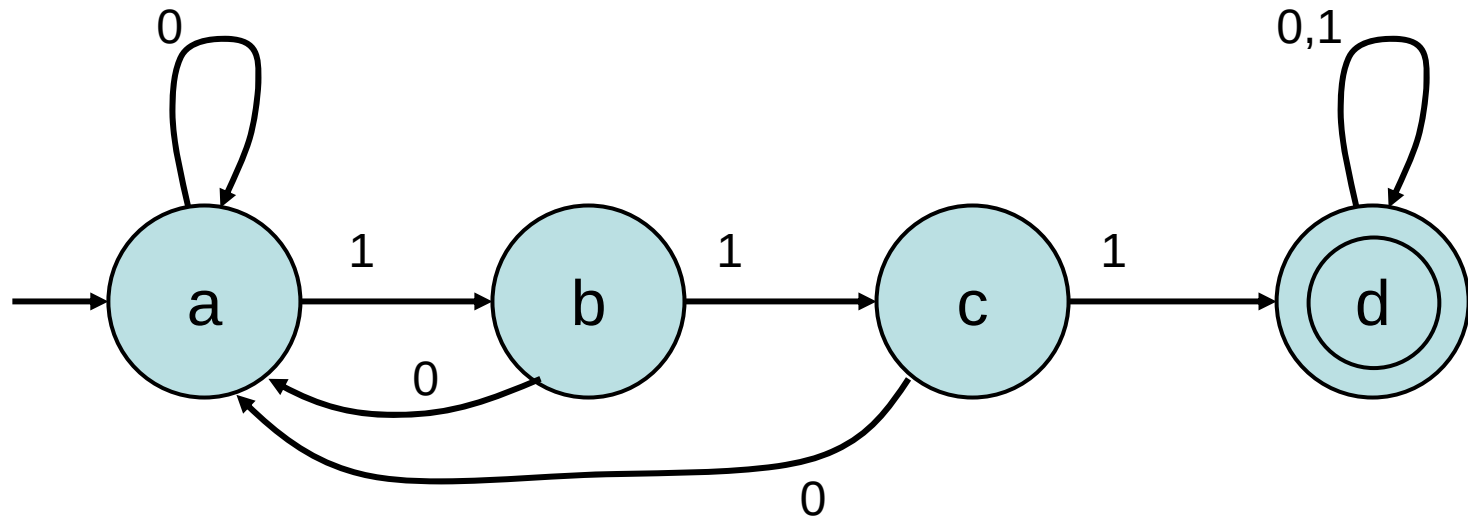


Accept state



Transition from a to b on
input symbol 1.
Allow self-loops

Example 1

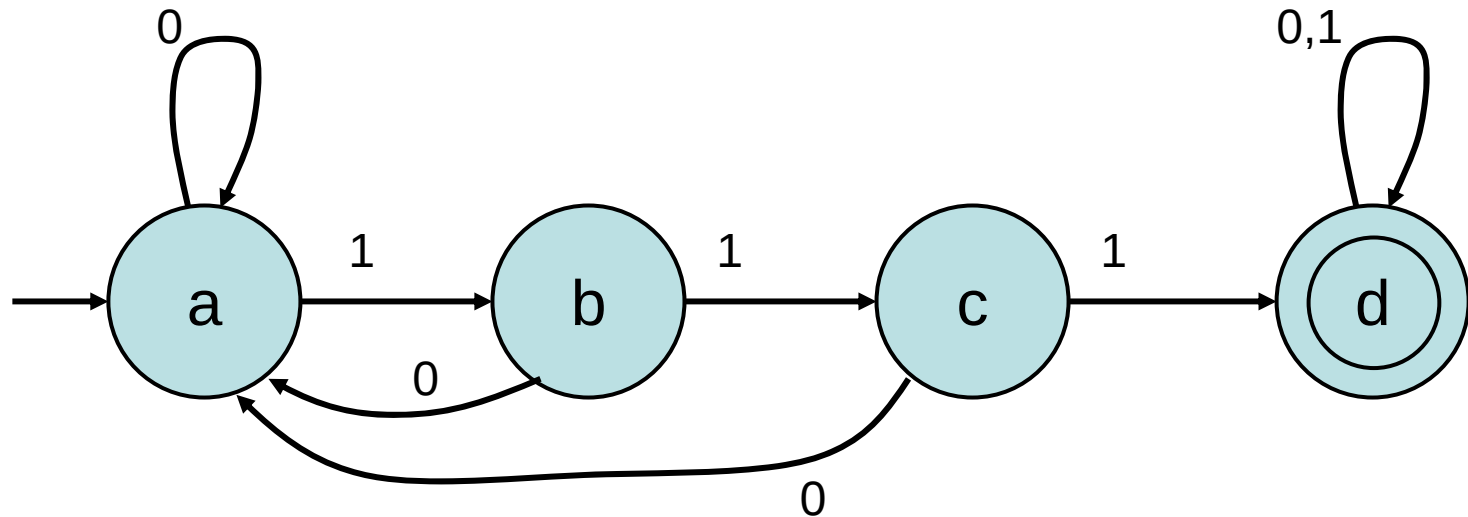


- **Example computation:**
 - Input word w : 1 0 1 1 0 1 1 1 0
 - States: a b a b c a b c d d
- We say that **M accepts w** , since w leads to d , an accepting state.

In general...

- A **FA M accepts a word w** if w causes M to follow a path from the start state to an accept state.
- Some terminology and notation:
 - Finite **alphabet** of symbols, usually called Σ .
 - In Example 1 (and often), $\Sigma = \{ 0,1 \}$.
 - **String (word) over Σ** : Finite sequence of symbols from Σ .
 - **Length** of w, $| w |$
 - ϵ , placeholder symbol for the **empty string**, $| \epsilon | = 0$
 - Σ^* , the set of all finite strings of symbols in Σ
 - **Concatenation** of strings w and x, written $w \circ x$ or $w x$.
 - **L(M), language recognized by M**:
$$\{ w \mid w \text{ is accepted by } M \}.$$
 - What is $L(M)$ for Example 1?

Example 1



- What is $L(M)$ for Example 1?
- $\{ w \in \{0,1\}^* \mid w \text{ contains } 111 \text{ as a substring} \}$
- Note: Substring refers to consecutive symbols.

Formal Definition of an FA

- An FA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma \rightarrow Q$ is the transition function,

The arguments of δ
are a state and an
alphabet symbol.

The result is a state.

- $q_0 \in Q$, is the start state, and
- $F \subseteq Q$ is the set of accepting, or final states.

Example 1

- What is the 5-tuple $(Q, \Sigma, \delta, q_0, F)$?
- $Q = \{ a, b, c, d \}$
- $\Sigma = \{ 0, 1 \}$
- δ is given by the state diagram, or alternatively, by a table:

- $q_0 = a$
- $F = \{ d \}$

	0	1
a	a	b
b	a	c
c	a	d
d	d	d

Formal definition of computation

- Extend the definition of δ to input strings and states:

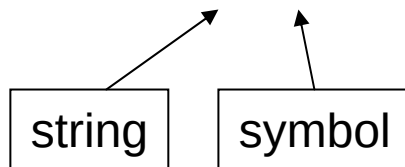
$\delta^*: Q \times \Sigma^* \rightarrow Q$, state and string yield a state

$\delta^*(q, w)$ = state that is reached by starting at q and following w .

- Defined recursively:

$$\delta^*(q, \varepsilon) = q$$

$$\delta^*(q, w a) = \delta(\delta^*(q, w), a)$$



- Or iteratively, compute $\delta^*(q, a_1 a_2 \dots a_k)$ by:

$s := q$

for $i = 1$ to k do $s := \delta(s, a_i)$

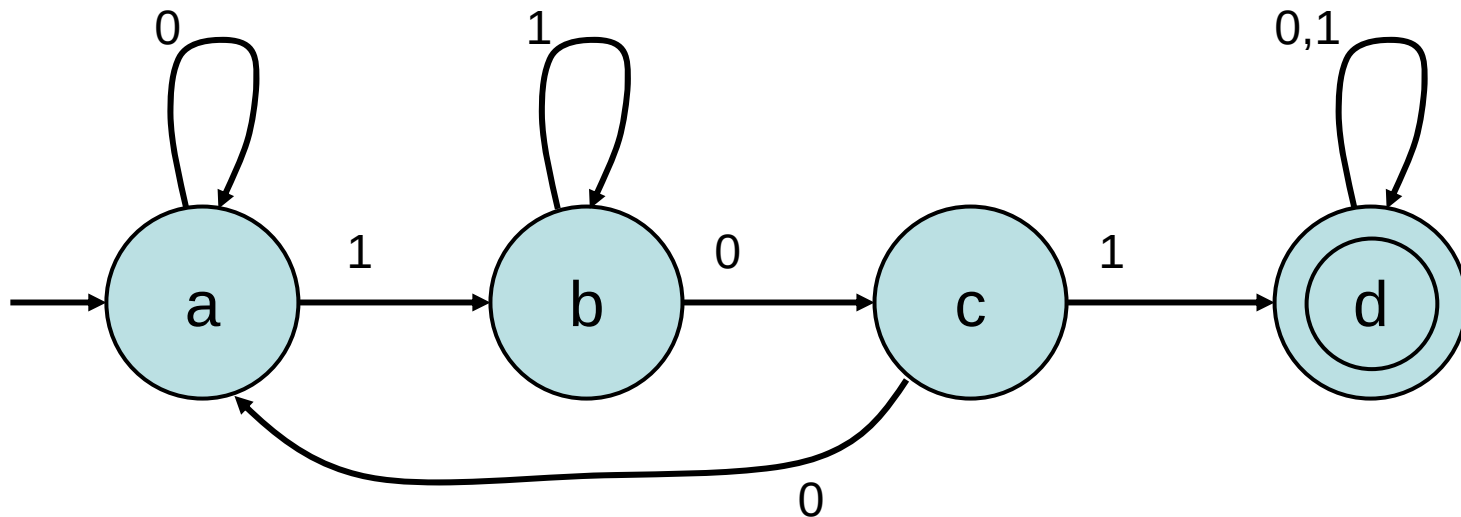
Formal definition of computation

- String w is **accepted** if $\delta^*(q_0, w) \in F$, that is, w leads from the start state to an accepting state.
- String w is **rejected** if it isn't accepted.
- A **language** is any set of strings over some alphabet.
- **$L(M)$** , language recognized by finite automaton $M = \{ w \mid w \text{ is accepted by } M \}$.
- A language is **regular**, or **FA-recognizable**, if it is recognized by some finite automaton.

Examples of Finite Automata

Example 2

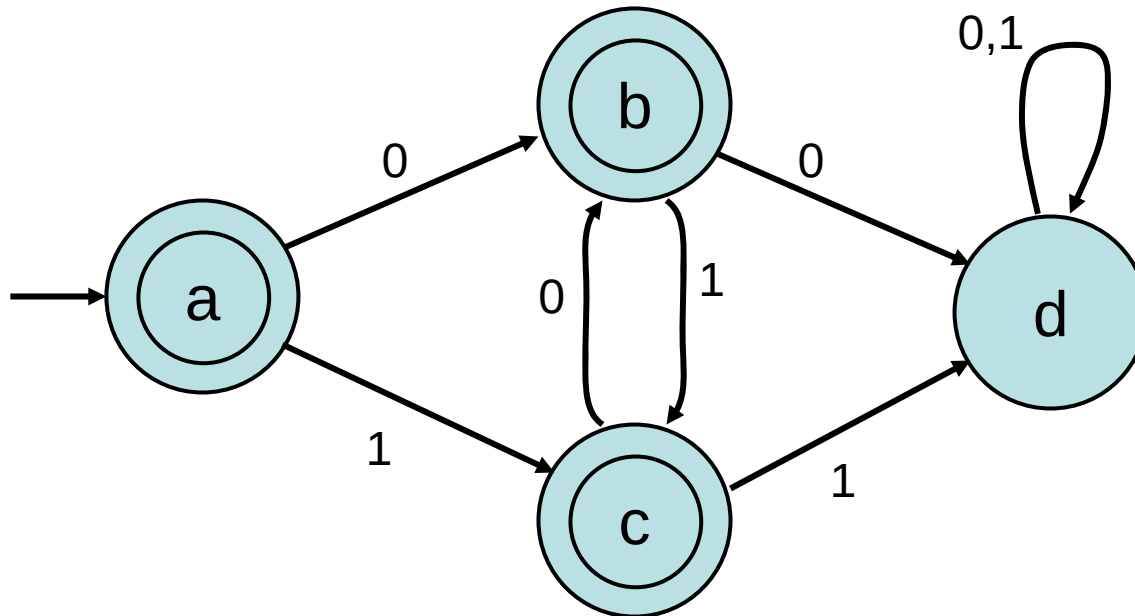
- Design an FA M with $L(M) = \{ w \in \{0,1\}^* \mid w \text{ contains } 101 \text{ as a substring} \}$.



- Failure from state b causes the machine to remain in state b .

Example 3

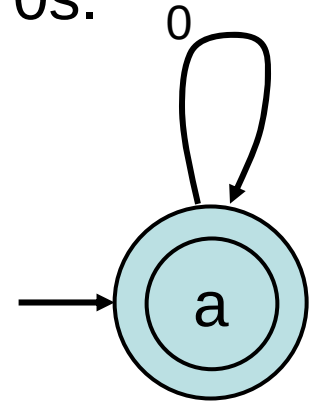
- $L = \{ w \in \{0,1\}^* \mid w \text{ doesn't contain either } 00 \text{ or } 11 \text{ as a substring} \}$.



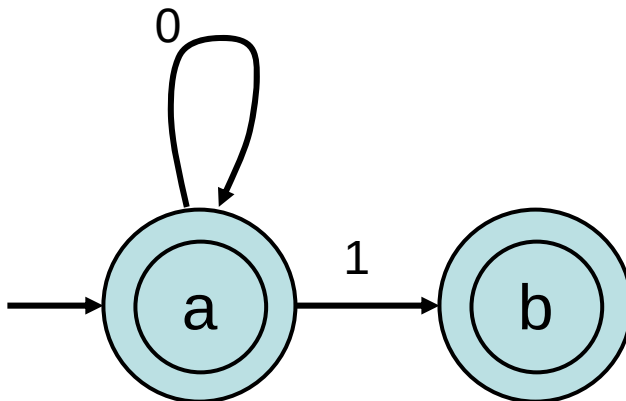
- State d is a **trap state** = a nonaccepting state that you can't leave.
- Sometimes we'll omit some arrows; by convention, they go to a trap state.

Example 4

- $L = \{ w \mid \text{all nonempty blocks of 1s in } w \text{ have odd length} \}$.
- E.g., ε , or 100111000011111, or any number of 0s.
- Initial 0s don't matter, so start with:



- Then 1 also leads to an accepting state, but it should be a different one, to “remember” that the string ends in one 1.

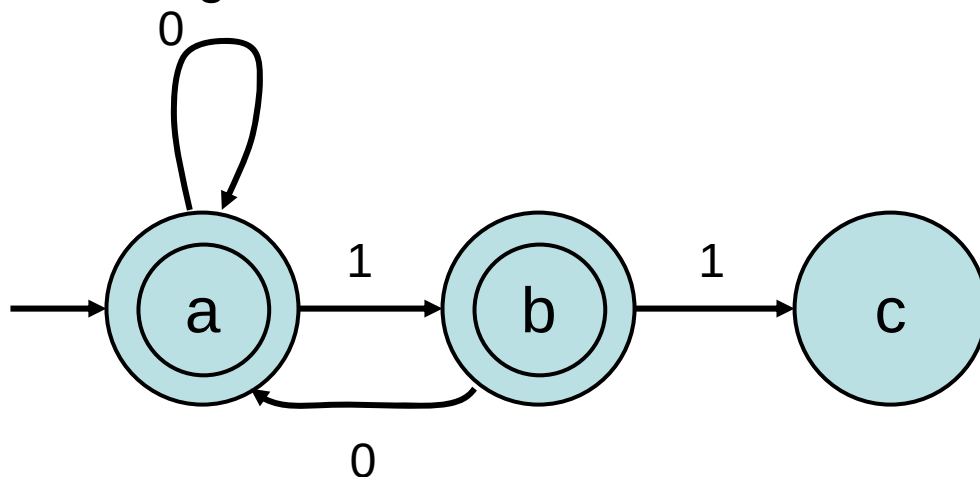
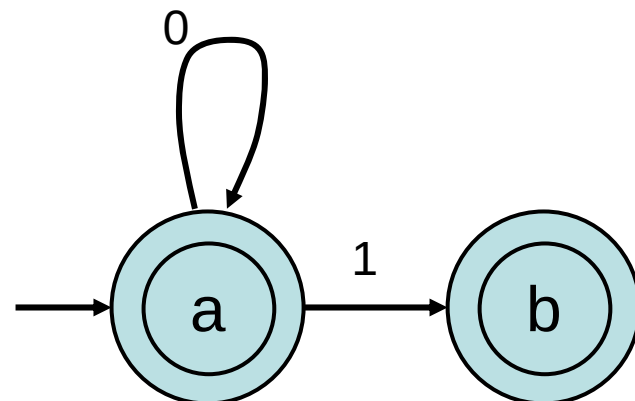


Example 4

- $L = \{ w \mid \text{all nonempty blocks of 1s in } w \text{ have odd length} \}$.

- From b:

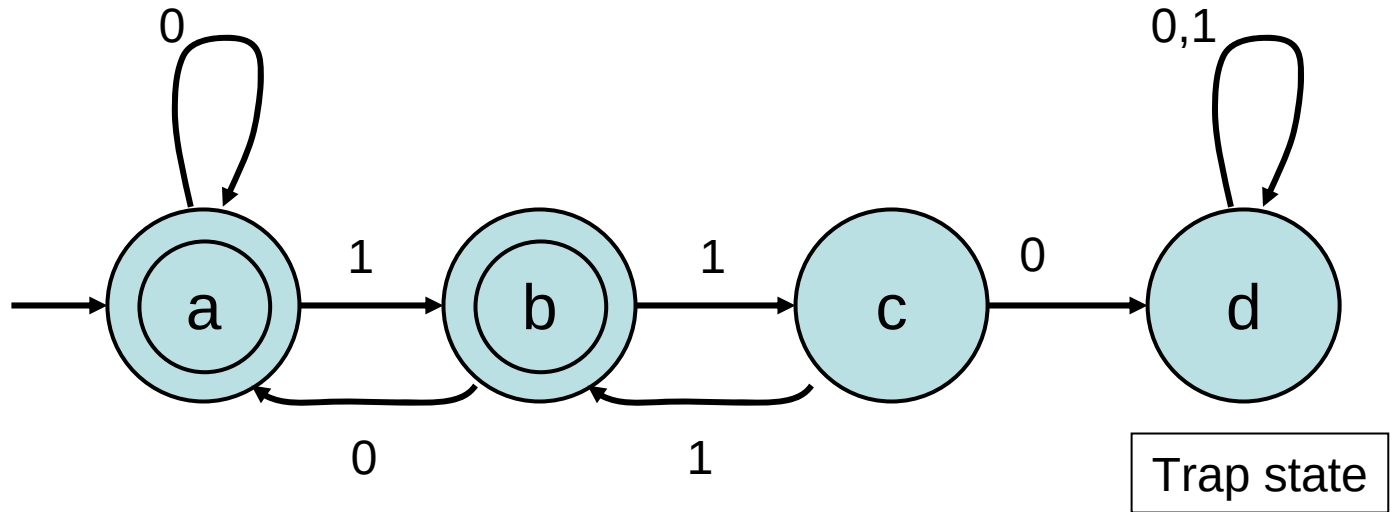
- 0 can return to a, which can represent either ε , or any string that is OK so far and ends with 0.
- 1 should go to a new **nonaccepting state**, meaning “the string ends with two 1s”.



- Note: c isn't a trap state---we can accept some extensions.

Example 4

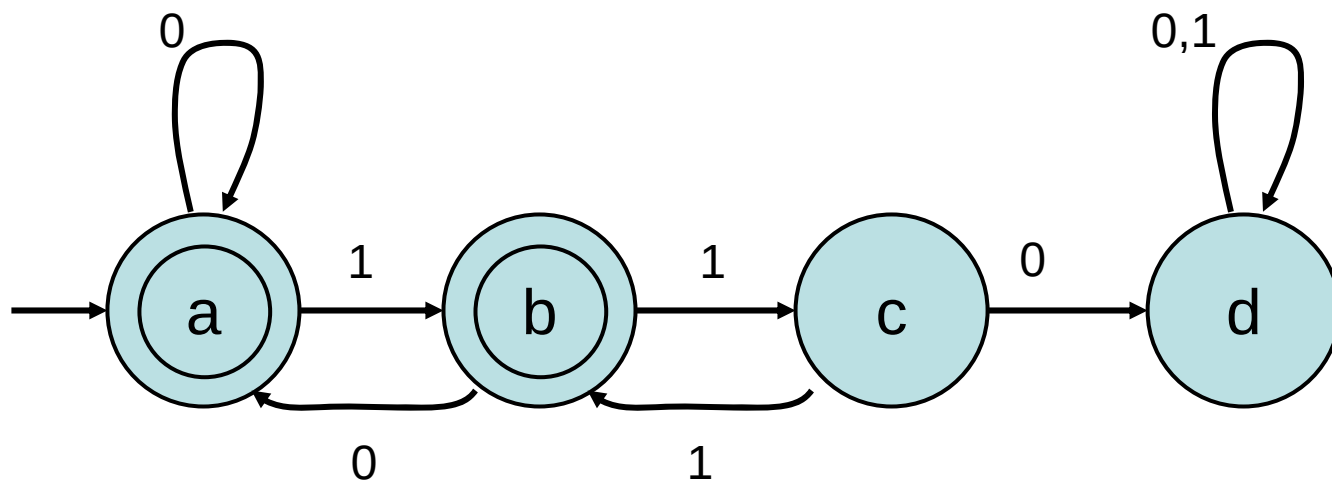
- $L = \{ w \mid \text{all nonempty blocks of 1s in } w \text{ have odd length} \}$.



- From c:
 - 1 can lead back to b, since future acceptance decisions are the same if the string so far ends with any odd number of 1s.
 - Reinterpret b as meaning “ends with an odd number of 1s”.
 - Reinterpret c as “ends with an even number of 1s”.
 - 0 means we must reject the current string and all extensions.

Example 4

- $L = \{ w \mid \text{all nonempty blocks of 1s in } w \text{ have odd length} \}.$



- Meanings of states (more precisely):
 - a: Either ε , or contains no **bad block** (even block of 1s followed by 0) so far and ends with 0.
 - b: No bad block so far, and ends with odd number of 1s.
 - c: No bad block so far, and ends with even number of 1s.
 - d: Contains a bad block.

Example 5

- $L = EQ = \{ w \mid w \text{ contains an equal number of 0s and 1s} \}$.
- No FA recognizes this language.
- Idea (not a proof):
 - Machine must “remember” how many 0s and 1s it has seen, or at least the difference between these numbers.
 - Since these numbers (and the difference) could be anything, there can’t be enough states to keep track.
 - So the machine will sometimes get confused and give a wrong answer.
- We’ll turn this into an actual proof next week.

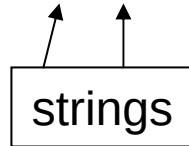
Language Operations

Language operations

- Operations that can be used to construct languages from other languages.
- Recall: A **language** is any set of strings.
- Since languages are **sets**, we can use the usual set operations:
 - Union, $L_1 \cup L_2$
 - Intersection, $L_1 \cap L_2$
 - Complement, L^c
 - Set difference, $L_1 - L_2$
- We also have new operations defined especially for sets of strings:
 - Concatenation, $L_1 \circ L_2$ or just $L_1 L_2$
 - Star, L^*

Concatenation

- $L_1 \circ L_2 = \{ x y \mid x \in L_1 \text{ and } y \in L_2 \}$

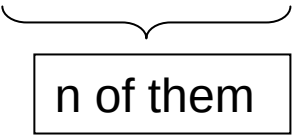


- Pick one string from each language and concatenate them.
- Example:
 $\Sigma = \{ 0, 1 \}$, $L_1 = \{ 0, 00 \}$, $L_2 = \{ 01, 001 \}$
 $L_1 \circ L_2 = \{ 001, 0001, 00001 \}$
- Notes:
 $|L_1 \circ L_2| \leq |L_1| \times |L_2|$, not necessarily equal.
 $L \circ L$ does not mean $\{ x x \mid x \in L \}$, but rather, $\{ x y \mid x \text{ and } y \text{ are both in } L \}$.

Concatenation

- $L_1 \circ L_2 = \{ x y \mid x \in L_1 \text{ and } y \in L_2 \}$
- Example:
 $\Sigma = \{ 0, 1 \}$, $L_1 = \{ 0, 00 \}$, $L_2 = \{ 01, 001 \}$
 $L_1 \circ L_2 = \{ 001, 0001, 00001 \}$
 $L_2 \circ L_2 = \{ 0101, 01001, 00101, 001001 \}$
- Example: $\emptyset \circ L$
 $\{ x y \mid x \in \emptyset \text{ and } y \in L \} = \emptyset$
- Example: $\{ \varepsilon \} \circ L$
 $\{ x y \mid x \in \{ \varepsilon \} \text{ and } y \in L \} = L$

Concatenation

- $L_1 \circ L_2 = \{ x y \mid x \in L_1 \text{ and } y \in L_2 \}$
- Write $L \circ L$ as L^2 ,
 $L \circ L \circ \dots \circ L$ as L^n , which is $\{ x_1 x_2 \dots x_n \mid \text{all } x\text{'s are in } L \}$


A diagram consisting of a horizontal line with a curly bracket above it. Below the line is a rectangular box containing the text "n of them".
- Example: $L = \{ 0, 11 \}$
 $L^3 = \{ 000, 0011, 0110, 01111, 1100, 11011, 11110, 111111 \}$
- Example: $L = \{ 0, 00 \}$
 $L^3 = \{ 000, 0000, 00000, 000000 \}$
- Boundary cases:
 $L^1 = L$
Define $L^0 = \{ \varepsilon \}$, for every L .
 - Implies that $L^0 L^n = \{ \varepsilon \} L^n = L^n$.
 - Special case of general rule $L^a L^b = L^{a+b}$.

The Star Operation

- $L^* = \{ x \mid x = y_1 y_2 \dots y_k \text{ for some } k \geq 0, \text{ where every } y \text{ is in } L \}$
 $= L^0 \cup L^1 \cup L^2 \cup \dots$
- Note: ε is in L^* for every L , since it's in L^0 .
- Example: What is $\emptyset^*$?
 - Apply the definition:

$$\emptyset^* = \emptyset^0 \cup \emptyset^1 \cup \emptyset^2 \cup \dots$$

The rest of these are just $\emptyset$.

This is $\{ \varepsilon \}$, by the convention that $L^0 = \{ \varepsilon \}$.

$$= \{ \varepsilon \}.$$

The Star Operation

- $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$
- Example: What is $\{a\}^*$?
 - Apply the definition:
$$\begin{aligned}\{a\}^* &= \{a\}^0 \cup \{a\}^1 \cup \{a\}^2 \cup \dots \\ &= \{\varepsilon\} \cup \{a\} \cup \{aa\} \cup \dots \\ &= \{\varepsilon, a, aa, aaa, \dots\}\end{aligned}$$
 - Abbreviate this to just a^* .
 - Note this is not just one string, but a set of strings---any number of a's.

The Star Operation

- $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$
- Example: What is Σ^* ?
 - We've already defined this to be the set of all finite strings over Σ .
 - But now it has a new formal definition:
$$\begin{aligned}\Sigma^* &= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \\ &= \{ \varepsilon \} \cup \{ \text{strings of length 1 over } \Sigma \} \\ &\quad \cup \{ \text{strings of length 2 over } \Sigma \} \\ &\quad \cup \dots \\ &= \{ \text{all finite strings over } \Sigma \}\end{aligned}$$
 - Consistent.

Summary: Language Operations

- Set operations: Union, intersection, complement, set difference
- New language operations: Concatenation, star
- Regular operations:
 - Of these six operations, we identify three as **regular operations**: union, concatenation, star.
 - We'll revisit these next time, when we define **regular expressions**.

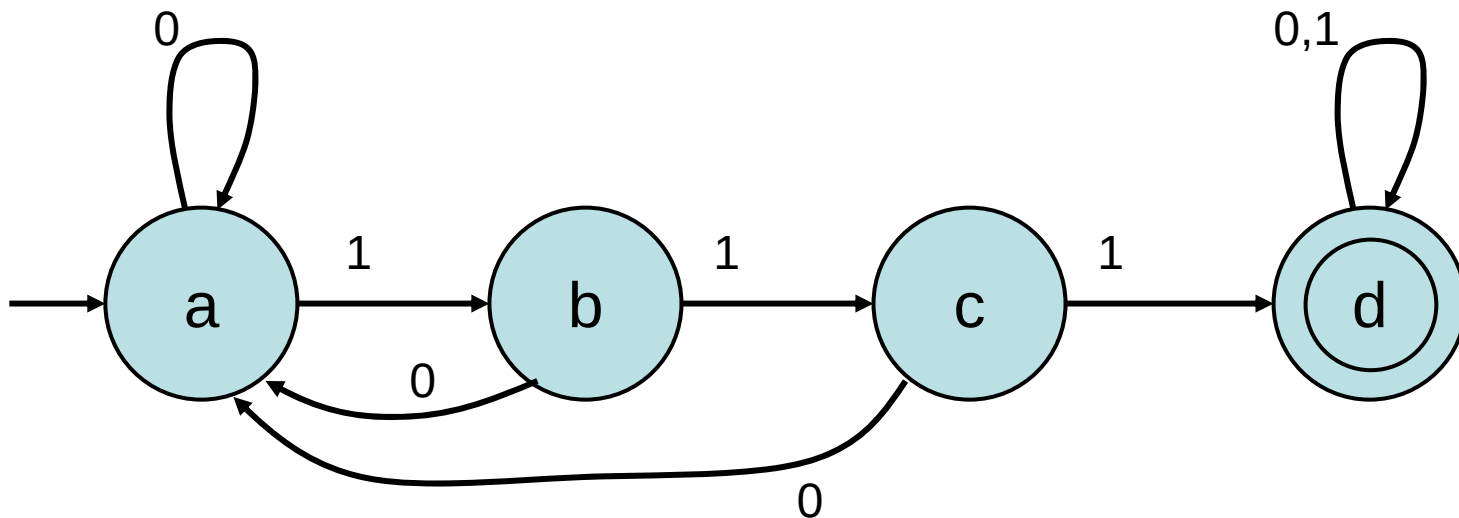
Closure of regular (FA-
recognizable) languages under
all six operations

Closure under operations

- The set of FA-recognizable languages is closed under all six operations (union, intersection, complement, set difference, concatenation, star).
- This means: If we start with FA-recognizable languages and apply any of these operations, we get another FA-recognizable language (for a different FA).
- **Theorem 1:** FA-recognizable languages are closed under complement.
- **Proof:**
 - Start with a language L_1 over alphabet Σ , recognized by some FA, M_1 .
 - Produce another FA, M_2 , with $L(M_2) = \Sigma^* - L(M_1)$.
 - Just interchange accepting and non-accepting states.

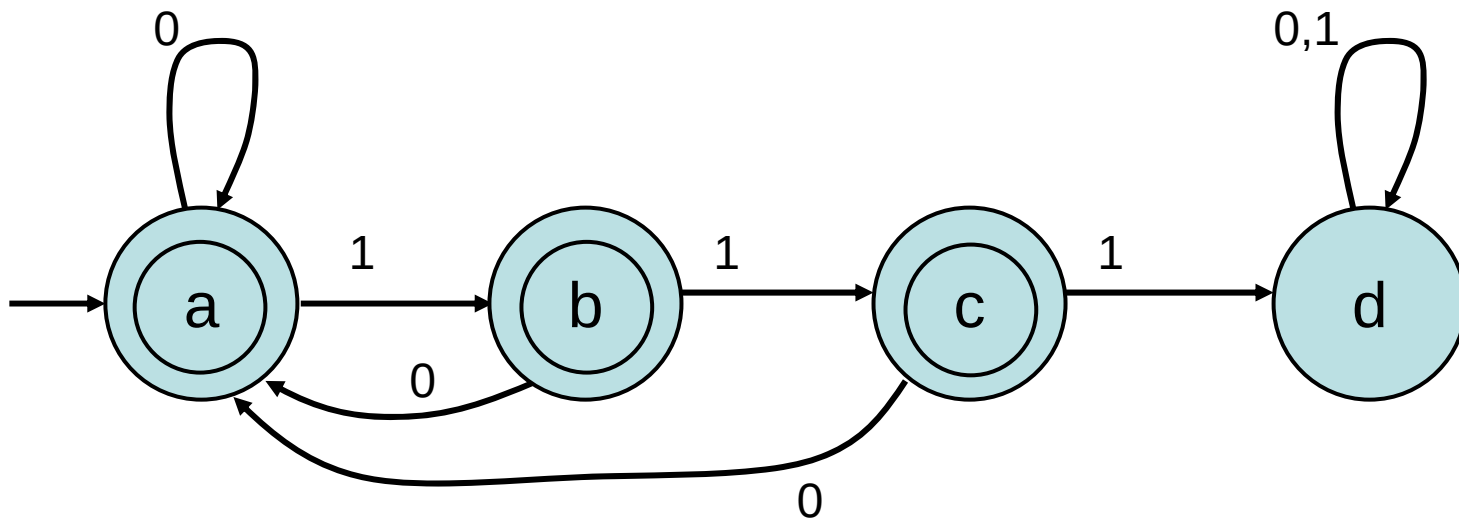
Closure under complement

- **Theorem 1:** FA-recognizable languages are closed under complement.
- **Proof:** Interchange accepting and non-accepting states.
- **Example:** FA for $\{ w \mid w \text{ does not contain } 111 \}$
 - Start with FA for $\{ w \mid w \text{ contains } 111 \}$:



Closure under complement

- **Theorem 1:** FA-recognizable languages are closed under complement.
- **Proof:** Interchange accepting and non-accepting states.
- **Example:** FA for $\{ w \mid w \text{ does not contain } 111 \}$
 - Interchange accepting and non-accepting states:



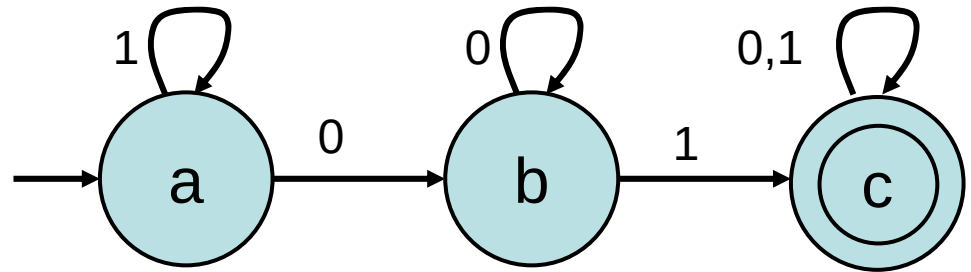
Closure under intersection

- **Theorem 2:** FA-recognizable languages are closed under intersection.
- **Proof:**
 - Start with FAs M_1 and M_2 for the same alphabet Σ .
 - Get another FA, M_3 , with $L(M_3) = L(M_1) \cap L(M_2)$.
 - Idea: Run M_1 and M_2 “in parallel” on the same input. If both reach accepting states, accept.
 - Example:
 - $L(M_1)$: Contains substring 01.
 - $L(M_2)$: Odd number of 1s.
 - $L(M_3)$: Contains 01 and has an odd number of 1s.

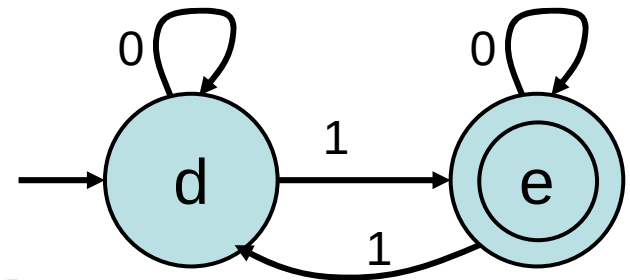
Closure under intersection

- Example:

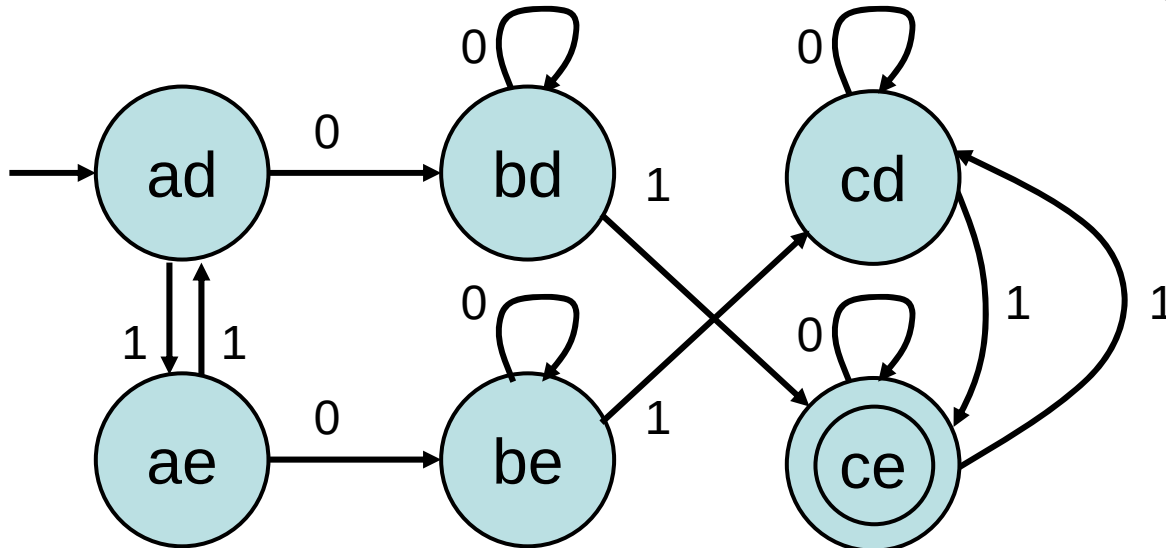
M_1 : Substring 01



M_2 : Odd number of 1s



M_3 :



Closure under intersection, general rule

- Assume:
 - $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$
 - $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$
- Define $M_3 = (Q_3, \Sigma, \delta_3, q_{03}, F_3)$, where
 - $Q_3 = Q_1 \times Q_2$
 - Cartesian product, $\{(q_1, q_2) \mid q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$
 - $\delta_3((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$
 - $q_{03} = (q_{01}, q_{02})$
 - $F_3 = F_1 \times F_2 = \{(q_1, q_2) \mid q_1 \in F_1 \text{ and } q_2 \in F_2\}$

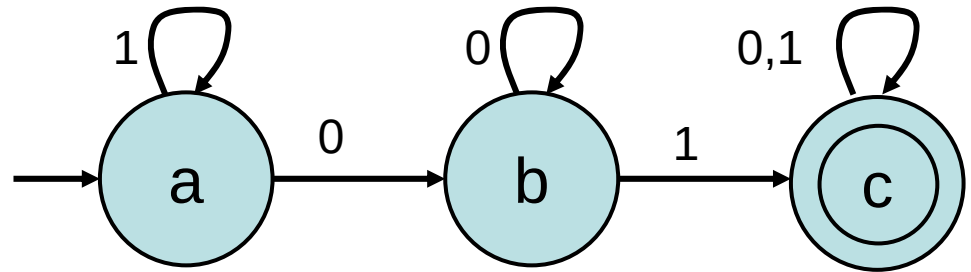
Closure under union

- **Theorem 3:** FA-recognizable languages are closed under union.
- **Proof:**
 - Similar to intersection.
 - Start with FAs M_1 and M_2 for the same alphabet Σ .
 - Get another FA, M_3 , with $L(M_3) = L(M_1) \cup L(M_2)$.
 - Idea: Run M_1 and M_2 “in parallel” on the same input. If **either reaches an accepting state**, accept.
 - Example:
 - $L(M_1)$: Contains substring 01.
 - $L(M_2)$: Odd number of 1s.
 - $L(M_3)$: Contains 01 **or** has an odd number of 1s.

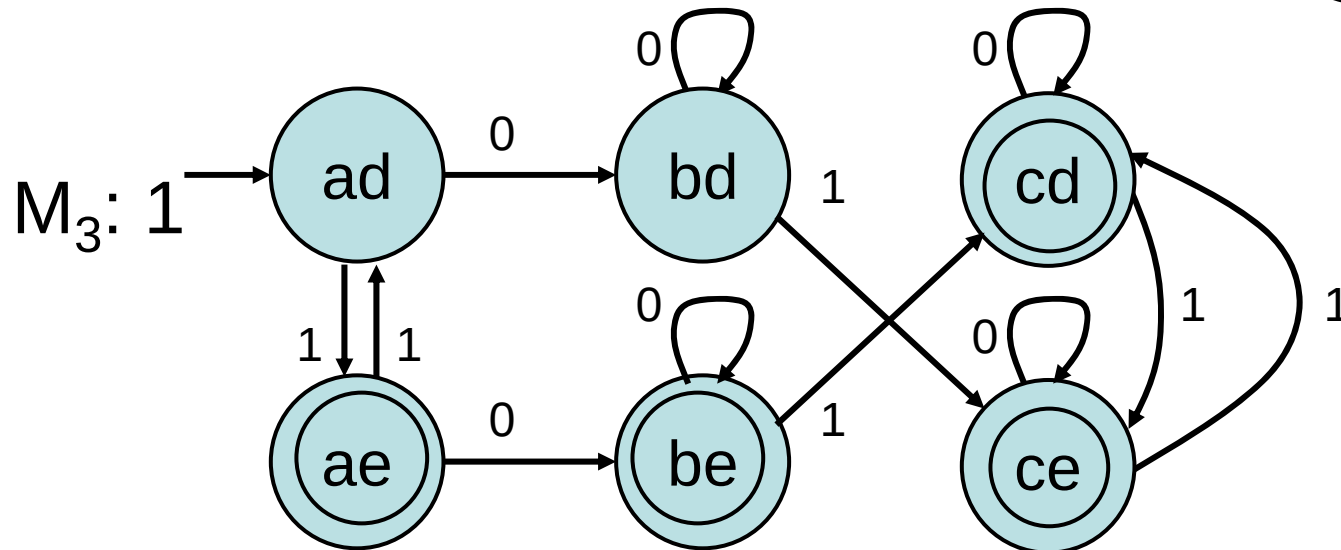
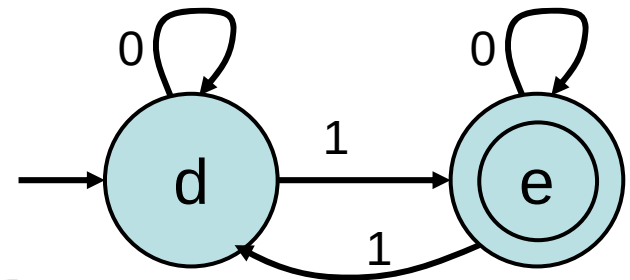
Closure under union

- Example:

M_1 : Substring 01



M_2 : Odd number of 1s



Closure under union, general rule

- Assume:
 - $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$
 - $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$
- Define $M_3 = (Q_3, \Sigma, \delta_3, q_{03}, F_3)$, where
 - $Q_3 = Q_1 \times Q_2$
 - Cartesian product, $\{(q_1, q_2) \mid q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$
 - $\delta_3((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$
 - $q_{03} = (q_{01}, q_{02})$
 - $F_3 = \{ (q_1, q_2) \mid q_1 \in F_1 \text{ or } q_2 \in F_2 \}$

Closure under set difference

- **Theorem 4:** FA-recognizable languages are closed under set difference.
- **Proof:**
 - Similar proof to those for union and intersection.
 - Alternatively, since $L_1 - L_2$ is the same as $L_1 \cap (L_2)^c$, we can just apply Theorems 2 and 3.

Closure under concatenation

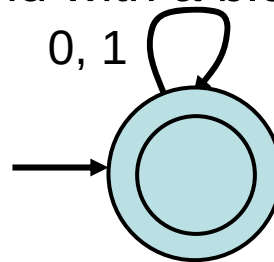
- **Theorem 5:** FA-recognizable languages are closed under concatenation.
- **Proof:**
 - Start with FAs M_1 and M_2 for the same alphabet Σ .
 - Get another FA, M_3 , with $L(M_3) = L(M_1) \circ L(M_2)$, which is $\{ x_1 x_2 \mid x_1 \in L(M_1) \text{ and } x_2 \in L(M_2) \}$
 - Idea: ???
 - Attach accepting states of M_1 somehow to the start state of M_2 .
 - But we have to be careful, since we don't know when we're done with the part of the string in $L(M_1)$ ---the string could go through accepting states of M_1 several times.

Closure under concatenation

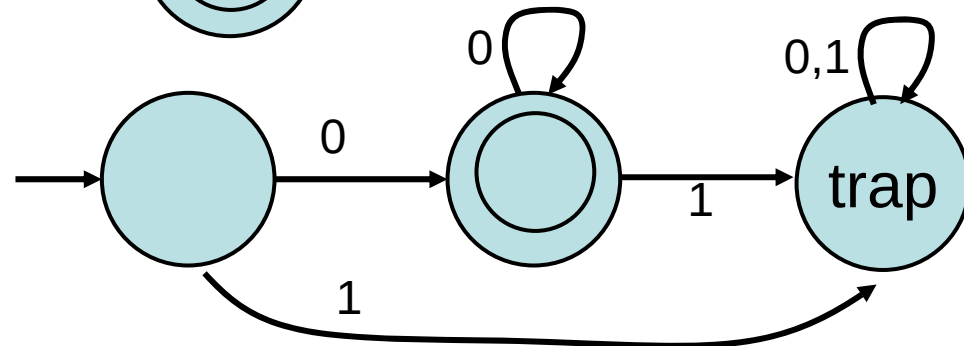
- **Theorem 5:** FA-recognizable languages are closed under concatenation.

- **Example:**

- $\Sigma = \{0, 1\}$, $L_1 = \Sigma^*$, $L_2 = \{0\}^+$ (just 0s, at least one).
- $L_1 L_2 =$ strings that end with a block of at least one 0
- M_1 :



- M_2 :



- How to combine?
- We seem to need to “guess” when to shift to M_2 .
- Leads to our next model, NFAs, which are FAs that can guess.

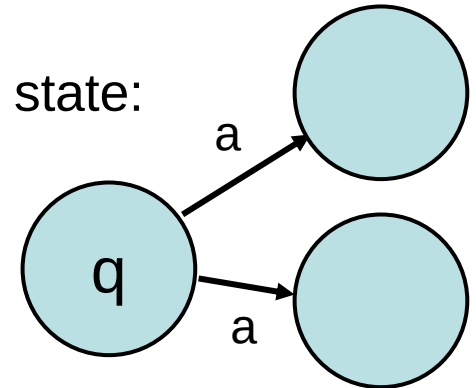
Closure under star

- **Theorem 6:** FA-recognizable languages are closed under star.
- **Proof:**
 - Start with FA M_1 .
 - Get another FA, M_2 , with $L(M_2) = L(M_1)^*$.
 - Same problems as for concatenation---need guessing.
 - ...
 - We'll define NFAs next, then return to complete the proofs of Theorems 5 and 6.

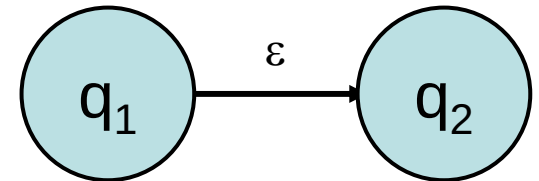
Nondeterministic Finite Automata

Nondeterministic Finite Automata

- Generalize FAs by adding **nondeterminism**, allowing several alternative computations on the same input string.
- Ordinary deterministic FAs follow one path on each input.
- Two changes:
 - Allow $\delta(q, a)$ to specify more than one successor state:



- Add ϵ -transitions, transitions made “for free”, without “consuming” any input symbols.



- Formally, combine these changes:

Formal Definition of an NFA

- An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma_\epsilon \rightarrow P(Q)$ is the transition function,

The arguments
are a state and either
an alphabet symbol or
 ϵ . Σ_ϵ means $\Sigma \cup \{\epsilon\}$.

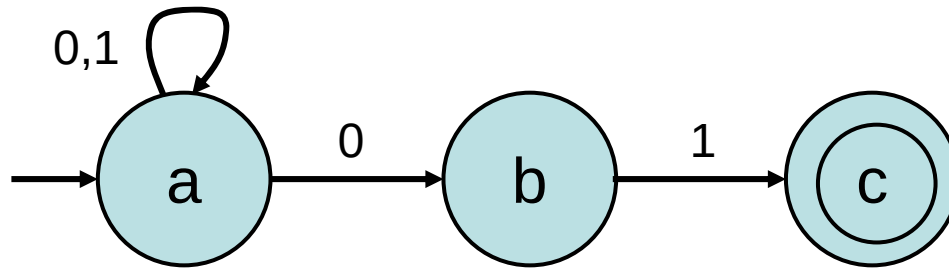
The result is a set of states.

- $q_0 \in Q$, is the start state, and
- $F \subseteq Q$ is the set of accepting, or final states.

Formal Definition of an NFA

- An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma_{\varepsilon} \rightarrow P(Q)$ is the transition function,
 - $q_0 \in Q$, is the start state, and
 - $F \subseteq Q$ is the set of accepting, or final states.
- How many states in $P(Q)$?
 $2^{|Q|}$
- Example: $Q = \{a, b, c\}$
 $P(Q) = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\} \}$

NFA Example 1



$Q = \{ a, b, c \}$

$\Sigma = \{ 0, 1 \}$

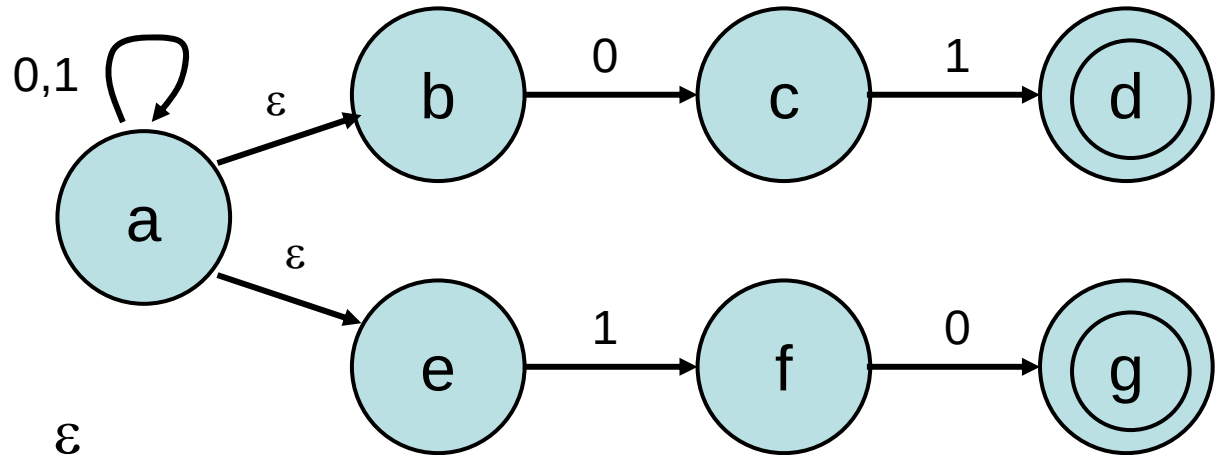
$q_0 = a$

$F = \{ c \}$

δ :

	0	1	ϵ
a	$\{a, b\}$	$\{a\}$	$\emptyset$
b	$\emptyset$	$\{c\}$	$\emptyset$
c	$\emptyset$	$\emptyset$	$\emptyset$

NFA Example 2



	0	1	ε
a	{a}	{a}	{b,c}
b	{c}	$\emptyset$	$\emptyset$
c	$\emptyset$	{d}	$\emptyset$
d	$\emptyset$	$\emptyset$	$\emptyset$
e	$\emptyset$	{f}	$\emptyset$
f	{g}	$\emptyset$	$\emptyset$
g	$\emptyset$	$\emptyset$	$\emptyset$

Next time...

- NFAs and how they compute
- NFAs vs. FAs
- Closure of regular languages under languages operations, revisited
- Regular expressions
- Regular expressions denote FA-recognizable languages.
- **Reading:** Sipser, Sections 1.2, 1.3

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 4
Nancy Lynch

Today

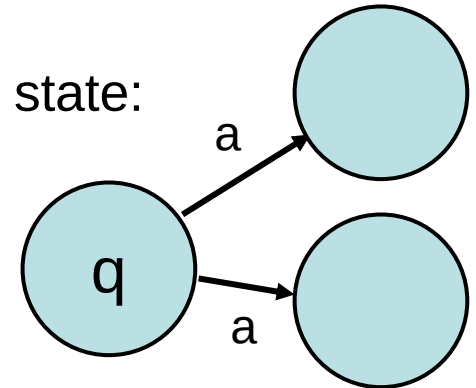
- Two more models of computation:
 - Nondeterministic Finite Automata (NFAs)
 - Add a guessing capability to FAs.
 - But provably equivalent to FAs.
 - Regular expressions
 - A different sort of model---expressions rather than machines.
 - Also provably equivalent.
- Topics:
 - Nondeterministic Finite Automata and the languages they recognize
 - NFAs vs. FAs
 - Closure of FA-recognizable languages under various operations, revisited
 - Regular expressions
 - Regular expressions denote FA-recognizable languages
- Reading: Sipser, Sections 1.2, 1.3
- Next: Section 1.4

Nondeterministic Finite Automata and the languages they recognize

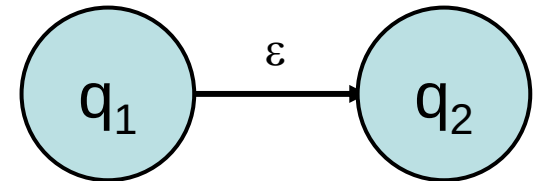
Nondeterministic Finite Automata

- Generalize FAs by adding **nondeterminism**, allowing several alternative computations on the same input string.
- Ordinary deterministic FAs follow one path on each input.
- Two changes:

- Allow $\delta(q, a)$ to specify more than one successor state:



- Add ϵ -transitions, transitions made “for free”, without “consuming” any input symbols.



- Formally, combine these changes:

Formal Definition of an NFA

- An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma_\epsilon \rightarrow P(Q)$ is the transition function,

The arguments
are a state and either
an alphabet symbol or
 ϵ . Σ_ϵ means $\Sigma \cup \{\epsilon\}$.

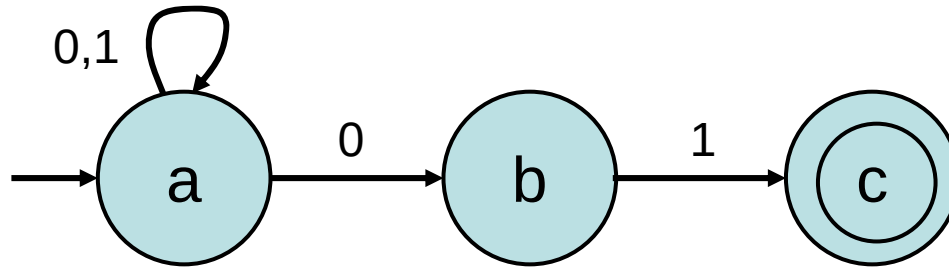
The result is a set of states.

- $q_0 \in Q$, is the start state, and
 - $F \subseteq Q$ is the set of accepting, or final states.

Formal Definition of an NFA

- An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma_{\varepsilon} \rightarrow P(Q)$ is the transition function,
 - $q_0 \in Q$, is the start state, and
 - $F \subseteq Q$ is the set of accepting, or final states.
- How many states in $P(Q)$?
 $2^{|Q|}$
- Example: $Q = \{a, b, c\}$
 $P(Q) = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\} \}$

NFA Example 1



$Q = \{ a, b, c \}$

$\Sigma = \{ 0, 1 \}$

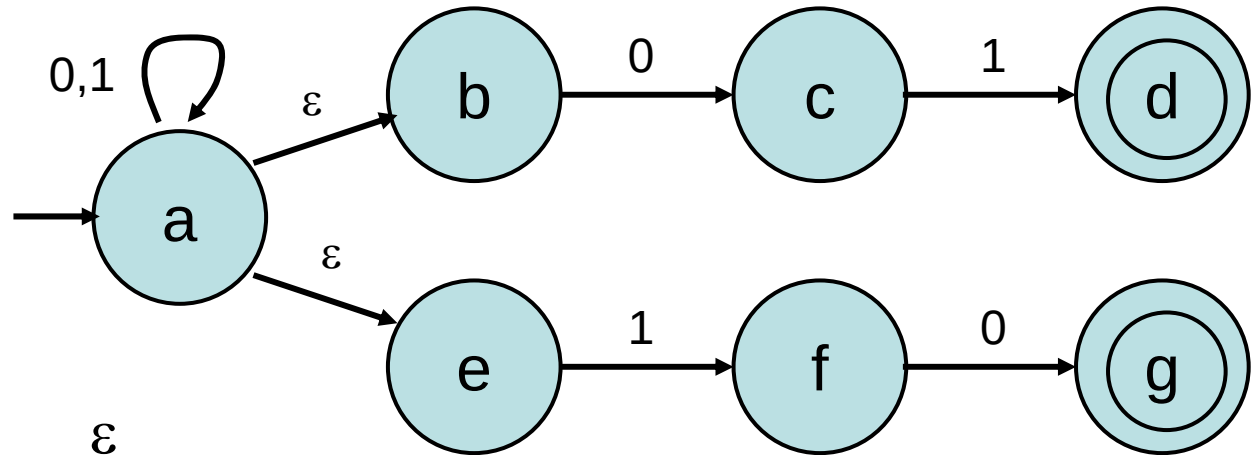
$q_0 = a$

$F = \{ c \}$

δ :

	0	1	ε
a	$\{a, b\}$	$\{a\}$	$\emptyset$
b	$\emptyset$	$\{c\}$	$\emptyset$
c	$\emptyset$	$\emptyset$	$\emptyset$

NFA Example 2



	0	1	ϵ
a	{a}	{a}	{b,c}
b	{c}	$\emptyset$	$\emptyset$
c	$\emptyset$	{d}	$\emptyset$
d	$\emptyset$	$\emptyset$	$\emptyset$
e	$\emptyset$	{f}	$\emptyset$
f	{g}	$\emptyset$	$\emptyset$
g	$\emptyset$	$\emptyset$	$\emptyset$

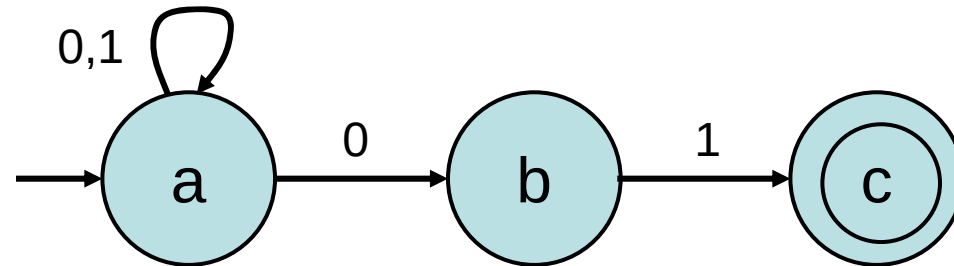
Nondeterministic Finite Automata

- NFAs are like DFAs with two additions:
 - Allow $\delta(q, a)$ to specify more than one successor state.
 - Add ε -transitions.
- Formally, an NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
 - Q is a finite set of states,
 - Σ is a finite set (alphabet) of input symbols,
 - $\delta: Q \times \Sigma_\varepsilon \rightarrow P(Q)$ is the transition function,

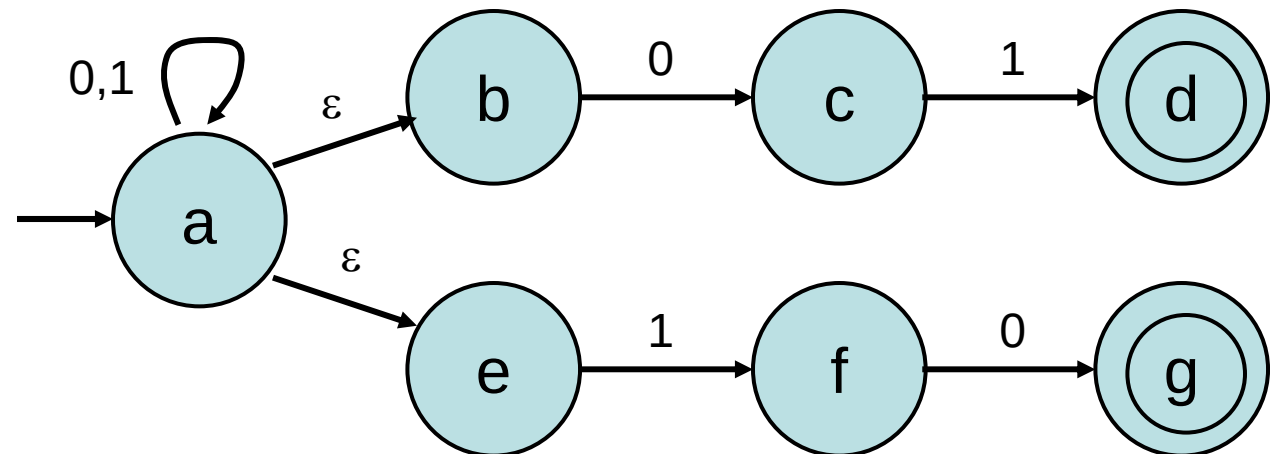
Σ_ε means $\Sigma \cup \{\varepsilon\}$.
 - $q_0 \in Q$, is the start state, and
 - $F \subseteq Q$ is the set of accepting, or final states.

NFA Examples

Example 1:



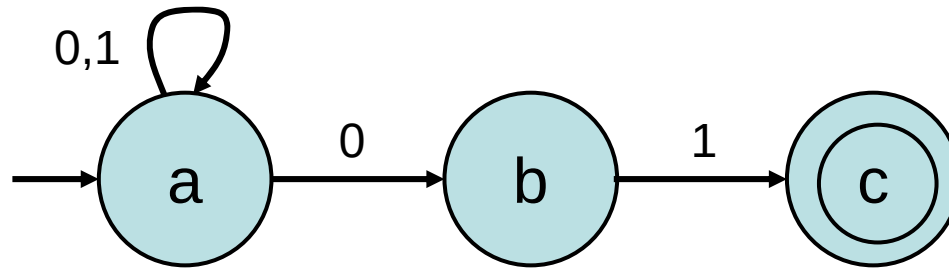
Example 2:



How NFAs compute

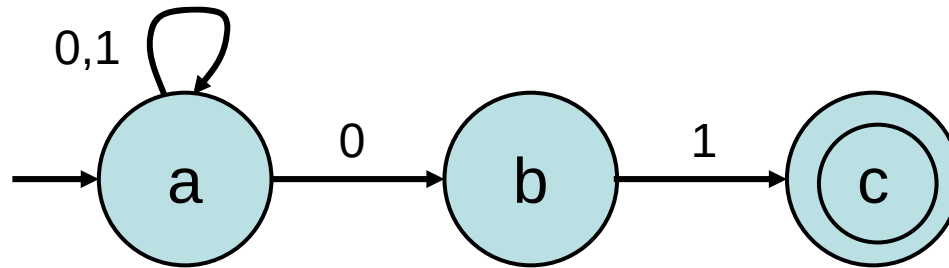
- Informally:
 - Follow allowed arrows in any possible way, while “consuming” the designated input symbols.
 - Optionally follow any ε arrow at any time, without consuming any input.
 - **Accepts** a string if **some** allowed sequence of transitions on that string leads to an accepting state.

Example 1



- $L(M) = \{ w \mid w \text{ ends with } 01 \}$
- M accepts exactly the strings in this set.
- Computations for input word $w = 101$:
 - Input word w : 1 0 1
 - States: a a a a
 - Or: a a b c
- Since c is an accepting state, M accepts 101

Example 1

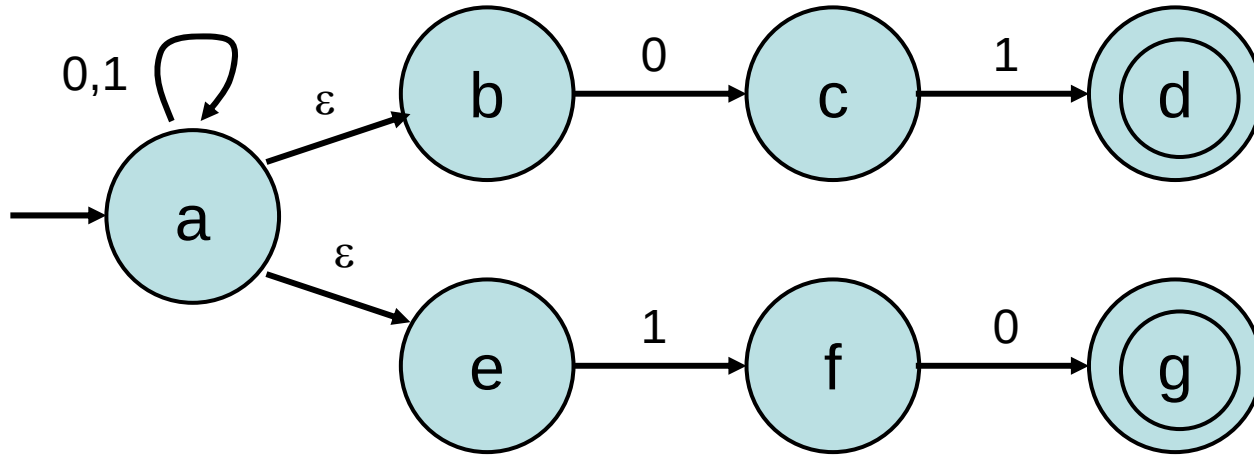


- Computations for input word $w = 0010$:
 - Possible states after 0: $\{a, b\}$
 - Then after another 0: $\{a, b\}$
 - After 1: $\{a, c\}$
 - After final 0: $\{a, b\}$
- Since neither a nor b is accepting, M does not accept 0010.

0 0 0

$\{a\} \xrightarrow{0} \{a, b\} \xrightarrow{0} \{a, b\} \xrightarrow{1} \{a, c\} \xrightarrow{0} \{a, b\}$

Example 2

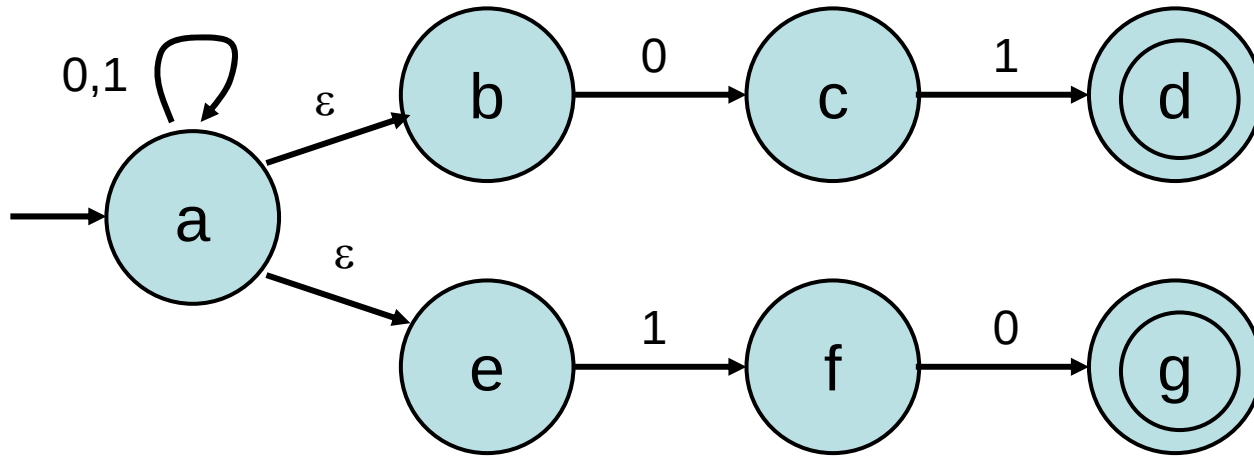


- $L(M) = \{ w \mid w \text{ ends with } 01 \text{ or } 10 \}$
- Computations for $w = 0010$:
 - Possible states after no input: $\{ a, b, e \}$
 - After 0: $\{ a, b, e, c \}$
 - After 0: $\{ a, b, e, c \}$
 - After 1: $\{ a, b, e, d, f \}$
 - After 0: $\{ a, b, e, c, g \}$
- Since g is accepting, M accepts 0010 .

0 0 1 0

$\{ a, b, e \} \xrightarrow{0} \{ a, b, e, c \} \xrightarrow{0} \{ a, b, e, c \} \xrightarrow{1} \{ a, b, e, d, f \} \xrightarrow{0} \{ a, b, e, c, g \}$

Example 2



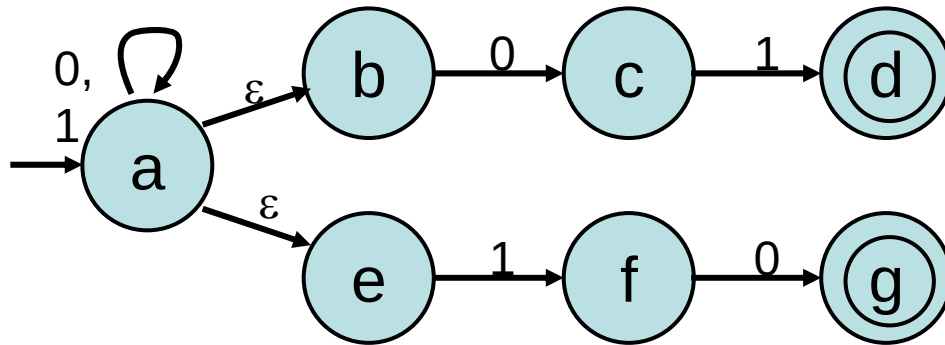
- Computations for $w = 0010$:

$$\begin{array}{ccccccc}
 & 0 & & 0 & & & \\
 \{a, b, e\} & \xrightarrow{\quad} & \{a, b, e, c\} & \xrightarrow{\quad} & \{a, b, e, c\} & & \\
 & 1 & & 0 & & & \\
 & \xrightarrow{\quad} & \{a, b, e, d, f\} & \xrightarrow{\quad} & \{a, b, e, c, g\} & &
 \end{array}$$

- Path to accepting state:

$$\begin{array}{ccccccccc}
 0 & 0 & \varepsilon & 1 & 0 & & & & \\
 a & \rightarrow & a & \rightarrow & a & \rightarrow & e & \rightarrow & f & \rightarrow & g
 \end{array}$$

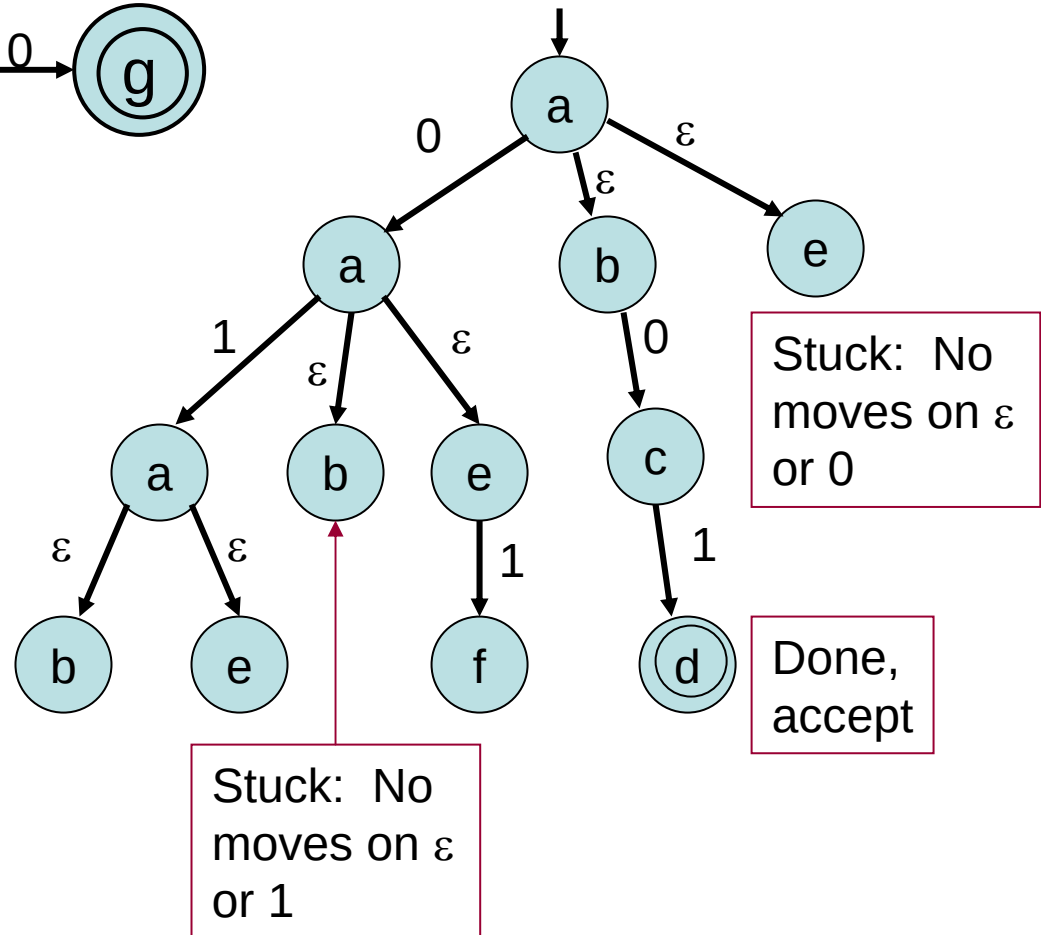
Viewing computations as a tree



Input $w = 01$

In general, accept if there is a path labeled by the entire input string, possibly interspersed with ϵ s, leading to an accepting state.

Here, leads to accepting state d .



Formal definition of computation

- Define $E(q)$ = set of states reachable from q using zero or more ε -moves (includes q itself).
- **Example 2:** $E(a) = \{ a, b, e \}$
- Define $\delta^*: Q \times \Sigma^* \rightarrow P(Q)$, state and string yield a set of states: $\delta^*(q, w)$ = states that can be reached from q by following w .
- **Defined iteratively:** Compute $\delta^*(q, a_1 a_2 \dots a_k)$ by:
 $S := E(q)$
 for $i = 1$ to k do
 $S := \bigcup_{r' \in \delta(r, a_i) \text{ for some } r \text{ in } S} E(r')$
- Or define recursively, LTTR.

Formal definition of computation

- $\delta^*(q, w)$ = states that can be reached from q by following w .
- String w is **accepted** if $\delta^*(q_0, w) \cap F \neq \emptyset$, that is, at least one of the possible end states is accepting.
- String w is **rejected** if it isn't accepted.
- **$L(M)$** , the language recognized by NFA M , = $\{ w \mid w \text{ is accepted by } M \}$.

NFAs vs. FAs

NFAs vs. DFAs

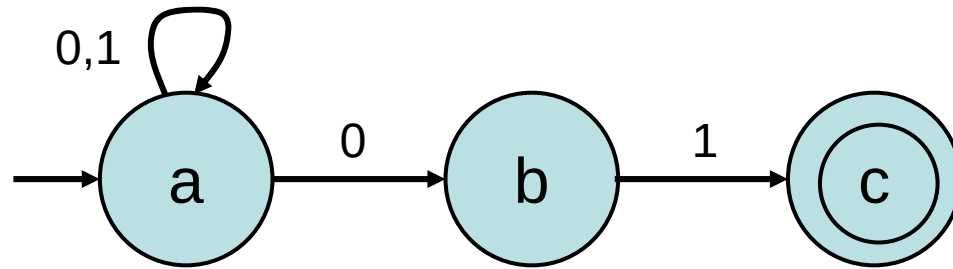
- **DFA** = Deterministic Finite Automaton, new name for ordinary Finite Automata (FA).
 - To emphasize the difference from NFAs.
- What languages are recognized by NFAs?
- Since DFAs are special cases of NFAs, NFAs recognize at least the DFA-recognizable (regular) languages.
- Nothing else!
- **Theorem:** If M is an NFA then $L(M)$ is DFA-recognizable.
- **Proof:**
 - Given NFA $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$, produce an equivalent DFA $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$.
 - Equivalent means they recognize the same language, $L(M_2) = L(M_1)$.
 - Each state of M_2 represents a set of states of M_1 : $Q_2 = P(Q_1)$.
 - Start state of M_2 is $E(\text{start state of } M_1) = \text{all states } M_1 \text{ could be in after scanning } \varepsilon$: $q_{02} = E(q_{01})$.

NFAs vs. DFAs

- **Theorem:** If M is an NFA then $L(M)$ is DFA-recognizable.
- **Proof:**
 - Given NFA $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$, produce an equivalent DFA $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$.
 - $Q_2 = P(Q_1)$
 - $q_{02} = E(q_{01})$
 - $F_2 = \{ S \subseteq Q_1 \mid S \cap F_1 \neq \emptyset \}$
 - Accepting states of M_2 are the sets that contain an accepting state of M_1 .
 - $\delta_2(S, a) = \bigcup_{r \in S} E(\delta_1(r, a))$
 - Starting from states in S , $\delta_2(S, a)$ gives all states M_1 could reach after a and possibly some ε -transitions.
 - M_2 recognizes $L(M_1)$: At any point in processing the string, the state of M_2 represents exactly the **set of states** that M_1 could be in.

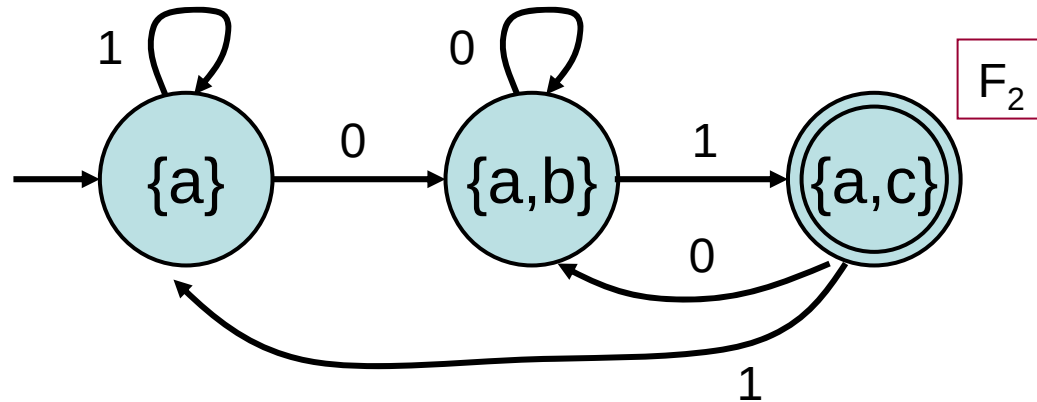
Example: NFA $\rightarrow$ DFA

- M_1 :



- States of M_2 : $\emptyset$, $\{a\}$, $\{b\}$, $\{c\}$, $\{a,b\}$, $\{a,c\}$, $\{b,c\}$, $\{a,b,c\}$

- δ_2 :

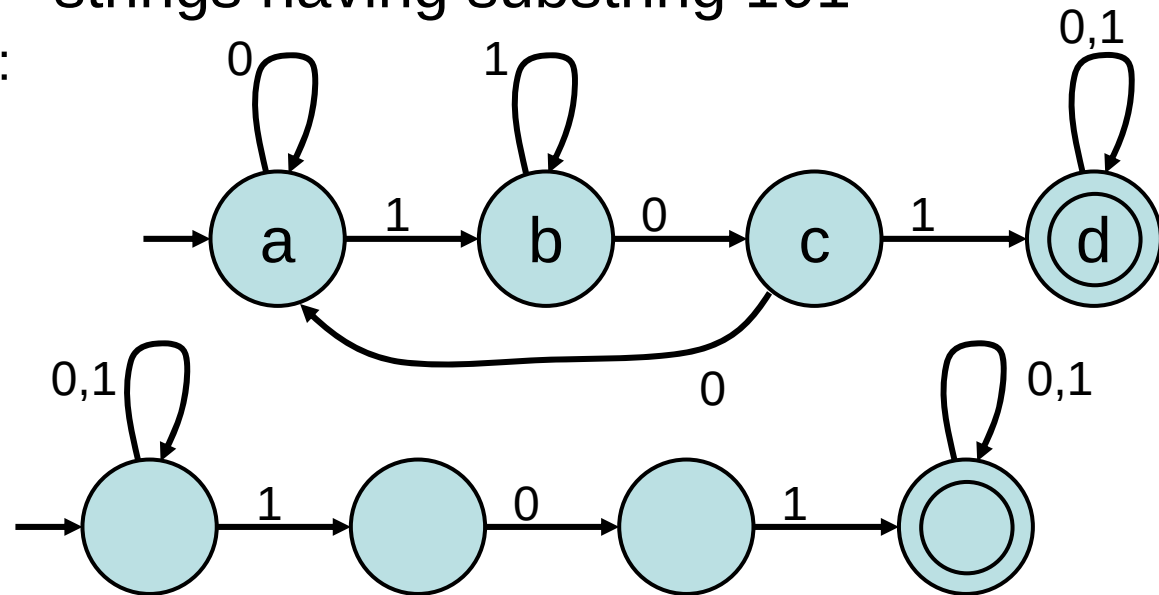


- Other 5 subsets aren't reachable from start state, don't bother drawing them.

NFAs vs. DFAs

- NFAs and DFAs have the same power.
- But sometimes NFAs are simpler than equivalent DFAs.
- **Example:** $L =$ strings ending in 01 or 10
 - Simple NFA, harder DFA (LTTR)
- **Example:** $L =$ strings having substring 101
 - Recall DFA:

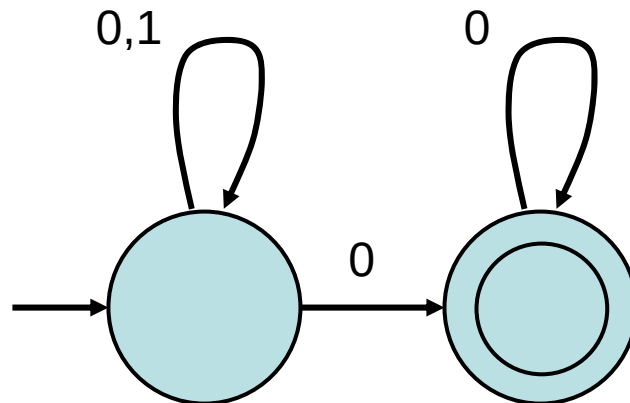
– NFA:



- Simpler---has the power to “guess” when to start matching.

NFAs vs. DFAs

- Which brings us back to last time.
- We got stuck in the proof of closure for DFA languages under concatenation:
- **Example:** $L = \{0, 1\}^* \{0\} \{0\}^*$



- NFA can guess when the critical 0 occurs.

Closure of regular (FA-recognizable) languages under various operations, revisited

Closure under operations

- The last example suggests we retry proofs of closure of FA languages under concatenation and star, this time using NFAs.
- OK since they have the same expressive power (recognize the same languages) as DFAs.
- We already proved closure under common set-theoretic operations---union, intersection, complement, difference---using DFAs.
- Got stuck on concatenation and star.
- First (warmup): Redo union proof in terms of NFAs.

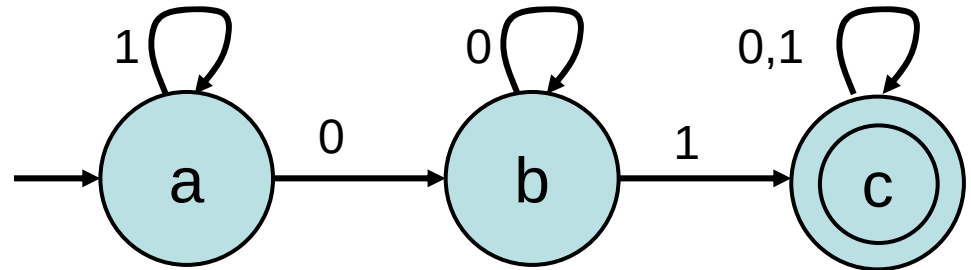
Closure under union

- **Theorem:** FA-recognizable languages are closed under union.
- **Old Proof:**
 - Start with DFAs M_1 and M_2 for the same alphabet Σ .
 - Get another DFA, M_3 , with $L(M_3) = L(M_1) \cup L(M_2)$.
 - Idea: Run M_1 and M_2 “in parallel” on the same input. If either reaches an accepting state, accept.

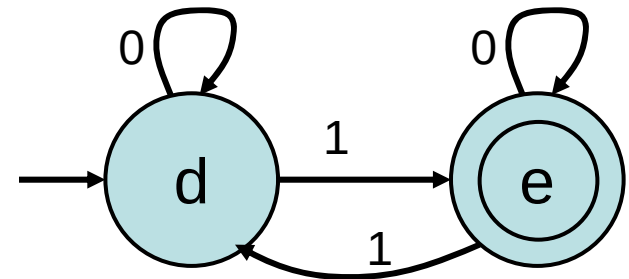
Closure under union

- Example:

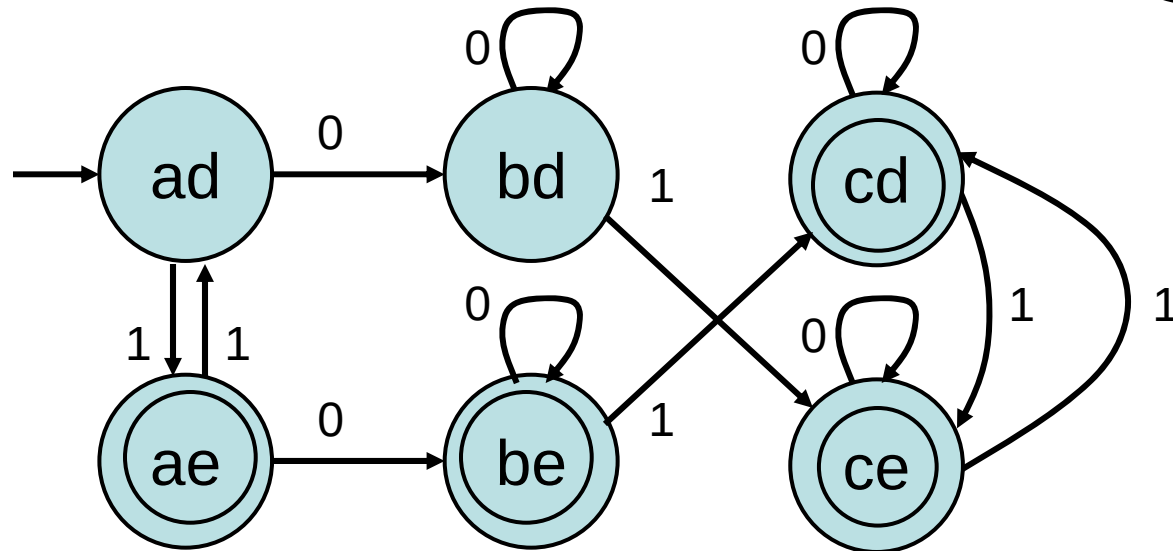
M_1 : Substring 01



M_2 : Odd number of 1s



M_3 :

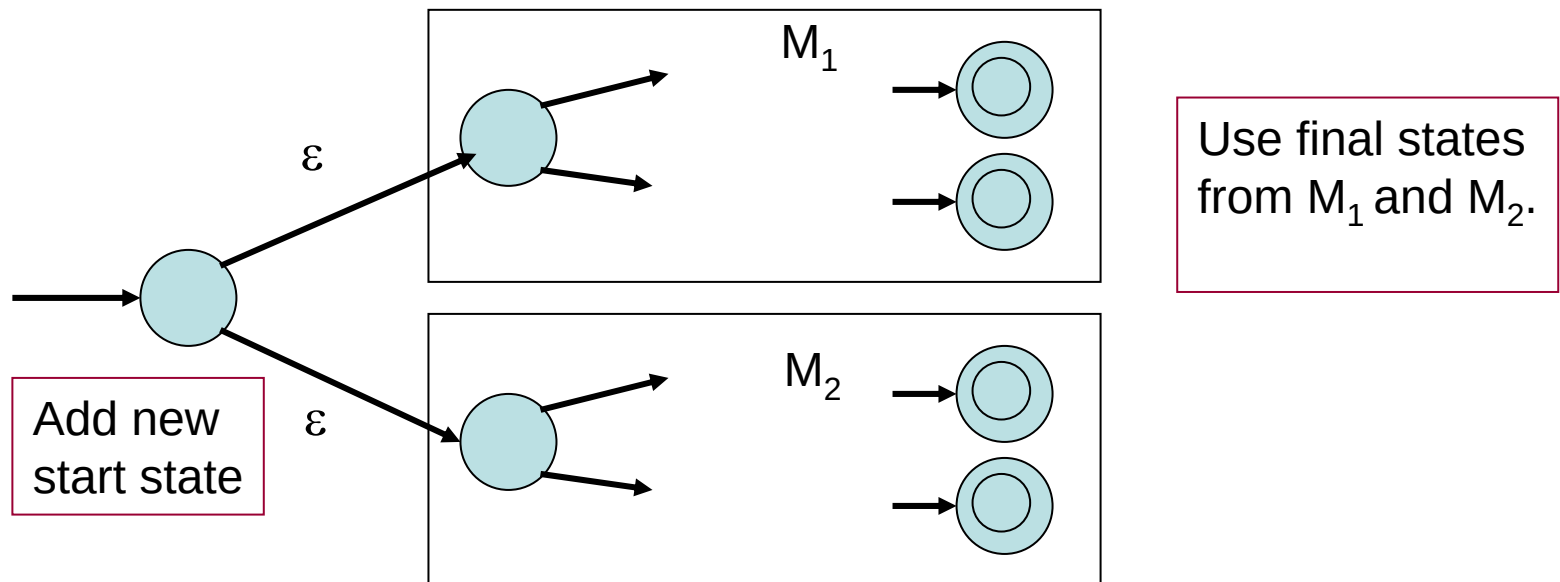


Closure under union, general rule

- Assume:
 - $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$
 - $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$
- Define $M_3 = (Q_3, \Sigma, \delta_3, q_{03}, F_3)$, where
 - $Q_3 = Q_1 \times Q_2$
 - Cartesian product, $\{(q_1, q_2) \mid q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$
 - $\delta_3((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$
 - $q_{03} = (q_{01}, q_{02})$
 - $F_3 = \{ (q_1, q_2) \mid q_1 \in F_1 \text{ or } q_2 \in F_2 \}$

Closure under union

- **Theorem:** FA-recognizable languages are closed under union.
- **New Proof:**
 - Start with NFAs M_1 and M_2 .
 - Get another NFA, M_3 , with $L(M_3) = L(M_1) \cup L(M_2)$.

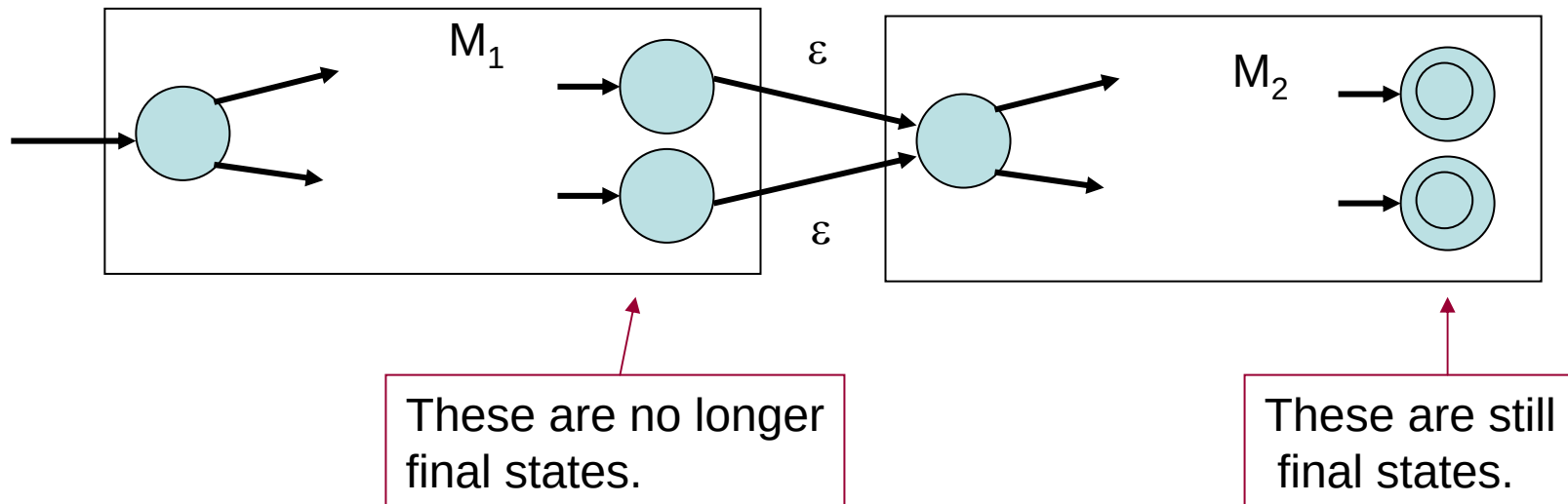


Closure under union

- **Theorem:** FA-recognizable languages are closed under union.
- **New Proof:** Simpler!
- Intersection:
 - NFAs don't seem to help.
- Concatenation, star:
 - Now try NFA-based constructions.

Closure under concatenation

- $L_1 \circ L_2 = \{ x y \mid x \in L_1 \text{ and } y \in L_2 \}$
- **Theorem:** FA-recognizable languages are closed under concatenation.
- **Proof:**
 - Start with NFAs M_1 and M_2 .
 - Get another NFA, M_3 , with $L(M_3) = L(M_1) \circ L(M_2)$.

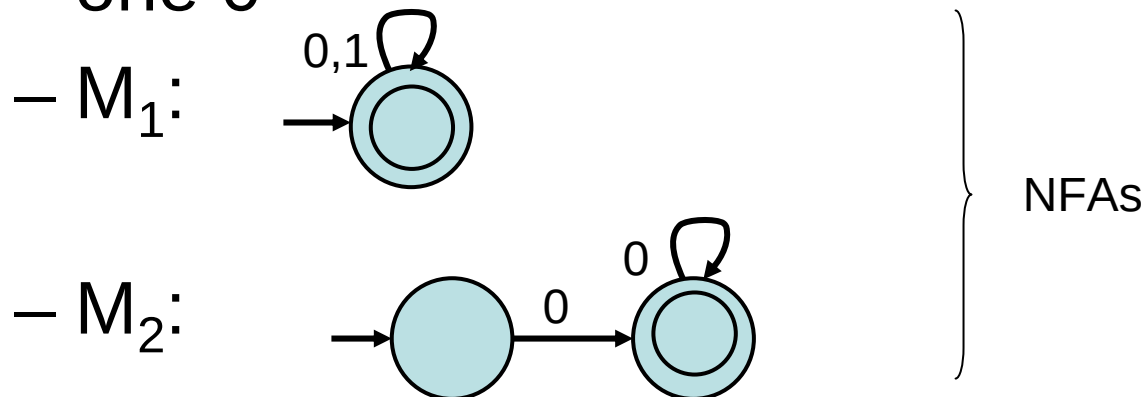


Closure under concatenation

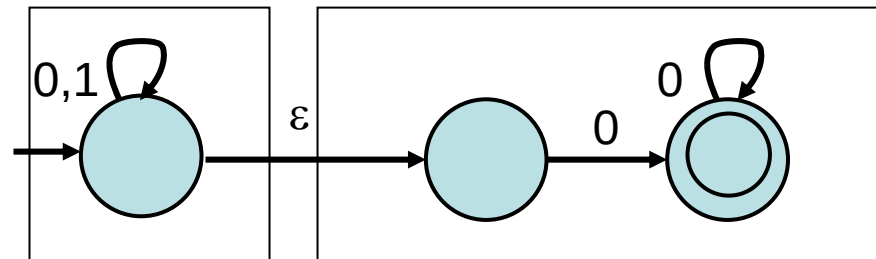
- Example:

- $\Sigma = \{0, 1\}$, $L_1 = \Sigma^*$, $L_2 = \{0\}^*$.

- $L_1 L_2 =$ strings that end with a block of at least one 0

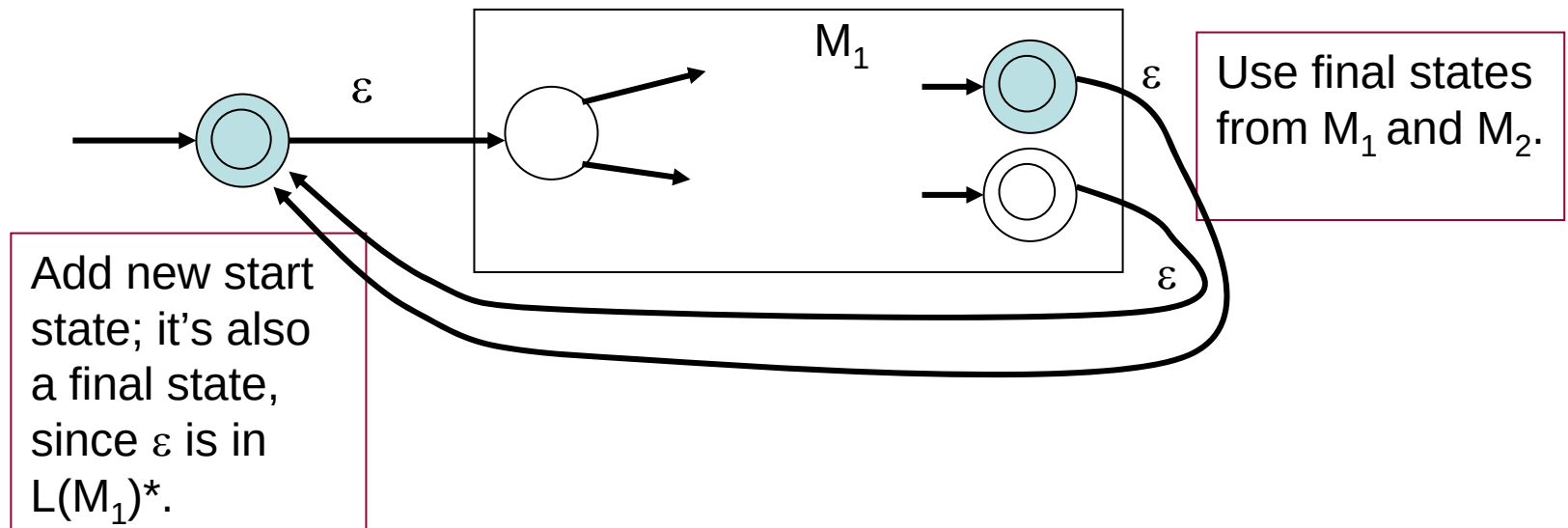


- Now combine:



Closure under star

- $L^* = \{ x \mid x = y_1 y_2 \dots y_k \text{ for some } k \geq 0, \text{ every } y \text{ in } L \}$
 $= L^0 \cup L^1 \cup L^2 \cup \dots$
- **Theorem:** FA-recognizable languages are closed under star.
- **Proof:**
 - Start with FA M_1 .
 - Get an NFA, M_2 , with $L(M_2) = L(M_1)^*$.

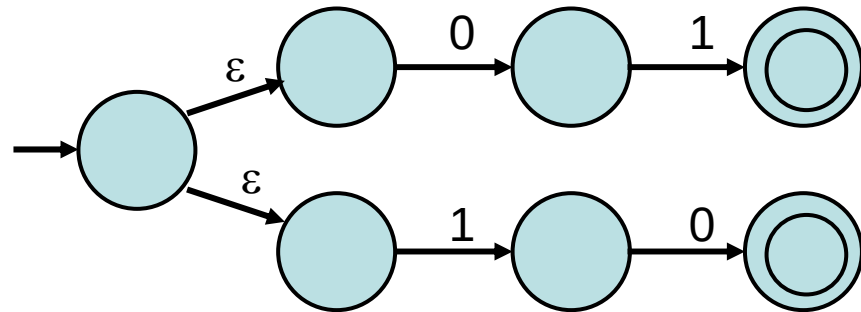


Closure under star

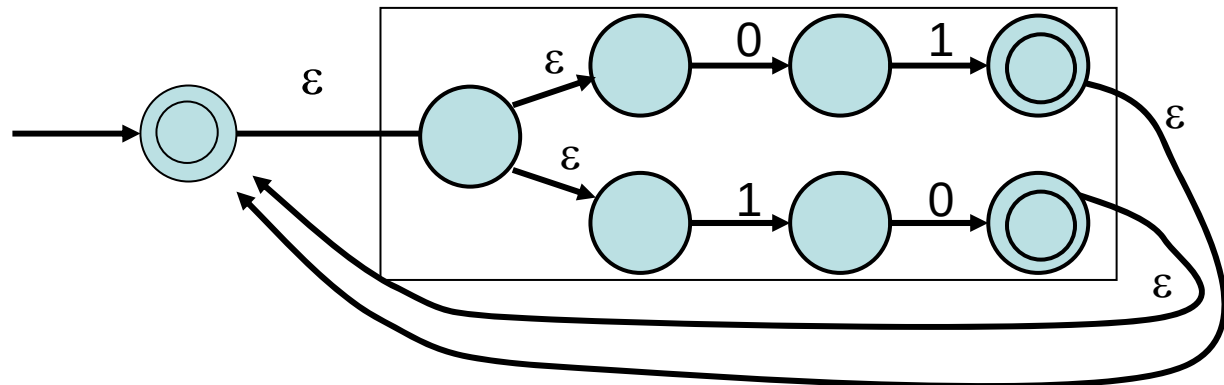
- Example:

- $\Sigma = \{0, 1\}$, $L_1 = \{01, 10\}$
- $(L_1)^* = \text{even-length strings where each pair consists of a 0 and a 1.}$

- M_1 :



- Construct M_2 :



Closure, summary

- FA-recognizable (regular) languages are closed under set operations, concatenation, and star.
- **Regular operations:** Union, concatenation, and star.
- Can be used to build regular expressions, which denote languages.
- E.g., regular expression $(0 \cup 1)^* 0 0^*$ denotes the language $\{0, 1\}^* \{0\} \{0\}^*$
- Study these next...

Regular Expressions

Regular expressions

- An algebraic-expression notation for describing (some) languages, rather than a machine representation.
- Languages described by regular expressions are exactly the FA-recognizable languages.
 - That's why FA-recognizable languages are called “regular”.
- **Definition:** R is a regular expression over alphabet Σ exactly if R is one of the following:
 - a , for some a in Σ ,
 - ε ,
 - $\emptyset$,
 - $(R_1 \cup R_2)$, where R_1 and R_2 are smaller regular expressions,
 - $(R_1 \circ R_2)$, where R_1 and R_2 are smaller regular expressions, or
 - (R_1^*) , where R_1 is a smaller regular expression.
- A recursive definition.

Regular expressions

- **Definition:** R is a regular expression over alphabet Σ exactly if R is one of the following:
 - a , for some a in Σ ,
 - ε ,
 - $\emptyset$,
 - $(R_1 \cup R_2)$, where R_1 and R_2 are smaller regular expressions,
 - $(R_1 \circ R_2)$, where R_1 and R_2 are smaller regular expressions, or
 - (R_1^*) , where R_1 is a smaller regular expression.
- These are just formal expressions---we haven't said yet what they "mean".
- **Example:** $((0 \cup 1) \circ \varepsilon)^* \cup 0$
- Abbreviations:
 - Sometimes omit $\circ$, use juxtaposition.
 - Sometimes omit parens, use precedence of operations: $*$ highest, then $\circ$, then $\cup$.
- **Example:** Abbreviate above as $((0 \cup 1) \varepsilon)^* \cup 0$
- **Example:** $(0 \cup 1)^* 111 (0 \cup 1)^*$

How regular expressions denote languages

- Define the languages recursively, based on the expression structure:
- **Definition:**
 - $L(a) = \{ a \}$; one string, with one symbol a .
 - $L(\varepsilon) = \{ \varepsilon \}$; one string, with no symbols.
 - $L(\emptyset) = \emptyset$; no strings.
 - $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$
 - $L(R_1 \circ R_2) = L(R_1) \circ L(R_2)$
 - $L(R_1^*) = (L(R_1))^*$
- **Example:** Expression $((0 \cup 1) \varepsilon)^* \cup 0$ denotes language $\{0, 1\}^* \cup \{0\} = \{0, 1\}^*$, all strings.
- **Example:** $(0 \cup 1)^* 111 (0 \cup 1)^*$ denotes $\{0, 1\}^* \{111\} \{0, 1\}^*$, all strings with substring 111.

More examples

- **Definition:**

- $L(a) = \{ a \}$; one string, with one symbol a .
- $L(\varepsilon) = \{ \varepsilon \}$; one string, with no symbols.
- $L(\emptyset) = \emptyset$; no strings.
- $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$
- $L(R_1 \circ R_2) = L(R_1) \circ L(R_2)$
- $L(R_1^*) = (L(R_1))^*$

- **Example:** L = strings over $\{ 0, 1 \}$ with odd number of 1s.

$$0^* 1 0^* (0^* 1 0^* 1 0^*)^*$$

- **Example:** L = strings with substring 01 or 10.

$$(0 \cup 1)^* 01 (0 \cup 1)^* \cup (0 \cup 1)^* 10 (0 \cup 1)^*$$

Abbreviate (writing Σ for $(0 \cup 1)$):

$$\Sigma^* 01 \Sigma^* \cup \Sigma^* 10 \Sigma^*$$

More examples

- **Example:** $L =$ strings with substring 01 or 10.

$$(0 \cup 1)^* 01 (0 \cup 1)^* \cup (0 \cup 1)^* 10 (0 \cup 1)^*$$

Abbreviate:

$$\Sigma^* 01 \Sigma^* \cup \Sigma^* 10 \Sigma^*$$

- **Example:** $L =$ strings with neither substring 01 or 10.

- Can't write complement.
- But can write: $0^* \cup 1^*$.

- **Example:** $L =$ strings with no more than two consecutive 0s or two consecutive 1s

- Would be easy if we could write complement.

$$(\varepsilon \cup 1 \cup 11) ((0 \cup 00) (1 \cup 11))^* (\varepsilon \cup 0 \cup 00)$$

- Alternate one or two of each.

More examples

- Regular expressions commonly used to specify syntax.
 - For (portions of) programming languages
 - Editors
 - Command languages like UNIX shell
- **Example:** Decimal numbers

$$D D^* . D^* \cup D^* . D D^*,$$

where D is the alphabet $\{ 0, \dots, 9 \}$

Need a digit either before or after the decimal point.

Regular Expressions Denote FA-Recognizable Languages

Languages denoted by regular expressions

- The languages denoted by regular expressions are exactly the regular (FA-recognizable) languages.
- **Theorem 1:** If R is a regular expression, then $L(R)$ is a regular language (recognized by a FA).
- **Proof:** Easy.
- **Theorem 2:** If L is a regular language, then there is a regular expression R with $L = L(R)$.
- **Proof:** Harder, more technical.

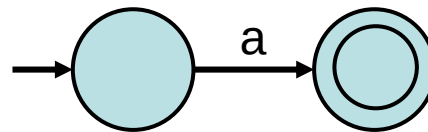
Theorem 1

- **Theorem 1:** If R is a regular expression, then $L(R)$ is a regular language (recognized by a FA).
- **Proof:**

- For each R , define an NFA M with $L(M) = L(R)$.
- Proceed by induction on the structure of R :
 - Show for the three base cases.
 - Show how to construct NFAs for more complex expressions from NFAs for their subexpressions.

- **Case 1: $R = a$**

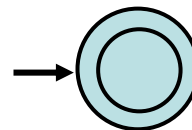
- $L(R) = \{ a \}$



Accepts only a .

- **Case 2: $R = \epsilon$**

- $L(R) = \{ \epsilon \}$



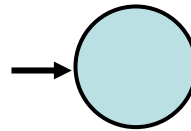
Accepts only ϵ .

Theorem 1

- **Theorem 1:** If R is a regular expression, then $L(R)$ is a regular language (recognized by a FA).
- **Proof:**

– Case 3: $R = \emptyset$

- $L(R) = \emptyset$

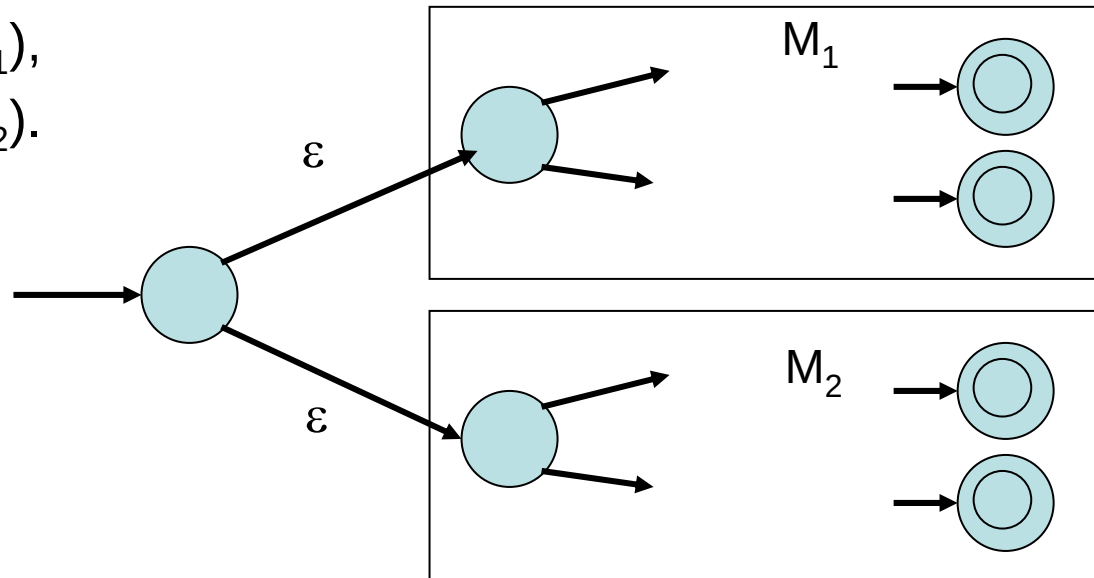


Accepts nothing.

– Case 4: $R = R_1 \cup R_2$

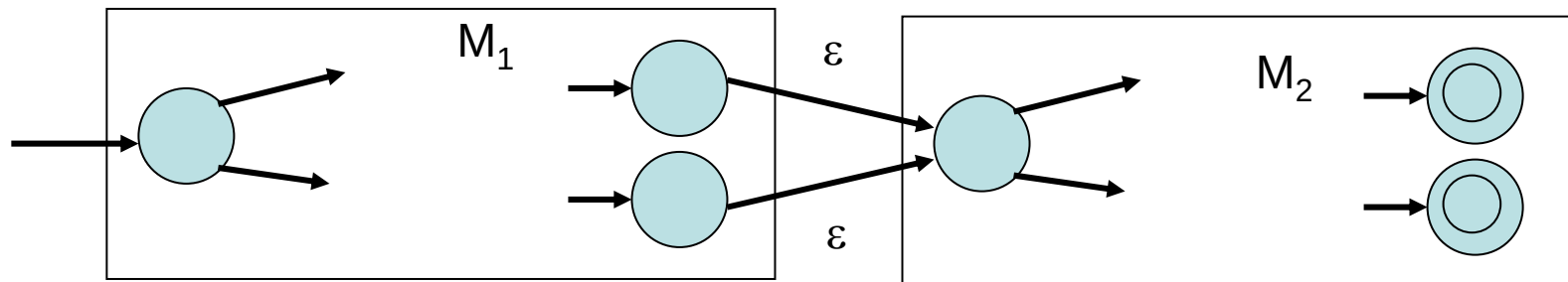
- M_1 recognizes $L(R_1)$,
- M_2 recognizes $L(R_2)$.

- Same construction we used to show regular languages are closed under union.



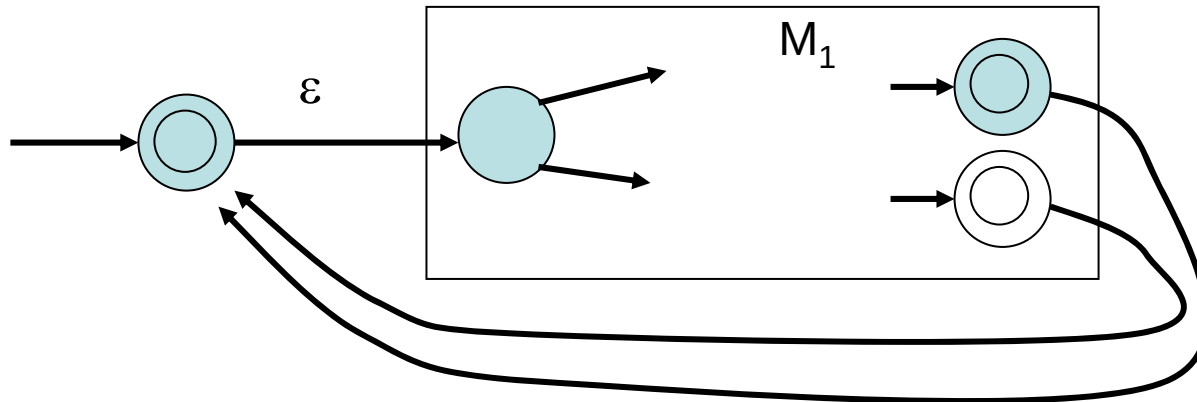
Theorem 1

- **Theorem 1:** If R is a regular expression, then $L(R)$ is a regular language (recognized by a FA).
- **Proof:**
 - **Case 5:** $R = R_1 \circ R_2$
 - M_1 recognizes $L(R_1)$,
 - M_2 recognizes $L(R_2)$.
 - Same construction we used to show regular languages are closed under concatenation.



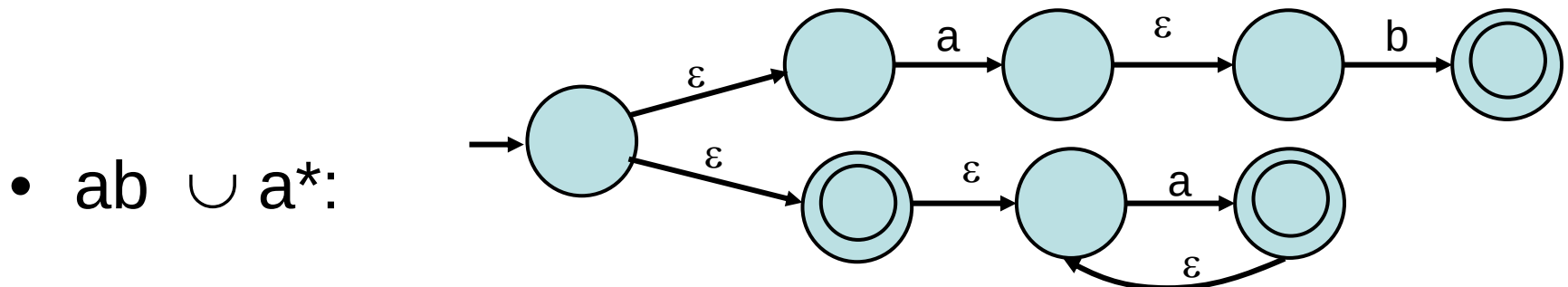
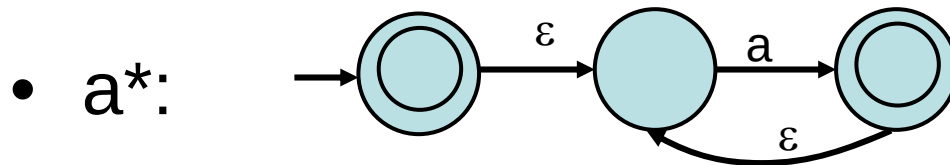
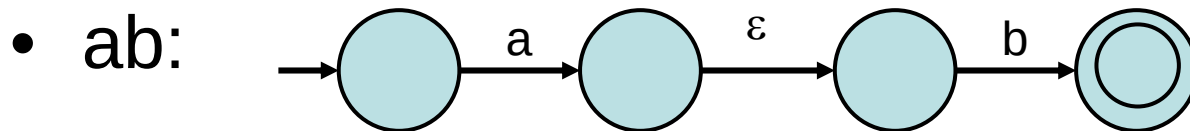
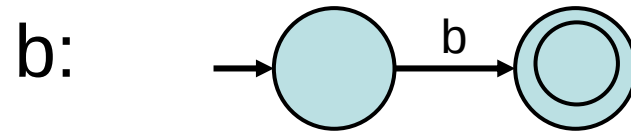
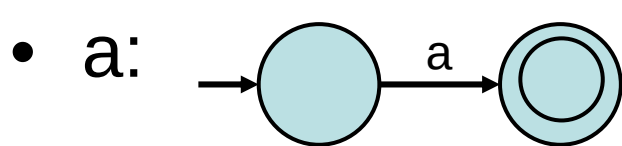
Theorem 1

- **Theorem 1:** If R is a regular expression, then $L(R)$ is a regular language (recognized by a FA).
- **Proof:**
 - **Case 6:** $R = (R_1)^*$
 - M_1 recognizes $L(R_1)$,
 - Same construction we used to show regular languages are closed under star.



Example for Theorem 1

- $L = ab \cup a^*$
- Construct machines recursively:

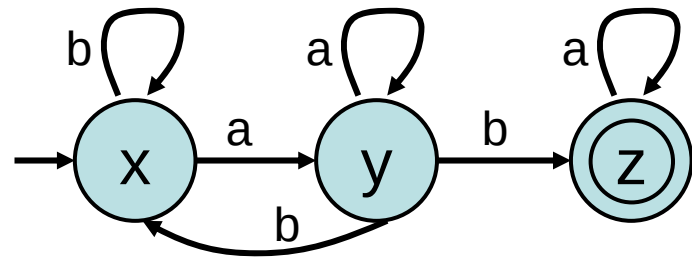


Theorem 2

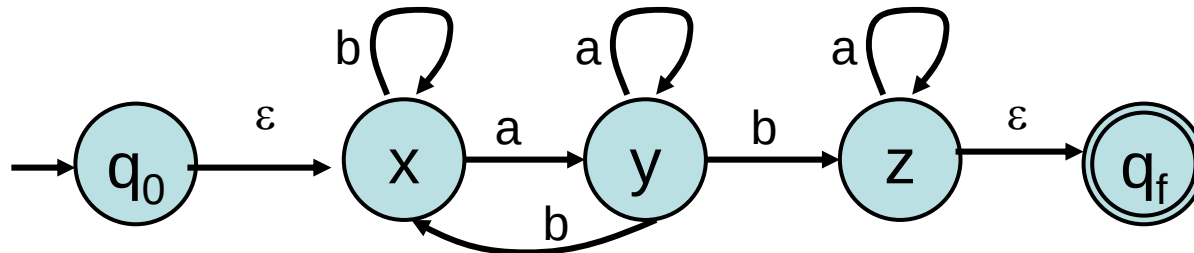
- **Theorem 2:** If L is a regular language, then there is a regular expression R with $L = L(R)$.

- **Proof:**

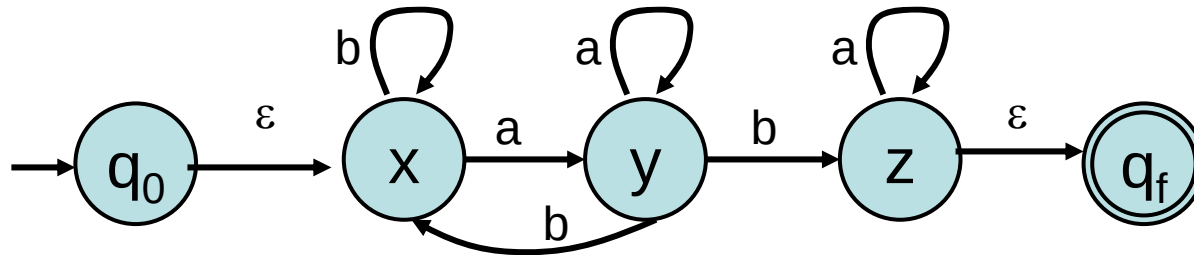
- For each NFA M , define a regular expression R with $L(R) = L(M)$.
- Show with an example:



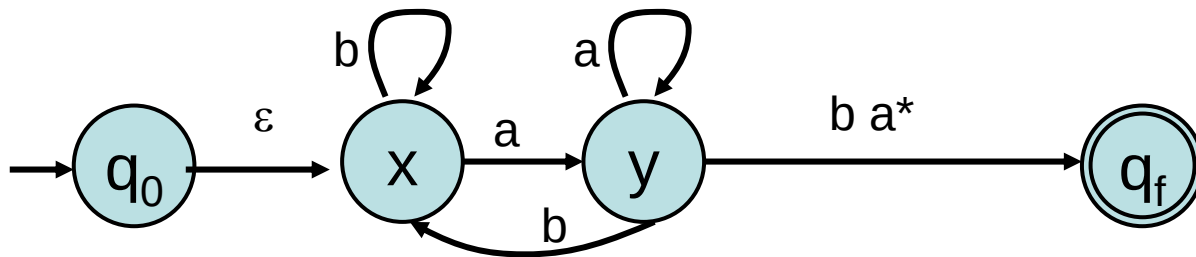
- Convert to a special form with only one final state, no incoming arrows to start state, no outgoing arrows from final state.



Theorem 2

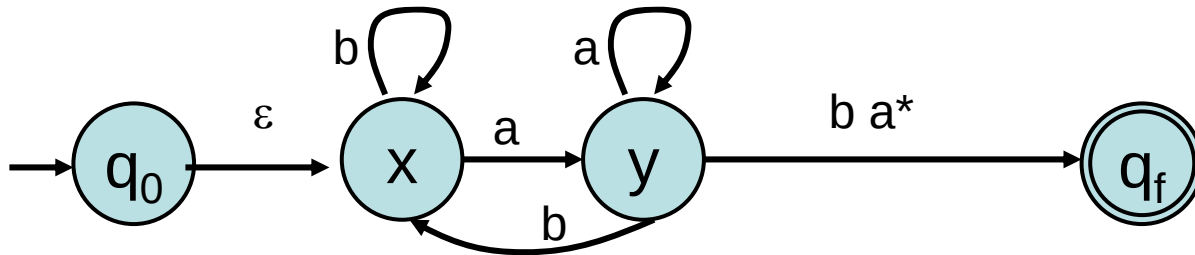


- Now remove states one at a time (any order), replacing labels of edges with more complicated regular expressions.
- First remove z:

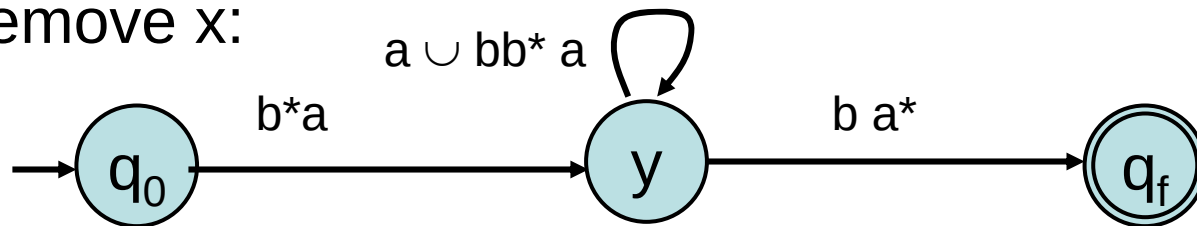


- New label $b a^*$ describes all strings that can move the machine from state y to state q_f , visiting (just) z any number of times.

Theorem 2

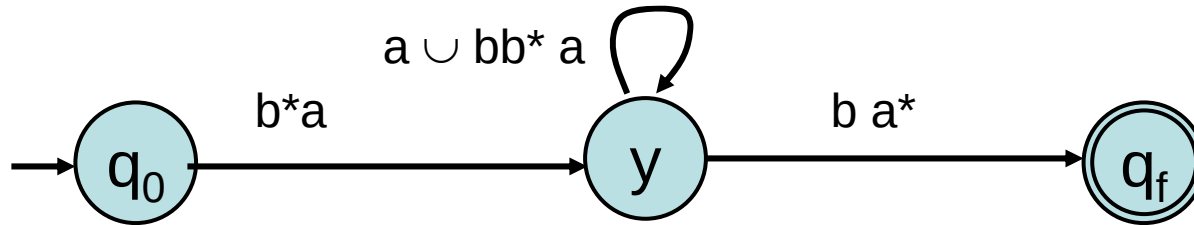


- Then remove x :

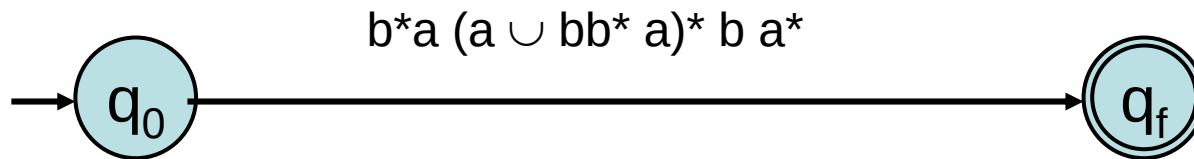


- New label b^*a describes all strings that can move the machine from q_0 to y , visiting (just) x any number of times.
- New label $a \cup bb^*a$ describes all strings that can move the machine from y to y , visiting (just) x any number of times.

Theorem 2



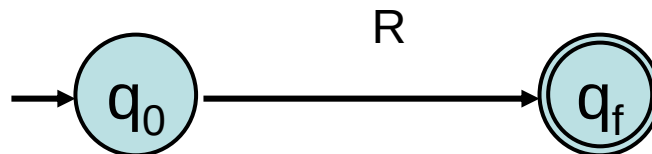
- Finally, remove y :



- New label describes all strings that can move the machine from q_0 to q_f , visiting (just) y any number of times.
- This final label is the needed regular expression.

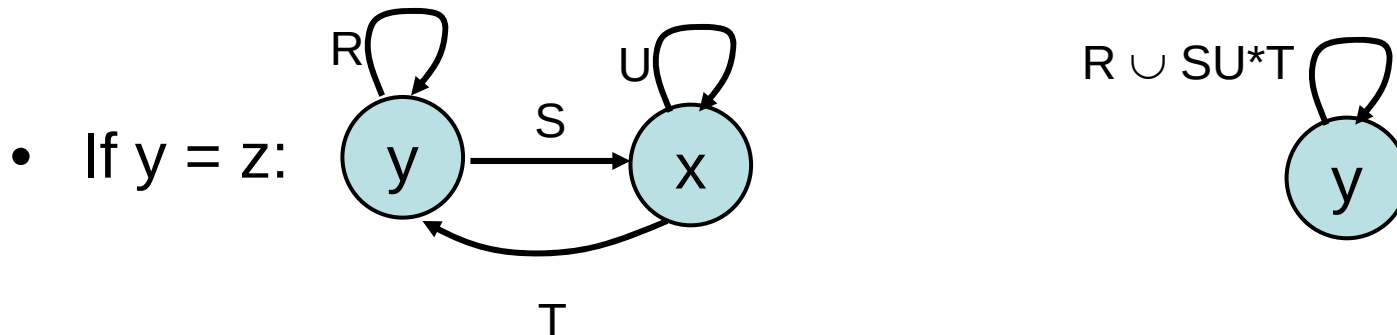
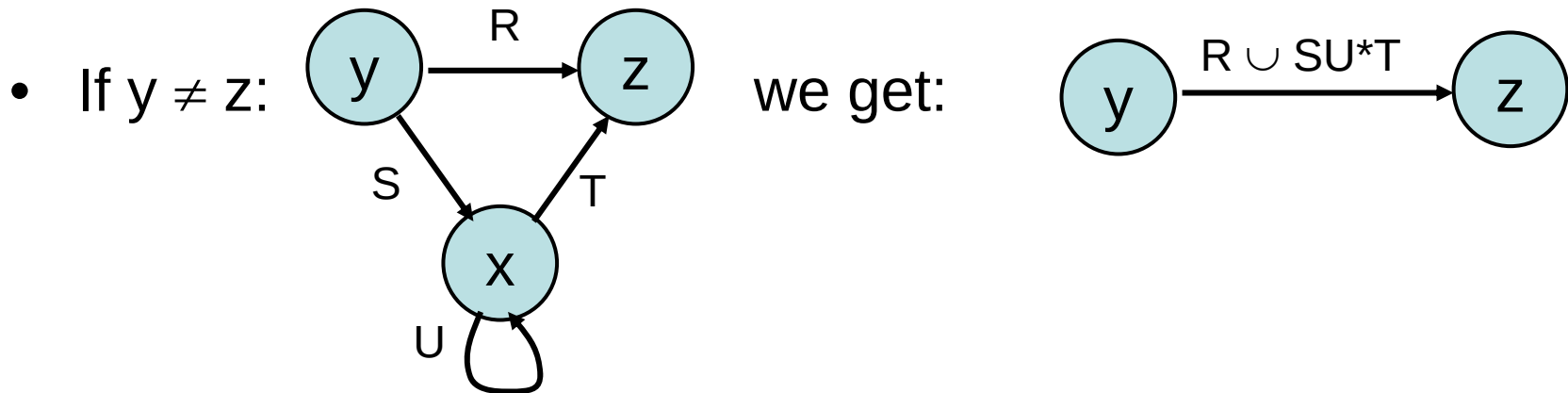
Theorem 2

- Define a **generalized NFA (gNFA)**.
 - Same as NFA, but:
 - Only one accept state, $\neq$ start state.
 - Start state has no incoming arrows, accept state no outgoing arrows.
 - Arrows are labeled with regular expressions.
 - How it computes: Follow an arrow labeled with a regular expression R while consuming a block of input that is a word in the language $L(R)$.
- Convert the original NFA M to a gNFA.
- Successively transform the gNFA to equivalent gNFAs (recognize same language), each time removing one state.
- When we have 2 states and one arrow, the regular expression R on the arrow is the final answer:



Theorem 2

- To remove a state x , consider every pair of other states, y and z , including $y = z$.
- New label for edge (y, z) is the union of two expressions:
 - What was there before, and
 - One for paths through (just) x .



Next time...

- Existence of non-regular languages
 - Showing specific languages aren't regular
 - The Pumping Lemma
 - Algorithms that answer questions about FAs.
-
- **Reading:** Sipser, Section 1.4; some pieces from 4.1

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 5
Nancy Lynch

Today

- Non-regular languages
- **Today's topics:**
 - Existence of non-regular languages
 - Showing some specific languages aren't regular
 - The Pumping Lemma
 - Examples
 - Algorithms that answer questions about FAs.
- **Reading:** Sipser, Sections 1.4, 4.1.
- **Next:**
 - Computability theory
 - Readings:
 - Sipser Chapter 3
 - The Nature of Computation, Chapter 8
 - GITCS notes, lecture 4

Existence of Non-Regular Languages

Existence of non-regular languages

- **Theorem:** There is a language over $\Sigma = \{ 0, 1 \}$ that is not regular.
- (Works for other alphabets too.)
- **Proof:**
 - Recall, a language is any (finite or infinite) set of (finite) strings.
 - It turns out that there are many more sets of finite strings than there are DFAs; so just based on cardinality, there must be some non-regular languages.
 - But, there are infinitely many sets of strings, and infinitely many DFAs---so what does it mean to say that one of these is “more” than the other?
 - Answer: There are different kinds of infinities:
 - Countably infinite sets, like the natural numbers or the integers.
 - Uncountably infinite sets, like the reals.
 - Also, different sizes of uncountable infinities.

Existence of non-regular languages

- **Theorem:** There is a language over $\Sigma = \{ 0, 1 \}$ that is not regular.
- **Proof:**
 - Follows from two claims:
 - **Claim 1:** The set of **all languages** over $\Sigma = \{ 0, 1 \}$ is uncountable, that is, it cannot be put into one-to-one correspondence with $\mathbb{N}$ (natural numbers).
 - **Claim 2:** The set of **regular languages** is countable.

Claim 1

- **Claim 1:** The set of all languages over $\Sigma = \{ 0, 1 \}$ is uncountable, that is, it cannot be put into one-to-one correspondence with $\mathbb{N}$.

- **Proof of Claim 1:** By contradiction.

- Suppose it is countable.
- Then we can put the set of all languages in one-to-one correspondence with $\mathbb{N}$, e.g.:

0		$\emptyset$	L_0
1		$\{ 0 \}$	L_1
2		All even-length strings (an infinite language)	L_2
3		All strings containing at least one 0	L_3
Etc.			

All (finite and infinite) sets of (finite) strings must appear in this list.

Claim 1

- **Claim 1:** The set of all languages over $\Sigma = \{ 0, 1 \}$ is uncountable, that is, it cannot be put into one-to-one correspondence with $\mathbb{N}$ (the natural numbers).
- **Proof, cont'd:**
 - Clarify:
 - Σ^* is the set of all (finite) strings over $\Sigma = \{ 0, 1 \}$.
 - $P(\Sigma^*)$ is the set of all sets of strings, or languages, over Σ .
 - Right column lists all languages, that is, all elements of $P(\Sigma^*)$.
 - Σ^* , the set of all finite strings, is countable:
 - We can list all finite strings in order of length, put them in one-to-one correspondence with $\mathbb{N}$.
 - E.g., $\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots$
 - Since there is a correspondence between $\mathbb{N}$ and Σ^* , and we assumed one between $\mathbb{N}$ and $P(\Sigma^*)$, there must be a correspondence between Σ^* and $P(\Sigma^*)$, e.g.:

Claim 1

ε	$\emptyset$	L_0
0	$\{ 0 \}$	L_1
1	All even-length strings (an infinite language)	L_2
00	All strings containing at least one 0.	L_3
Etc.		

- Call the correspondence f , so we have $f(\varepsilon) = L_0$, $f(0) = L_1$, $f(1) = L_2$, etc.
- Now define **D, the diagonal set of strings**:
 $D = \{ w \in \Sigma^* \mid w \text{ is not in } f(w) \}$
- Examples:
 - ε is in D, because ε is not in $\emptyset$
 - 0 is not in D, because 0 is in $\{ 0 \}$
 - 1 is in D, because 1 is not an even-length string.
 - 00 is not in D, because 00 contains at least one 0.
 - Etc.

Claim 1

- Now the twist...
- Since the right column includes all subsets of Σ^* , D itself appears somewhere.
- That is, $D = f(x)$ for some string x .
 $x \dots\dots\dots D = \{ w \mid w \text{ is not in } f(w) \}$
- **Tricky question:** Is this string x in D or not?
- Two possibilities:
 - If x is in D , then x is not in $f(x)$ by definition of D , so x is not in D since $D = f(x)$.
 - If x is not in D , then x is in $f(x)$ by definition of D , so x is in D since $D = f(x)$.
- Either way, a contradiction.
- Implies that no such mapping f exists.
- So there is no correspondence between N and $P(\Sigma^*)$.
- So $P(\Sigma^*)$, the set of languages over Σ , is uncountable.

Claim 2

- **Claim 2:** The set of regular languages is countable.
- **Proof:**
 - Each regular language is recognized by some DFA.
 - Each DFA has a finite description: states, start states, transitions,...
 - Can write each of these using standard names for states, without changing the language.
 - Can enumerate these “standard form” DFAs in order of length.
 - Leads to an enumeration of the regular languages.
- Since $P(\Sigma^*)$, the set of all languages, is uncountable, whereas the set of regular languages is countable, some language must be non-regular.
- In fact, by considering different kinds of infinity, one can prove that “most” languages are non-regular.

Showing specific languages are
non-regular

Showing languages are non-regular

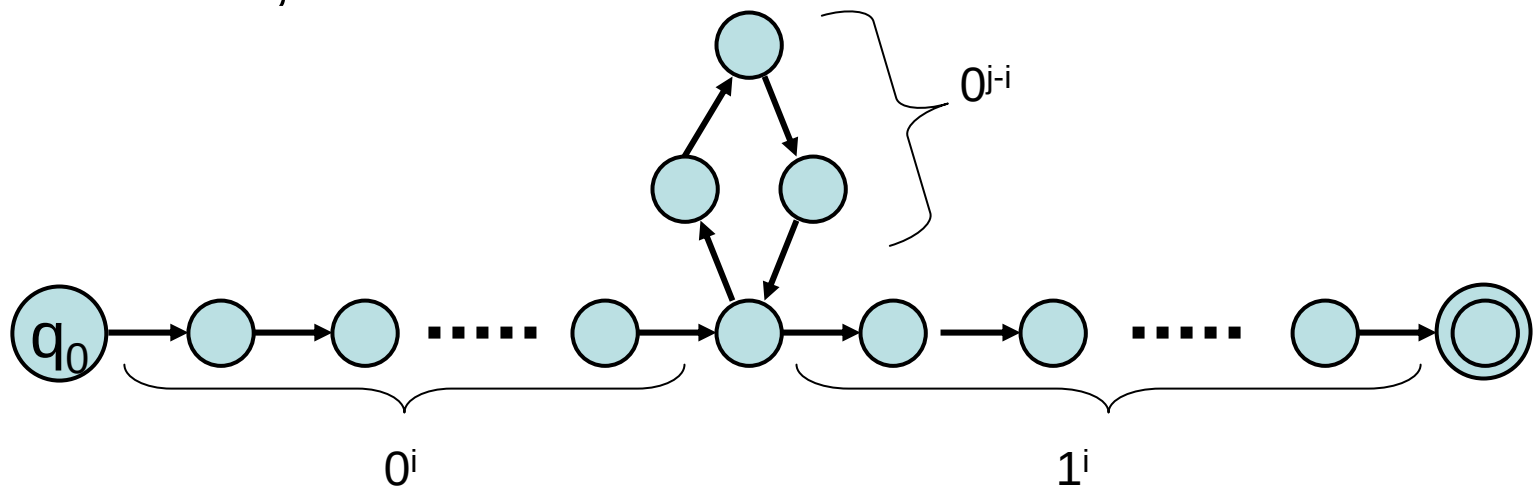
- **Basic tool: Pigeonhole Principle:** If you put $> n$ pigeons into n holes, then some hole has > 1 pigeon.
- **Example 1:** $L_1 = \{ 0^n 1^n \mid n > 0 \}$ is non-regular
 - E.g., 0011 is in L_1 , 011 is not.
 - Show by contradiction, using Pigeonhole Principle.
 - Assume L_1 is regular.
 - Then there is a DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizing L_1 .
 - Now define:
 - Pigeons = all strings in 0^* .
 - Holes = states in Q .
 - Put pigeon 0^i into hole $\delta^*(q_0, 0^i)$, that is, the hole corresponding to the state reached by input 0^i .

Showing languages are non-regular

- **Example 1:** $L_1 = \{ 0^n 1^n \mid n \geq 0 \}$ is non-regular
 - Assume L_1 is regular.
 - Then there is a DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizing L_1 .
 - Define:
 - Pigeons = all strings in 0^* .
 - Holes = states in Q .
 - Put pigeon 0^i into hole $\delta^*(q_0, 0^i)$, that is, the hole corresponding to the state reached by input 0^i .
 - There are $|Q|$ holes, but $> |Q|$ pigeons (actually, infinitely many).
 - So by Pigeonhole Principle, 2 pigeons must be put in the same hole, say 0^i and 0^j with $i < j$.
 - That is, 0^i and 0^j lead to the same state.
 - Then since M accepts $0^i 1^i$, it also accepts $0^j 1^i$, which is incorrect, contradiction.

Showing languages are non-regular

- **Example 1:** $L_1 = \{ 0^n 1^n \mid n \geq 0 \}$ is non-regular
 - Assume L_1 is regular.
 - Then there is a DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizing L_1 .
 - 0^i and 0^j lead to the same state.
 - Then since M accepts $0^i 1^i$, it also accepts $0^j 1^i$, which is incorrect, contradiction.



$0^i 1^i$ leads to the final state, so $0^j 1^i$ does also.

Showing languages are non-regular

- **Example 2:** $L_2 = \{ 010010001 \dots 0^i 1 \mid i \text{ is any positive integer} \}$ is non-regular
 - Show by contradiction, using Pigeonhole Principle.
 - Assume L_2 is regular, so there is a DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizing L_2 .
 - Define:
 - Pigeons = all strings in L_2 .
 - Holes = states.
 - Put pigeon string into hole corresponding to the state it leads to.
 - By the Pigeonhole Principle, two pigeons share a hole, say $01 \dots 0^i 1$ and $01 \dots 0^j 1$, where $j > i$.
 - So $01 \dots 0^i 1$ and $01 \dots 0^j 1$ lead to the same state.
 - M accepts $01 \dots 0^i 1 0^{j-i} 1$.
 - So M accepts $01 \dots 0^j 1 0^{j-i} 1$, incorrect, contradiction.

Showing languages are non-regular

- **Example 3:** $L_3 = \{ w w \mid w \in \{ 0, 1 \}^* \}$ is non-regular
 - Show by contradiction, using Pigeonhole Principle.
 - Assume L_3 is regular, so there is a DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognizing L_3 .
 - Define:
 - Pigeons = strings of the form 0^i1 where i is a nonnegative integer; that is, $1, 01, 001, \dots$
 - Holes = states.
 - Put pigeon string into hole corresponding to the state it leads to.
 - By the Pigeonhole Principle, two pigeons share a hole, say 0^i1 and 0^j1 , where $j > i$.
 - So 0^i1 and 0^j1 lead to the same state.
 - M accepts 0^i10^i1 .
 - So M accepts 0^j10^i1 , incorrect, contradiction.

The Pumping Lemma

Pumping Lemma

- Use Pigeonhole Principle (PHP) to prove a general result that can be used to show many languages are non-regular.
- **Theorem (Pumping Lemma):**
 - Let L be a regular language, recognized by a DFA with p states.
 - Let $x \in L$ with $|x| \geq p$.
 - Then x can be written as $x = u v w$ where $|v| \geq 1$, so that for all $m \geq 0$, $u v^m w \in L$.
 - In fact, it is possible to subdivide x in a particular way, with the total length of u and v being at most p : $|u v| \leq p$.
- That is, we can take any sufficiently long word in the language, and find some piece that can be added in any number of times to get other words in the language (“pumping up”).
- Or, we could remove the piece (“pumping down”).
- And this piece could be chosen to be near the beginning of the word.

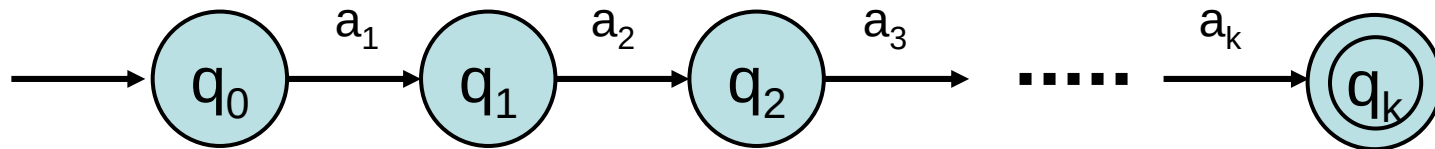
Pumping Lemma

- **Theorem (Pumping Lemma):**

- Let L be a regular language, recognized by a DFA with p states.
- Let $x \in L$ with $|x| \geq p$.
- Then x can be written as $x = u v w$ where $|v| \geq 1$, so that for all $m \geq 0$, $u v^m w \in L$.

- **Proof** (of the basic lemma):

- Consider $x \in L$ with $|x| \geq p$.
- Write $x = a_1 a_2 a_3 \dots a_k$ in L , where $k \geq p$.
- Suppose x passes through states $q_0, q_1, \dots, q_k$, where q_0 is the start state and q_k is an accept state.



- Since there are at least $p+1$ state occurrences and M has only p states, two state occurrences must be the same, by PHP.
- Say $q_i = q_j$ for some $i < j$.

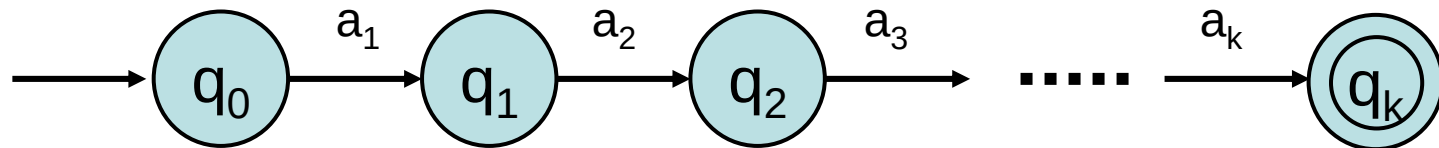
Pumping Lemma

- **Theorem (Pumping Lemma):**

- Let L be a regular language, recognized by a DFA with p states.
- Let $x \in L$ with $|x| \geq p$.
- Then x can be written as $x = u v w$ where $|v| \geq 1$, so that for all $m \geq 0$, $u v^m w \in L$.

- **Proof:**

- Assume $x = a_1 a_2 a_3 \dots a_k$ in L , where $k \geq p$.



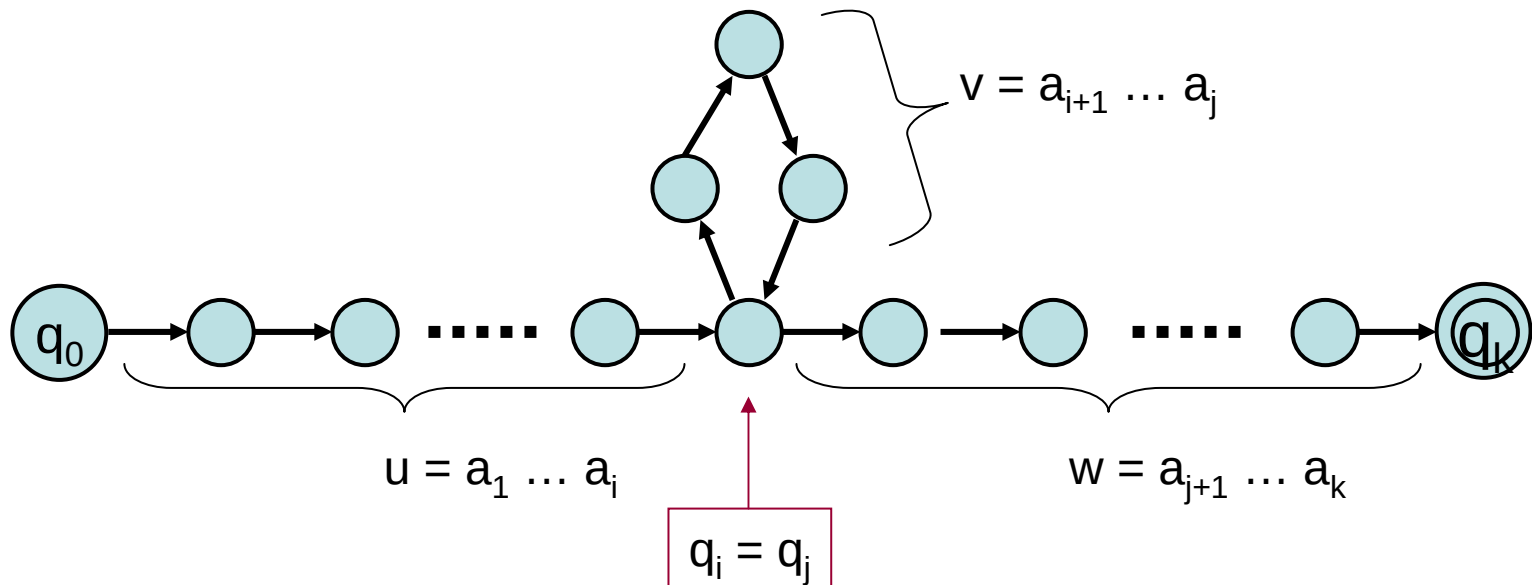
- $q_i = q_j$, $i < j$.
- Write $u = a_1 \dots a_i$, $v = a_{i+1} \dots a_j$, and $w = a_{j+1} \dots a_k$.

- **Claim this works:**

- $x = u v w$, obviously.
- $|v| = |a_{i+1} \dots a_j| \geq 1$, since $i < j$.
- $u v^m w$ is accepted, since it follows the loop m times (possibly 0 times).

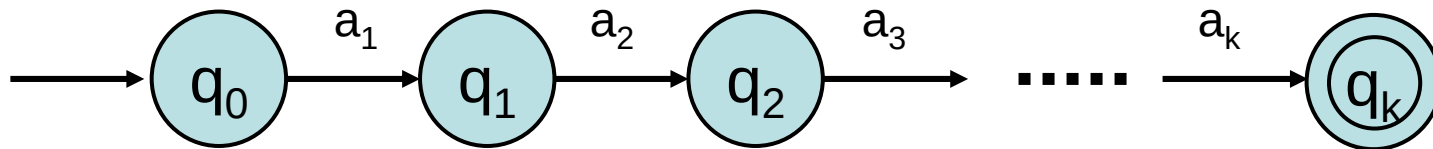
The loop

- $u v^m w$ is accepted, since it follows the loop m times (possibly 0 times).



Getting the extra condition

- **Theorem (Pumping Lemma):**
 - Let L be a regular language, recognized by a DFA with p states.
 - Let $x \in L$ with $|x| \geq p$.
 - Then x can be written as $x = u v w$ where $|v| \geq 1$, so that for all $m \geq 0$, $u v^m w \in L$.
 - In fact, it is possible to subdivide x in a particular way, with the total length of u and v being at most p : $|u v| \leq p$.
- **Proof:**
 - Consider $x \in L$ with $|x| \geq p$.
 - Write $x = a_1 a_2 a_3 \dots a_k$ in L , where $k \geq p$.
 - Suppose x passes through states $q_0, q_1, \dots, q_k$.



- Two state occurrences must be the same, by PHP.
- We can choose these two occurrences to be among the first $p+1$.
- Then $|u v| \leq p$.

Example 1, revisited

- $L_1 = \{ 0^n 1^n \mid n \geq 0 \}$ is non-regular.
- Suppose there is a DFA for L_1 with p states.
- We pick a particular word x in L_1 and pump it to get a contradiction.
- Choose $x = 0^p 1^p$, where p is the number of states.
- Then the Pumping Lemma says that x can be written as $u v w$, with $|v| \geq 1$, so that $u v v w$ is also in L_1 .
 - We're using $m = 2$ here.
- We get a contradiction, by considering three cases:
 - v consists of 0s only: Then $u v v w$ contains at least one extra 0, the same 1s, can't match.
 - v consists of 1s only: At least one extra 1, can't match.
 - v consists of a mix of 0s and 1s: Then $u v v w$ contains a 1 before a 0, so $u v v w$ can't be in L_1 .

Example 3, revisited

- $L_3 = \{ w w \mid w \in \{ 0, 1 \}^* \}$ is non-regular.
- Suppose there is a DFA for L_3 with p states.
- Pick a word x in L_3 and pump it to get a contradiction.
- Choose $x = 0^p 1 0^p 1$, where p is the number of states.
- Pumping Lemma says that x can be written as $u v w$, with $|v| \geq 1$, so that $u v^m w$ is also in L_3 , for every m .
- But so what?
 - The PL might give us $v = x$, $u = w = \varepsilon$.
 - Then adding in v any number of times, or removing v , yields a string in L_3 .
 - E.g., if $x = 001001$, and $v = x$, then $u v v w = 001001001001$, which is in L_3 .
 - No contradiction here.

Example 3, revisited

- $L_3 = \{ w w \mid w \in \{ 0, 1 \}^* \}$ is non-regular.
- Choose $x = 0^p 1 0^p 1$, where p is the number of states.
- Pumping Lemma says that x can be written as $u v w$, with $|v| \geq 1$, so that $u v^m w$ is also in L_3 , for every m .
- No contradiction here.
- So we use the extra condition, making the repeating part appear near the beginning: $|u v| \leq p$.
- This implies that uv must contain only 0s.
- Then $u v v w$ does yield a contradiction: it adds in at least one 0, in the first part only, yielding unequal-length runs of 0s.

Example 3, revisited

- $L_3 = \{ w w \mid w \in \{ 0, 1 \}^* \}$ is non-regular.
- Choose $x = 0^p 1 0^p 1$, where p is the number of states.
- Then x can be written as $u v w$, with $|v| \geq 1$, so that $u v^m w$ is also in L_3 , for every m , and so that $|u v| \leq p$.
- This implies that uv must contain only 0s.
- Then $u v v w$ does yield a contradiction: it adds in at least one 0, in the first part only, yielding unequal-length runs of 0s.
- **Note:** It was important to pick the right string to pump.
 - E.g., if we chose $x = 010101\dots$, an even number of repetitions of 01, then we could pump all we want and not get a contradiction.
 - The PL might give us $x = u v w$ with $v = 0101$.
 - Adding in 0101 any number of times yields a string in L_3 .

More Examples

Example 4: Palindromes

- $L_4 = \text{PAL} = \{ w \in \{0,1\}^* \mid w = w^R \}$ is non-regular.
- Suppose there is a DFA for PAL with p states.
- Pick a word x in PAL and pump it to get a contradiction.
- Choose $x = 0^p 1 0^p$; clearly x is in PAL
- The Pumping Lemma yields $x = u v w$, $|v| \geq 1$, $|uv| \leq p$, and $u v^m w$ in PAL for every m .
- Thus, the pumping part is near the beginning of x .
- Since $|uv| \leq p$, uv consists of 0s only.
- Since $|v| \geq 1$, v contains at least one 0.
- Then $u v v w$ must be in PAL.
- But this can't be, because we added at least one 0 in the first part and not in the second part.

Example 5

- $L_5 = EQ = \{ w \in \{0,1\}^* \mid w \text{ contains the same number of 0s and 1s} \}$ is non-regular.
- Suppose there is a DFA for EQ with p states.
- Choose $x = 0^p 1^p$ to pump; clearly x is in EQ.
- The Pumping Lemma yields $x = u v w$, $|v| \geq 1$, $|uv| \leq p$, and $u v^m w$ in EQ for every m .
- Since $|uv| \leq p$, uv consists of 0s only.
- Since $|v| \geq 1$, v contains at least one 0.
- Then $u v v w$ is supposed to be in EQ, but it isn't.

Example 5

- $L_5 = EQ = \{ w \in \{0,1\}^* \mid w \text{ contains the same number of 0s and 1s} \}$ is non-regular.
- Alternative proof:
 - By contradiction.
 - Suppose that EQ is regular.
 - Then $EQ \cap 0^*1^*$ is also regular. Why?
 - Because 0^*1^* is regular, and the class of regular languages is closed under intersection.
 - But $EQ \cap 0^*1^* = \{ 0^n1^n \mid n \geq 0 \} = L_1$, which we have already proved is non-regular.
 - Contradiction.

Example 6

- A non-regular unary language, $\Sigma = \{ 1 \}$.
- $L_6 = \{ 1^n \mid n \text{ is a prime number} \}$ is non-regular.
- Suppose L_6 is regular, p = number of states in accepting DFA.
- Let $n \geq p$ be a prime number, choose $x = 1^n$.
- The Pumping Lemma yields $x = u v w$, $|v| \geq 1$, and $u v^m w$ in L_6 for every m .
- So we have $x = 1^n = 1^a 1^b 1^c$, where $u = 1^a$, $v = 1^b$, $w = 1^c$.
- Since $u v^m w$ in L_6 for every m , we have that every number of the form $n + k b$ is prime, for every nonnegative integer k .
- But that can't be true:
 - Consider $k = n$.
 - Then $n + k b = n + n b = n(1+b)$, which is not prime (since $b \geq 1$).

Example 7: Pumping down

- $L_7 = \{ 0^i 1^j \mid i > j \}$ is non-regular.
- It doesn't work to pump up within the initial block of 0s---wouldn't produce something outside L_7 .
- But we can **pump down**, if we choose the right x .
- Choose $x = 0^{p+1} 1^p$, obviously in L_7 .
- Then $x = u v w$, $|v| \geq 1$, $|uv| \leq p$, and every $u v^m w$, for any $m \geq 0$, is in L_7 .
- Considering $m = 0$, we know that $u w$ is in L_7 .
- v consists of just 0s, and contains at least one 0.
- So removing v removes at least one 0, which yields a string that is not in L_7 .

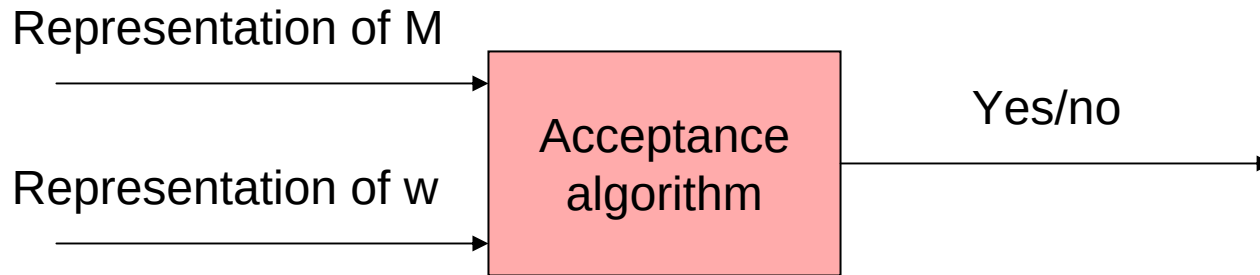
Algorithms that Answer Questions about FAs

Answering questions about FAs

- We can ask general questions about DFAs, NFAs, and regular expressions and try to answer them **algorithmically**, that is, by procedures that could be programmed in some ordinary programming language.
- Represent the DFAs, etc., by strings in some standard way, e.g., tuples with some encoding of a transition table.
- Sample questions:
 - **Acceptance**: Does a given DFA M accept a given input string w ?
 - **Nonemptiness**: Does DFA M accept any strings at all?
 - **Totality**: Does M accept all strings?
 - **Nonempty intersection**: Do $L(M_1)$ and $L(M_2)$ have any string in common?
 - **Subset**: Is $L(M_1)$ a subset of $L(M_2)$?
 - **Equivalence**: Is $L(M_1) = L(M_2)$?
 - **Finiteness**: Is $L(M)$ a finite set?
 - **Optimality**: Does M have the smallest number of states for a DFA that recognizes $L(M)$?

Acceptance problem

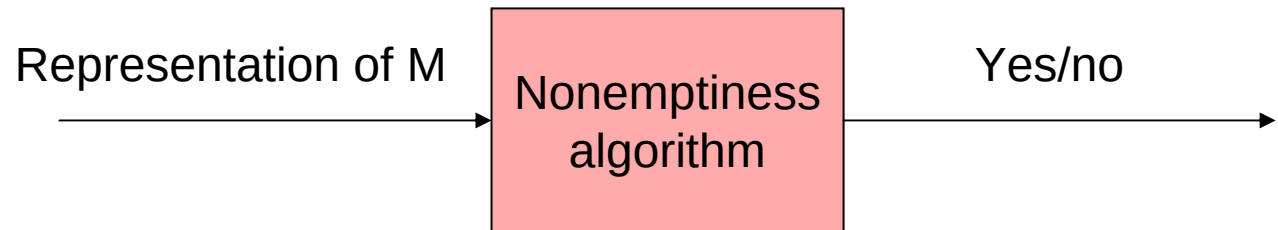
- Does a given DFA M accept a given input string w ?



- Need representation for w as well as M , since DFAs have different input alphabets, whereas the program has a fixed alphabet.
- Algorithm:**
 - Emulate M on w to see if it ends up at an accepting state.
 - Do the emulation using table lookup for each step.

Nonemptiness problem

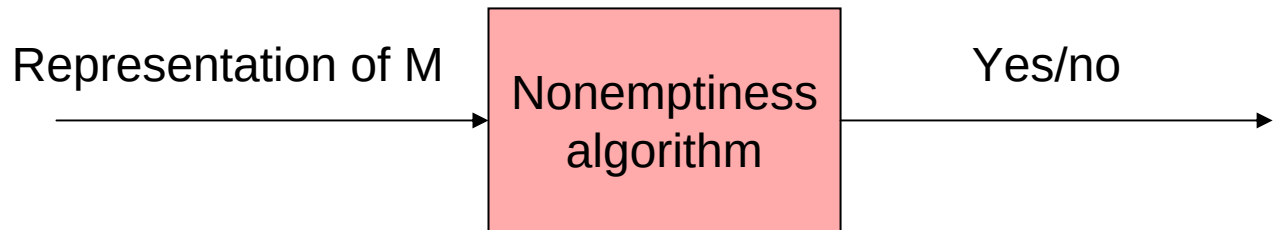
- Does DFA M accept any strings at all?
- That is, is $L(M) \neq \emptyset$?



- Note that $L(M) \neq \emptyset$ if and only if there is a path from the start state q_0 to an accepting state.
- **Algorithm 1:**
 - Search the DFA digraph from q_0 , using some standard search method like BFS or DFS, until you stop finding new states.
 - See if any accepting state has been reached.
- **Algorithm 2:**
 - If M accepts anything, it accepts some string of length $< n$, where n is the number of states. Try all these strings.

Nonemptiness problem

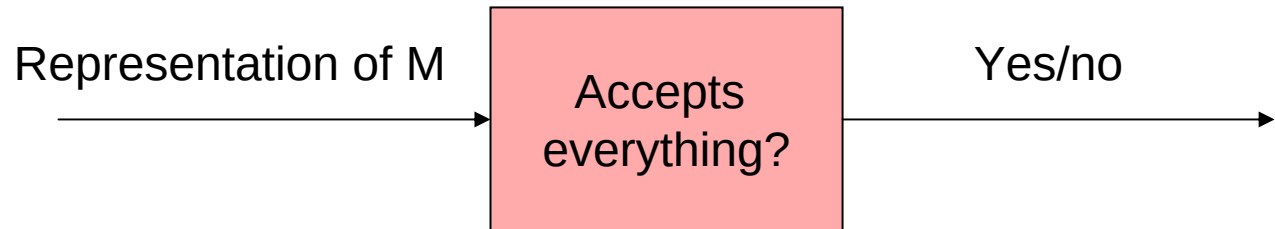
- Does DFA M accept any strings at all?
- That is, is $L(M) \neq \emptyset$?



- **Algorithm 2:**
 - If M accepts anything, it accepts some string of length $< n$, where n is the number of states. Try all these strings.
- But why is it true that, if M accepts anything, then it accepts some string of length $< n$?
 - Otherwise consider the shortest w accepted; must have $|w| \geq n$.
 - Then the states encountered in processing w must repeat somewhere, by Pigeonhole Principle.
 - Short-circuit the intervening segment and get a shorter accepted word, contradicting the assumption that w is shortest.

Totality problem

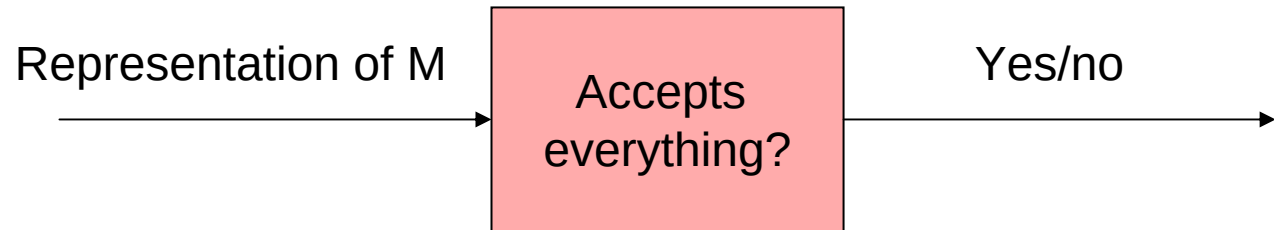
- Does M accept all strings?
- That is, is $L(M) = \Sigma^*$?



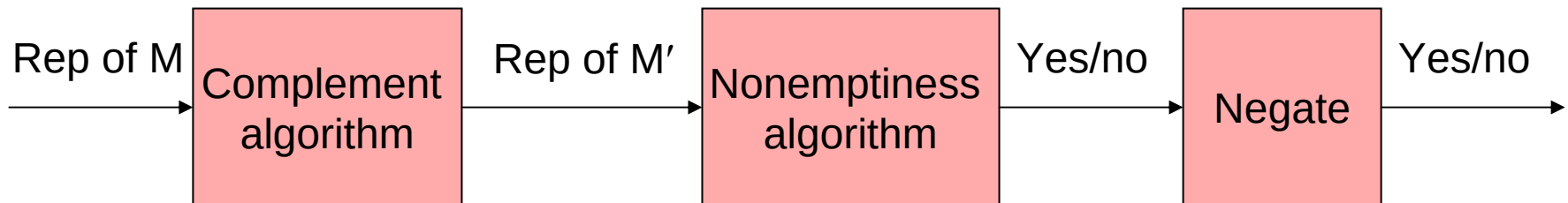
- We can't try all strings...
- Note that $L(M) = \Sigma^*$ if and only if there is no path from the start state q_0 to a nonaccepting state.
- **Algorithm 1:**
 - Search to see if there is a path to a nonaccepting state and give the opposite answer.
- **Algorithm 2:**
 - Transform M into a machine M' with $L(M') = (L(M))^c$.
 - Ask if $L(M')$ is nonempty, and give the opposite answer.

Totality problem

- Does M accept all strings?
- That is, is $L(M) = \Sigma^*$?

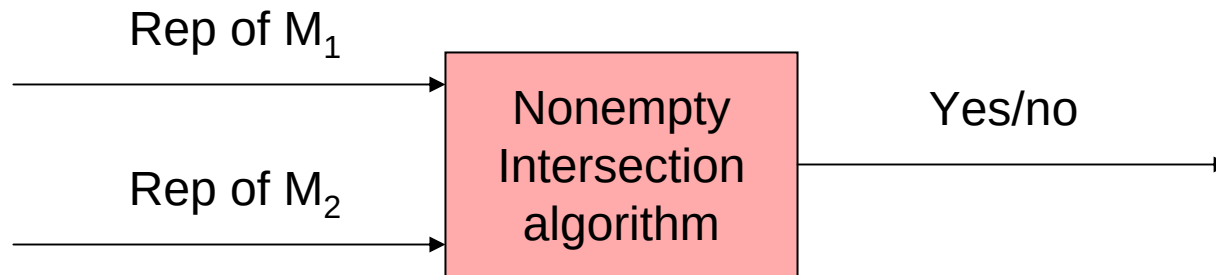


- Algorithm 2:
 - Transform M into a machine M' with $L(M') = (L(M))^c$.
 - Ask if $L(M')$ is nonempty, and give the opposite answer.
 - Both steps can be done with programs.



Nonempty intersection problem

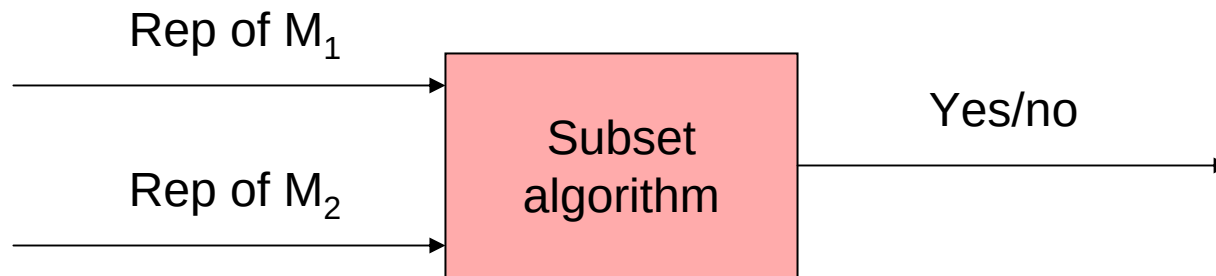
- Do $L(M_1)$ and $L(M_2)$ have any string in common?
- That is, is $L(M_1) \cap L(M_2) \neq \emptyset$?



- **Algorithm:**
 - Get a DFA M_3 (algorithmically) that recognizes $L(M_1) \cap L(M_2)$.
 - Ask if $L(M_3)$ is nonempty and give the same answer.

Subset problem

- Is $L(M_1) \subseteq L(M_2)$?
- Note that $L(M_1) \subseteq L(M_2)$ if and only if $L(M_1) \cap (L(M_2))^c = \emptyset$.



- **Algorithm:**
 - Get a DFA M_3 (algorithmically) that recognizes $(L(M_2))^c$.
 - Get another DFA M_4 (algorithmically) that recognizes $L(M_1) \cap (L(M_2))^c$.
 - Ask if $L(M_4)$ is empty and give the same answer.

Equivalence problem

- Is $L(M_1) = L(M_2)$?
- Note that $L(M_1) = L(M_2)$ if and only if both $L(M_1) \subseteq L(M_2)$ and $L(M_2) \subseteq L(M_1)$.
- Algorithm:
 - Test whether $L(M_1) \subseteq L(M_2)$ (algorithmically).
 - Test whether $L(M_2) \subseteq L(M_1)$ (algorithmically).
 - Say yes iff both say yes.

Finiteness problem

- Is $L(M)$ a finite set?
- Can't try all words...
- As for the nonemptiness test, we would like to find a limited range of lengths to test for membership, sufficient to answer the question.
- Pumping Lemma is useful here!
- **Claim 1:** If M accepts even one string of length $\geq n$ (the number of states), then $L(M)$ is infinite.
 - Because we can pump up that string repeatedly.
- But we can't try all strings of length $\geq n$...???
- **Claim 2:** If $L(M)$ is infinite, then M accepts at least one string x with $n \leq |x| < 2n$.
- With these claims, we have an easy algorithm:

Finiteness problem

- Is $L(M)$ a finite set?
- **Claim 1:** If M accepts even one string of length $\geq n$ (the number of states), then $L(M)$ is infinite.
- **Claim 2:** If $L(M)$ is infinite, then M accepts at least one string x with $n \leq |x| < 2n$.
- **Algorithm** (assuming Claims 1 and 2):
 - Try all strings of lengths $n, \dots, 2n-1$.
 - If any are in $L(M)$, then $L(M)$ is infinite.
 - By Claim 1.
 - If none are in $L(M)$, then $L(M)$ is finite.
 - By Claim 2.

Finiteness problem

- **Claim 2:** If $L(M)$ is infinite, then M accepts at least one string x with $n \leq |x| < 2n$.
- **Proof of Claim 2:**
 - Since $L(M)$ is infinite, it includes a string of length $\geq 2n$.
 - Choose one, x , of minimum length $\geq 2n$.
 - Apply the Pumping Lemma to x , writing $x = uvw$ with $|uv| \leq n$ and $|v| \geq 1$.
 - Pumping down, we know that uw is in $L(M)$.
 - Show that $n \leq |uw| < 2n$:
 - $|uw| \geq n$:
 - Because $|uvw| = |x| \geq 2n$, and $|v| \leq |uv| \leq n$.
 - $|uw| < 2n$:
 - Suppose not, so $|uw| \geq 2n$.
 - Impossible because uw is shorter than x and x was chosen to be a minimum length string in $L(M)$ with length $\geq 2n$.

Optimality problem

- Does M have the smallest number of states for a DFA that recognizes $L(M)$?
- Algorithm 1:
 - Enumerate all DFAs (up to isomorphism) with fewer states than M and test all for equivalence with M .
 - How to enumerate:
 - Use canonical state names $q_0, q_1, q_2, \dots$
 - For each fixed number n of states, where $n < \text{number of states of } M$, list all machines with states $q_0, \dots, q_{n-1}$.
 - List these by considering all possible collections of arrows, all choices of accept states.
 - Details LTTR.
- Algorithm 2:
 - Apply a **state minimization algorithm** for DFAs to M (see Sipser, Exercise 7.40).
 - Merges states of M , as far as possible, while maintaining equivalence.
 - Can prove that such an approach in fact yields the minimum number of states.

Questions about NFAs

- Sample questions:
 - **Acceptance:** Does a given NFA M accept a given input string w ?
 - **Nonemptiness:** Does NFA M accept any strings at all?
 - **Totality:** Does M accept all strings?
 - **Nonempty intersection:** Do $L(M_1)$ and $L(M_2)$ have any string in common?
 - **Subset:** Is $L(M_1)$ a subset of $L(M_2)$?
 - **Equivalence:** Is $L(M_1) = L(M_2)$?
 - **Finiteness:** Is $L(M)$ a finite set?
 - **Optimality:** Does M have the smallest number of states for an NFA that recognizes $L(M)$?
- Can answer all but the last simply by translating to DFAs and answering the same question.
- Optimality: List and test equivalence.

Questions about regular expressions

- Sample questions:
 - **Acceptance:** Does the language denoted by a given regular expression R include a given input string w ?
 - **Nonemptiness:** Is $L(R) \neq \emptyset$?
 - **Totality:** Is $L(R) = \Sigma^*$?
 - **Nonempty intersection:** Is $L(R_1) \cap L(R_2) \neq \emptyset$?
 - **Subset:** Is $L(R_1) \subseteq L(R_2)$?
 - **Equivalence:** Is $L(R_1) = L(R_2)$?
 - **Finiteness:** Is $L(R)$ a finite set?
 - **Optimality:** Is R the shortest regular expression whose language is $L(R)$?
- Can answer all but the last simply by translating to DFAs and answering the same question.
- Optimality: List and test equivalence.

Next time...

- Computability theory
- Readings:
 - Sipser Chapter 3
 - The Nature of Computation, Chapter 8
 - GITCS notes, lecture 4

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 4

*Lecturer: Scott Aaronson**Scribe: Aseem Kishore*

1 Previously in 6.089...

Last lecture, we talked about two different models of computation, *finite automata* and *circuits*. Finite automata allowed us to recognize many properties of an arbitrarily long input, while circuits allowed us to represent any Boolean function.

However, both models had significant limitations. Circuits were limited in hardware—we have to know how big an input is before we can make a circuit for it—while finite automata were limited by their memory—which also had to be known in advance of a problem.

2 Turing Machines

How can we generalize finite automata to overcome their limitations? A first idea is to let them move backwards on the tape as well as forwards. This is a good start, but by itself it actually provides no extra power. To see why, suppose a two-way finite automaton is in some state a , going forward at some point x on the tape. At this point, if it goes backwards on the tape and then returns to x , that simply induces some function $a \Rightarrow f(a)$, where f depends on the earlier part of the tape. But this means that we can simulate the two-way machine by a one-way machine, which simply keeps track of the whole function f instead of just the state a .¹ Thus, being able to move in two directions provides no additional power on its own.

What we really need is a machine that can not only move backwards and forwards, but also *write* to the tape and *halt* at any time of its choosing. And that's what a Turing machine is. The ability to write essentially gives Turing machines an unlimited memory, since any information that can't fit in the machine's internal state can always be written to the tape. The ability to halt at discretion means that Turing machines aren't "tied to the input" the way finite automata are, but can do as much auxiliary computation as they need.

So at any given point on the tape, a Turing machine faces three questions:

1. Change state?
2. Write to the tape?
3. Move left, move right, or halt?

The machine's answer to these questions is a function of its current state and the current input on the tape.

A clear example of these features overcoming the limitations of finite automata is a Turing machine's ability to solve the palindrome problem. By using a simple back-and-forth process, a

¹Note that the number of functions $f(a)$ mapping states to states grows exponentially with the number of states in the original machine.

Turing machine can repeatedly check that a letter at one end exists at the opposite end, and by marking letters that it has seen, the machine ensures it continuously narrows its scope. (Here we're assuming that additional symbols are available besides just 0 and 1.) This algorithm takes $O(n^2)$ time (interestingly, there is a proof that argues that this is the best a Turing machine can do).

Likewise, addition of integers is also possible, as are multiplying and some other mathematical operations. (We won't prove that!) Searching for non-regular patterns also becomes possible. But perhaps the most interesting thing a Turing machine can do is to emulate another Turing machine!

3 Universal Turing Machines

In his 1936 paper "On Computable Numbers" (in some sense, the founding document of computer science), Turing proved that we can build a Turing machine U that acts as an interpreter for other Turing machines. In other words, U 's input tape can contain a description of *another* Turing machine, which is then simulated step by step. Such a machine U is called a *Universal Turing machine*. If a universal machine *didn't* exist, then in general we would need to build new hardware every time we wanted to solve a new problem: there wouldn't even be the concept of *software*. This is why Professor Aaronson refers to Turing's universality result as the "existence of the software industry lemma"!

A question was brought up in class as to how this can be, if the machine being interpreted may require more states than the interpreting Turing machine has. It turns out that universal Turing machines aren't limited by their states, because they can always keep extra state on blank sections of the tape. They can thus emulate a machine with any number of states, but themselves requiring only a few states. (In fact, there is a popular parlor-type competition to find a universal Turing machine that uses as few states and symbols as possible. Recently, one student actually came up with such a machine that uses only two states and a three-symbol alphabet. To be fair, however, the machine required the inputs in a special format, which required some pre-computation, so a question arises as to how much of the work is being done by the machine versus beforehand by the pre-computation.)

4 The Church-Turing Thesis

Related to the idea of universal machines is the so-called *Church-Turing thesis*, which claims that anything we would naturally regard as "computable" is actually computable by a Turing machine. Intuitively, given any "reasonable" model of computation you like (RAM machines, cellular automata, etc.), you can write compilers and interpreters that translate programs back and forth between that model and the Turing machine model. It's never been completely clear how to interpret this thesis: is it a claim about the laws of physics? about human reasoning powers? about the computers that we actually build? about math or philosophy?

Regardless of its status, the Church-Turing Thesis was such a powerful idea that Gödel declared, "one has for the first time succeeded in giving an absolute definition to an interesting epistemological notion."

But as we'll see, even Turing machines have their limitations.

5 Halting is a problem

Suppose we have a Turing machine that never halts. Can we make a Turing machine that can detect this? In other words, can we make an infinite loop detector? This is called the *Halting problem*.

The benefits of such a machine would be widespread. For example, we could then prove or disprove Goldbach's Conjecture, which says that all even numbers 4 or greater are the sum of two primes. We could do this by writing a machine that iterated over all even numbers to test this conjecture:

```
for i = 2 to infinity:
  if 2*i is not a sum of two primes
    then HALT
```

We would then simply plug this program into our infinite-loop-detecting Turing machine. If the machine detected a halt, we'd know the program must eventually encounter a number for which Goldbach's conjecture is false. But if it detected no halt, then we'd know the conjecture was true.

It turns out that such an infinite loop detector can't exist. This was also proved in Turing's paper, by an amazingly simple proof that's now part of the intellectual heritage of computer science.²:

We argue by contradiction. Let P be a Turing machine that solves the halting problem. In other words, given an input machine M , $P(M)$ accepts if $M(0)$ halts, and rejects if $M(0)$ instead runs forever. Here $P(M)$ means P run with an encoding of M on its input tape, and $M(0)$ means M run with all 0's on its input tape. Then we can easily modify P to produce a new Turing machine Q , such that $Q(M)$ runs forever if $M(M)$ halts, or halts if $M(M)$ runs forever.

Then the question becomes: what happens with $Q(Q)$? If $Q(Q)$ halts, then $Q(Q)$ runs forever, and if $Q(Q)$ runs forever, then $Q(Q)$ halts. The only possible conclusion is that the machine P can't have existed in the first place.

In other words, we've shown that the halting problem is *undecidable*—that is, whether another machine halts or not is not something that is *computable* by Turing machine. We can also prove general uncomputability in other ways. Before we do so, we need to lay some groundwork.

6 There are multiple infinities

In the 1880's, Georg Cantor discovered the extraordinary fact that there are different degrees of infinity. In particular, the infinity of real numbers is greater than the infinity of integers.

For simplicity, let's only talk about positive integers, and real numbers in the interval $[0, 1]$. We can associate every such real number with an infinite binary string: for example, 0.0011101001.... A technicality is that some real numbers can be represented in two ways: for example, $0.100\bar{0}$ is equivalent to $0.011\bar{1}$. But we can easily handle this, for example by disallowing an infinity of trailing 1's.

To prove that there are more real numbers than integers, we'll argue by contradiction: suppose the two infinities are the same. If this is true, then we must be able to create a one-to-one association, pairing off every positive integer with a real number $x \in [0, 1]$. We can arrange this association like so:

²This proof also exists as a poem by Geoffrey K. Pullum entitled "Scooping the Loop Snooper": <http://www.lel.ed.ac.uk/~gpullum/loopsnoop.html>

1: 0.0000... (rational)
2: 0.1000...
3: 0.0100...
4: 0.101001000100001... (irrational)
5: 0.110010110001001...

We can imagine doing this for all positive integers. However, we note that we can construct another real number whose n^{th} digit is the opposite of the n^{th} digit of the n^{th} number. For example, using the above association, we would get 0.11110...

This means that, contrary to assumption, there were additional real numbers in $[0, 1]$ not in our original list. Since every mapping will leave real numbers left over, we conclude that there are more real numbers than integers.

If we try to apply the same proof with rational numbers instead of real numbers, we fail. This is because the rational numbers are *countable*; that is, each rational number can be represented by a finite-length string, so we actually can create a one-to-one association of integers to rational numbers.

7 Infinitely many unsolvable problems

We can use the existence of these multiple infinities to prove that there are uncomputable problems. We'll begin by showing that the number of possible Turing machines is the smallest infinity, the infinity of integers.

We can define a Turing machine as a set of states and a set of transitions from each state to another state (where the transitions are based on the symbol being read). A crucial aspect of this definition is that both sets are finite.

Because of this, the number of Turing machines is *countable*. That is, we can “flatten” each machine into one finite-length string that describes it, and we can place these strings into a one-to-one association with integers, just as we can with rational numbers.

The number of *problems*, on the other hand, is a greater infinity: namely, the infinity of real numbers. This is because we can define a problem as a function that maps every input $x \in 0, 1^*$ to an output (0 or 1). But since there are infinitely many inputs, to specify such a function requires an infinite number of bits. So just like with Cantor's proof, we can show that the infinity of problems is greater than the infinity of Turing machines.

The upshot is that there are far more problems than there are Turing machines to solve them. From this perspective, the set of computable problems is just a tiny island in a huge sea of unsolvability. Admittedly, most of the unsolvable problems are not things that human beings will ever care about, or even be able to define. On the other hand, Turing's proof of the unsolvability of the halting problem shows that at least *some* problems we care about are unsolvable.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 7
Nancy Lynch

Today

- Basic computability theory
- **Topics:**
 - Decidable and recognizable languages
 - Recursively enumerable languages
 - Turing Machines that solve problems involving FAs
 - Undecidability of the Turing machine acceptance problem
 - Undecidability of the Turing machine halting problem
- **Reading:** Sipser, Sections 3.1, 3.2, Chapter 4
- **Next:** Sections 5.1, 5.2

Decidable and Recognizable Languages

Decidable and recognizable languages

- Last time, we began studying the important notion of **computability**.
- As a concrete model of computation, we introduced basic one-tape, one-head Turing machines.
- Also discussed some variants.
- Claimed they are all **equivalent**, so the notion of computability is **robust**.
- Today: Look more carefully at the notions of computability and equivalence.

Decidable and recognizable languages

- Assume: TM has accepting state q_{acc} and rejecting state q_{rej} .
 - **Definition:** TM M **recognizes** language L provided that $L = \{ w \mid M \text{ on } w \text{ reaches } q_{acc} \} = \{ w \mid M \text{ accepts } w \}$.
 - Another important notion of computability:
 - **Definition:** TM M **decides** language L provided that both of the following hold:
 - On every w , M eventually reaches either q_{acc} or q_{rej} .
 - $L = \{ w \mid M \text{ on } w \text{ reaches } q_{acc} \}$.
 - Thus, **if M recognizes L** , then:
 - Words in L lead to q_{acc} .
 - Words not in L either lead to q_{rej} or never halt (“loop”).
 - Whereas **if M decides L** , then:
 - Words in L lead to q_{acc} .
 - Words not in L lead to q_{rej} .
- 
- Always halts

Decidable and recognizable languages

- **Theorem 1:** If M decides L then M recognizes L .
- Obviously.
- But not necessarily vice versa.
- In fact, these two notions define **different language classes**:
- **Definition:**
 - L is **Turing-recognizable** if there is some TM that recognizes L .
 - L is **Turing-decidable** if there is some TM that decides L .
- The classes of Turing-recognizable and Turing-decidable languages are different.
- **Theorem 2:** If L is Turing-decidable then L is Turing-recognizable.
- Obviously.
- But the other direction does not hold---there are languages that are Turing-recognizable but not Turing-decidable.
- We'll see some examples soon.

Decidable and recognizable languages

- **Theorem 3:** If L is Turing-decidable then L^c is T-decidable.
- **Proof:**
 - Suppose that M decides L .
 - Design a new machine M' that behaves just like M , but:
 - If M accepts, M' rejects.
 - If M rejects, M' accepts.
 - Formally, can do this by interchanging q_{acc} and q_{rej} .
 - Then M' decides L^c .

Decidable and recognizable languages

- A basic connection between Turing-recognizable and Turing-decidable languages:
- **Theorem 4:** L is Turing decidable if and only if L and L^c are both Turing-recognizable.
- **Proof:** $\Rightarrow$
 - Suppose that L is Turing-decidable.
 - Then L is Turing-recognizable, by Theorem 2.
 - Also, L^c is Turing-decidable, by Theorem 3.
 - So L^c is Turing-recognizable, by Theorem 2.
- **Proof:** $\Leftarrow$
 - Given M_1 recognizing L , and M_2 recognizing L^c .
 - Produce a Turing Machine M that decides whether or not its input w is in L or L^c .

Decidable and recognizable languages

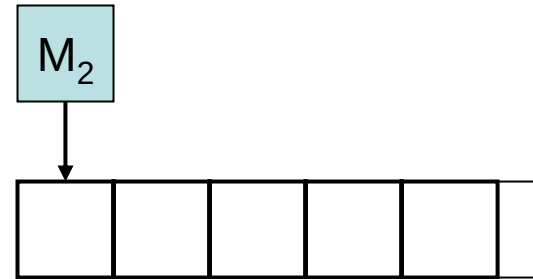
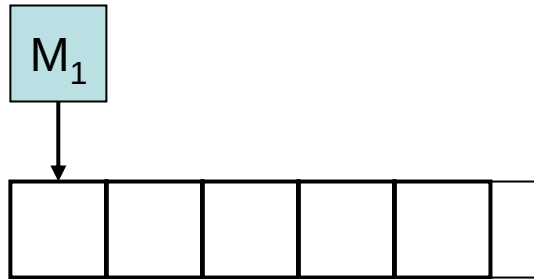
- **Theorem 4:** L is Turing decidable if and only if L and L^c are both Turing-recognizable.
- **Proof:** $\Leftarrow$
 - Given M_1 recognizing L , and M_2 recognizing L^c .
 - Produce a Turing Machine M that decides whether or not its input w is in L or L^c .
 - **Idea:** Run both M_1 and M_2 on w .
 - One must accept.
 - If M_1 accepts, then M accepts.
 - If M_2 accepts, then M rejects.
 - But, we can't run M_1 and M_2 one after the other because the first one might never halt.
 - Run them in parallel, until one accepts?
 - How? We don't have a parallel Turing Machine model.

Decidable and recognizable languages

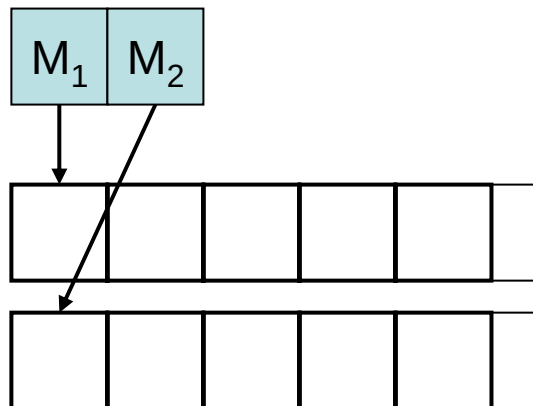
- **Theorem 4:** L is Turing decidable if and only if L and L^c are both Turing-recognizable.

- **Proof:** $\Leftarrow$

- M_1 recognizes L , and M_2 recognizes L^c .

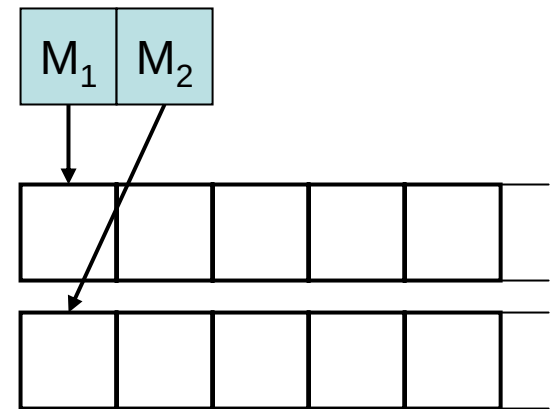


- Let M be a 2-tape Turing Machine:



Decidable and recognizable languages

- **Theorem 4:** L is Turing decidable if and only if L and L^c are both Turing-recognizable.
- **Proof:** $\Leftarrow$
 - M copies input from 1st tape to 2nd tape.
 - Then emulates M_1 and M_2 together, step-by-step.
 - No interaction between them.
 - M 's finite-state control keeps track of states of M_1 and M_2 ; thus, Q includes $Q_1 \times Q_2$.
 - Also includes new start, accept, and reject states and whatever else is needed for bookkeeping.



Language Classification

- Four possibilities:
 - L and L^c are both Turing-recognizable.
 - Equivalently, L is Turing-decidable.
 - L is Turing-recognizable, L^c is not.
 - L^c is Turing-recognizable, L is not.
 - Neither L nor L^c is Turing-recognizable.
- All four possibilities occur, as we will see.
- How do we know that there are languages L that are neither Turing-recognizable nor co-Turing-recognizable?
- Cardinality argument:
 - There are uncountably many languages.
 - There are only countably many Turing-recognizable languages and only countably many co-Turing-recognizable languages.
 - Because there are only countably many Turing machines (up to renaming).

Examples

- **Example:** Every regular language L is decidable.
 - Let M be a DFA with $L(M) = L$.
 - Design a Turing machine M' that simulates M .
 - If, after processing the input, the simulated M is in an accepting state, M' accepts; else M' rejects.

Examples

- **Example:** Let X = be the set of binary representations of natural numbers for which the following procedure halts:
 - while $x \neq 1$ do
 - if x is odd then $x := 3x + 1$
 - if x is even then $x := x/2$
 - halt
- Obviously, X is Turing-recognizable: just simulate this procedure and accept if/when it halts.
- Is it decidable? (?)

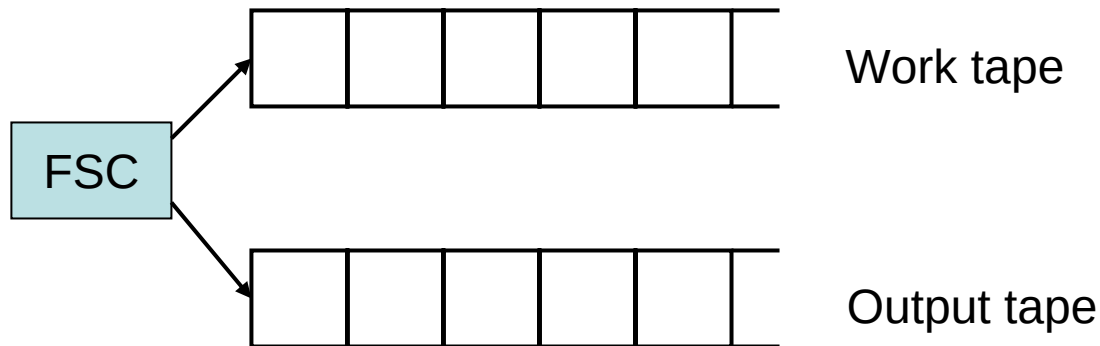
Closure Properties

- **Theorem 5:** The set of Turing-recognizable languages is closed under set union and intersection.
- **Proof:**
 - Run both machines in parallel.
 - For union, accept if either accepts.
 - For intersection, accept if both accept.
- However, the set of Turing-recognizable languages is not closed under complement.
- As we will soon see.
- **Theorem 6:** The set of Turing-decidable languages is closed under union, intersection, and complement.
- **Theorem 7:** Both the Turing-recognizable and Turing-decidable languages are closed under concatenation and star (HW).

Recursively Enumerable Languages

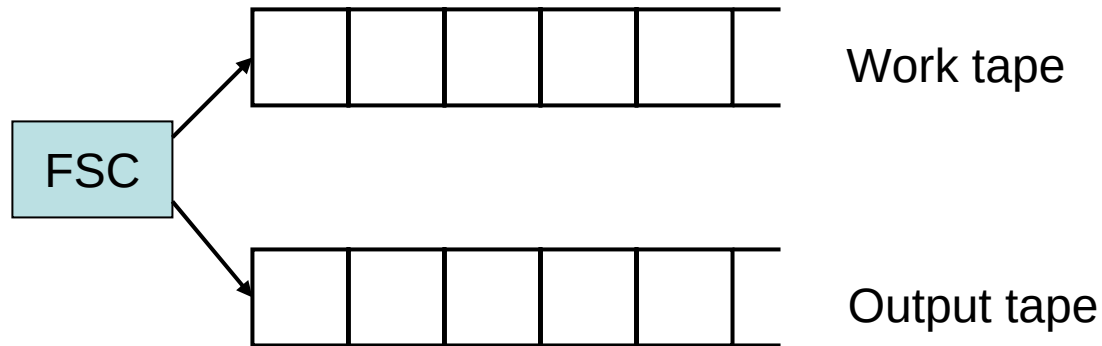
Recursively enumerable languages

- Yet another kind of computability for Turing Machines.
- An **enumerator** is a Turing Machine variant:



- Starts with a blank work tape (no input).
- Prints a sequence of finite strings (possibly infinitely many) on output tape.
- More specifically, e.g.:
 - Enters a special state q_{print} , where contents of work tape, up to first blank, are copied to output tape, followed by blank as a separator.
 - Then machine continues.
 - No accept or reject states.

Recursively enumerable languages



- Starts with a blank work tape (no input).
- Prints a sequence of finite strings (possibly infinitely many) on output tape.
- It may print the same string more than once.
- If E is an enumerator, then define
$$L(E) = \{ x \mid x \text{ is printed by } E \}.$$
- If $L = L(E)$ for some enumerator E , then we say that L is **recursively enumerable** (r.e.).

Recursively enumerable languages

- Interesting connection between recursive enumerability and Turing recognizability:
- **Theorem 8:** L is recursively enumerable if and only if L is Turing-recognizable.
- **Proof:** $\Rightarrow$
 - Given E , an enumerator for L , construct Turing machine M to recognize L .
 - **M : On input x :**
 - M simulates E (on no input, as usual).
 - Whenever E prints, M checks to see if the new output is x .
 - If it ever sees x , M accepts.
 - Otherwise, M keeps going forever.

Recursively enumerable languages

- **Theorem 8:** L is recursively enumerable if and only if L is Turing-recognizable.
- **Proof:** $\Leftarrow$
 - Given M , a Turing machine that recognizes L , construct E to enumerate L .
 - **Idea:**
 - Simulate M on all inputs.
 - If/when any simulated execution reaches q_{acc} , print out the associated input.
 - As before, we can't run M on all inputs sequentially, because some computations might not terminate.
 - So we must run them **in parallel**.
 - But this time we must run **infinitely many** computations, so we can't just use a multitape Turing machine.

Recursively enumerable languages

- **Theorem 8:** L is recursively enumerable if and only if L is Turing-recognizable.
- **Proof:** $\Leftarrow$
 - Given M , a Turing machine that recognizes L , construct E to enumerate L .
 - Simulate M on all inputs; when any simulated execution reaches q_{acc} , print out the associated input.
 - New trick: Dovetailing
 - Run 1 step for 1st input string, ε .
 - Run 2 steps for 1st and 2nd inputs, ε and 0.
 - Run 3 steps for 1st, 2nd, and 3rd inputs, ε , 0 and 1.
 - ...
 - Run more and more steps for more and more inputs.
 - Eventually succeeds in reaching q_{acc} for each accepting computation of M , so enumerates all elements of L .

Recursively enumerable languages

- **Theorem 8:** L is recursively enumerable if and only if L is Turing-recognizable.
- **Proof:** $\Leftarrow$
 - Simulate M on all inputs; when any simulated execution reaches q_{acc} , print out the associated input.
 - Dovetail all computations of M .
 - Complicated bookkeeping, messy to work out in detail.
 - But can do algorithmically, hence on a Turing machine.

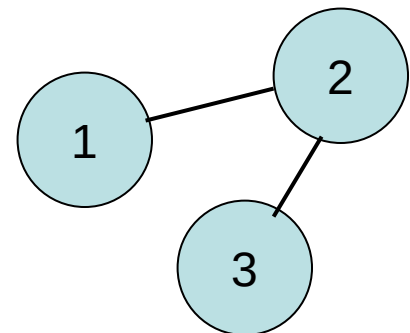
Turing Machines that solve problems
for other domains besides strings

Turing Machines that solve problems for other domains

- [Sipser Section 4.1]
- Our examples of computability by Turing machines have so far involved properties of strings, and numbers represented by strings.
- We can also consider computability by TMs for other domains, such as graphs or DFAs.
- **Graphs:**
 - Consider the problem of whether a given graph has a cycle of length > 2 .
 - Can formalize this problem as a language (set of strings) by encoding graphs as strings over some finite alphabet.
 - Graph = (V, E) , V = vertices, E = edges, undirected.

Turing Machines that solve graph problems

- Consider the problem of whether a given graph has a cycle of length > 2 .
- Formalize as a language (set of strings) by encoding graphs as strings over some finite alphabet.
- Graph = (V, E) , V = vertices, E = edges, undirected.
- A standard encoding:
 - Vertices = positive integers (represented in binary)
 - Edges = pairs of positive integers
 - Graph = list of vertices, list of edges.
- **Example:** $((1, 2, 3), ((1, 2), (2, 3)))$
- Write $\langle G \rangle$ for the encoding of G .



Turing Machines that solve graph problems

- Consider the problem of whether a given graph has a cycle of length > 2 .
- Graph = (V, E) , V = vertices, E = edges, undirected.
- Write $\langle G \rangle$ for the encoding of G .
- Using this representation for the input, we can write an algorithm to determine whether or not a given graph G has a cycle, and formalize the algorithm using a Turing machine.
 - E.g., search and look for repeated vertices.
- So cyclicity is a decidable property of graphs.

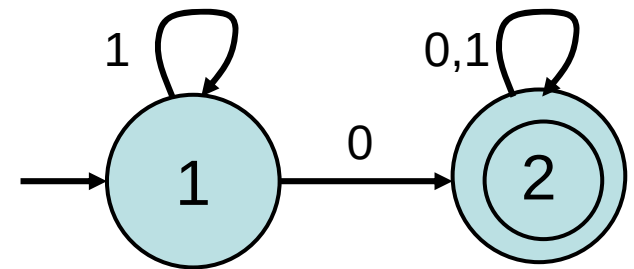
Turing Machines that solve problems for other domains

- We can also consider computability for domains that are **sets of machines**:

- **DFAs:**

- Encode DFAs using bit strings, by defining standard naming schemes for states and alphabet symbols.
- Then a DFA tuple is again a list.

- **Example:**



Encode as:

$$\underbrace{(1, 2)}_Q, \underbrace{(0, 1)}_\Sigma, \underbrace{((1, 1, 1), (1, 0, 2), (2, 0, 2), (2, 1, 2))}_\delta, \underbrace{(1)}_{q_0}, \underbrace{(2)}_F$$

- Encode the list using bit strings.
- Write **<M>** for the encoding of M.
- So we can define languages whose elements are (bit strings representing) DFAs.

Turing Machines that solve DFA problems

- **Example:** $L_1 = \{ \langle M \rangle \mid L(M) = \emptyset \}$ is Turing-decidable
- Elements of L_1 are bit-string representations of DFAs that accept nothing (emptiness problem).
- Already described an algorithm to decide this, based on searching to determine whether any accepting state is reachable from the start state.
- Could formalize this (painfully) as a Turing machine.
- Proves that L_1 is Turing-decidable.
- Similarly, all the other decision problems we considered for DFAs, NFAs, and regular expressions are Turing-decidable (not just Turing-recognizable).
- Just represent the inputs using standard encodings and formalize the algorithms that we've already discussed, using Turing machines.

Turing Machines that solve DFA problems

- **Example:** Equivalence for DFAs
 $L_2 = \{ \langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) \}$ is Turing-decidable.
- Elements of L_2 are bit-string representations of pairs of DFAs that recognize the same language.
- Note that the domain we encode is **pairs of DFAs**.
- Already described an algorithm to decide this, based on testing inclusion both ways; to test whether $L(M_1) \subseteq L(M_2)$, just test whether $L(M_1) \cap (L(M_2))^c = \emptyset$.
- Formalize as a Turing machine.
- Proves that L_2 is Turing-decidable.

Turing Machines that solve DFA problems

- **Example:** Acceptance for DFAs
 $L_3 = \{ \langle M, w \rangle \mid w \in L(M) \}$ is Turing-decidable.
- Domain is **(DFA, input) pairs**.
- Algorithm simply runs M on w .
- Formalize as a Turing machine.
- Proves that L_3 is Turing-decidable.

Moving on...

- Now, things get more complicated: we consider inputs that are **encodings of Turing machines** rather than DFAs.
- In other words, we will discuss Turing machines that decide questions about Turing machines!

Undecidability of the Turing Machine Acceptance Problem

Undecidability of TM Acceptance Problem

- Now (and for a while), we will focus on showing that certain languages are **not Turing-decidable**, and that some are **not even Turing-recognizable**.
- It's easy to see that such languages **exist**, based on cardinality considerations.
- Now we will show some **specific languages** are not Turing decidable, and not Turing-recognizable.
- These languages will express questions about Turing machines.

Undecidability of TM Acceptance

- We have been discussing **decidability of problems involving DFAs**, e.g.:
 - $\{ \langle M \rangle \mid M \text{ is a DFA and } L(M) = \emptyset \}$, decidable by Turing machine that searches M 's digraph.
 - $\{ \langle M, w \rangle \mid M \text{ is a DFA, } w \text{ is a word in } M\text{'s alphabet, and } w \in L(M) \}$, decidable by a Turing machine that emulates M on w .
- Turing machines compute only on strings, but we can regard them as computing on DFAs by **encoding the DFAs as strings** (using a standard encoding).
- Now we consider **encoding Turing machines as strings**, and allowing other Turing machines to compute on these strings.
- Encoding of Turing machines: Standard state names, lists, etc., similar to DFA encoding.
- $\langle M \rangle$ = encoding of Turing machine M .
- $\langle M, w \rangle$ = encoding Turing machine + input string
- Etc.

Problems we will consider

- $\text{Acc}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a (basic) Turing machine, } w \text{ is a word in } M\text{'s alphabet, and } M \text{ accepts } w \}$.
 - $\text{Halt}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a Turing machine, } w \text{ is a word in } M\text{'s alphabet, and } M \text{ halts (either accepts or rejects) on } w \}$.
 - $\text{Empty}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a Turing machine and } L(M) = \emptyset \}$
 - Recall: $L(M)$ refers to the set of strings M accepts.
 - Etc.
-
- Thus, we can formulate questions about Turing machines as languages.
 - Then we can ask if they are Turing-decidable; that is, can some particular TM answer these questions about all (basic) TMs?
 - We'll prove that they cannot.

The Acceptance Problem

- $\text{Acc}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a (basic) Turing machine and } M \text{ accepts } w \}$.
- **Theorem 1:** Acc_{TM} is Turing-recognizable.
- **Proof:**
 - Construct a TM U that recognizes Acc_{TM} .
 - **U :** On input $\langle M, w \rangle$:
 - Simulate M on input w .
 - If M accepts, accept.
 - If M rejects, reject.
 - Otherwise, U loops forever.
 - Then U accepts exactly $\langle M, w \rangle$ encodings for which M accepts w .
- U is sometimes called a **universal Turing machine** because it runs all TMs.
 - Like an interpreter for a programming language.

The Acceptance Problem

- $\text{Acc}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$.
- **U: On input $\langle M, w \rangle$:**
 - Simulate M on input w.
 - If M accepts, accept.
 - If M rejects, reject.
 - Otherwise, U loops forever.
- **U recognizes Acc_{TM} .**
- U is a **universal Turing machine** because it runs all TMs.
- U uses a fixed, finite set of states, and set of alphabet symbols, but still simulates TMs with arbitrarily many states and symbols.
 - All encoded using the fixed symbols, decoded during emulation.

The Acceptance Problem

- $\text{Acc}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$.
- **U: On input $\langle M, w \rangle$:**
 - Simulate M on input w.
 - If M accepts, accept.
 - If M rejects, reject.
 - Otherwise, U loops forever.
- **U recognizes Acc_{TM} .**
- **Does U decide Acc_{TM} ?**
- **No.**
 - If M loops forever on w, U loops forever on $\langle M, w \rangle$, never accepts or rejects.
 - To decide, U would have to **detect when M is looping** and reject.
 - Seems difficult...

Undecidability of Acceptance

- **Theorem 2:** Acc_{TM} is not Turing-decidable.
- **Proof:**
 - Assume that Acc_{TM} is Turing-decidable and produce a contradiction.
 - Similar to the diagonalization argument that shows that we can't enumerate all languages.
 - Since (we assume) Acc_{TM} is Turing-decidable, there must be a particular TM H that decides Acc_{TM} :
 - $H(\langle M, w \rangle)$:
 - accepts if M accepts w ,
 - rejects if M rejects w ,
 - rejects if M loops on w .

Undecidability of Acceptance

- **Theorem 2:** Acc_{TM} is not Turing-decidable.
- **Proof, cont'd:**
 - $H(\langle M, w \rangle)$ accepts if M accepts w , rejects if M rejects w or if M loops on w .
 - Use H to construct another TM H' that decides a special case of the same language.
 - Instead of considering whether M halts on an arbitrary w , just consider M on its own representation:
 - $H'(\langle M \rangle)$:
 - accepts if M accepts $\langle M \rangle$,
 - rejects if M rejects $\langle M \rangle$ or if M loops on $\langle M \rangle$.
 - If H exists, then so does H' : H' simply runs H on certain arguments.

Undecidability of Acceptance

- **Theorem 2:** Acc_{TM} is not Turing-decidable.
- **Proof, cont'd:**
 - $H'(<M>)$:
 - accepts if M accepts $<M>$,
 - rejects if M rejects $<M>$ or if M loops on $<M>$.
 - Now define D (the diagonal machine) to do the opposite of H' :
 - $D(<M>)$:
 - rejects if M accepts $<M>$,
 - accepts if M rejects $<M>$ or if M loops on $<M>$.
 - If H' exists, then so does D : D runs H' and outputs the opposite.

Undecidability of Acceptance

- **Theorem 2:** Acc_{TM} is not Turing-decidable.
- **Proof, cont'd:**
 - $D(\langle M \rangle)$:
 - rejects if M accepts $\langle M \rangle$,
 - accepts if M rejects $\langle M \rangle$ or if M loops on $\langle M \rangle$.
 - Now, what happens if we run D on $\langle D \rangle$?
 - Plug in D for M :
 - $D(\langle D \rangle)$:
 - rejects if D accepts $\langle D \rangle$,
 - accepts if D rejects $\langle D \rangle$ or if D loops on $\langle D \rangle$.
 - Then D accepts $\langle D \rangle$ if and only if D does not accept $\langle D \rangle$, contradiction!
 - So Acc_{TM} is not Turing-decidable.
 - !!!

Diagonalization Proofs

- This undecidability proof for Acc_{TM} is an example of a **diagonalization proof**.
- Earlier, we used diagonalization to show that the set of all languages is not countable.
- Consider a big matrix, with TMs labeling rows and strings that represent TMs labeling columns.
- The major diagonal describes results for $M(\langle M \rangle)$, for all M .
- D is a **diagonal machine**, constructed explicitly to differ from the diagonal entries: $D(\langle M \rangle)$'s result differs from $M(\langle M \rangle)$'s.
- Implies that D itself can't appear as a label for a row in the matrix, a contradiction since the matrix is supposed to include all TMs.

Summary: Acc_{TM}

- We have shown that $\text{Acc}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a Turing machine and } M \text{ accepts } w \}$ is Turing-recognizable but not Turing-decidable.
- **Corollary:** $(\text{Acc}_{\text{TM}})^c$ is not Turing-recognizable.
- **Proof:**
 - By Theorem 4.
 - If Acc_{TM} and $(\text{Acc}_{\text{TM}})^c$ were both Turing-recognizable, then Acc_{TM} would be Turing-decidable.

Undecidability of the Turing Machine Halting Problem

The Halting Problem

- **Halt_{TM}** = { $\langle M, w \rangle$ | M is a Turing machine and M halts on (either accepts or rejects) w }.
- Compare with **Acc_{TM}** = { $\langle M, w \rangle$ | M is a Turing machine and M accepts w }.
- **Terminology caution:** Sipser calls Acc_{TM} the “halting problem”, and calls Halt_{TM} just Halt_{TM}.
- **Theorem:** Halt_{TM} is not Turing-decidable.
- **Proof:**
 - Let’s not use diagonalization.
 - Rather, take advantage of diagonalization work already done for Acc_{TM}, using new method: **reduction**.
 - Prove that, if we could decide Halt_{TM}, then we could decide Acc_{TM}.
 - Reduction is a very powerful, useful technique for showing undecidability; we’ll use it several times.
 - Also useful (later) to show inherent complexity results.

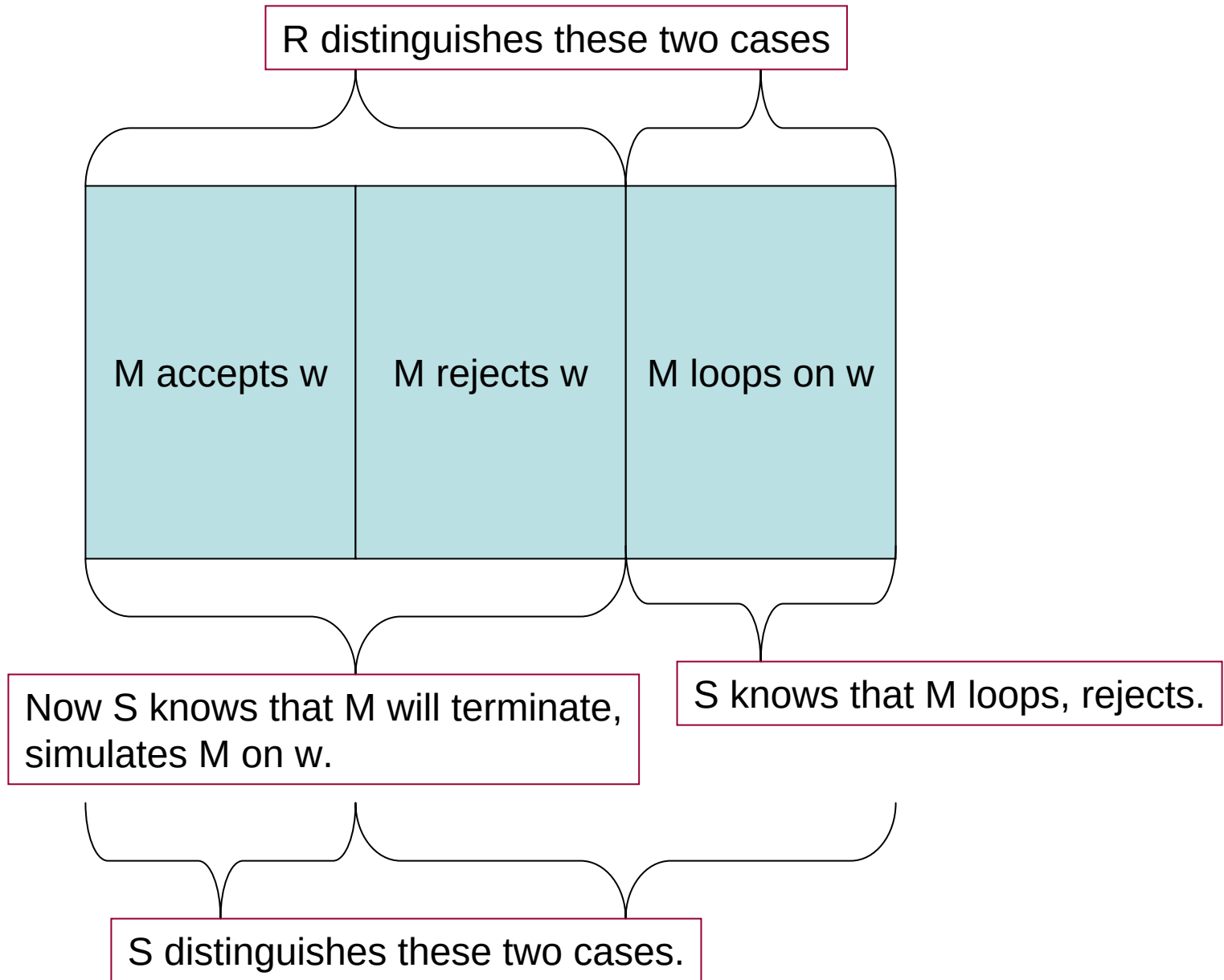
The Halting Problem

- $\text{Halt}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ halts on (accepts or rejects) } w \}$.
- **Theorem:** Halt_{TM} is not Turing-decidable.
- **Proof:**
 - Suppose for contradiction that Halt_{TM} is Turing-decidable, say by Turing machine R :
 - $R(\langle M, w \rangle)$:
 - accepts if M halts on (accepts or rejects) w ,
 - rejects if M loops (neither accepts nor rejects) on w .
 - Using R , define new TM S to decide Acc_{TM} :
 - S : On input $\langle M, w \rangle$:
 - Run R on $\langle M, w \rangle$; R must either accept or reject; can't loop, by definition of R .
 - If R accepts then M must halt (accept or reject) on w . Then simulate M on w , knowing this must terminate. If M accepts, accept. If M rejects, reject.
 - If R rejects, then reject.

The Halting Problem

- **Theorem:** Halt_{TM} is not Turing-decidable.
- **Proof:**
 - Suppose Halt_{TM} is Turing-decidable by TM R.
 - **S:** On input $\langle M, w \rangle$:
 - Run R on $\langle M, w \rangle$; R must either accept or reject; can't loop, by definition of R.
 - If R accepts then M must halt (accept or reject) on w. Then simulate M on w, knowing this must terminate. If M accepts, accept. If rejects, reject.
 - If R rejects, then reject.
 - Claim S decides Acc_{TM} : 3 cases:
 - If M accepts w, then R accepts $\langle M, w \rangle$, and the simulation leads S to accept.
 - If M rejects w, then R accepts $\langle M, w \rangle$, and the simulation leads S to reject.
 - If M loops on w, then R rejects $\langle M, w \rangle$, and S rejects.
 - That's what's supposed to happen in three cases, for Acc_{TM} .

The Three Cases



The Halting Problem

- **Theorem:** Halt_{TM} is not Turing-decidable.
- **Proof:**
 - Suppose Halt_{TM} is Turing-decidable by TM R.
 - **S:** On input $\langle M, w \rangle$:
 - Run R on $\langle M, w \rangle$; R must either accept or reject; can't loop, by definition of R.
 - If R accepts then M must halt (accept or reject) on w. Then simulate M on w, knowing this must terminate. If M accepts, accept. If rejects, reject.
 - If R rejects, then reject.
 - S decides Acc_{TM} .
 - So Acc_{TM} is decidable, contradiction.
 - Therefore, Halt_{TM} is not Turing-decidable.

The Halting Problem

- **Theorem:** Halt_{TM} is not Turing-decidable.
- Also:
- **Theorem:** Halt_{TM} is Turing-recognizable.
- So:
- **Corollary:** $(\text{Halt}_{\text{TM}})^c$ is not Turing-recognizable.

Next time...

- More undecidable problems:
 - About Turing machines:
 - Emptiness, etc.
 - About other things:
 - Post Correspondence Problem (a string matching problem).
- **Reading:** Sipser Sections 4.2, 5.1.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 8
Nancy Lynch

Today

- More undecidable problems:
 - About Turing machines: Emptiness, etc.
 - About other things: Post Correspondence Problem.
- **Topics:**
 - Undecidable problems about Turing machines.
 - The Post Correspondence Problem: Definition
 - Computation histories
 - First proof attempt
 - Second attempt: Undecidability of modified PCP (MPCP)
 - Finish undecidability of PCP
- **Reading:** Sipser Sections 4.2, 5.1.

Undecidable Problems about Turing Machines

Undecidable Problems about Turing Machines

- We already showed that Acc_{TM} and Halt_{TM} are not Turing-decidable (and their complements are not even Turing-recognizable).
- Now consider some other problems:
 - $\text{Acc01}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM that accepts the string } 01 \}$
 - $\text{Empty}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM that accepts no strings} \}$
 - $\text{Reg}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is regular} \}$
 - EQ_{TM} , equivalence for TMs, $= \{ \langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are TMs and } L(M_1) = L(M_2) \}$

Acc01_{TM}

- $\text{Acc01}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ accepts the string } 01 \}$
- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- This might seem surprising---it seems simpler than the general acceptance problem, since it involves just one particular string.
- **Proof attempt:**
 - Try a reduction---show if you could decide Acc01_{TM} then you could decide general acceptance problem Acc_{TM} .
 - Let R be a TM that decides Acc01_{TM} ; design S to decide Acc_{TM} .
 - **S :** On input $\langle M, w \rangle$:
 - Run R on $\langle M \rangle$.
 - If R accepts... ??? Gives useful information only if $w = 01$.
 - Doesn't work.

Acc01_{TM}

- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- **Proof attempt:**
 - Let R be a TM that decides Acc01_{TM}.
 - S: On input $\langle M, w \rangle$:
 - Run R on $\langle M \rangle$.
 - If R accepts... ???
 - Doesn't work.
- How can we use information about what a machine does on 01 to help decide what a given machine M will do on an arbitrary w?
- **Idea:** Consider a different machine---modify M.

Acc01_{TM}

- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- **Proof:**
 - Let R be a TM that decides Acc01_{TM}.; design S to decide Acc_{TM}.
 - **S:** On input $\langle M, w \rangle$:
 - Instead of running M on w, S constructs a new machine $M'_{M,w}$ that depends on M and w.
 - $M'_{M,w}$: On any input x, ignores x and runs M on w.
 - Thus, the new machine is the same as M, but hard-wires in the given input w.
 - More precisely:

Acc01_{TM}

- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- **Proof:**
 - R decides Acc01_{TM}.; design S to decide Acc_{TM}.
 - S: On input $\langle M, w \rangle$:
 - Step 1: Construct a new machine $\langle M'_{M,w} \rangle$, where
 - $M'_{M,w}$: On input x :
 - Run M on w and accept/reject if M does.
 - Step 2: Run R on $\langle M'_{M,w} \rangle$, and accept/reject if R does.
 - Note that S can construct $\langle M'_{M,w} \rangle$ algorithmically, from inputs M and w .

Acc01_{TM}

- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- **Proof:**
 - R decides Acc01_{TM}.; design S to decide Acc_{TM}.
 - **S:** On input $\langle M, w \rangle$:
 - **Step 1:** Construct a new machine $\langle M'_{M,w} \rangle$, where
 - $M'_{M,w}$: On input x :
 - Run M on w and accept/reject if M does.
 - **Step 2:** Run R on $\langle M'_{M,w} \rangle$, and accept/reject if R does.
 - Running R on $\langle M'_{M,w} \rangle$ tells us whether or not $M'_{M,w}$ accepts 01.
 - **Claim:** $M'_{M,w}$ accepts 01 if and only if M accepts w .
 - $M'_{M,w}$ always behaves the same, ignoring its own input and simulating M on w .
 - If $M'_{M,w}$ accepts 01 (or anything else), then M accepts w .
 - If M accepts w , then $M'_{M,w}$ accepts 01 (and everything else).
 - So S gives the right answer for whether M accepts w .

Acc01_{TM}

- **Theorem 1:** Acc01_{TM} is not Turing-decidable.
- **Theorem:** Acc01_{TM} is Turing-recognizable.
- **Corollary:** (Acc01_{TM})^c is not Turing-recognizable.

Empty_{TM}

- $\text{Empty}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset \}$
- **Theorem 2:** Empty_{TM} is not Turing-decidable.
- **Proof:**
 - Reduce Acc_{TM} to Empty_{TM} .
 - Modify the given machine M : Given $\langle M, w \rangle$, construct a new machine M' so that asking whether $L(M') = \emptyset$ gives the right answer to whether M accepts w :
 - Specifically, **M accepts w if and only if $L(M') \neq \emptyset$.**
 - Use the same machine M' as for Acc01_{TM} .
 - **S:** On input $\langle M, w \rangle$:
 - Step 1: Construct $\langle M'_{M,w} \rangle$ as before, which acts on every input just like M on w .
 - Step 2: Ask whether $L(M'_{M,w}) = \emptyset$ and output the opposite answer.

Empty_{TM}

- **Theorem 2:** Empty_{TM} is not Turing-decidable.
- **Proof:**
 - Reduce Acc_{TM} to Empty_{TM}.
 - **S:** On input $\langle M, w \rangle$:
 - Step 1: Construct $\langle M'_{M,w} \rangle$ as before, which acts on every input just like M on w .
 - Step 2: Ask whether $L(M'_{M,w}) = \emptyset$ and output the opposite answer.
 - Now M accepts w
 - if and only if $M'_{M,w}$ accepts everything
 - if and only if $M'_{M,w}$ accepts something
 - if and only if $L(M'_{M,w}) \neq \emptyset$.
 - So S decides Acc_{TM}, contradiction.
 - So Empty_{TM} is not Turing-decidable.

Empty_{TM}

- **Theorem 2:** Empty_{TM} is not Turing-decidable.
- **Theorem:** (Empty_{TM})^c is Turing-recognizable.
- **Proof:** On input $\langle M \rangle$, run M on all inputs, dovetailed, accept if any accept.
- **Corollary:** Empty_{TM} is not Turing-recognizable

Reg_{TM}

- $\text{Reg}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is regular} \}$
- That is, given a TM, we want to know whether its language is also recognized by some DFA.
- For some, the answer is yes: TM that recognizes 0^*1^*
- For some, no: TM that recognizes $\{0^n1^n \mid n \geq 0\}$
- We can prove that there is no algorithm to decide whether the answer is yes or no.
- **Theorem 3:** Reg_{TM} is not Turing-decidable.
- **Proof:**
 - Reduce Acc_{TM} to Reg_{TM} .
 - Assume TM R that decides Reg_{TM} , design S to decide Acc_{TM} .
 - **S:** On input $\langle M, w \rangle$:
 - Step 1: Construct a new machine $\langle M'_{M,w} \rangle$ that accepts a regular language if and only if M accepts w .
 - Tricky...

Reg_{TM}

- $\text{Reg}_{\text{TM}} = \{ \langle M \rangle \mid L(M) \text{ is regular} \}$
- **Theorem 3:** Reg_{TM} is not Turing-decidable.
- **Proof:**
 - Assume R decides Reg_{TM} , design S to decide Acc_{TM} .
 - S : On input $\langle M, w \rangle$:
 - Step 1: Construct a new machine $\langle M'_{M,w} \rangle$ that accepts a regular language if and only if M accepts w .
 - $M'_{M,w}$: On input x :
 - If x is of the form $0^n 1^n$, then accept.
 - If x is not of this form, then run M on w and accept if M accepts.
 - Step 2: Run R on input $\langle M'_{M,w} \rangle$, and accept/reject if R does.

Reg_{TM}

- **Theorem 3:** Reg_{TM} is not Turing-decidable.
- **Proof:**
 - S: On input $\langle M, w \rangle$:
 - Step 1: Construct a new machine $\langle M'_{M,w} \rangle$ that accepts a regular language if and only if M accepts w .
 - $M'_{M,w}$: On input x :
 - If x is of the form $0^n 1^n$, then accept.
 - If x is not of this form, then run M on w and accept if M accepts.
 - Step 2: Run R on input $\langle M'_{M,w} \rangle$, and accept/reject if R does.
 - If M accepts w , then $M'_{M,w}$ accepts everything, hence recognizes the regular language $\{0,1\}^*$.
 - If M does not accept w , then $M'_{M,w}$ accepts exactly the strings of the form $0^n 1^n$, which constitute a non-regular language.
 - Thus, M accepts w iff $M'_{M,w}$ recognizes a regular language.

And more questions

- Many more questions about what TMs compute can be proved undecidable using the same method.
- One more example: $EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are basic TMs that recognize the same language} \}$
- **Theorem 4:** EQ_{TM} is not Turing-decidable.
- **Proof:**
 - Reduce $Empty_{TM}$ to EQ_{TM} .
 - Assume R is a TM that decides EQ_{TM} ; design S to decide $Empty_{TM}$.
 - Define any particular TM $M_{\emptyset}$ with $L(M) = \emptyset$ (M accepts nothing).
 - **S :** On input $\langle M \rangle$:
 - Run R on input $\langle M, M_{\emptyset} \rangle$; accept/reject if R does.
 - R tells whether $\langle M, M_{\emptyset} \rangle \in EQ_{TM}$, that is, whether $L(M) = L(M_{\emptyset}) = \emptyset$.

An Undecidable Problem not involving Turing Machines

Post Correspondence Problem

- A simple string-matching problem.
- Given a finite set of “tile types”, e.g.:

$$\left\{ \begin{pmatrix} a \\ a b \end{pmatrix} \begin{pmatrix} c a \\ a b \end{pmatrix} \begin{pmatrix} b \\ c \end{pmatrix} \begin{pmatrix} b d \\ d \end{pmatrix} \right\}$$

- Is there a nonempty finite sequence of tiles (allowing repetitions, and not necessarily using all the tile types) for which the concatenation of top strings = concatenation of bottom strings?
- **Example:** $\begin{pmatrix} a \\ a b \end{pmatrix} \begin{pmatrix} b d \\ d \end{pmatrix}$ or $\begin{pmatrix} a \\ a b \end{pmatrix} \begin{pmatrix} b \\ c \end{pmatrix} \begin{pmatrix} c a \\ a b \end{pmatrix} \begin{pmatrix} b d \\ d \end{pmatrix}$
- No limit on length, but must be finite.
- Call such a sequence a **match**, or **correspondence**.
- **Post Correspondence Problem (PCP) =**
 $\{ \langle T \rangle \mid T \text{ is a finite set of tile types that has a match} \}$

Post Correspondence Problem

- Given a finite set of tile types, is there a nonempty finite sequence of tiles for which the concatenation of top strings = concatenation of bottom strings?
- Call sequence a **match**, or **correspondence**.
- **Post Correspondence Problem (PCP) =**
 $\{ \langle T \rangle \mid T \text{ is a finite set of tile types that has a match} \}$.
- **Theorem:** PCP is undecidable.
- **Proof:**
 - Reduce Acc_{TM} to PCP.
 - Previous reductions involved reducing one question about TMs (usually Acc_{TM}) to another question about TMs.
 - Now we reduce TM acceptance to a question about matching strings.
 - Do this by encoding TM computations using strings...

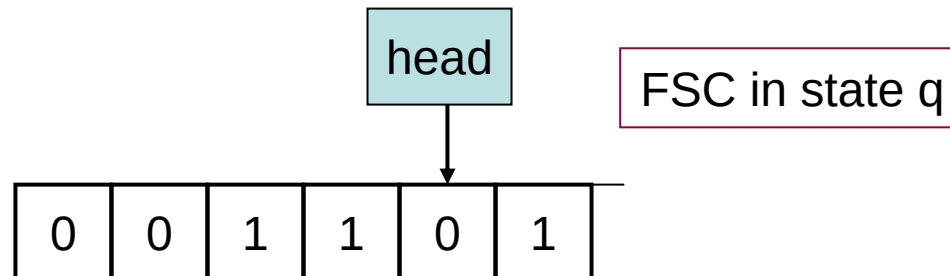
Computation Histories

Computation Histories

- **Computation History (CH):** A formal, stylized way of representing the computation of a TM on a particular input.
- **Configuration:**
 - Instantaneous snapshot of the TM's computation.
 - Includes current state, current tape contents, current head position.
 - Write in standard form: $w_1 q w_2$, where w_1 and w_2 are strings of tape symbols and q is a state.
 - Meaning:
 - $w_1 w_2$ is the string on the non-blank portion of the tape, perhaps part of the blank portion (rest assumed blank).
 - w_1 is the portion of the string strictly to the left of the head.
 - w_2 is the portion directly under the head and to the right.
 - q is the current state.

Configurations

- **Configuration:**
 - $w_1 q w_2$, where w_1 and w_2 are strings of tape symbols and q is a state.
 - Meaning:
 - $w_1 w_2$ is the string on the non-blank portion of the tape, perhaps part of the blank portion (rest assumed blank).
 - w_1 is the portion of the string strictly to the left of the head.
 - w_2 is the portion directly under the head and to the right.
 - q is the current state.
- **Example:** 0011q01 represents TM configuration:



Computation Histories

- TM begins in a **starting configuration**, of the form $q_0 w$, where w is the input string, and moves through a series of configurations, following the transition function.
- **Computation History of TM M on input w :**
 - A (finite or infinite) sequence of configs $C_1, C_2, C_3, \dots, C_k, \dots$, where
 - $C_1, C_2, \dots$ are configurations of M .
 - C_1 is the starting configuration with input w .
 - Each C_{i+1} follows from C_i using M 's transition function.
- **Accepting CH:** Finite CH ending in accepting configuration.
- **Rejecting CH:** Finite CH ending in rejecting configuration.
- Represent CH as a string $\# C_1 \# C_2 \# \dots \# C_k \#$, where $\#$ is a special separator symbol.
- **Claim:** M accepts w iff there is an accepting CH of M on w .

Undecidability of PCP: First Attempt

First attempt

- **Theorem:** PCP is undecidable.
- **Proof attempt:**
 - Reduce Acc_{TM} to PCP, that is, show that, if we can decide PCP, then we can decide Acc_{TM} .
 - Given $\langle M, w \rangle$, construct a finite set $T_{M,w}$ of tile types such that $T_{M,w}$ has a match iff M accepts w .
 - That is, $T_{M,w}$ has a match iff there is an accepting CH of M on w .
 - Write the accepting CH twice:

$$\begin{array}{ccccccccccc} \# & C_1 & \# & C_2 & \# & C_3 & \# & \dots & \# & C_k & \# \\ \# & C_1 & \# & C_2 & \# & C_3 & \# & \dots & \# & C_k & \# \end{array}$$
 - Split along boundaries of successive configurations:

$$\left| \begin{array}{ccccccccccc} \# & \boxed{C_1} & \# & \boxed{C_2} & \# & \boxed{C_3} & \# & \dots & \# & \boxed{C_k} & \# \\ \# & \boxed{C_1} & \# & \boxed{C_2} & \# & \boxed{C_3} & \# & \dots & \# & \boxed{C_k} & \# \end{array} \right|$$

First attempt

- Given $\langle M, w \rangle$, construct a finite set $T_{M,w}$ of tile types s.t.
 $T_{M,w}$ has a match iff there is an accepting CH of M on w .
- Write the accepting CH twice, and split along boundaries of successive configurations:

$$\left| \begin{array}{c} \# \boxed{C_1} \# \boxed{C_2} \# \boxed{C_3} \# \dots \# \boxed{C_k} \# \\ \# \boxed{C_1} \# \boxed{C_2} \# \boxed{C_3} \# \dots \# \boxed{C_k} \# \end{array} \right|$$

– What tiles do we need?

– Try $T_{M,w} = \left\{ \begin{pmatrix} \# \\ \# C_1 \end{pmatrix} \begin{pmatrix} C_k \# \\ \# \end{pmatrix} \begin{pmatrix} C_i \# \\ \# C_j \end{pmatrix} \right\}$ where

- C_1 = starting configuration for M on w ,
- C_k = accepting configuration (can assume unique, because we can assume accepting machine cleans up its tape).
- C_j follows from C_i by rules of M (one step).

First attempt

$$- T_{M,w} = \left\{ \begin{pmatrix} \# \\ \# C_1 \end{pmatrix} \begin{pmatrix} C_k \# \\ \# \end{pmatrix} \begin{pmatrix} C_i \# \\ \# C_j \end{pmatrix} \right\}$$

- C_1 = starting configuration for M on w ,
- C_k = accepting configuration.
- C_j follows from C_i by rules of M (one step).
- M accepts w iff $T_{M,w}$ has a match.
- But there is a problem:
 - $T_{M,w}$ has infinitely many tile types $T_{M,w}$, because M has infinitely many configurations.
 - Configuration has tape contents, state, head position---infinitely many possibilities.
 - Of course, in any particular accepting computation, only finitely many configurations appear.
 - But we don't know what these are ahead of time.
 - So we can't pick a single finite set of tiles.

First attempt

- M accepts w iff $T_{M,w}$ has a match.
- But:
 - $T_{M,w}$ has infinitely many tile types $T_{M,w}$, because M has infinitely many configurations.
 - In any particular accepting computation, only finitely many configurations appear.
 - But we can't pick a single finite set of tiles for all computations.
- New insight:
 - Represent infinitely many configurations with finitely many tiles.
 - Going from one configuration to the next involves changing only a few “local” things:
 - State
 - Contents of one tape cell
 - Position of head, by at most 1
 - So let tiles represent small pieces of configs, not entire configs.

Undecidability of Modified PCP

Undecidability of Modified PCP

- **Modified PCP (MPCP):** Like PCP, but we're given not just a finite set of tiles, but also a designated tile that must start the match.
- $\text{MPCP} = \{ \langle T, t \rangle \mid T \text{ is a finite set of tiles, } t \text{ is a tile in } T, \text{ and there is a match for } T \text{ starting with } t \}$.
- **Theorem:** MPCP is undecidable.
- Later, we remove the requirement to start with t :
- **Theorem:** PCP is undecidable.
- **Proof:**
 - By reducing MPCP to PCP.
 - If PCP were decidable, MPCP would be also, contradiction.

MPCP is undecidable

- Reduce Acc_{TM} to MPCP.
- Given $\langle M, w \rangle$, construct $(T_{M,w}, t_{M,w})$, an instance of MPCP.
- 7 kinds of tiles:
- **Type 1 tile:**
$$\left(\begin{array}{c} \# \\ \# q_0 w_1 w_2 \dots w_n \# \end{array} \right)$$
 - $w = w_1 w_2 \dots w_n$
 - $q_0 w_1 w_2 \dots w_n$ is the starting configuration for input w .
 - Bottom string is long, but there's only one tile like this.
 - Tile depends on w , which is OK.
 - Make this the initial tile $t_{M,w}$.

MPCP is undecidable

- Now consider how M goes from one configuration to the next.
- E.g., by moving right: $\delta(q,a) = (q',b,R)$.
- Config changes using this transition look like (e.g.):
 - $w_1 w_2 q a w_3 \rightarrow w_1 w_2 b q' w_3$.
 - Only change is to replace $q a$ by $b q'$.
- **Type 2 tiles:**
 - For each transition of the form $\delta(q,a) = (q',b,R)$:

$$\begin{pmatrix} q & a \\ b & q' \end{pmatrix}$$

MPCP is undecidable

- E.g., moving left: $\delta(q,a) = (q',b,L)$.
- **Type 3 tile:**
 - For each transition of the form $\delta(q,a) = (q',b,L)$, and every symbol c in the tape alphabet Γ :

$$\begin{pmatrix} c & q & a \\ q' & c & b \end{pmatrix}$$

- Include arbitrary c because it could be anything.
- Notice, only finitely many tiles (so far).

MPCP is undecidable

- Now, to match unchanged portions of 2 consecutive configurations:
- **Type 4 tile:**
 - For every symbol a in the tape alphabet Γ :

$$\begin{pmatrix} a \\ a \end{pmatrix}$$

- Still only finitely many tiles.

MPCP is undecidable

- What can we do with the tiles we have so far?
- **Example: Partial match**
 - Suppose the starting configuration is $q_0 1 1 0$ and the first move is $(q_0, 1) \rightarrow (q_4, 0, R)$.
 - Then the next configuration is $0 q_4 1 0$.
 - We can start the match with tile 1: $\begin{pmatrix} \# \\ \# q_0 110 \# \end{pmatrix}$
 - Continue with type 2 tile: $\begin{pmatrix} q_0 1 \\ 0 q_4 \end{pmatrix}$
 - Use type 4 tiles for the 2 unchanged symbols: $\begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
 - Yields: $\# \begin{array}{|c|c|c|c|} \hline q_0 & 1 & 1 & 0 \\ \hline \end{array} \#$
 $\# \begin{array}{|c|c|c|c|c|c|c|c|} \hline q_0 & 1 & 1 & 0 & \# & 0 & q_4 & 1 & 0 \\ \hline \end{array} \#$

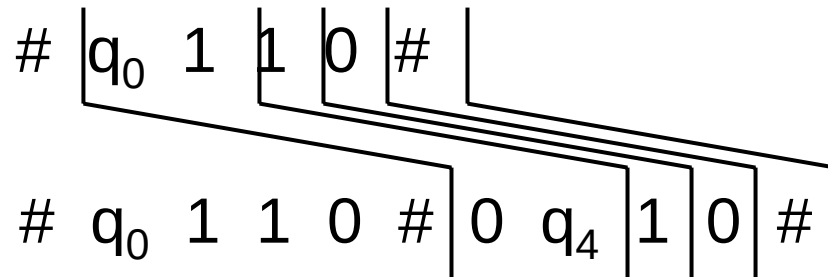
MPCP is undecidable

- Now we put in the separators.
- Type 5 tiles:

$$\begin{pmatrix} \# \\ \# \end{pmatrix} \quad \begin{pmatrix} \# \\ \text{--} \# \end{pmatrix}$$

Allows us to add extra spaces at right end as needed---lets the configuration size grow.

- Example: Extend previous match:



MPCP is undecidable

- How does this end?

- **Type 6 tiles:**

- For every a in Γ :

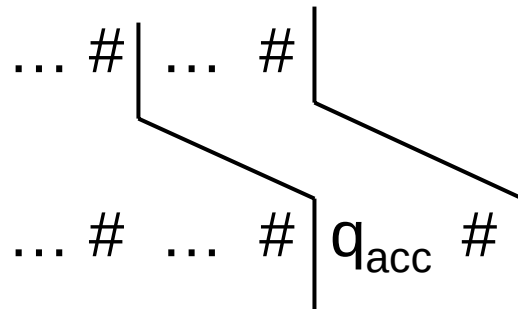
$$\begin{pmatrix} a & q_{\text{acc}} \\ & q_{\text{acc}} \end{pmatrix} \begin{pmatrix} q_{\text{acc}} & a \\ & q_{\text{acc}} \end{pmatrix}$$

- A trick...

- Adds “pseudo-steps” to the end of the computation, where the state “eats” adjacent symbols in the top row.

- Yields one symbol less in each successive bottom configuration.

- Do this until the remaining bottom “configuration” is $q_{\text{acc}} \#$:



MPCP is undecidable

- To finish off:
- Type 7 tile:

$$\begin{pmatrix} q_{\text{acc}} \# \# \\ \# \end{pmatrix}$$

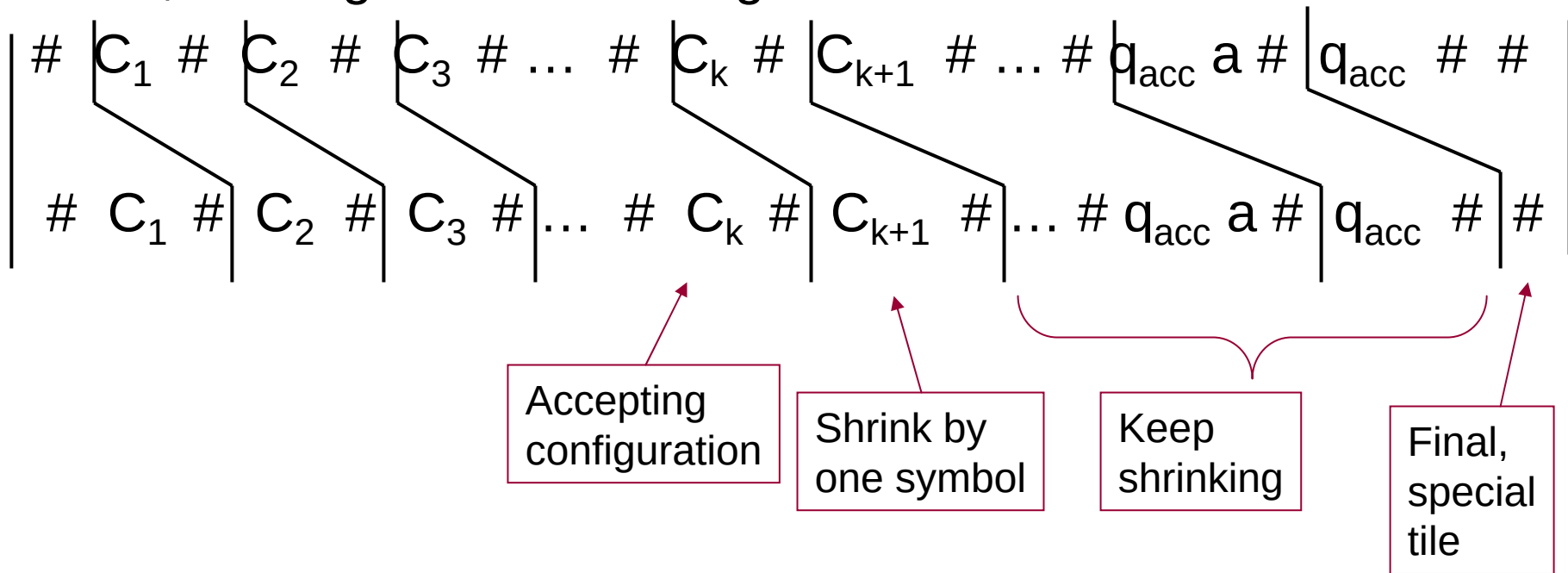
- That completes the definition of $T_{M,w}$ and $t_{M,w}$.
- Note that $T_{M,w}$, for a given M and w , is a finite set of tiles.

MPCP is undecidable

- That completes the definition of $T_{M,w}$ and $t_{M,w}$.
- Note that $T_{M,w}$, for a given M and w , is a finite set of tiles.
- Why does this work?
- Must show:
 - If M accepts w , then $T_{M,w}$ has a match beginning with $t_{M,w}$, that is, $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$.
 - If $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$, then M accepts w .
- If M accepts w , then there is an accepting computation history, which can be described by a match using the given tiles, starting from the distinguished initial tile:

MPCP is undecidable

- If M accepts w , then there is an accepting computation history, which can be described by a match using the given tiles, starting from the distinguished initial tile:



- So $T_{M,w}$ has a match beginning with $t_{M,w}$, that is, $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$.

MPCP is undecidable

- If $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$, that is, if $T_{M,w}$ has a match beginning with the designated tile $t_{M,w}$, then M accepts w .
- The rules are designed so the only way we can get a match beginning with the designated tile:

$$\left(\begin{array}{c} \# \\ \# q_0 w_1 w_2 \dots w_n \# \end{array} \right)$$

is to have an actual accepting computation of M on w . Hand-wave, in the book, LTTR.

- Combining the two directions, we get:
 M accepts w iff $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$, that is,
 $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$ iff $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$.

MPCP is undecidable

- $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$ iff $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$.
- **Theorem:** MPCP is undecidable.
- **Proof:**
 - By contradiction.
 - Assume MPCP is decidable, and decide Acc_{TM} , using S:
 - **S:** On input $\langle M, w \rangle$:
 - Step 1: Construct $\langle T_{M,w}, t_{M,w} \rangle$, instance of MPCP, as described.
 - Step 2: Use MPCP to decide if $T_{M,w}$ has a match beginning with $t_{M,w}$. If so, accept; if not, reject.
 - Thus, if MPCP is decidable, then also Acc_{TM} is decidable, contradiction.

Undecidability of (Unmodified) PCP

Undecidability of PCP

- We showed that MPCP, in which the input is a set of tiles + designated input tile, is undecidable, by reducing Acc_{TM} to MPCP.
- Now we want:
- **Theorem:** PCP is undecidable.
- Why doesn't our construction reduce Acc_{TM} to PCP?
- $T_{M,v}$ has trivial matches, e.g., just
$$\begin{pmatrix} a \\ a \end{pmatrix}$$
- **Proof of the theorem:**
 - To show that PCP is undecidable, reduce MPCP to PCP, that is, show that if PCP is decidable, then so is MPCP.

Undecidability of PCP

- **Theorem:** PCP is undecidable.
- **Proof:**
 - Reduce MPCP to PCP.
 - To decide MPCP using PCP, suppose we are given:
 - T: $\left\{ \begin{pmatrix} u_1 \\ v_1 \end{pmatrix} \begin{pmatrix} u_2 \\ v_2 \end{pmatrix} \dots \begin{pmatrix} u_k \\ v_k \end{pmatrix} \right\}$
 - t: $\begin{pmatrix} u_1 \\ v_1 \end{pmatrix}$
 - We want to know if there is a match beginning with t.
 - Construct an instance T' of ordinary PCP that has a match (starting with any tile) iff T has a match starting with t.

Undecidability of PCP

- Given $T: \left\{ \begin{pmatrix} u_1 \\ v_1 \end{pmatrix} \begin{pmatrix} u_2 \\ v_2 \end{pmatrix} \dots \begin{pmatrix} u_k \\ v_k \end{pmatrix} \right\}$ $t: \begin{pmatrix} u_1 \\ v_1 \end{pmatrix}$
- Construct an instance T' of PCP that has a match iff T has a match starting with t .
- Construction (technical):
 - Add 2 new alphabet symbols, ♥ and ♦
 - If $u = u_1 u_2 \dots u_n$ then define:
 - ♥ $u = ♥ u_1 ♥ u_2 \dots ♥ u_n$
 - $u ♥ = u_1 ♥ u_2 \dots ♥ u_n ♥$
 - ♥ $u ♥ = ♥ u_1 ♥ u_2 \dots ♥ u_n ♥$
 - Instance T' of PCP:

$$\left\{ \begin{pmatrix} ♥ u_1 \\ ♥ v_1 ♥ \end{pmatrix} \begin{pmatrix} ♥ u_1 \\ v_1 ♥ \end{pmatrix} \begin{pmatrix} ♥ u_2 \\ v_2 ♥ \end{pmatrix} \dots \begin{pmatrix} ♥ u_k \\ v_k ♥ \end{pmatrix} \begin{pmatrix} ♥ ♦ \\ ♦ \end{pmatrix} \right\}$$

Undecidability of PCP

- **Claim:** T has a match starting with t iff T' has any match.

$\Rightarrow$ Suppose T has a match starting with t: $\begin{pmatrix} u_1 \\ v_1 \end{pmatrix}$
 Mimic this match with T' tiles, starting with $\begin{pmatrix} \heartsuit u_1 \\ \heartsuit v_1 \heartsuit \end{pmatrix}$
 and ending with $\begin{pmatrix} \heartsuit \spadesuit \\ \spadesuit \end{pmatrix}$

Yields the same matching strings, with $\heartsuit$ s interspersed, and with $\spadesuit$ at the end.

$\Leftarrow$ If T' has any match, it must begin with $\begin{pmatrix} \heartsuit u_1 \\ \heartsuit v_1 \heartsuit \end{pmatrix}$
 because that's the only tile in which top and bottom start with the same symbol.

Other tiles are like T tiles but with extra $\heartsuit$ s.

Stripping out $\heartsuit$ s yields match for T beginning with t.

Undecidability of PCP

- So, to decide MPCP using a decider for PCP:
- Given instance $\langle T, t \rangle$ for MPCP,
 - Step 1: Construct instance T' for PCP, as above.
 - Step 2: Ask decider for PCP whether T' has any match.
 - If so, answer yes for $\langle T, t \rangle$.
 - If not, answer no.
- Since we already know MPCP is undecidable, so is PCP.

Next time...

- Mapping reducibility
- Rice's Theorem
- Reading:
 - Sipser Section 5.3, Problems 5.28-5.30.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 9
Nancy Lynch

Today

- Mapping reducibility and Rice's Theorem
- We've seen several undecidability proofs.
- Today we'll extract some of the key ideas of those proofs and present them as general, abstract definitions and theorems.
- Two main ideas:
 - A formal definition of **reducibility** from one language to another. Captures many of the reduction arguments we have seen.
 - **Rice's Theorem**, a general theorem about undecidability of properties of Turing machine behavior (or program behavior).

Today

- Mapping reducibility and Rice's Theorem
- **Topics:**
 - Computable functions.
 - Mapping reducibility, $\leq_m$
 - Applications of $\leq_m$ to show undecidability and non-recognizability of languages.
 - Rice's Theorem
 - Applications of Rice's Theorem
- **Reading:**
 - Sipser Section 5.3, Problems 5.28-5.30.

Computable Functions

Computable Functions

- These are needed to define mapping reducibility, $\leq_m$.
- **Definition:** A function $f: \Sigma_1^* \rightarrow \Sigma_2^*$ is **computable** if there is a Turing machine (or program) such that, for every w in Σ_1^* , M on input w halts with just $f(w)$ on its tape.
- To be definite, use basic TM model, except replace q_{acc} and q_{rej} states with one q_{halt} state.
- So far in this course, we've focused on accept/reject decisions, which let TMs **decide language membership**.
- That's the same as computing functions from Σ^* to $\{ \text{accept}, \text{reject} \}$.
- Now generalize to compute functions that produce strings.

Total vs. partial computability

- We require f to be **total** = defined for every string.
- Could also define **partial computable** (= **partial recursive**) functions, which are defined on some subset of Σ_1^* .
- Then M should not halt if $f(w)$ is undefined.

Computable functions

- **Example 1: Computing prime numbers.**
 - $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$
 - On input w that is a binary representation of positive integer i , result is the standard binary representation of the i^{th} prime number.
 - On inputs representing 0, result is the empty string ε .
 - Probably don't care what the result is in this case, but totality requires that we define something.
 - For instance:
 - $f(\varepsilon) = f(0) = f(00) = \varepsilon$
 - $f(1) = f(01) = f(001) = 10$ (binary rep of 2, first prime)
 - $f(10) = f(010) = 11$ (3, second prime)
 - $f(11) = 101$ (5, third prime)
 - $f(100) = 111$ (7, fourth prime)
 - Computable, e.g., by sieve algorithm.

Computable functions

- **Example 2: Reverse machine.**
 - $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$
 - On input $w = \langle M \rangle$, where M is a (basic) Turing machine, $f(w) = \langle M' \rangle$, where M' is a Turing machine that accepts exactly the reverses of the words accepted by M .
 - $L(M') = \{ w^R \mid w \in L(M) \}$
 - On inputs w that don't represent TMs, $f(w) = \varepsilon$.
 - Computable:
 - M' reverses its input and then simulates M .
 - Can compute description of M' from description of M .

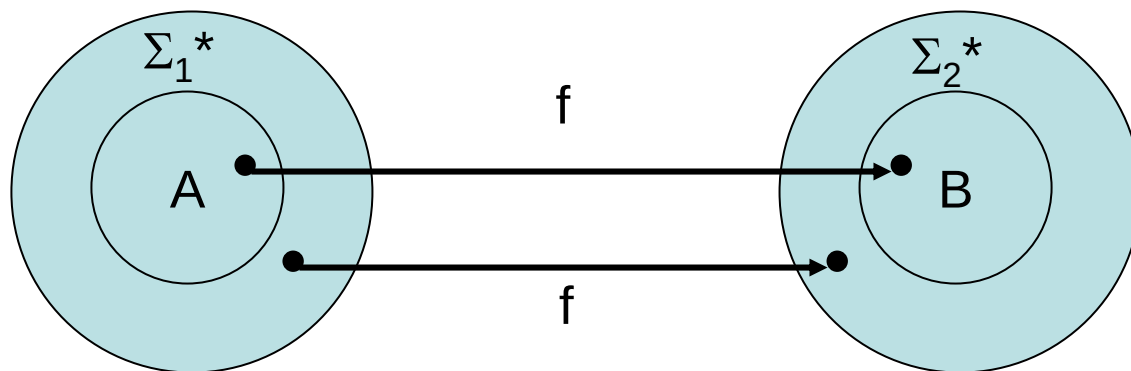
Computable functions

- Example 3: Transformations of DFAs, etc.
 - We studied several algorithmic transformations of DFAs and NFAs:
 - NFA $\rightarrow$ equivalent DFA
 - DFA for $L \rightarrow$ DFA for L^c
 - DFA for $L \rightarrow$ DFA for $\{ w^R \mid w \in L \}$
 - Etc.
 - All of these transformations can be formalized as computable functions (from machine representations to machine representations)

Mapping Reducibility

Mapping Reducibility

- **Definition:** Let $A \subseteq \Sigma_1^*$, $B \subseteq \Sigma_2^*$ be languages. Then **A is mapping-reducible to B**, $A \leq_m B$, provided that there is a computable function $f: \Sigma_1^* \rightarrow \Sigma_2^*$ such that, for every string w in Σ_1^* , $w \in A$ if and only if $f(w) \in B$.
- Two things to show for “if and only if”:



- We've already seen many instance of $\leq_m$ in the reductions we've used to prove undecidability and non-recognizability, e.g.:

Mapping reducibility examples

- Example: $\text{Acc}_{\text{TM}} \leq_m \text{Acc01}_{\text{TM}}$

Accepts the string 01, possibly others

- $\langle M, w \rangle \rightarrow \langle M'_{M,w} \rangle$, by computable function f .
- $M'_{M,w}$ behaves as follows: If M accepts w then it accepts everything; otherwise it accepts nothing.
- This f demonstrates mapping reducibility because:
 - If $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$ then $\langle M'_{M,w} \rangle \in \text{Acc01}_{\text{TM}}$.
 - If $\langle M, w \rangle \notin \text{Acc}_{\text{TM}}$ then $\langle M'_{M,w} \rangle \notin \text{Acc01}_{\text{TM}}$.
 - Thus, we have “if and only if”, as needed.
 - And f is computable.
- Technicality: Must also map inputs not of the form $\langle M, w \rangle$ somewhere.

Mapping reducibility examples

- Example: $\text{Acc}_{\text{TM}} \leq_m (\text{E}_{\text{TM}})^c$

Nonemptiness, $\{ M \mid M \text{ accepts some string} \}$

- $\langle M, w \rangle \rightarrow \langle M'_{M,w} \rangle$, by computable function f .
- Use same f as before: If M accepts w then $M'_{M,w}$ accepts everything; otherwise it accepts nothing.
- But now we must show something different:
 - If $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$ then $\langle M'_{M,w} \rangle \in (\text{E}_{\text{TM}})^c$.
 - Accepts something, in fact, accepts everything.
 - If $\langle M, w \rangle \notin \text{Acc}_{\text{TM}}$ then $\langle M'_{M,w} \rangle \in \text{E}_{\text{TM}}$.
 - Accepts nothing.
 - f is computable.
- Note: We didn't show $\text{Acc}_{\text{TM}} \leq_m \text{E}_{\text{TM}}$.
 - Reversed the sense of the answer (took the complement).

Mapping reducibility examples

- Example: $\text{Acc}_{\text{TM}} \leq_m \text{REG}_{\text{TM}}$.

↑
TMs accepting a regular language

- $\langle M, w \rangle \rightarrow \langle M'_{M,w} \rangle$, by computable function f .
- We defined f so that: If M accepts w then $M'_{M,w}$ accepts everything; otherwise it accepts exactly the strings of the form $0^n 1^n$, $n \geq 0$.
- So $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$
iff $M'_{M,w}$ accepts a regular language
iff $\langle M'_{M,w} \rangle \in \text{REG}_{\text{TM}}$.

Mapping reducibility examples

- Example: $\text{Acc}_{\text{TM}} \leq_m \text{MPCP}$.

Modified Post Correspondence Problem



- $\langle M, w \rangle \rightarrow \langle T_{M,w}, t_{M,w} \rangle$, by computable function f , where $\langle T_{M,w}, t_{M,w} \rangle$ is an instance of MPCP (set of tiles + distinguished tile).
- We defined f so that $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$
iff $T_{M,w}$ has a match starting with $t_{M,w}$
iff $\langle T_{M,w}, t_{M,w} \rangle \in \text{MPCP}$
- Example: $\text{Acc}_{\text{TM}} \leq_m \text{PCP}$.
- $\langle M, w \rangle \rightarrow \langle T_{M,w} \rangle$ where $\langle M, w \rangle \in \text{Acc}_{\text{TM}}$ iff $T_{M,w}$ has a match iff $\langle T_{M,w} \rangle \in \text{PCP}$.

Basic Theorems about $\leq_m$

- **Theorem 1:** If $A \leq_m B$ and B is Turing-decidable then A is Turing-decidable.
- **Proof:**
 - To decide if $w \in A$:
 - Compute $f(w)$
 - Can be done by a TM, since f is computable.
 - Decide whether $f(w) \in B$.
 - Can be done by a TM, since B is decidable.
 - Output the answer.
- **Corollary 2:** If $A \leq_m B$ and A is undecidable then B is undecidable.
- So undecidability of Acc_{TM} implies undecidability of E_{TM} , REG_{TM} , MPCP , etc.

Basic Theorems about $\leq_m$

- **Theorem 3:** If $A \leq_m B$ and B is Turing-recognizable then A is Turing-recognizable.
- **Proof:** On input w :
 - Compute $f(w)$.
 - Run a TM that recognizes B on input $f(w)$.
 - If this TM ever accepts, accept.
- **Corollary 4:** If $A \leq_m B$ and A is not Turing-recognizable then B is not Turing-recognizable.
- **Theorem 5:** $A \leq_m B$ if and only if $A^c \leq_m B^c$.
- **Proof:** Use same f .
- **Theorem 6:** If $A \leq_m B$ and $B \leq_m C$ then $A \leq_m C$.
- **Proof:** Compose the two functions.

Basic Theorems about $\leq_m$

- **Theorem 6:** If $A \leq_m B$ and $B \leq_m C$ then $A \leq_m C$.
- **Example: PCP**
 - Showed $\text{Acc}_{\text{TM}} \leq_m \text{MPCP}$.
 - Showed $\text{MPCP} \leq_m \text{PCP}$.
 - Conclude from Theorem 6 that $\text{Acc}_{\text{TM}} \leq_m \text{PCP}$.

More Applications of Mapping Reducibility

Applications of $\leq_m$

- We have already used $\leq_m$ to show undecidability; now use it to show non-Turing-recognizability.
- **Example: Acc01_{TM}**
 - We already know that Acc01_{TM} is Turing-recognizable.
 - Now show that $(\text{Acc01}_{\text{TM}})^c$ is not Turing-recognizable.
 - We showed that $\text{Acc}_{\text{TM}} \leq_m \text{Acc01}_{\text{TM}}$.
 - So $(\text{Acc}_{\text{TM}})^c \leq_m (\text{Acc01}_{\text{TM}})^c$, by Theorem 5.
 - We also already know that $(\text{Acc}_{\text{TM}})^c$ is not Turing recognizable.
 - So $(\text{Acc01}_{\text{TM}})^c$ is not Turing-recognizable, by Corollary 4.

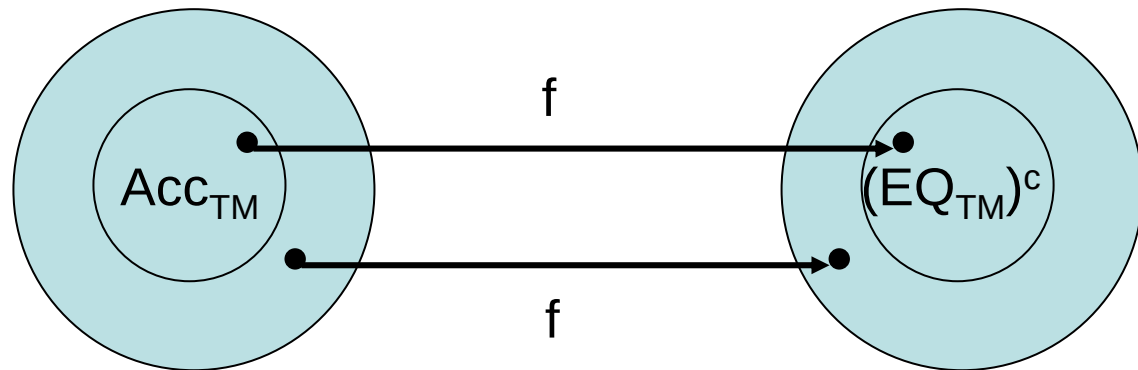
Applications of $\leq_m$

- Now an example of a language that is not Turing-recognizable and whose complement is also not Turing-recognizable.
- That is, it's neither Turing-recognizable nor **co-Turing-recognizable**.
- **Example:** $EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are TMs and } L(M_1) = L(M_2) \}$
 - Important in practice, e.g.:
 - Compare two versions of the “same” program.
 - Compare the result of a compiler optimization to the original un-optimized compiler output.
- **Theorem 7:** EQ_{TM} is not Turing-recognizable.
- **Theorem 8:** $(EQ_{TM})^c$ is not Turing-recognizable.

Applications of $\leq_m$

- $EQ_{TM} = \{ \langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) \}$
- **Theorem 7:** EQ_{TM} is not Turing-recognizable.
- **Proof:**

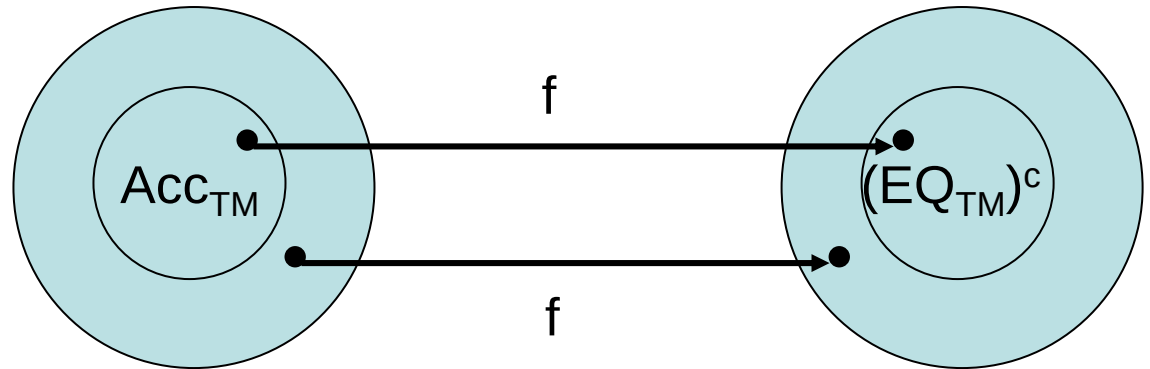
- Show $(Acc_{TM})^c \leq_m EQ_{TM}$ and use Corollary 4.
 - Already showed $(Acc_{TM})^c$ is not Turing-recognizable.
- Equivalently, show $Acc_{TM} \leq_m (EQ_{TM})^c$.
 - Equivalent by Theorem 5.
- Need:



- Accepting iff not equivalent.

EQ_{TM} is not Turing-recognizable.

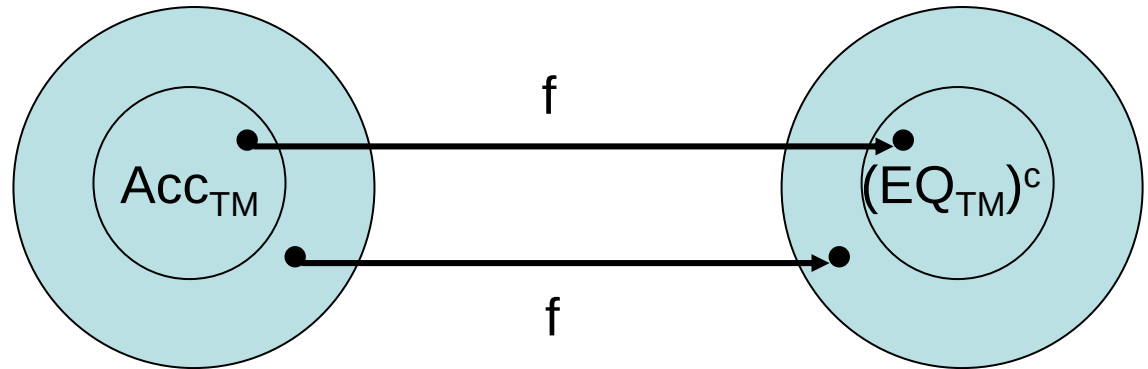
- $Acc_{TM} \leq_m (EQ_{TM})^c$:



- Define $f(x)$ so that $x \in Acc_{TM}$ iff $f(x) \in (EQ_{TM})^c$.
- If x is not of the form $\langle M, w \rangle$ define $f(x) = \langle M_0, M_0 \rangle$, where M_0 is any particular TM.
- Then $x \notin Acc_{TM}$ and $f(x) \in EQ_{TM}$, which fits our requirements.
- So now assume that $x = \langle M, w \rangle$.
- Then define $f(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 always rejects, and
 - M_2 ignores its input, runs M on w , and accepts iff M accepts w .
- Claim:** $x \in Acc_{TM}$ iff $f(x) \in (EQ_{TM})^c$.

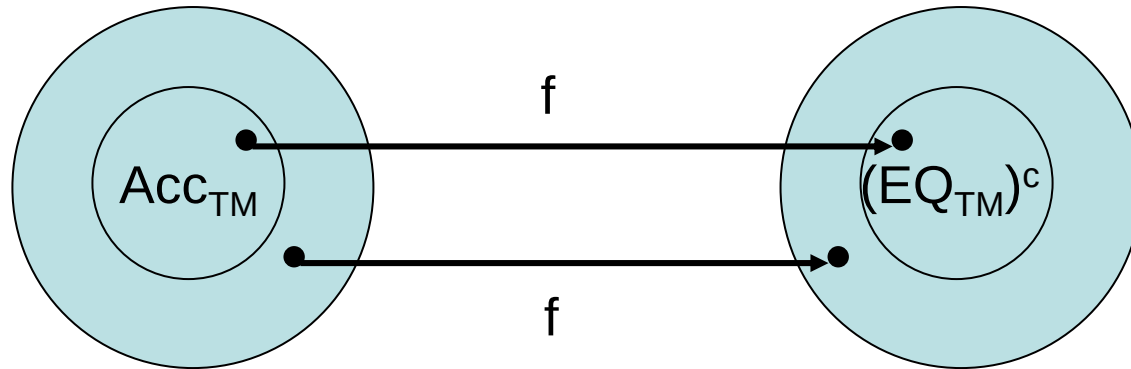
EQ_{TM} is not Turing-recognizable.

- $Acc_{TM} \leq_m (EQ_{TM})^c$:



- Assume $x = \langle M, w \rangle$, define $f(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 always rejects, and
 - M_2 ignores its input, runs M on w , and accepts iff M accepts w .
- **Claim:** $x \in Acc_{TM}$ iff $f(x) \in (EQ_{TM})^c$.
- **Proof:**
 - If $x \in Acc_{TM}$, then M accepts w , so M_2 accepts everything, so $\langle M_1, M_2 \rangle \notin EQ_{TM}$, so $\langle M_1, M_2 \rangle \in (EQ_{TM})^c$.
 - If $x \notin Acc_{TM}$, then M does not accept w , so M_2 accepts nothing, so $\langle M_1, M_2 \rangle \in EQ_{TM}$, so $\langle M_1, M_2 \rangle \notin (EQ_{TM})^c$.

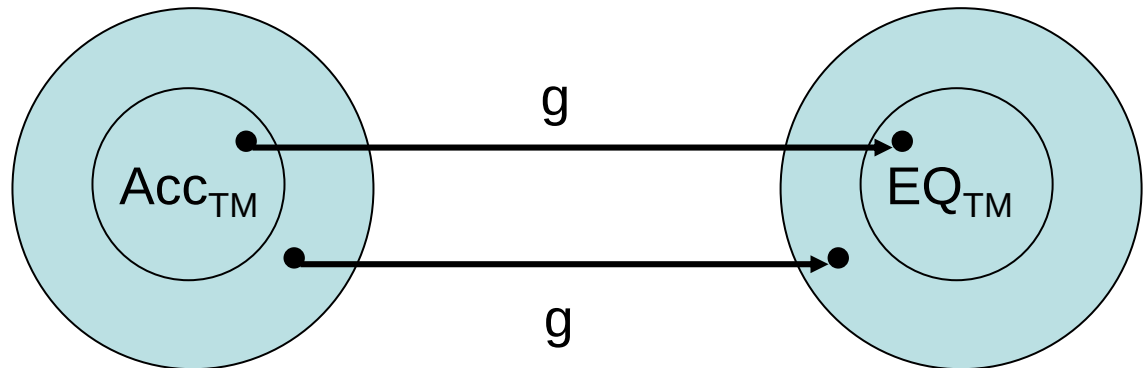
EQ_{TM} is not Turing-recognizable.



- Assume $x = \langle M, w \rangle$, define $f(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 always rejects, and
 - M_2 ignores its input, runs M on w , and accepts iff M accepts w .
- **Claim:** $x \in Acc_{TM}$ iff $f(x) \in (EQ_{TM})^c$.
- Therefore, $Acc_{TM} \leq_m (EQ_{TM})^c$ using f .
- So $(Acc_{TM})^c \leq_m EQ_{TM}$ by Theorem 5.
- So EQ_{TM} is not Turing-recognizable, by Corollary 4.

Applications of $\leq_m$

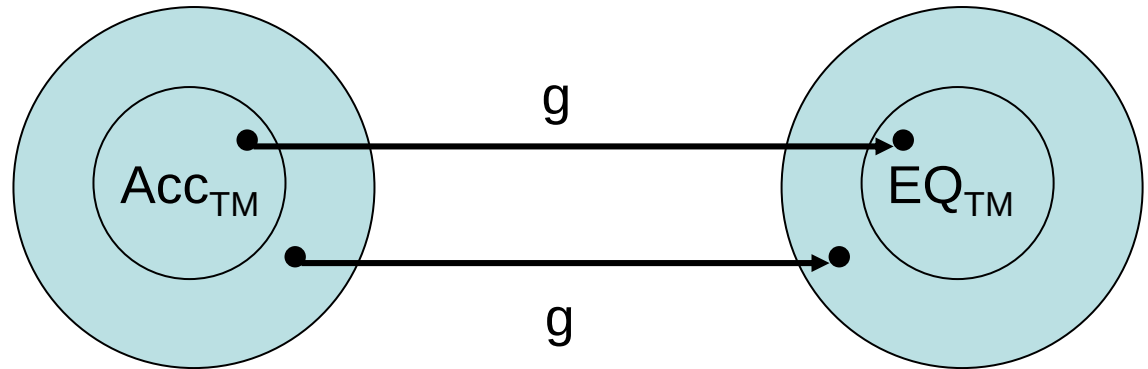
- We have proved:
- **Theorem 7:** EQ_{TM} is not Turing-recognizable.
- It turns out that the complement isn't T-recognizable either!
- **Theorem 8:** $(EQ_{TM})^c$ is not Turing-recognizable.
- **Proof:** Show $(Acc_{TM})^c \leq_m (EQ_{TM})^c$ and use Corollary 4.
 - We know $(Acc_{TM})^c$ is not Turing-recognizable.
 - Equivalently, show $Acc_{TM} \leq_m EQ_{TM}$.
 - Need:



- Accepting iff equivalent.

$(EQ_{TM})^c$ is not Turing-recognizable.

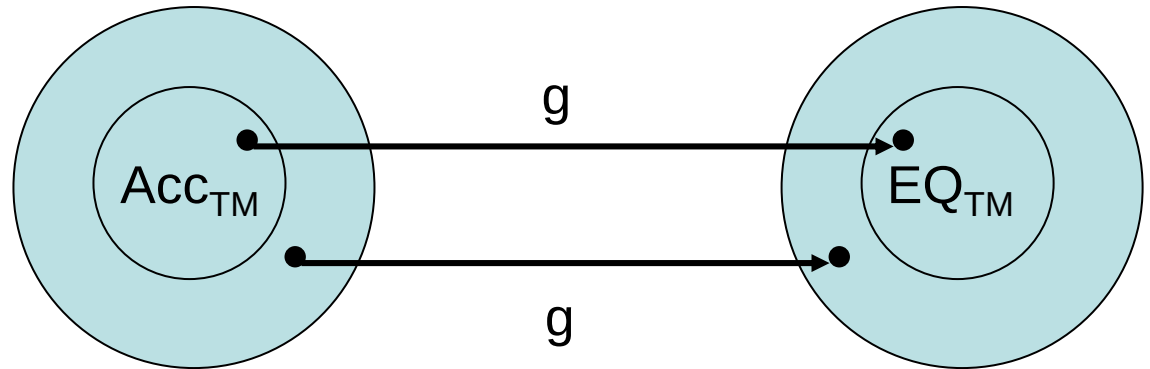
- $Acc_{TM} \leq_m EQ_{TM}$:



- Define $g(x)$ so that $x \in Acc_{TM}$ iff $f(x) \in EQ_{TM}$.
- If x is not of the form $\langle M, w \rangle$ define $f(x) = \langle M_0, M_0' \rangle$, where $L(M_0) \neq L(M_0')$.
- Then $x \notin Acc_{TM}$ and $g(x) \notin EQ_{TM}$, as required.
- So now assume $x = \langle M, w \rangle$.
- Define $g(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 accepts everything, and
 - M_2 ignores its input, runs M on w , accepts iff M does (as before).
- **Claim:** $x \in Acc_{TM}$ iff $g(x) \in EQ_{TM}$.

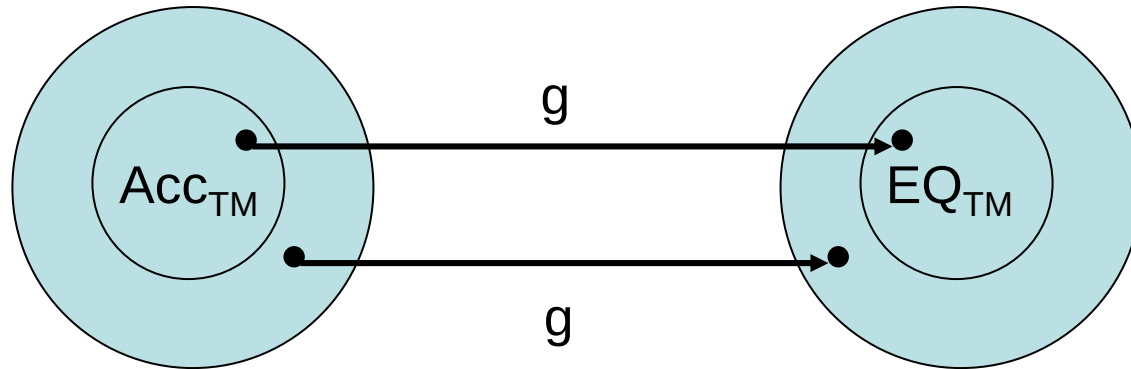
$(EQ_{TM})^c$ is not Turing-recognizable.

- $Acc_{TM} \leq_m EQ_{TM}$:



- Assume $x = \langle M, w \rangle$, define $g(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 accepts everything, and
 - M_2 ignores its input, runs M on w , and accepts iff M does.
- **Claim:** $x \in Acc_{TM}$ iff $g(x) \in EQ_{TM}$.
- **Proof:**
 - If $x \in Acc_{TM}$, then M_1 and M_2 both accept everything, so $\langle M_1, M_2 \rangle \in EQ_{TM}$.
 - If $x \notin Acc_{TM}$, then M_1 accepts everything and M_2 accepts nothing, so $\langle M_1, M_2 \rangle \notin EQ_{TM}$.

$(EQ_{TM})^c$ is not Turing-recognizable.



- Assume $x = \langle M, w \rangle$, define $g(x) = \langle M_1, M_2 \rangle$, where:
 - M_1 accepts everything, and
 - M_2 ignores its input, runs M on w , and accepts iff M does.
- **Claim:** $x \in Acc_{TM}$ iff $g(x) \in EQ_{TM}$.
- Therefore, $Acc_{TM} \leq_m EQ_{TM}$ using g .
- So $(Acc_{TM})^c \leq_m (EQ_{TM})^c$ by Theorem 5.
- So $(EQ_{TM})^c$ is not Turing-recognizable, by Corollary 4.

Rice's Theorem

Rice's Theorem

- We've seen many undecidability results for properties of TMs, e.g., for:
 - $\text{Acc01}_{\text{TM}} = \{ \langle M \rangle \mid 01 \in L(M) \}$
 - $E_{\text{TM}} = \{ \langle M \rangle \mid L(M) = \emptyset \}$
 - $\text{REG}_{\text{TM}} = \{ \langle M \rangle \mid L(M) \text{ is a regular language} \}$
- These are all **properties of the language recognized by the machine**.
- Contrast with:
 - $\{ \langle M \rangle \mid M \text{ never tries to move left off the left end of the tape} \}$
 - $\{ \langle M \rangle \mid M \text{ has more than 20 states} \}$
- **Rice's Theorem** says (essentially) that **any property of the language recognized by a TM is undecidable**.
- Very powerful theorem.
- Covers many problems besides the ones above, e.g.:
 - $\{ \langle M \rangle \mid L(M) \text{ is a finite set} \}$
 - $\{ \langle M \rangle \mid L(M) \text{ contains some palindrome} \}$
 - ...

Rice's Theorem

- Rice's Theorem says (essentially) that any property of the language recognized by a TM is undecidable.
- Technicality: Restrict to **nontrivial properties**.
- Define a set P of languages, to be a **nontrivial property of Turing-recognizable languages** provided that
 - There is some TM M_1 such that $L(M_1) \in P$, and
 - There is some TM M_2 such that $L(M_2) \notin P$.
- Equivalently:
 - There is some Turing-recognizable language L_1 in P , and
 - There is some Turing recognizable language L_2 not in P .
- **Rice's Theorem:** Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- !

Rice's Theorem

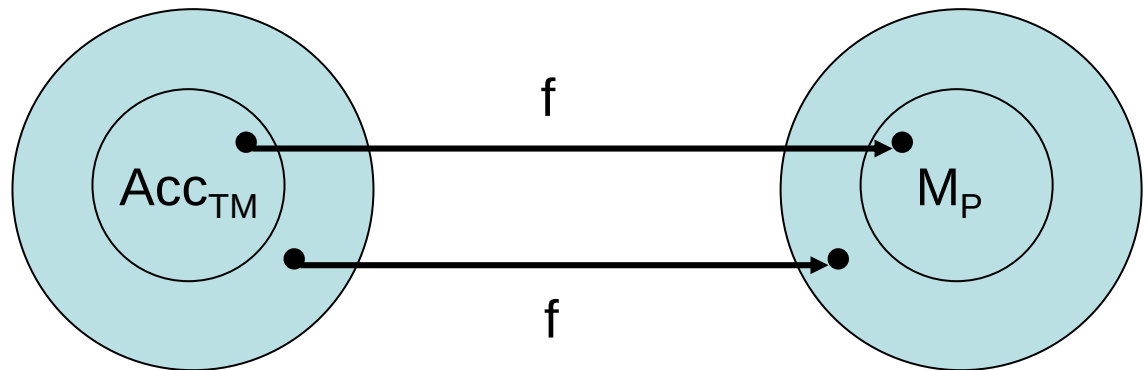
- P is a **nontrivial property of T-recog. languages** if:
 - There is some TM M_1 such that $L(M_1) \in P$, and
 - There is some TM M_2 such that $L(M_2) \notin P$.
- **Rice's Theorem:** Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- **Proof:**
 - Show $\text{Acc}_{\text{TM}} \leq_m M_P$.
 - Suppose WLOG that the empty language does not satisfy P , that is, $\emptyset \notin P$.
 - Why is this WLOG?
 - Otherwise, work with P^c instead of P .
 - Then $\emptyset \notin P^c$, continue the proof using P^c .
 - Conclude that M_{P^c} is undecidable.
 - Implies that M_P is undecidable.

Rice's Theorem

- **Rice's Theorem:** Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.

- **Proof:**

- Show $\text{Acc}_{\text{TM}} \leq_m M_P$.
- Suppose $\emptyset \notin P$.
- Need:



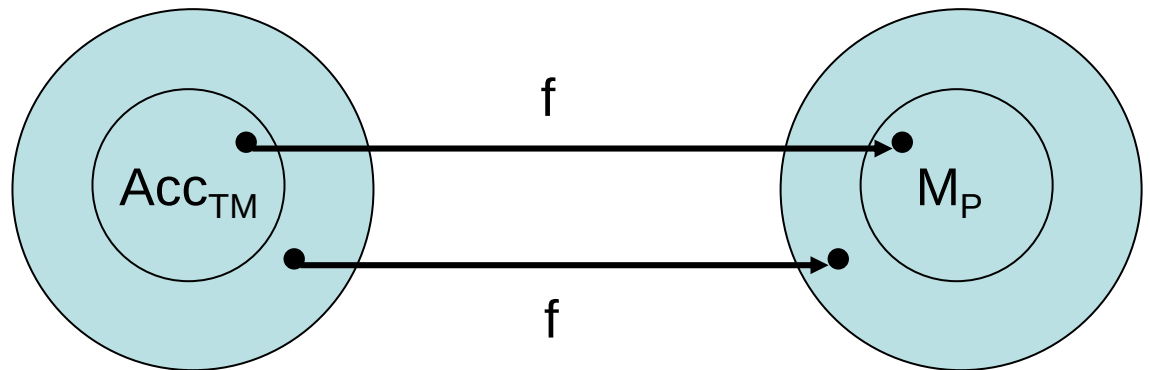
- Let M_1 be any TM such that $L(M_1) \in P$, so $\langle M_1 \rangle \in M_P$.
 - How do we know such M_1 exists?
 - Because P is nontrivial.

Rice's Theorem

- **Rice's Theorem:** Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.

- **Proof:**

- Show $\text{Acc}_{\text{TM}} \leq_m M_P$.
- Suppose $\emptyset \notin P$.
- Need:



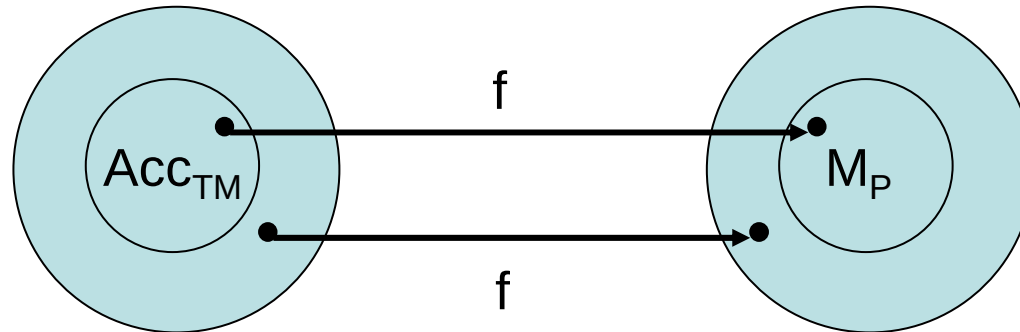
- Let M_1 be any TM such that $L(M_1) \in P$, so $\langle M_1 \rangle \in M_P$.
- Let M_2 be any TM such that $L(M_2) = \emptyset$, so $\langle M_2 \rangle \notin M_P$.

Rice's Theorem

- **Rice's Theorem:** Let P be a nontrivial property. Then $M_P = \{ \langle M \rangle \mid L(M) \in P \}$ is undecidable.

- **Proof:**

- Need:

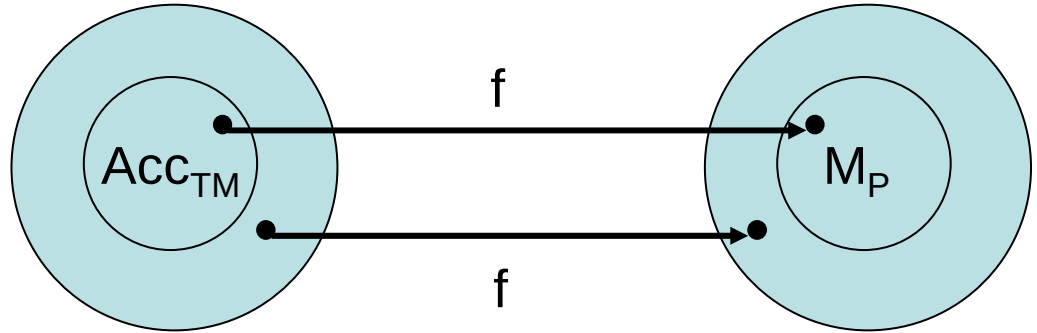


- Let M_1 be any TM such that $L(M_1) \in P$, so $\langle M_1 \rangle \in M_P$.
 - Let M_2 be any TM such that $L(M_2) = \emptyset$, so $\langle M_2 \rangle \notin M_P$.
 - Define $f(x)$:
 - If x isn't of the form $\langle M, w \rangle$, return something $\notin M_P$, like $\langle M_2 \rangle$.
 - If $x = \langle M, w \rangle$, then $f(x) = \langle M'_{M,w} \rangle$, where:
 - $M'_{M,w}$: On input y :
 - ...

Rice's Theorem

- **Proof:**

- Show $\text{Acc}_{\text{TM}} \leq_m M_P$.



- $L(M_1) \in P$, so $\langle M_1 \rangle \in M_P$.

- $L(M_2) = \emptyset$, so $\langle M_2 \rangle \notin M_P$.

- Define $f(x)$:

- If $x = \langle M, w \rangle$, then $f(x) = \langle M'_{M,w} \rangle$, where:

- $M'_{M,w}$: On input y :

- Run M on w .

- If M accepts w then run M_1 on y , accept if M_1 accepts y .

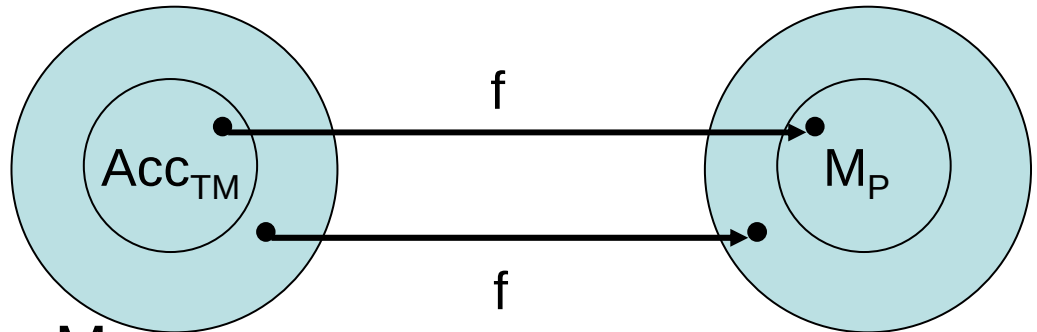
- (If M doesn't accept w or M_1 doesn't accept y , loop forever.)

- Tricky...

Rice's Theorem

- **Proof:**

- Show $\text{Acc}_{\text{TM}} \leq_m M_P$.



- $L(M_1) \in P$, so $\langle M_1 \rangle \in M_P$.

- $L(M_2) = \emptyset$, so $\langle M_2 \rangle \notin M_P$.

- If $x = \langle M, w \rangle$, then $f(x) = \langle M'_{M,w} \rangle$, where:

- $M'_{M,w}$: On input y :

- Run M on w .

- If M accepts w then run M_1 on y and accept if M_1 accepts y .

- Claim $x \in \text{Acc}_{\text{TM}}$ if and only if $f(x) \in M_P$.

- If $x = \langle M, w \rangle \in \text{Acc}_{\text{TM}}$ then $L(M'_{M,w}) = L(M_1) \in P$, so $f(x) \in M_P$.

- If $x = \langle M, w \rangle \notin \text{Acc}_{\text{TM}}$ then $L(M'_{M,w}) = \emptyset \notin P$, so $f(x) \notin M_P$.

- Therefore, $\text{Acc}_{\text{TM}} \leq_m M_P$ using f .

- So M_P is undecidable, by Corollary 2.

Rice's Theorem

- We have proved:
- **Rice's Theorem:** Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- Note:
 - Rice proves **undecidability**, doesn't prove **non-Turing-recognizability**.
 - The sets M_P may be Turing-recognizable.
- **Example: $P =$ languages that contain 01**
 - Then $M_P = \{ \langle M \rangle \mid 01 \in L(M) \} = \text{Acc01}_{\text{TM}}$.
 - Rice implies that M_P is undecidable.
 - But we already know that $M_P = \text{Acc01}_{\text{TM}}$ is Turing-recognizable.
 - For a given input $\langle M \rangle$, a TM/program can simulate M on 01 and accept iff this simulation accepts.

More Applications of Rice's Theorem

Applications of Rice's Theorem

- **Example 1: Using Rice**
 - $\{ \langle M \rangle \mid M \text{ is a TM that accepts at least 37 different strings} \}$
 - Rice implies that this is undecidable.
 - This set = M_P , where P = “the language contains at least 37 different strings”
 - P is a language property.
 - Nontrivial, since some TM-recognizable languages satisfy it and some don't.

Applications of Rice's Theorem

- **Example 2:** Property that isn't a language property and is decidable
 - $\{ \langle M \rangle \mid M \text{ is a TM that has at least 37 states} \}$
 - Not a language property, but a property of a machine's structure.
 - So Rice doesn't apply.
 - Obviously decidable, since we can determine the number of states given the TM description.

Applications of Rice's Theorem

- **Example 3:** Another property that isn't a language property and is decidable
 - $\{ \langle M \rangle \mid M \text{ is a TM that runs for at most 37 steps on input } 01 \}$
 - Not a language property, not a property of a machine's structure.
 - Rice doesn't apply.
 - Obviously decidable, since, given the TM description, we can just simulate it for 37 steps.

Applications of Rice's Theorem

- **Example 4:** Undecidable property for which Rice's Theorem doesn't work to prove undecidability
 - $\text{Acc01SQ} = \{ \langle M \rangle \mid M \text{ is a TM that accepts the string } 01 \text{ in exactly a perfect square number of steps} \}$
 - Not a language property, Rice doesn't apply.
 - Can prove undecidable by showing $\text{Acc01}_{\text{TM}} \leq_m \text{Acc01SQ}$.
 - Acc01_{TM} is the set of TMs that accept 01 in **any number** of steps.
 - $\text{Acc01SQ}_{\text{TM}}$ is the set of TMs that accept 01 in **a perfect square number** of steps.
 - Design mapping f so that M accepts 01 iff $f(M) = \langle M' \rangle$ where M' accepts 01 in a perfect square number of steps.
 - $f(\langle M \rangle) = \langle M' \rangle$ where...

Applications of Rice's Theorem

- **Example 4:** Undecidable property for which Rice doesn't work to prove undecidability
 - $\text{Acc01SQ} = \{ \langle M \rangle \mid M \text{ is a TM that accepts the string } 01 \text{ in exactly a perfect square number of steps} \}$
 - Show $\text{Acc01}_{\text{TM}} \leq_m \text{Acc01SQ}$.
 - Design f so M accepts 01 iff $f(M) = \langle M' \rangle$ where M' accepts 01 in a perfect square number of steps.
 - $f(\langle M \rangle) = \langle M' \rangle$ where:
 - M' : On input x :
 - If $x \neq 01$, then reject.
 - If $x = 01$, then simulate M on 01. If M accepts 01, then accept, but just after doing enough extra steps to ensure that the total number of steps is a perfect square.
 - $\langle M \rangle \in \text{Acc01}_{\text{TM}}$ iff M' accepts 01 in a perfect square number of steps, iff $f(\langle M \rangle) \in \text{Acc01SQ}$.
 - So $\text{Acc01}_{\text{TM}} \leq_m \text{Acc01SQ}$, so Acc01SQ is undecidable.

Applications of Rice's Theorem

- **Example 5:** Trivial language property
 - $\{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is recognized by some TM having an even number of states} \}$
 - This is a language property.
 - So it might seem that Rice should apply...
 - But, it's a **trivial** language property: Every Turing-recognizable language is recognized by some TM having an even number of states.
 - Could always add an extra, unreachable state.
 - Decidable or undecidable?
 - Decidable (of course), since it's the set of all TMs.

Applications of Rice's Theorem

- Example 6:
 - $\{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is recognized by some TM having at most 37 states and at most 37 tape symbols} \}$
 - A language property.
 - Is it nontrivial?
 - Yes, some languages satisfy it and some don't.
 - So Rice applies, showing that it's undecidable.
 - Note: This isn't $\{ \langle M \rangle \mid M \text{ is a TM that has at most 37 states and at most 37 tape symbols} \}$
 - That's decidable.
 - What about $\{ \langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is recognized by some TM having at least 37 states and at least 37 tape symbols} \}$?
 - Trivial---all Turing-recognizable languages are recognized by some such machine.

Next time...

- The Recursion Theorem
- Reading:
 - Sipser Section 6.1

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and
Complexity
Or, Great Ideas in Theoretical
Computer Science
Spring, 2010

Class 10
Nancy Lynch

Today

- Final topic in computability theory: **Self-Reference and the Recursion Theorem**
- Consider adding to TMs (or programs) a new, powerful capability to “know” and use their own descriptions.
- The Recursion Theorem says that this apparent extra power does not add anything to the basic computability model: these self-referencing machines can be transformed into ordinary non-self-referencing TMs.

Today

- Self-Reference and the Recursion Theorem
- Topics:
 - Self-referencing machines and programs
 - Statement of the Recursion Theorem
 - Applications of the Recursion Theorem
 - Proof of the Recursion Theorem: Special case
 - Proof of the Recursion Theorem: General case
- Reading:
 - Sipser, Section 6.1

Self-referencing machines and programs

Self-referencing machines/programs

- Consider the following program P_1 .
- P_1 :
 - Obtain $\langle P_1 \rangle$
 - Output $\langle P_1 \rangle$
- P_1 simply outputs its own representation, as a string.
- Simplest example of a machine/program that uses its own description.

Self-referencing machines/programs

- A more interesting example:
- P_2 : On input w :
 - If $w = \varepsilon$ then output 0
 - Else
 - Obtain $\langle P_2 \rangle$
 - Run P_2 on $\text{tail}(w)$
 - If P_2 on $\text{tail}(w)$ outputs a number n then output $n+1$.
- What does P_2 compute?
- It computes $|w|$, the length of its input.
- Uses the recursive style common in LISP, Scheme, other recursive programming languages.
- We assume that, once we have the representation of a machine, we can simulate it on a given input.
- E.g., if P_2 gets $\langle P_2 \rangle$, it can simulate P_2 on any input.

Self-referencing machines/programs

- One more example:
- P_3 : On input w :
 - Obtain $\langle P_3 \rangle$
 - Run P_3 on w
 - If P_3 on w outputs a number n then output $n+1$.
- A valid self-referencing program.
- What does P_3 compute?
- Seems contradictory: if P_3 on w outputs n then P_3 on w outputs $n+1$.
- But according to the usual semantics of recursive calls, it never halts, so there's no contradiction.
- P_3 computes a partial function that isn't defined anywhere.

Statement of the Recursion Theorem

The Recursion Theorem

- Used to justify self-referential programs like P_1 , P_2 , P_3 , by asserting that they have corresponding (equivalent) basic TMs.

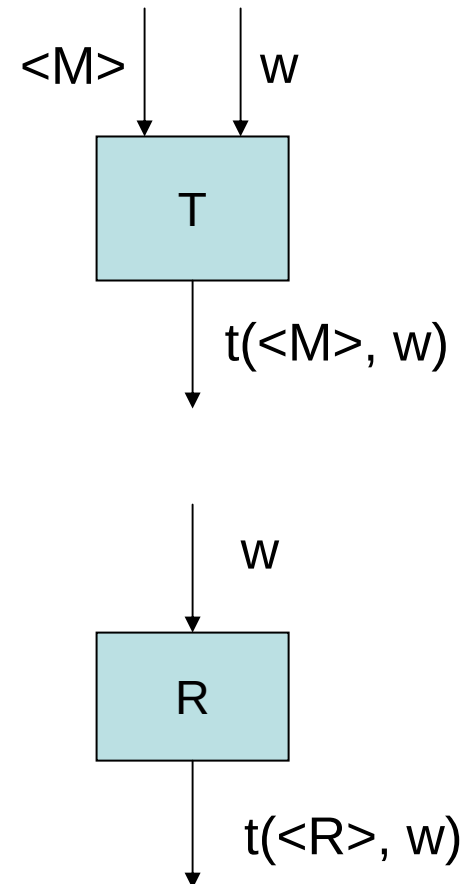
- **Recursion Theorem (Sipser Theorem 6.3):**

Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$.

Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.

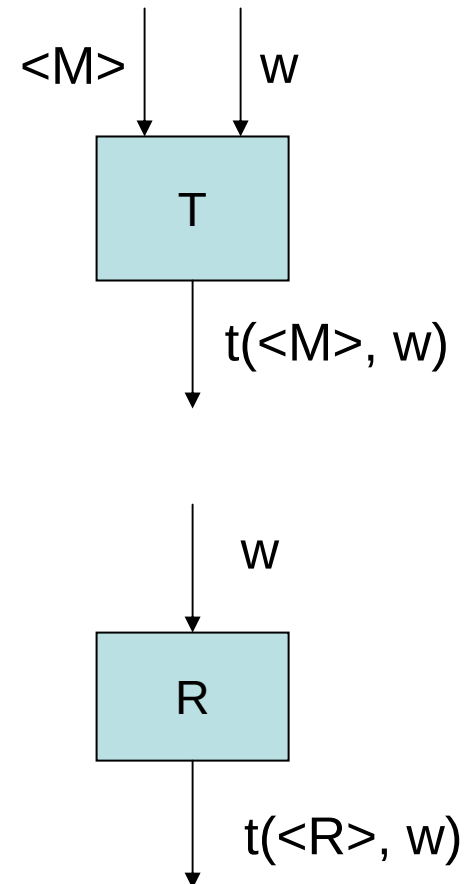
The Recursion Theorem

- **Recursion Theorem:** Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.
- Thus, T is a TM that takes 2 inputs.
- Think of the first as the description of some arbitrary 1-input TM M .
- Then R behaves like T , but with the first input set to $\langle R \rangle$, the description of R itself.
- Thus, R uses its own representation.



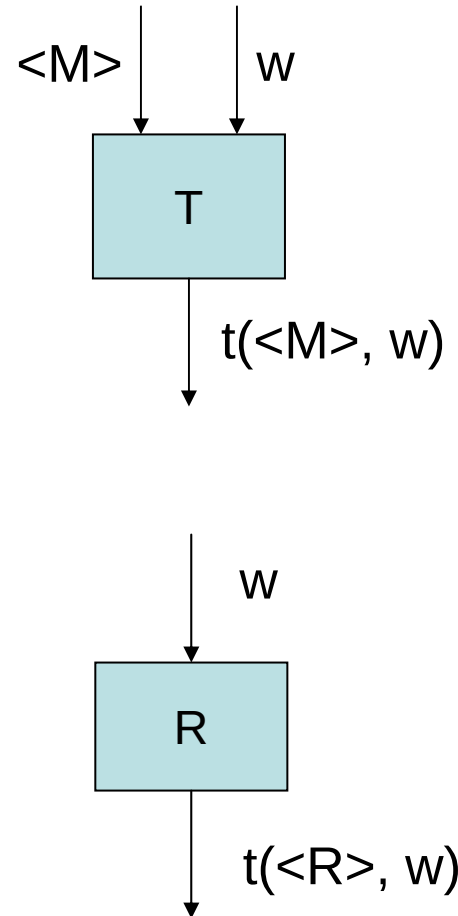
The Recursion Theorem

- **Recursion Theorem:** Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.
- **Example:** P_2 , revisited
 - Computes length of input.
 - What are T and R ?
 - Here is a version of P_2 with an extra input $\langle M \rangle$:
 - T_2 : On inputs $\langle M \rangle$ and w :
 - If $w = \varepsilon$ then output 0
 - Else run M on $\text{tail}(w)$; if it outputs n then output $n+1$.



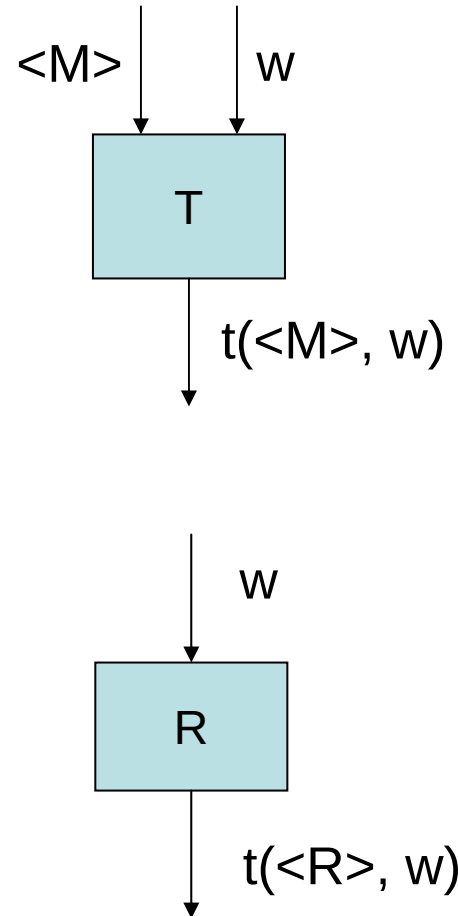
The Recursion Theorem

- Example: P_2 , revisited
 - T_2 : On inputs $\langle M \rangle$ and w :
 - If $w = \varepsilon$ then output 0
 - Else run M on $\text{tail}(w)$; if it outputs n then output $n+1$.
 - T_2 produces different results, depending on what M does.
 - E.g., if M always loops:
 - T_2 outputs 0 on input $w = \varepsilon$ and loops on every other input.
 - E.g., if M always halts and outputs 1:
 - T_2 outputs 0 on input $w = \varepsilon$ and outputs 2 on every other input.



The Recursion Theorem

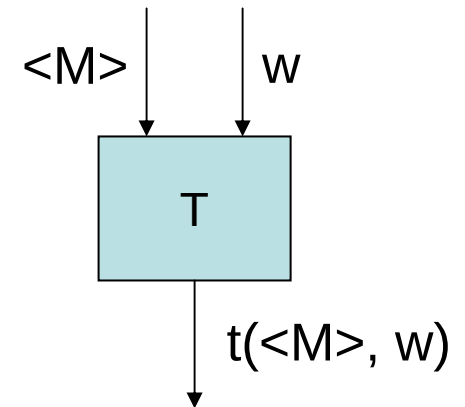
- Example: P_2 , revisited
 - T_2 : On inputs $\langle M \rangle$ and w :
 - If $w = \varepsilon$ then output 0
 - Else run M on $\text{tail}(w)$; if it outputs n then output $n+1$.
 - Recursion Theorem says there is a TM R computing $t(\langle R \rangle, w)$ ---just like T_2 but with input $\langle M \rangle$ set to $\langle R \rangle$ for the same R .
 - This R is just P_2 as defined earlier.



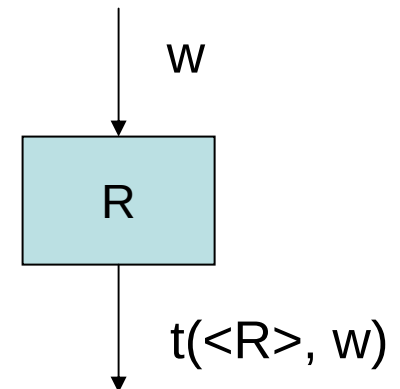
The Recursion Theorem

- Recursion Theorem (Sipser Theorem 6.3):

Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$.



Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.



Applications of the Recursion Theorem

Applications of Recursion Theorem

- The Recursion Theorem can be used to show various negative results, e.g., undecidability results.
- **Application 1: Acc_{TM} is undecidable**
 - We already know this, but the Recursion Theorem provides a new proof.
 - Suppose for contradiction that D is a TM that decides Acc_{TM} .
 - Construct another machine R using self-reference (justified by the Recursion Theorem):
- **R : On input w :**
 - Obtain $\langle R \rangle$ (using Recursion Theorem)
 - Run D on input $\langle R, w \rangle$ (we can construct $\langle R, w \rangle$ from $\langle R \rangle$ and w)
 - Do the opposite of what D does:
 - If D accepts $\langle R, w \rangle$ then reject.
 - If D rejects $\langle R, w \rangle$ then accept.

Application 1: Acc_{TM} is undecidable

- Suppose for contradiction that D decides Acc_{TM} .
- R : On input w :
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R, w \rangle$
 - Do the opposite of what D does:
 - If D accepts $\langle R, w \rangle$ then reject.
 - If D rejects $\langle R, w \rangle$ then accept.
- RT says that $\text{TM } R$ exists, assuming decider D exists.
- Formally, to apply RT , use the 2-input machine T :
- T : On inputs $\langle M \rangle$ and w :
 - Run D on input $\langle M, w \rangle$
 - Do the opposite of what D does:
 - If D accepts $\langle M, w \rangle$ then reject.
 - If D rejects $\langle M, w \rangle$ then accept.

Application 1: Acc_{TM} is undecidable

- Suppose for contradiction that D decides Acc_{TM} .
- R : On input w :
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R, w \rangle$
 - Do the opposite of what D does:
 - If D accepts $\langle R, w \rangle$ then reject.
 - If D rejects $\langle R, w \rangle$ then accept.
- Now get a contradiction:
 - If R accepts w , then
 - D accepts $\langle R, w \rangle$ since D is a decider for Acc_{TM} , so
 - R rejects w by definition of R .
 - If R does not accept w , then
 - D rejects $\langle R, w \rangle$ since D is a decider for Acc_{TM} , so
 - R accepts w by definition of R .
- Contradiction. So D can't exist, so Acc_{TM} is undecidable.

Applications of Recursion Theorem

- **Application 2: Acc01_{TM} is undecidable**
 - Similar to the previous example.
 - Suppose for contradiction that D is a TM that decides Acc01_{TM} .
 - Construct another machine R using the Recursion Theorem:
- **R : On input w : (ignores its input)**
 - Obtain $\langle R \rangle$ (using RT)
 - Run D on input $\langle R \rangle$
 - Do the opposite of what D does:
 - If D accepts $\langle R \rangle$ then reject.
 - If D rejects $\langle R \rangle$ then accept.
- RT says that R exists, assuming decider D exists.

Application 2: Acc01_{TM} is undecidable

- Suppose for contradiction that D decides Acc01_{TM} .
- R : On input w :
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R \rangle$
 - Do the opposite of what D does:
 - If D accepts $\langle R \rangle$ then reject.
 - If D rejects $\langle R \rangle$ then accept.
- Now get a contradiction, based on what R does on input 01 :
 - If R accepts 01 , then
 - D accepts $\langle R \rangle$ since D is a decider for Acc01_{TM} , so
 - R rejects 01 (and everything else), by definition of R .
 - If R does not accept 01 , then
 - D rejects $\langle R \rangle$ since D is a decider for Acc01_{TM} , so
 - R accepts 01 (and everything else), by definition of R .
- Contradiction. So D can't exist, so Acc01_{TM} is undecidable.

Applications of Recursion Theorem

- Application 3: Using Recursion Theorem to prove Rice's Theorem
 - Rice's Theorem: Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
 - Nontriviality: There is some M_1 with $L(M_1) \in P$, and some M_2 with $L(M_2) \notin P$.
 - Implies lots of things are undecidable.
 - We already proved this; now, a new proof using the Recursion Theorem.
 - Suppose for contradiction that D is a TM that decides M_P .
 - Construct machine R using the Recursion Theorem:...

Application 3: Using Recursion Theorem to prove Rice's Theorem

- Rice's Theorem: Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- Nontriviality: $L(M_1) \in P$, $L(M_2) \notin P$.
- D decides M_P .
- **R: On input w :**
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R \rangle$
 - If D accepts $\langle R \rangle$ then run M_2 on input w and do the same thing.
 - If D rejects $\langle R \rangle$ then run M_1 on input w and do the same thing.
- M_1 and M_2 are as above, in the nontriviality definition.
- R exists, by the Recursion Theorem.
- Get contradiction by considering whether or not $L(R) \in P$:

Application 3: Using Recursion Theorem to prove Rice's Theorem

- Rice's Theorem: Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- $L(M_1) \in P, L(M_2) \notin P$.
- D decides M_P .
- **R: On input w :**
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R \rangle$
 - If D accepts $\langle R \rangle$ then run M_2 on input w and do the same thing.
 - If D rejects $\langle R \rangle$ then run M_1 on input w and do the same thing.
- Get contradiction by considering whether or not $L(R) \in P$:
 - If $L(R) \in P$, then
 - D accepts $\langle R \rangle$, since D decides M_P , so
 - $L(R) = L(M_2)$ by definition of R , so
 - $L(R) \notin P$.

Application 3: Using Recursion Theorem to prove Rice's Theorem

- Rice's Theorem: Let P be a nontrivial property of Turing-recognizable languages. Let $M_P = \{ \langle M \rangle \mid L(M) \in P \}$. Then M_P is undecidable.
- $L(M_1) \in P, L(M_2) \notin P$.
- D decides M_P .
- **R: On input w :**
 - Obtain $\langle R \rangle$
 - Run D on input $\langle R \rangle$
 - If D accepts $\langle R \rangle$ then run M_2 on input w and do the same thing.
 - If D rejects $\langle R \rangle$ then run M_1 on input w and do the same thing.
- Get contradiction by considering whether or not $L(R) \in P$:
 - If $L(R) \notin P$, then
 - D rejects $\langle R \rangle$, since D decides M_P , so
 - $L(R) = L(M_1)$ by definition of R , so
 - $L(R) \in P$.
- Contradiction!

Applications of Recursion Theorem

- Application 4: Showing non-Turing-recognizability
 - Define $\text{MIN}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a “minimal” TM, that is, no TM with a shorter encoding recognizes the same language} \}$.
 - **Theorem:** MIN_{TM} is not Turing-recognizable.
 - Note: This doesn't follow from Rice:
 - Requires non-T-recognizability, not just undecidability.
 - Besides, it's not a language property.
 - **Proof:**
 - Assume for contradiction that MIN_{TM} is Turing-recognizable.
 - Then it's **enumerable**, say by enumerator TM E.
 - Define TM R, using the Recursion Theorem:
 - **R:** On input w: ...

Application 4: Non-Turing-recognizability

- $\text{MIN}_{\text{TM}} = \{ \langle M \rangle \mid M \text{ is a "minimal" TM} \}$.
- **Theorem:** MIN_{TM} is not Turing-recognizable.
- **Proof:**
 - Assume that MIN_{TM} is Turing-recognizable.
 - Then it's **enumerable**, say by enumerator TM E.
 - **R:** On input w :
 - Obtain $\langle R \rangle$.
 - Run E, producing list $\langle M_1 \rangle, \langle M_2 \rangle, \dots$ of all minimal TMs, until you find some $\langle M_i \rangle$ with $|\langle M_i \rangle|$ strictly greater than $|\langle R \rangle|$.
 - That is, until you find a TM with a rep bigger than yours.
 - Run $M_i(w)$ and do the same thing.
 - **Contradiction:**
 - $L(R) = L(M_i)$
 - $|\langle R \rangle|$ less than $|\langle M_i \rangle|$
 - Therefore, M_i is not minimal, and should not be in the list.

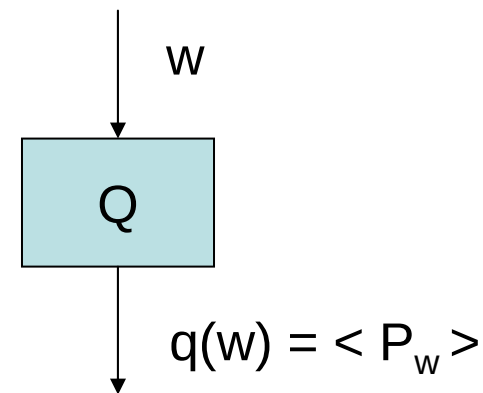
Proof of the Recursion Theorem: Special case

Proof of Recursion Theorem: Special Case

- Start with easier first step: Produce a TM corresponding to P_1 :
- P_1 :
 - Obtain $\langle P_1 \rangle$
 - Output $\langle P_1 \rangle$
- P_1 outputs its own description.

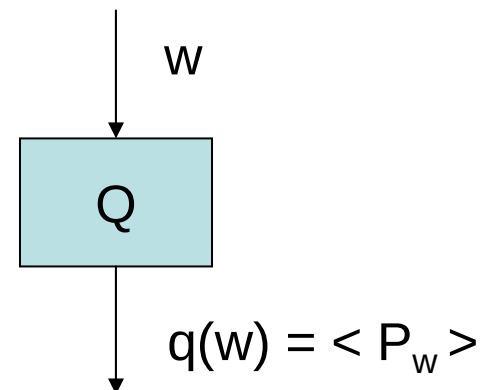
- **Lemma: (Sipser Lemma 6.1):** There is a computable function $q: \Sigma^* \rightarrow \Sigma^*$ such that, for any string w , $q(w)$ is the description of a TM P_w that just prints out w and halts.

- **Proof:** Straightforward construction.
Can hard-wire w in the FSC of P_w .

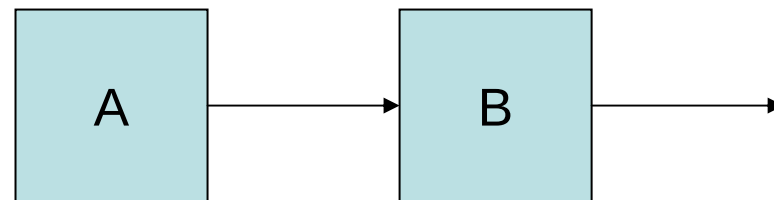


Proof of RT: Special Case

- **Lemma: (Sipser Lemma 6.1):** There is a computable function $q: \Sigma^* \rightarrow \Sigma^*$ such that, for any string w , $q(w)$ is the description of a TM P_w that just prints out w and halts.

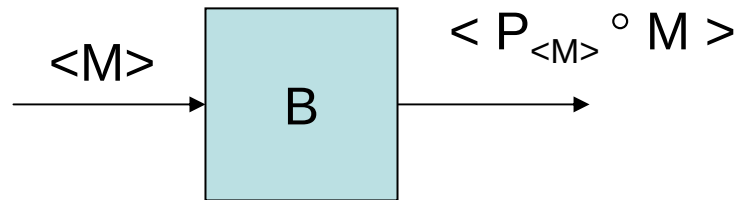


- Now, back to the machine that outputs its own description...
- Consists of 2 sub-machines, A and B.

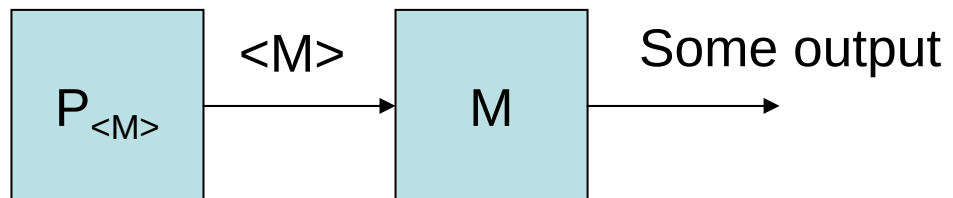


- Output of A feeds into B.
- Write as $A \circ B$.

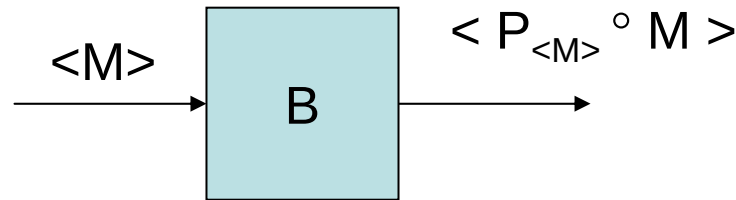
Construction of B



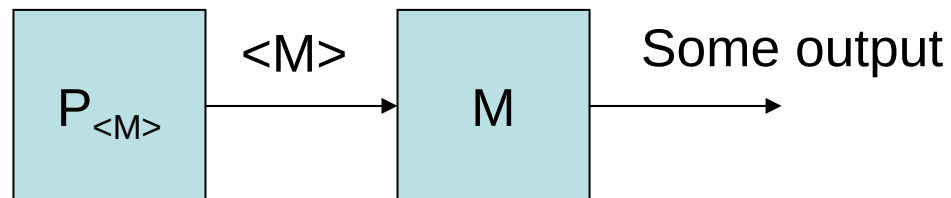
- B expects its input to be the representation $\langle M \rangle$ of a 1-input TM (a function-computing TM, not a language recognizer).
 - If not, we don't care what B does.
- B outputs the encoding of the combination of two machines, $P_{\langle M \rangle}$ and M .
- The first machine is $P_{\langle M \rangle}$, which simply outputs $\langle M \rangle$.
- The second is the input machine M .
- $P_{\langle M \rangle} \circ M$:



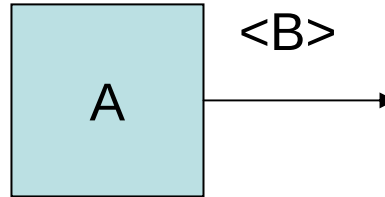
Construction of B



- How can B generate $\langle P_{\langle M \rangle} \circ M \rangle$?
 - B can generate a description of $P_{\langle M \rangle}$, that is, $\langle P_{\langle M \rangle} \rangle$, by Lemma 6.1.
 - B can generate a description of M, that is, $\langle M \rangle$, since it already has $\langle M \rangle$ as its input.
 - Once B has descriptions of $P_{\langle M \rangle}$ and M, it can combine them into a single description of the combined machine $P_{\langle M \rangle} \circ M$, that is, $\langle P_{\langle M \rangle} \circ M \rangle$.



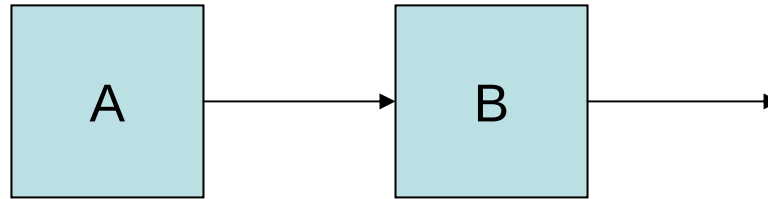
Construction of A



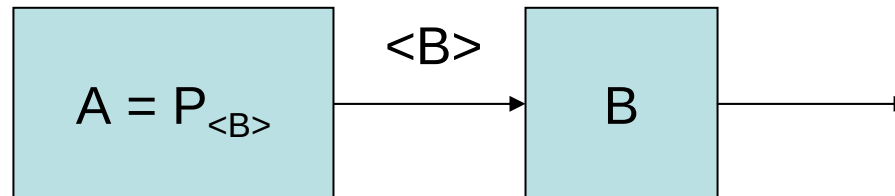
- A is $P_{\langle B \rangle}$, the machine that just outputs $\langle B \rangle$, where B is the complicated machine constructed above.
- A has no input, just outputs $\langle B \rangle$.

Combining the Pieces

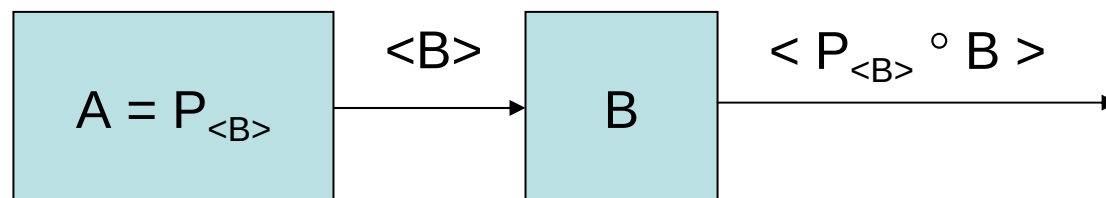
- $A \circ B$:



- Claim $A \circ B$ outputs its own description, which is $\langle A \circ B \rangle$.
- Check this...
- A is $P_{\langle B \rangle}$, so the output from A to B is $\langle B \rangle$:

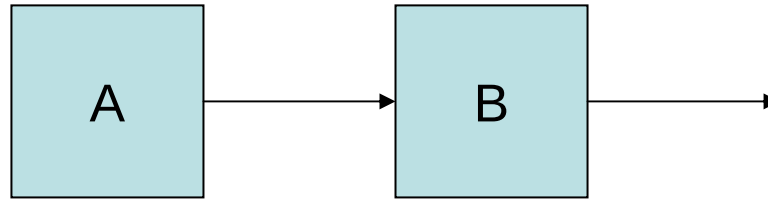


- Substituting B for M in B's output:

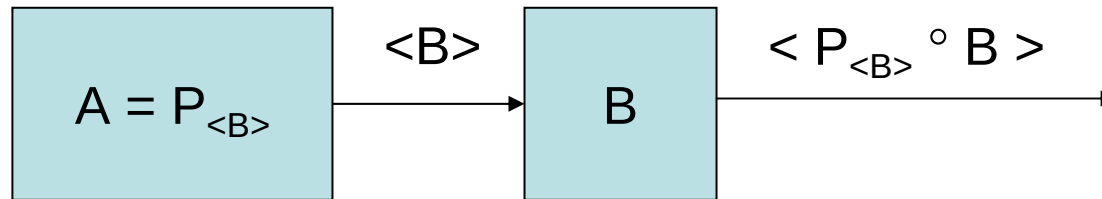


Combining the Pieces

- $A \circ B$:



- Claim $A \circ B$ outputs its own description, which is $\langle A \circ B \rangle$.

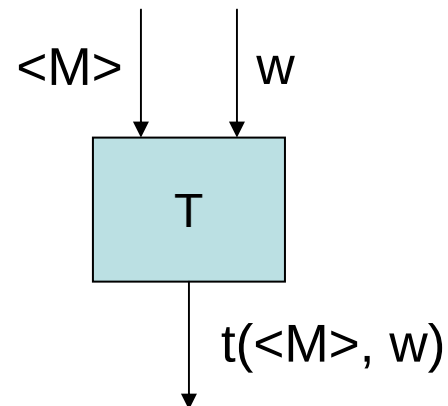


- The output of $A \circ B$ is, therefore, $\langle P_{\langle B \rangle} \circ B \rangle = \langle A \circ B \rangle$.
- As needed!
- $A \circ B$ outputs its own description, $\langle A \circ B \rangle$.

Proof of the Recursion Theorem: General case

Proof of the RT: General case

- So, we have a machine that **outputs its own description**.
- A curiosity---this is not the general RT.
- RT says not just that:
 - There is a TM that outputs its own description.
- But that:
 - There are TMs that can **use** their own descriptions, in “arbitrary ways”.
- The “arbitrary ways” are captured by the machine T in the RT statement.

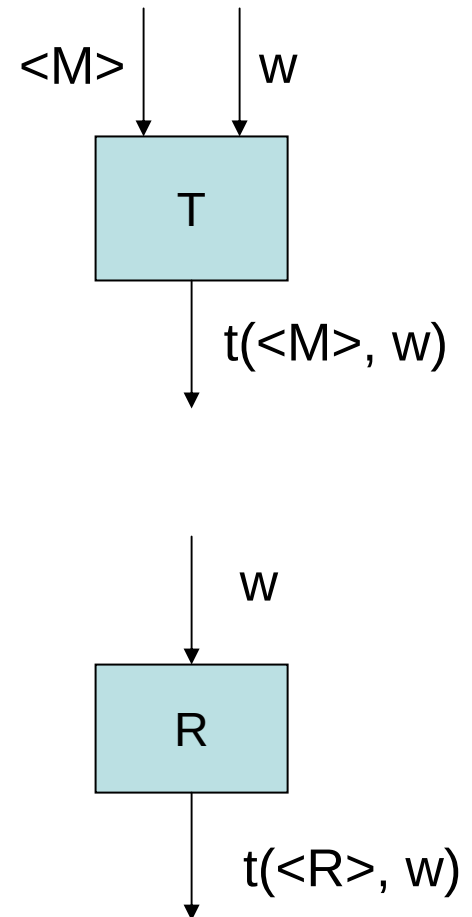


The Recursion Theorem

- Recursion Theorem:

Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$.

Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.



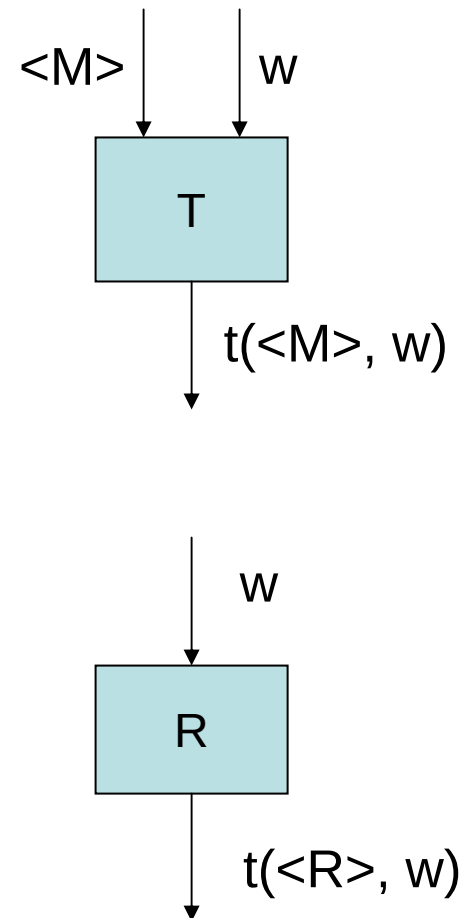
The Recursion Theorem

- **Recursion Theorem:**

Let T be a TM that computes a (possibly partial) 2-argument function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$.

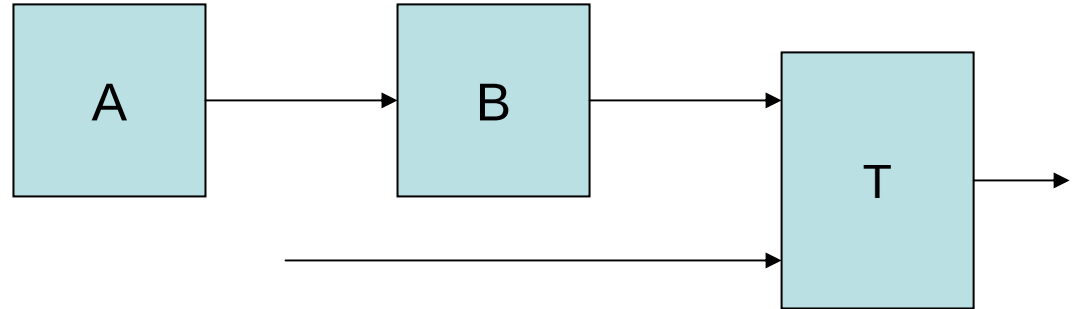
Then there is another TM R that computes the function $r: \Sigma^* \rightarrow \Sigma^*$, where for any w , $r(w) = t(\langle R \rangle, w)$.

- Construct R from:
 - The given T , and
 - Variants of A and B from the special-case proof.



Proof of RT: General Case

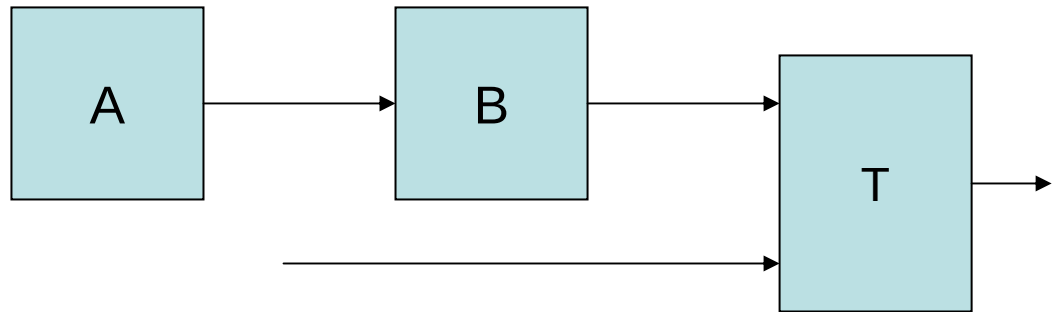
- R looks like:



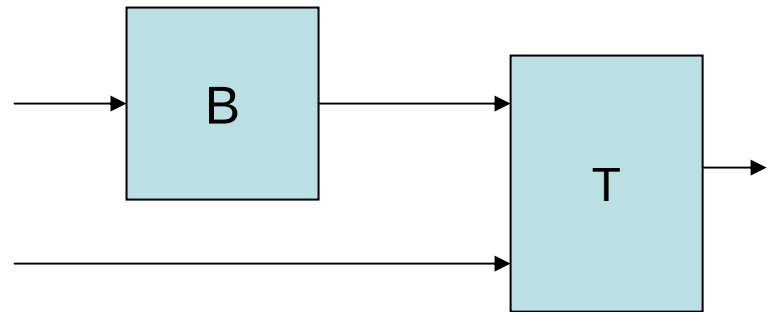
- Write this as $(A \circ B)^{\circ 1} T$
 - The $\circ 1$ means that the output from $(A \circ B)$ connects to the first (top) input line of T.

Proof of RT: General Case

- $R = (A \circ B)^{\circ 1} T$

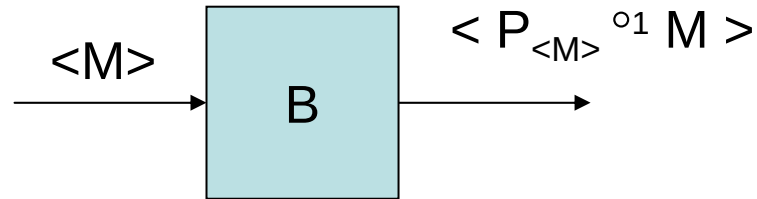


- New A: $P_{\langle B^{\circ 1} T \rangle}$, where $B^{\circ 1} T$ means:

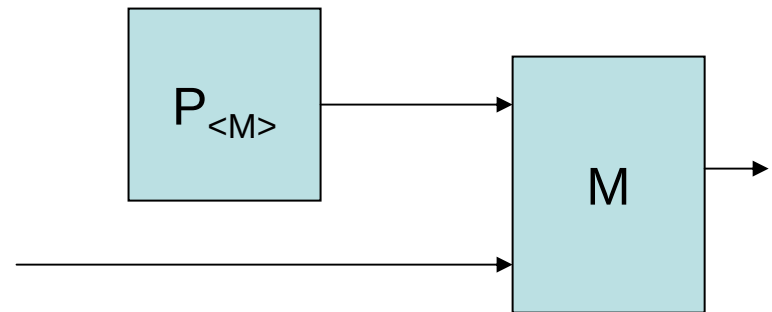


Proof of RT: General Case

- New B:

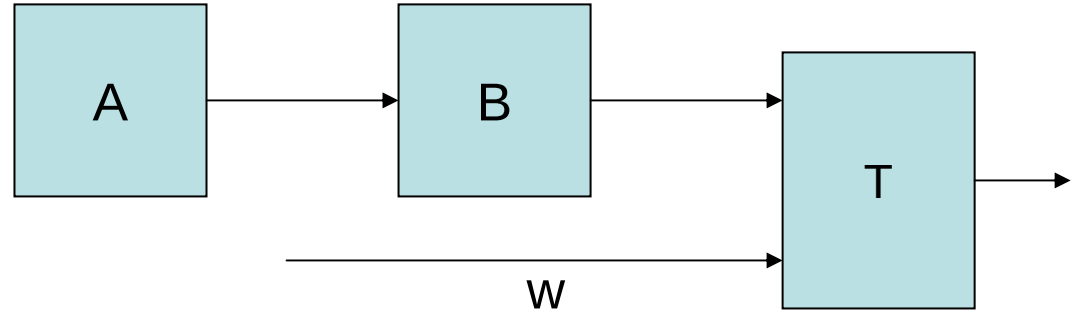


- Like B in the special case, but now M is a 2-input TM.
- $P_{\langle M \rangle} \circ^1 M$: 1-input TM, which uses output of $P_{\langle M \rangle}$ as first input of M .

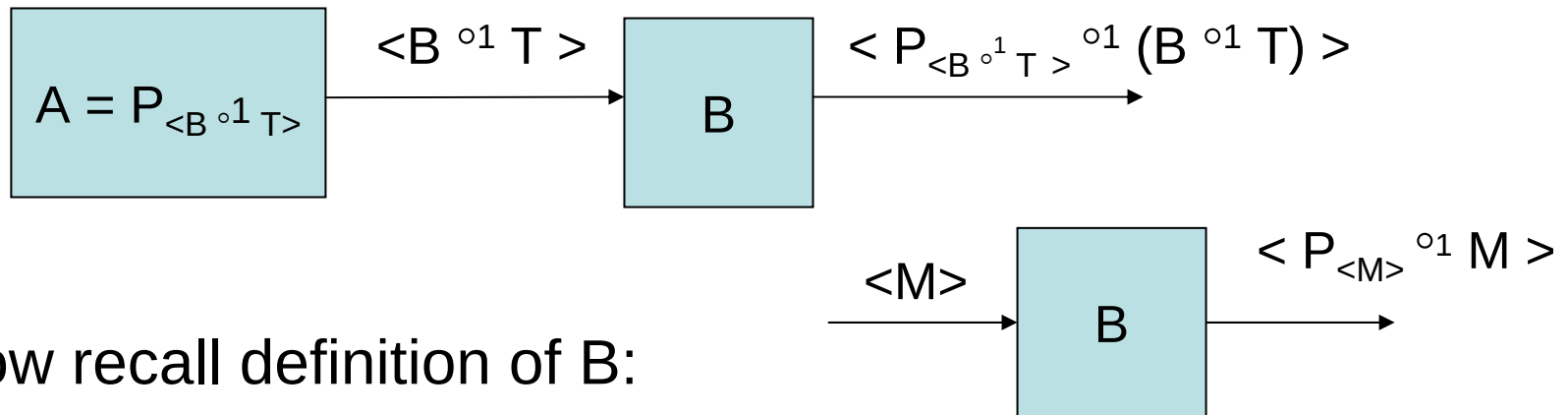


Combining the Pieces

- $R = (A \circ B)^{\circ 1} T$

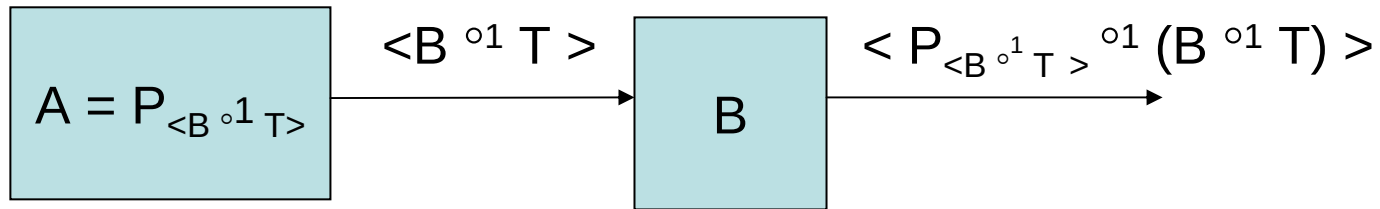


- Claim R outputs $t(\langle R \rangle, w)$:
- A is $P_{\langle B^{\circ 1} T \rangle}$, so the output from A to B is $\langle B^{\circ 1} T \rangle$:

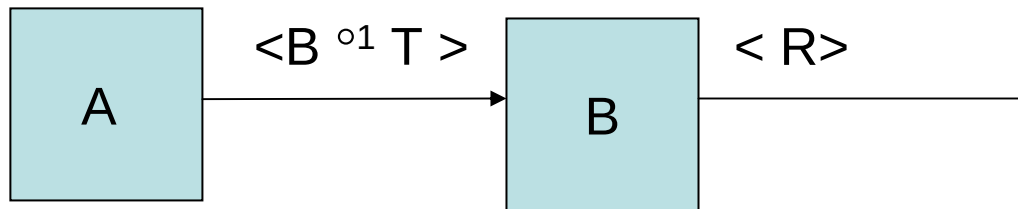


- Now recall definition of B :
- Plug in $B^{\circ 1} T$ for M in B 's input, and obtain output for B .

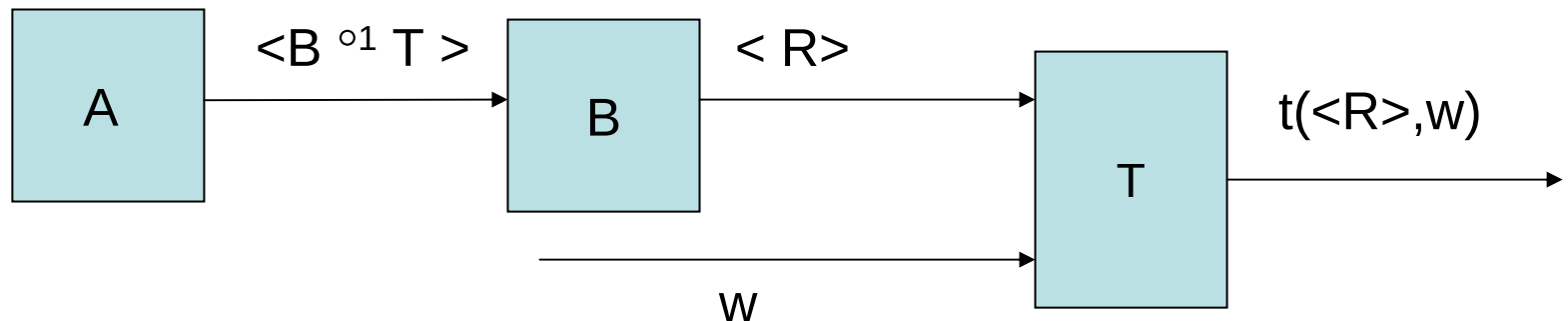
Combining the Pieces



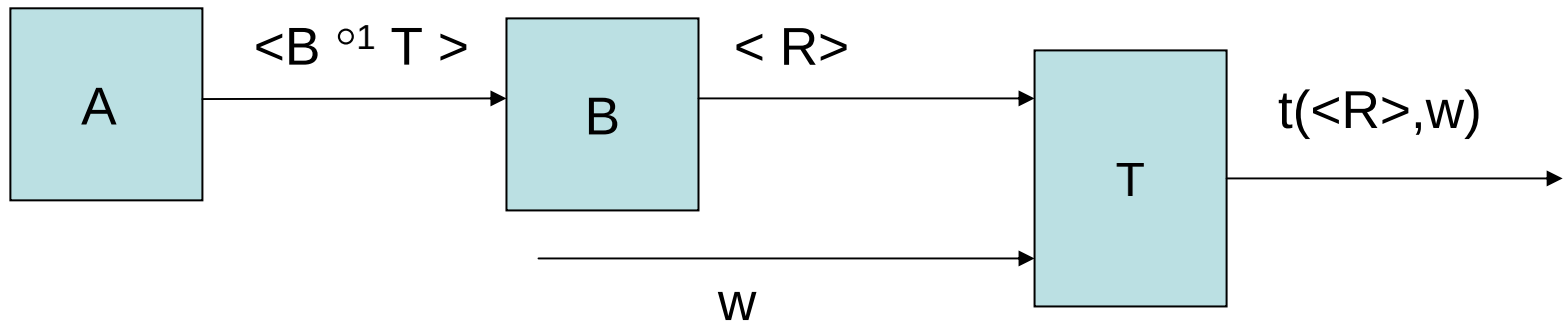
- B's output = $\langle A \circ 1_T \rangle (B \circ 1_T) \rangle = \langle R \rangle$:



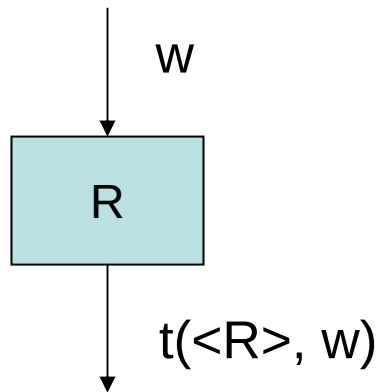
- Now combine with T , plugging in R for M in T 's input:



Combining the Pieces



- Thus, $R = (A \circ B) \circ^1 T$, on input w , produces $t(\langle R \rangle, w)$, as needed for the Recursion Theorem.



Next time...

- More on computability theory
- **Reading:**
 - "Computing Machinery and Intelligence" by Alan Turing:

<http://www.loebner.net/Prizef/TuringArticle.html>

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 15

Lecturer: Scott Aaronson

Scribe: Tiffany Wang

1 Administrivia

Midterms have been graded and the class average was 67. Grades will be normalized so that the average roughly corresponds to a B. The solutions will be posted on the website soon.

Pset4 will be handed out on Thursday.

2 Recap

2.1 Probabilistic Computation

We previously examined probabilistic computation methods and the different probabilistic complexity classes, as seen in Figure 1.

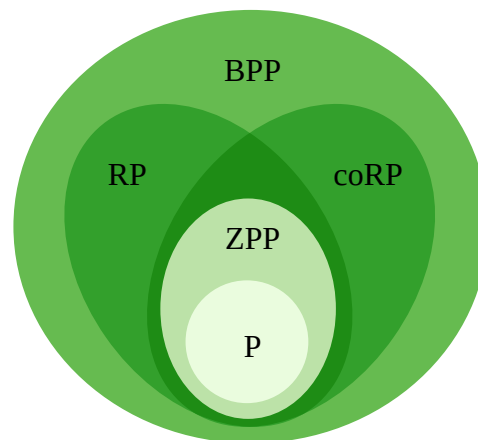


Figure by MIT OpenCourseWare.

Figure 1. Probabilistic Complexity Classes

P: Polynomial time - Problems that can be solved deterministically in polynomial time.

ZPP: Zero-error Probabilistic Polynomial (Expected Polynomial) time - Problems that can be solved efficiently but with 50% chance that the algorithm does not produce an answer and must be run again. If the algorithm *does* produce an answer it is guaranteed to be correct.

RP: Randomized Polynomial time - Problems for which if the answer is *NO*, the algorithm always outputs *NO*. Otherwise, if the answer is *YES*, the algorithm outputs *YES* at least 50% of the time. Hence there is an asymmetry between *YES* and *NO* outputs.

coRP: Complement of *RP* - These are problems for which there's a polynomial-time algorithm that always outputs *YES* if the answer is *YES* and outputs *NO* at least 50% of the time if the answer

is *NO*.

BPP: Bounded-error Probabilistic Polynomial time - Problems where if the answer is *YES*, the algorithm accepts with probability $\geq \frac{2}{3}$, and if the answer is *NO*, the algorithm accepts with probability $\leq \frac{1}{3}$.

2.2 Amplification and Chernoff Bound

The question that arises is whether the boundary values $\frac{1}{3}$ and $\frac{2}{3}$ have any particular significance. One of the nice things about using a probabilistic algorithm is that as long as there is a noticeable gap between the probability of accepting if the answer is *YES* and the probability of accepting if the answer is *NO*, that gap can be amplified by repeatedly running the algorithm.

For example, if you have an algorithm that outputs a wrong answer with $\Pr \leq \frac{1}{3}$, then you can repeat the algorithm hundreds of times and just take the majority answer. The probability of obtaining a wrong answer becomes astronomically small (there's a much greater chance of an asteroid destroying your computer).

This notion of amplification can be proven using a tool known as the *Chernoff Bound*. The Chernoff Bound states that given a set of independent events, the number of events that will happen is heavily concentrated about the expected value of the number of occurring events.

So given an algorithm that outputs a wrong answer with $\Pr = \frac{1}{3}$, repeating the algorithm 10,000 times would produce an expected number of 3333.3... wrong answers. The number of wrong answers will not be exactly the expected value, but the probability of getting a number far from the expected value (say 5,000) is very small.

2.3 P vs. BPP

There exists a fundamental question as to whether every probabilistic algorithm can be replaced by a deterministic one, or *derandomized*. In terms of complexity, the question is whether $P=BPP$, which is almost as deep a question as $P=NP$.

There is currently a very strong belief that derandomization is possible in general, but no one yet knows how to prove it.

3 Derandomization

Although derandomization has yet to be proven in the general case, it *has* been proven for some spectacular special cases: cases where for decades, the only known efficient solutions came from randomized algorithms. Indeed, this has been one of the big success stories in theoretical computer science in the last 10 years.

3.1 AKS Primality Test

In 2002, Agrawal, Kayal, and Saxena of the Indian Institute of Technology Kanpur developed a deterministic polynomial-time algorithm for testing whether an integer is prime or composite, also known as the AKS primality test.

For several decades prior, there existed good algorithms to test primality, but all were probabilistic. The problem was first known to be in the class RP, and then later shown to be in the class ZPP. It was also shown that the problem was in the class P, but only assuming that the Generalized Riemann Hypothesis was true. The problem was also known to be solvable deterministically in $n^{O(\log \log \log n)}$ time (which is *slightly* more than polynomial). Ultimately, it was nice to have the final answer and the discovery was an exciting thing to be alive for in the world of theoretical computer science.

The basic idea behind AKS is related to Pascal's Triangle. As seen in Figure 2, in every prime-numbered row, the numbers in Pascal's Triangle are all a multiple of the row number. On the other hand, in every composite-numbered row, the numbers are *not* all multiples of the row number.

0:	1
1:	1 1
2:	1 2 1
3:	1 3 3 1
4:	1 4 6 4 1
5:	1 5 10 10 5 1
6:	1 6 15 20 15 5 1
7:	1 7 21 35 35 21 7 1
8:	1 8 28 56 70 56 28 8 1

Figure 2. Pascal's Triangle and Prime Numbers

So to test the primality of an integer N , can we just check whether or not all the numbers in the N^{th} row of Pascal's Triangle are multiples of N ? The problem is that there are exponentially many numbers to check, and checking all of them would be no more efficient than trial division.

Looking at the expression $(x+a)^N$, which has coefficients determined by the N^{th} row of Pascal's Triangle, AKS noticed that the relationship $(x+a)^N = x^N + a^N \pmod N$ holds if and only if N is prime. This is because if N is prime, then all the “middle” coefficients will be divisible by N (and therefore disappear when we reduce mod N), while if N is composite then some middle coefficients will not be divisible by N . What this means is that the primality testing problem can be mapped to an instance of the polynomial identity testing problem: given two algebraic formulas, decide whether or not they represent the same polynomial.

In order to determine whether $(x+a)^N = x^N + a^N \pmod N$, one approach would be to plug in many random values of a and see if we get the same result each time. However, since the number of terms would still be exponential, we need to evaluate the expression not only mod N , but also mod a random polynomial:

$$(x+a)^N = x^N + a^N \pmod N, x^r - 1.$$

It turns out that this solution method works; on the other hand, it still depends on the use of randomness (the thing we're trying to eliminate).

The tour de force of the AKS paper was to show that if N is composite, then it is only necessary to try some small number of deterministically-chosen values of a and r until a pair is found such that the equation is not satisfied. This immediately leads to a method for distinguishing prime numbers from composite ones in deterministic polynomial time.

3.2 Trapped in a Maze

Problem: Given a maze (an undirected graph) with a given start vertex and end vertex, is the end vertex reachable or not?

Proposed solution from the floor: Depth-first search.

In a maze, this is the equivalent of wandering around the maze and leaving bread crumbs to mark paths that have already been explored. This solution runs in polynomial time, but the problem is that it requires breadcrumbs, or translated into complexity terms, a large amount of memory. The hope would be to solve the undirected connectivity problem in LOGSPACE: that is, to find a way out of the maze while remembering only $O(\log n)$ bits at any one time. (Why $O(\log n)$? That's the amount of information needed even to write down where you are; thus, it's essentially the best you can hope for.)

Proposed solution from the floor: Follow the right wall.

The trouble is that, if you were always following the right wall, it would be simple to create a maze that placed you in an infinite loop.

Simple-minded solution: Just wander around the maze randomly.

Now we're talking! Admittedly, in a *directed* graph it could take an exponential time for a random walk to reach the end vertex. For example, at each intersection of the graph shown in Figure 3, you advance forward with $\text{Pr}=\frac{1}{2}$ and return to the starting point with $\text{Pr}=\frac{1}{2}$. The chance that you make n consecutive correct choices to advance all the way to the end is exponentially small, so it will take exponential time to reach the end vertex.

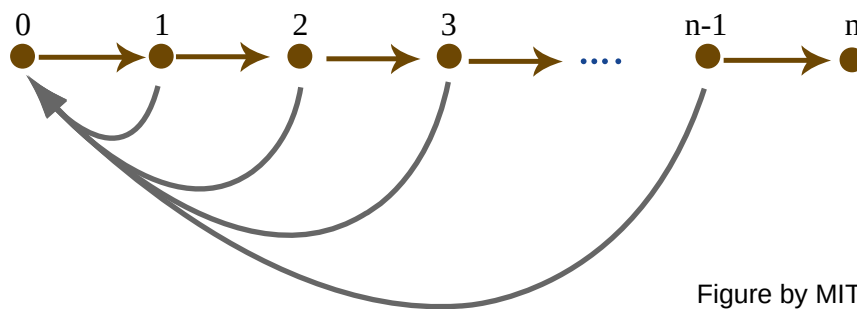


Figure by MIT OpenCourseWare.

Figure 3. Exponential Time Directed Graph

In 1979, Aleliunas et al. showed that randomly wandering through an undirected graph will get you out with high probability after $O(n^3)$ steps. After $O(n^3)$ steps, with high probability you will

have visited every vertex, regardless of the structure of the graph.

However, this still leaves the question of whether there's a *deterministic* algorithm to get out of a maze using only $O(\log n)$ bits of memory. Finally, in 2005, Omer Reingold proved that by making *pseudorandom* path selections based on a somewhat complicated set of rules, the maze problem can be solved deterministically in LOGSPACE. At each step, the rule is a function of the outcome of the previous application of the rules.

4 New Unit: Cryptography

4.1 History

Cryptography is a 3,000-year old black art that has been completely transformed over the last few decades by ideas from theoretical computer science. Cryptography is perhaps the best example of a field in which the concepts of theoretical computer science have real practical applications: problems are designed to be hard, the worst case assumptions are the right assumptions, and computationally intractable problems are there because we put them there.

For more on the history of cryptography, a great reference is David Kahn's *The Codebreakers*, which was written before people even knew about the biggest cryptographic story of all: the breaking of the German naval code in World War II, by Alan Turing and others.

4.2 Caesar Cipher

One of the oldest cryptographic codes used in history is the "Caesar cipher." In this cryptosystem, a plaintext message is converted into ciphertext by simply adding 3 to each letter of the message, wrapping around to A after you reach Z. Thus A becomes D, Z becomes C, and DEMOCRITUS becomes GHPRFULWXV.

Clearly this encryption system can easily be broken by anyone who can subtract mod 26. As an amusing side note, just a couple years ago, the head of the Sicilian mafia was finally caught after 40 years because he was using the Caesar cipher to send messages to his subordinates.

4.3 Substitution Cipher

A slightly more complicated cryptographic encoding is to scramble the letters of a message according to a random rule which permutes all the letters of the alphabet. For example, substituting every A with an X and every S with a V.

This encoding can also be easily broken by performing a frequency analysis of the letters appearing in the ciphertext.

4.4 One-Time Pad

It was not until the 1920's that a "serious" cryptosystem was devised. Gilbert Sandford Vernam, an American businessman, proposed what is known today as the one-time pad.

Under the one-time pad cryptosystem, the plaintext message is represented by a binary string M which is then XOR-ed with a random binary key, K , of the same length. As seen in Figure 4,

the ciphertext C is equal to the bitwise sum of M and K , mod 2.

$$\begin{array}{r}
 M : 111010110001 \\
 \oplus K : 011011110101 \\
 \hline
 C : 100001011010
 \end{array}$$

Figure 4. One-time Pad Encryption

Assuming that the recipient is a trusted party who shares the knowledge of the key, the ciphertext can be decrypted by performing another XOR operation: $C \oplus K = M \oplus K \oplus K = M$. See Figure 5.

$$\begin{array}{r}
 C : 100001011010 \\
 \oplus K : 011011110101 \\
 \hline
 M : 111010110001
 \end{array}$$

Figure 5. One-Time Pad Decryption

To an eavesdropper who does not have knowledge of the key, the ciphertext appears to be nonsense since XOR-ing any string of bits with a random string just produces another random string. There is no way to guess what the ciphertext may be encoding because any binary key could have been used.

As a result of this, the one-time pad is a provably unbreakable cryptographic encoding, but only if used correctly. The problem with using the one-time pad is that it literally is a “one-time” encryption. If the same key is ever used to encrypt more than one message, then the cryptosystem is no longer secure. For example, if we sent another message M_2 encrypted with the same key K to produce C_2 , the eavesdropper could obtain a combination of the messages: $C_1 \oplus C_2 = M_1 \oplus K \oplus M_2 \oplus K = M_1 \oplus M_2$. If the eavesdropper had any idea of what either of the messages may have contained, the eavesdropper could learn about the other plaintext message, and indeed obtain the key K .

As a concrete example, Soviet spies during the Cold War used the one-time pad to encrypt their messages and occasionally slipped up and re-used keys. As a result, the NSA, through its VENONA project, was able to decipher some of the ciphertext and even gather enough information to catch Julius and Ethel Rosenberg.

4.5 Shannon’s Theorem

As we saw, the one-time pad has the severe shortcoming that the number of messages that can be encrypted is limited by the amount of key available.

Is it possible to have a cryptographic code which is unbreakable (in the same absolute sense that the one-time pad is unbreakable), yet uses a key that is much smaller than the message?

In the 1940s, Claude Shannon proved that a perfectly secure cryptographic code requires the encryption key to be at least as long as the message that is sent.

Proof: Given an encryption function: $e_k : \{0, 1\}_{plaintext}^n \rightarrow \{0, 1\}_{ciphertext}^m$.

For all keys k , e_k must be an injective function (provide a one-to-one mapping between plaintext and ciphertext). Every plaintext must map to a different ciphertext, otherwise there would be no way of decrypting the message.

This immediately implies that for a given ciphertext, C , that was encrypted with a key of r bits, the number of possible plaintexts that could have produced C is at most 2^r (the number of possible keys). If $r < n$, then the number of possible plaintexts that could have generated C is smaller than the total number of possible plaintext messages. So if the adversary had unlimited computational power, the adversary could try all possible values of the key and rule out all plaintexts that could not have encrypted to C . The adversary would thus have learned something about the plaintext, making the cryptosystem insecure. Therefore the encryption key must be at least as long as the message for a perfectly secure cryptosystem.

The key loophole in Shannon's argument is the assumption that the adversary has unlimited computational power. For a practical cryptosystem, we can exploit computational complexity theory, and in particular the assumption that the adversary is a polynomial-time Turing machine that does not have unlimited computational power. More on this next time...

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and Complexity Or, GITCS

Class 12
Nancy Lynch

Today: Complexity Theory

- **First part of the course: Basic models of computation**
 - Circuits, decision trees
 - DFAs, NFAs:
 - Restricted notion of computation: no auxiliary memory, just one pass over input.
 - Yields restricted class of languages: regular languages.
- **Second part: Computability**
 - Very general notion of computation.
 - Machine models like Turing machines, or programs in general (idealized) programming languages.
 - Unlimited storage, multiple passes over input, compute arbitrarily long, possibly never halt.
 - Yields large language classes: Turing-recognizable = enumerable, and Turing-decidable.
- **Third part: Complexity theory**

Complexity Theory

- First part of the course: Basic models of computation
- Second part: Computability
- **Third part: Complexity theory**
 - A middle ground.
 - Restrict the general TM model by limiting its use of resources:
 - Computing time (number of steps).
 - Space = storage (number of tape squares used).
 - Leads to interesting subclasses of the Turing-decidable languages, based on specific bounds on amounts of resources used.
 - Compare:
 - Computability theory answers the question “What languages are computable (at all)?”
 - Complexity theory answers “What languages are computable with particular restrictions on amount of resources?”

Complexity Theory

- Topics

- Examples of time complexity analysis (informal).
- Asymptotic function notation: O , o , Ω , Θ
- Time complexity classes
- P , polynomial time
- Languages not in P
- Hierarchy theorems

- Reading:

- Sipser, Sections 7.1, 7.2, and a bit from 9.1.

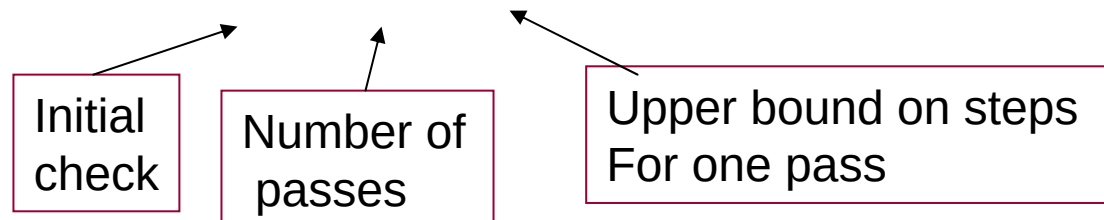
- Next:

- Midterm, then Section 7.3 (after the break).

Examples of time complexity analysis

Examples of time complexity analysis

- Consider a basic 1-tape Turing machine M that decides membership in the language $L = \{0^k 1^k \mid k \geq 0\}$:
 - M first checks that its input is in $0^* 1^*$, using one left-to-right pass.
 - Returns to the beginning (left).
 - Then does repeated passes, each time crossing off one 0 and one 1, until it runs out of at least one of them.
 - If it runs out of both on the same pass, accepts, else rejects.
- **Q:** How much time until M halts?
- Depends on the particular input.
- **Example: $0111\dots 1110$ (length n)**
 - Approximately n steps to reject---not in $0^* 1^*$,
- **Example: $00\dots 011\dots 1$ ($n/2$ 0s and $n/2$ 1s)**
 - Approximately (at most) $2n + (n/2) 2n = 2n + n^2$ steps to accept.



Time complexity analysis

- $L(M) = \{0^k 1^k \mid k \geq 0\}$.
- Time until M halts depends on the particular input.
- $0111\dots 1110$ (length n)
 - Approximately n steps to reject---not in 0^*1^* ,
- $00\dots 011\dots 1$ ($n/2$ 0s and $n/2$ 1s)
 - Approximately (at most) $2n + n^2$ steps to accept.
- It's too complicated to determine exactly how many steps are required for every input.
- So instead, we:
 - Get a close upper bound, not an exact step count.
 - Express the bound as a function of the input length n , thus grouping together all inputs of the same length and considering the max.
 - Often ignore constant factors and low-order terms.
- So, we describe the time complexity of M as $O(n^2)$.
 - At most some constant times n^2 .

Time complexity analysis

- $L(M) = \{0^k 1^k \mid k \geq 0\}$.
- Time complexity of machine $M = O(n^2)$.
- Q: Can we do better with a multitape machine?
- Yes, with 2 tapes:
 - After checking $0^* 1^*$, the machine copies the 0s to the second tape.
 - Then moves 2 heads together, one scanning the 0s on the second tape and one scanning the 1s on the first tape.
 - Check that all the symbols match.
 - Time $O(n)$, proportional to n .

Time complexity analysis

- $L(M) = \{0^k 1^k \mid k \geq 0\}$.
- 1-tape machine: $O(n^2)$, 2-tape machine: $O(n)$.
- Q: Can we beat $O(n^2)$ with a 1-tape machine?
- Yes, can get $O(n \log n)$:
 - First check 0^*1^* , as before, $O(n)$ steps.
 - Then perform **marking** phases, as long as some unmarked 0 and some unmarked 1 remain.
 - In each marking phase:
 - Scan to see whether $\#$ of unmarked 0s $\equiv \#$ of unmarked 1s, mod 2.
 - That is, see whether they have the same parity.
 - If not, then reject, else continue.
 - Scan again, marking every other 0 starting with the first and every other 1 starting with the first.
 - After all phases are complete:
 - If just 0s or just 1s remain, then reject
 - If no unmarked symbols remain, then accept.

Time complexity analysis

- $O(n \log n)$ algorithm:
 - Check 0^*1^* .
 - Perform **marking** phases, as long as some unmarked 0 and some unmarked 1 remain.
 - In each marking phase:
 - Scan to see if # of unmarked 0s $\equiv$ # of unmarked 1s, mod 2; if not, then reject, else continue.
 - Scan again, marking every other 0 starting with the first and every other 1 starting with the first.
 - If just 0s or just 1s remain, then reject, else accept.
- **Example: $00\dots011\dots1$ (25 0s and 25 1s)**
 - Correct form, 0^*1^* .
 - Phase 1: Same parity (odd), marking leaves 12 0s and 12 1s.
 - Phase 2: Same parity (even), marking leaves 6, 6.
 - Phase 3: Same parity (even), marking leaves 3, 3.
 - Phase 4: Same parity (odd), marking leaves 1,1.
 - Phase 5: Same parity (odd), marking leaves 0,0
 - Accept

Time complexity analysis

- **Example: 00...011...1** (25 0s and 25 1s)
 - Correct form, 0^*1^* .
 - Phase 1: Same parity (odd), marking leaves 12 0s and 12 1s.
 - Phase 2: Same parity (even), marking leaves 6, 6.
 - Phase 3: Same parity (even), marking leaves 3, 3.
 - Phase 4: Same parity (odd), marking leaves 1,1.
 - Phase 5: Same parity (odd), marking leaves 0,0
 - Accept
- Odd parity leads to remainder 1 on division by 2, even parity leads to remainder 0.
- Can read off odd-even parity designations to get binary representations of the numbers, starting with final phase for high-order bit:
 - 5: odd; 4: odd; 3: even; 2: even; 1: odd
 - Yields 1 1 0 0 1, binary representation of 25
- If the algorithm accepts, it means the 2 numbers have the same binary representation, so they are equal.

Time complexity analysis

- Example: 00...011...1 (17 0s and 25 1s)
 - Correct form, 0^*1^* .
 - Phase 1: Same parity (odd), marking leaves 8 0s and 12 1s.
 - Phase 2: Same parity (even), marking leaves 4, 6.
 - Phase 3: Same parity (even), marking leaves 2, 3.
 - Phase 4: Different parity, reject
 - Don't complete this, so don't generate the complete binary representation of either number.

Time complexity analysis

- Algorithm
 - Check 0^*1^* .
 - Perform **marking** phases, as long as some unmarked 0 and some unmarked 1 remain.
 - In each marking phase:
 - Scan to see if $\# \text{ of unmarked 0s} \equiv \# \text{ of unmarked 1s} \pmod{2}$; if not, then reject, else continue.
 - Scan again, marking every other 0 starting with the first and every other 1 starting with the first.
 - If just 0s or just 1s remain, then reject, else accept.
- **Complexity analysis:**
 - Number of phases is $O(\log_2 n)$, since we (approximately) halve the number of unmarked 0s and unmarked 1s at each phase.
 - Time for each phase: $O(n)$.
 - Total: $O(n \log n)$.
- This analysis is informal; now define O , etc., more carefully and then revisit the example.

Asymptotic function notation:

O, o, Ω, Θ

Asymptotic function notation

- **Definition: O (big- O)**

- Let f, g be two functions: $N \rightarrow \mathbb{R}^{\geq 0}$.
- We write $f(n) = O(g(n))$, and say “ $f(n)$ is big- O of $g(n)$ ” if the following holds:
 - There is a positive real c , and a positive integer n_0 , such that $f(n) \leq c g(n)$ for every $n \geq n_0$.
 - That is, $f(n)$ is bounded from above by a constant times $g(n)$, for all sufficiently large n .

- Often used for complexity upper bounds.

- **Example:** $n + 2 = O(n)$; can use $c = 2$, $n_0 = 2$.

- **Example:** $3n^2 + n = O(n^2)$; can use $c = 4$, $n_0 = 1$.

- **Example:** Any degree- k polynomial with nonnegative coefficients, $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 = O(n^k)$

- Thus, $3n^4 + 6n^2 + 17 = O(n^4)$.

More big-O examples

- Definition:
 - Let $f, g: \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$
 - $f(n) = O(g(n))$ means that there is a positive real c , and a positive integer n_0 , such that $f(n) \leq c g(n)$ for every $n \geq n_0$.
- **Example:** $3n^4 = O(n^7)$, though this is not the tightest possible statement.
- **Example:** $3n^7 \neq O(n^4)$.
- **Example:** $\log_2(n) = O(\log_e(n))$; $\log_a(n) = O(\log_b(n))$ for any a and b
 - Because logs to different bases differ by a constant factor.
- **Example:** $2^{3+n} = O(2^n)$, because $2^{3+n} = 8 \times 2^n$
- **Example:** $3^n \neq O(2^n)$

Other notation

- **Definition: Ω (big-Omega)**
 - Let f, g be two functions: $N \rightarrow \mathbb{R}^{\geq 0}$
 - We write $f(n) = \Omega(g(n))$, and say “ $f(n)$ is big-Omega of $g(n)$ ” if the following holds:
 - There is a positive real c , and a positive integer n_0 , such that $f(n) \geq c g(n)$ for every $n \geq n_0$.
 - That is, $f(n)$ is bounded from below by a positive constant times $g(n)$, for all sufficiently large n .
- Used for complexity lower bounds.
- **Example:** $3n^2 + 4n \log(n) = \Omega(n^2)$
- **Example:** $3n^7 = \Omega(n^4)$.
- **Example:** $\log_e(n) = \Omega(\log_2(n))$
- **Example:** $3^n = \Omega(2^n)$

Other notation

- **Definition: Θ (Theta)**
 - Let f, g be two functions: $N \rightarrow \mathbb{R}^{\geq 0}$
 - We write $f(n) = \Theta(g(n))$, and say “ $f(n)$ is Theta of $g(n)$ ” if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
 - Equivalently, there exist positive reals c_1, c_2 , and positive integer n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for every $n \geq n_0$.
- **Example:** $3n^2 + 4n \log(n) = \Theta(n^2)$
- **Example:** $3n^4 = \Theta(n^4)$.
- **Example:** $3n^7 \neq \Theta(n^4)$.
- **Example:** $\log_e(n) = \Theta(\log_2(n))$
- **Example:** $3^n \neq \Theta(2^n)$

Plugging asymptotics into formulas

- Sometimes we write things like $2^{\Theta(\log_2 n)}$
- What does this mean?
- Means the exponent is some function $f(n)$ that is $\Theta(\log n)$, that is, $c_1 \log(n) \leq f(n) \leq c_2 \log(n)$ for every $n \geq n_0$.
- So $2^{c_1 \log(n)} \leq 2^{\Theta(\log_2 n)} \leq 2^{c_2 \log(n)}$
- In other words, $n^{c_1} \leq 2^{\Theta(\log_2 n)} \leq n^{c_2}$
- Same as $n^{\Theta(1)}$.

Other notation

- **Definition: o (Little-o)**
 - Let f, g be two functions: $N \rightarrow \mathbb{R}^{\geq 0}$
 - We write $f(n) = o(g(n))$, and say “ $f(n)$ is little-o of $g(n)$ ” if **for every positive real c , there is some positive integer n_0 , such that $f(n) < c g(n)$ for every $n \geq n_0$.**
 - In other words, no matter what constant c we choose, for sufficiently large n , $f(n)$ is less than $g(n)$.
 - In other words, $f(n)$ grows at a slower rate than any constant times $g(n)$.
 - In other words, $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- **Example:** $3n^4 = o(n^7)$
- **Example:** $\sqrt{n} = o(n)$
- **Example:** $n \log n = o(n^2)$
- **Example:** $2^n = o(3^n)$

Back to the TM running times...

- Running times (worst case over all inputs of the same length n) of the 3 TMs described earlier:
 - Simple 1-tape algorithm: $\Theta(n^2)$
 - 2-tape algorithm: $\Theta(n)$
 - More clever 1-tape algorithm: $\Theta(n \log n)$
- More precisely, consider any Turing machine M that decides a language.
- Define the running time function $t_M(n)$ to be:
 - $\max_{w \in \Sigma^n} t'_M(w)$, where
 - $t'_M(w)$ is the exact running time (number of steps) of M on input w .
- Then for these three machines, $t_M(n)$ is $\Theta(n^2)$, $\Theta(n)$, and $\Theta(n \log n)$, respectively.

Time Complexity Classes

Time Complexity Classes

- Classify decidable languages according to upper bounds on the running time for TMs that decide them.
- **Definition:** Let $t: \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$ be a (total) function. Then $\text{TIME}(t(n))$ is the set of languages:
 $\{ L \mid L \text{ is decided by some } O(t(n))\text{-time Turing machine} \}$
- Call this a “time-bounded complexity class”.
- **Notes:**
 - Notice the O ---allows some slack.
 - To be careful, we need to specify which kind of TM model we are talking about; assume basic 1-tape.
- **Complexity Theory studies:**
 - Which languages are in which complexity classes.
 - E.g., is the language PRIMES in $\text{TIME}(n^5)$?
 - How complexity classes are related to each other.
 - E.g., is $\text{TIME}(n^5) = \text{TIME}(n^6)$, or are there languages that can be decided in time $O(n^6)$ but not in time $O(n^5)$?

Time Complexity Classes

- A problem: Running times are model-dependent.
- E.g., $L = \{0^k 1^k \mid k \geq 0\}$:
 - On 1-tape TM, can decide in time $O(n \log n)$.
 - On 2-tape TM, can decide in time $O(n)$.
- To be definite, we'll define the complexity classes in terms of 1-tape TMs (as Sipser does); others use multi-tape, or other models like Random-Access Machines (RAMs).
- **Q:** Is this difference important?
- Only up to a point:
 - If $L \in \text{TIME}(f(n))$ based on any “standard” machine model, then also $L \in \text{TIME}(g(n))$, where $g(n) = O(p(f(n)))$ for some polynomial p , based on any other “standard” machine model.
 - Running times for L in any two standard models are polynomial-related.
- **Example:** Single-tape vs. multi-tape Turing machines

Time Complexity Classes

- If $L \in \text{TIME}(f(n))$ based on any “standard” machine model, then also $L \in \text{TIME}(g(n))$, where $g(n) = O(p(f(n)))$ for some polynomial p , based on any other “standard” machine model.
- **Example:** 1-tape vs. multi-tape Turing machines
 - 1-tape $\rightarrow$ multi-tape with no increase in complexity.
 - Multi-tape $\rightarrow$ 1-tape: If $t(n) \geq n$ then every $t(n)$ -time multi-tape TM has an equivalent $O(t^2(n))$ -time 1-tape TM.
 - **Proof idea:**
 - 1-tape TM simulates multi-tape TM.
 - Simulates each step of multi-tape TM using 2 scans over non-blank portion of tapes, visiting all heads, making all changes.
 - **Q:** What is the time complexity of the simulating 1-tape TM? That is, how many steps does the 1-tape TM use to simulate the $t(n)$ steps of the multi-tape machine?

Time Complexity Classes

- **Example:** 1-tape vs. multi-tape Turing machines
 - Multi-tape $\rightarrow$ 1-tape: If $t(n) \geq n$ then every $t(n)$ -time multi-tape TM has an equivalent $O(t^2(n))$ -time 1-tape TM.
 - 1-tape TM simulates multi-tape TM; simulates each step using 2 scans over non-blank portion of tapes, visiting all heads, making all changes.
 - Q: What is the time complexity of the 1-tape TM?
 - Q: How big can the non-blank portion of the multi-tape TM's tapes become?
 - Initially n , for the input.
 - In $t(n)$ steps, no bigger than $t(n)$, because that's how far the heads can travel (starts at left).
 - So the number of steps by the 1-tape TM is at most:

$$t(n) \times c t(n), \text{ hence } O(t^2(n)).$$

Number of steps of
multi-tape machine

Steps taken by the scans,
to emulate one step of the
multi-tape machine.

Time Complexity Classes

- If $L \in \text{TIME}(f(n))$ based on any “standard” machine model, then also $L \in \text{TIME}(g(n))$, where $g(n) = O(p(f(n)))$ for some polynomial p , based on any other “standard” machine model.
- Slightly-idealized versions of real computers, programs in standard languages, other “reasonable” machine models, can be emulated by basic TMs with only polynomial increase in running time.
- Important exception: Nondeterministic Turing machines (or other nondeterministic computing models)
 - For nondeterministic TMs, running time is usually measured by max number of steps on any branch.
 - A bound of $t(n)$ on the maximum number of steps on any branch translates into $2^{O(t(n))}$ steps for basic deterministic TMs.

P, Polynomial Time

P, Polynomial Time

- A formal way to define **fast computability**.
- Because of simulation results, polynomial differences are considered to be unimportant for (deterministic) TMs.
- So our definition of fast computability ignores polynomial differences.
- **Definition:** The class P of languages that are decidable in polynomial time is defined by:
$$P = \cup_{p \text{ a poly}} \text{TIME}(p(n)) = \cup_{k \geq 0} \text{TIME}(n^k)$$
- Notes:
 - These time-bounded language classes are defined with respect to basic (1-tape, 1-head) Turing machines.
 - Simulation results imply that we could have used any “reasonable” deterministic computing model and get the same language class.
 - Robust notion.

P, Polynomial Time

- Definition: The class P of languages that are decidable in polynomial time is defined by:
$$P = \cup_{p \text{ a poly}} \text{TIME}(p(n)) = \cup_{k \geq 0} \text{TIME}(n^k)$$
- P plays a role in complexity theory loosely analogous to that of decidable languages in computability.
- Recall Church-Turing thesis:
 - If L is decidable using some reasonable model of computation, then it is decidable using any reasonable model of computation.
- **Modified Church-Turing thesis:**
 - If L is decidable **in polynomial time** using some reasonable **deterministic** model of computation, then it is decidable **in polynomial time** using any reasonable **deterministic** model of computation.
- This is not a theorem---rather, a philosophical statement.
- Can think of this as defining what a reasonable model is.
- We'll focus on the class P for much of our work on complexity theory.

P, Polynomial Time

- We'll focus on the class P for much of our work on complexity theory.
- **Q:** Why is P a good language class to study?
- It's **model-independent** (for reasonable models).
- It's **scalable**:
 - Constant-factor dependence on input size.
 - E.g., an input that's twice as long requires only c times as much time, for some constant c (depends on degree of the polynomial).
 - E.g., consider time bound n^3 .
 - Input of length n takes time n^3 .
 - Input of length $2n$ takes time $(2n)^3 = 8n^3$, $c = 8$.
 - Works for all polynomials, any degree.

P, Polynomial Time

- Q: Why is P a good language class to study?
- It's model-independent (for reasonable models).
- It's scalable.
- It has **nice composition properties**:
 - Composing two polynomials yields another polynomial.
 - This property will be useful later, when we define **polynomial-time reducibilities**.
 - Preview: $A \leq_p B$ means that there exists a polynomial-time computable function f such that $x \in A$ if and only if $f(x) \in B$.
 - Desirable theorem: $A \leq_p B$ and $B \in P$ imply $A \in P$.
 - Proof:
 - Suppose B is decidable in time $O(n^k)$.
 - Suppose the reducibility function f is computable in time $O(n^l)$.

P, Polynomial Time

- P has nice composition properties:
 - $A \leq_p B$ means that there's a polynomial-time computable function f such that $x \in A$ if and only if $f(x) \in B$.
 - Desirable theorem: $A \leq_p B$ and $B \in P$ imply $A \in P$.
 - Proof:
 - Suppose B is decidable in time $O(n^k)$, and f is computable in time $O(n^l)$.
 - How much time does it take to decide membership in A by reduction to B ?
 - Given x of length n , time to compute $f(x)$ is $O(n^l)$.
 - Moreover, $|f(x)| = O(n^l)$, since there's not enough time to generate a bigger result.
 - Now run B 's decision procedure on $f(x)$.
 - Takes time $O(|f(x)|^k) = O((n^l)^k) = O(n^{lk})$.
 - Another polynomial, so A is decidable in poly time, so $A \in P$

P, Polynomial Time

- Q: Why is P a good language class to study?
 - It's model-independent (for reasonable models).
 - It's scalable.
 - It has nice composition properties.
- Q: What are some limitations?
 - Includes too much:
 - Allows polynomials with arbitrarily large exponents and coefficients.
 - Time $10,000,000 n^{10,000,000}$ isn't really feasible.
 - In practice, running times are usually low degree polynomials, up to about $O(n^4)$.
 - On the other hand, proving a non-polynomial lower bound is likely to be meaningful.

P, Polynomial Time

- Q: Why is P a good language class to study?
 - It's model-independent (for reasonable models).
 - It's scalable.
 - It has nice composition properties.
- Q: What are some limitations?
 - Includes too much.
 - Excludes some things:
 - Considers **worst case time complexity** only.
 - Some algorithms may work well enough in most cases, or in common cases, even though the worst case is exponential.
 - **Random choices**, with membership being decided with high probability rather than with certainty.
 - **Quantum computing**.

P, Polynomial Time

- **Example:** A language in P.
 - $\text{PATH} = \{ \langle G, s, t \rangle \mid G = (V, E) \text{ is a digraph that has a directed path from } s \text{ to } t \}$
 - Represent G by adjacency matrix ($|V|$ rows and $|V|$ columns, 1 indicates an edge, 0 indicates no edge).
 - **Brute-force algorithm:** Try all paths of length $\leq |V|$.
 - Exponential running time in input size, not polynomial.
 - **Better algorithm:** BFS of G starting from s .
 - Mark new nodes accessible from already-marked nodes, until no new nodes are found.
 - Then see if t is marked.
 - Complexity analysis:
 - At most $|V|$ phases are executed.
 - Each phase takes polynomial time to explore marked nodes and their outgoing edges.

A Language Not in P

A Language Not in P

- **Q:** Is every language in P?
- No, because $P \subseteq$ decidable languages, and not every language is decidable.
- **Q:** Is every decidable language in P?
- No again, but it takes some work to show this.
- **Theorem:** For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- **Proof:**
 - Fix computable function t .
 - Define language **Acc(t)**
 $= \{ \langle M \rangle \mid M \text{ is a basic TM and } M \text{ accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}.$
 - **Claim 1:** Acc(t) is decidable.
 - **Claim 2:** Acc(t) is not decided by any basic TM in $\leq t(n)$ steps.

A Language Not in P

- **Theorem:** For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- **Proof:**
 - $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$.
 - **Claim 1:** $\text{Acc}(t)$ is decidable.
 - Given $\langle M \rangle$, simulate M on $\langle M \rangle$ for $t(|\langle M \rangle|)$ simulated steps and see if it accepts.
 - **Claim 2:** $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
 - Use a diagonalization proof, like that for Acc_{TM} .
 - Assume $\text{Acc}(t)$ is decided in time $\leq t(n)$ by some basic TM.
 - Here, $n = |\langle M \rangle|$ for input $\langle M \rangle$.

A Language Not in P

- **Theorem:** For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$.
- **Claim 2:** $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
- **Proof:**
 - Assume $\text{Acc}(t)$ is decided in time $\leq t(n)$ by some basic TM.
 - Then $\text{Acc}(t)^c$ is decided in time $\leq t(n)$, by another basic TM.
 - Interchange q_{acc} and q_{rej} states.
 - Let M_0 be a basic TM that decides $\text{Acc}(t)^c$ in time $\leq t(n)$.
 - That means $t(n)$ steps of M_0 , not $t(n)$ simulated steps.
 - Thus, for every basic Turing machine M :
 - If $\langle M \rangle \in \text{Acc}(t)^c$, then M_0 accepts $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - If $\langle M \rangle \in \text{Acc}(t)$, then M_0 rejects $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.

A Language Not in P

- **Theorem:** For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$.
- **Claim 2:** $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
- **Proof:**
 - Assume $\text{Acc}(t)$ is decided in time $\leq t(n)$ by some basic TM.
 - $\text{Acc}(t)^c$ is decided in time $\leq t(n)$, by basic TM M_0 .
 - Thus, for every basic Turing machine M :
 - If $\langle M \rangle \in \text{Acc}(t)^c$, then M_0 accepts $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - If $\langle M \rangle \in \text{Acc}(t)$, then M_0 rejects $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - Thus, for every basic Turing machine M :
 - $\langle M \rangle \in \text{Acc}(t)^c$ iff M_0 accepts $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.

A Language Not in P

- **Theorem:** For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$.
- **Claim 2:** $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
- **Proof:**
 - Assume $\text{Acc}(t)$ is decided in time $\leq t(n)$ by some basic TM.
 - $\text{Acc}(t)^c$ is decided in time $\leq t(n)$, by basic TM M_0 .
 - For every basic Turing machine M :
 $\langle M \rangle \in \text{Acc}(t)^c$ iff M_0 accepts $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - However, by definition of $\text{Acc}(t)$, for every basic TM M :
 $\langle M \rangle \in \text{Acc}(t)^c$ iff M does not accept $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.

A Language Not in P

- **Claim 2:** $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
- **Proof:**
 - Assume $\text{Acc}(t)$ is decided in time $\leq t(n)$ by some basic TM.
 - $\text{Acc}(t)^c$ is decided in time $\leq t(n)$, by basic TM M_0 .
 - For every basic Turing machine M :
 - $\langle M \rangle \in \text{Acc}(t)^c$ iff M_0 accepts $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - $\langle M \rangle \in \text{Acc}(t)$ iff M does not accept $\langle M \rangle$ in time $\leq t(|\langle M \rangle|)$.
 - Now plug in M_0 for M in both statements:
 - $\langle M_0 \rangle \in \text{Acc}(t)^c$ iff M_0 accepts $\langle M_0 \rangle$ in time $\leq t(|\langle M_0 \rangle|)$.
 - $\langle M_0 \rangle \in \text{Acc}(t)$ iff M_0 does not accept $\langle M_0 \rangle$ in time $\leq t(|\langle M_0 \rangle|)$.
 - Contradiction!

A Language Not in P

- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}.$
- We have proved:
- Theorem: For any computable function t , there is a language that is decidable, but cannot be decided by any basic Turing machine in time $t(n)$.
- Proof:
 - Claim 1: $\text{Acc}(t)$ is decidable.
 - Claim 2: $\text{Acc}(t)$ is not decided by any basic TM in $\leq t(n)$ steps.
- Thus, for every computable function $t(n)$, no matter how large (exponential, double-exponential,...), there are decidable languages not decidable in time $t(n)$.
- In particular, there are decidable languages not in P.

Hierarchy Theorems

Hierarchy Theorems

- Simplified summary, from Sipser Section 9.1.
- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$
- We have just proved that, for any computable function t , the language $\text{Acc}(t)$ is decidable, but cannot be decided by any basic TM in time $t(n)$.
- **Q:** How much time does it take to compute $\text{Acc}(t)$?
- More than $t(n)$, but how much more?
- Technical assumption: t is “time-constructible”, meaning it can be computed in an amount of time that is not much bigger than t itself.
 - Examples: Typical functions, like polynomials, exponentials, double-exponentials,...

Hierarchy Theorems

- $\text{Acc}(t) = \{ \langle M \rangle \mid M \text{ is a basic TM that accepts } \langle M \rangle \text{ in } \leq t(|\langle M \rangle|) \text{ steps} \}$
- **Q:** How much time does it take to compute $\text{Acc}(t)$?
- **Theorem (informal statement):** If t is any time-constructible function, then $\text{Acc}(t)$ can be decided by a basic TM in time not much bigger than $t(n)$.
 - E.g., approximately $t^2(n)$.
 - Sipser (Theorem 9.10) gives a tighter bound.
- **Q:** Why exactly does it take much more than $t(n)$ time to run an arbitrary machine M on $\langle M \rangle$ for $t(|\langle M \rangle|)$ simulated steps?
- We must simulate an arbitrary machine M using a fixed “universal” TM, with a fixed state set, fixed alphabet, etc.

Hierarchy Theorems

- **Theorem (informal):** If t is any time-constructible function, then $\text{Acc}(t)$ can be decided by a basic TM in time not much bigger than $t(n)$.
 - E.g., approximately $t^2(n)$.
- Implies that there is:
 - A language decidable in time n^2 but not time n .
 - A language decidable in time n^6 but not time n^3 .
 - A language decidable in time 4^n but not time 2^n .
- Extend this reasoning to show:
 - $\text{TIME}(n) \neq \text{TIME}(n^2) \neq \text{TIME}(n^4) \dots$
 $\neq \text{TIME}(2^n) \neq \text{TIME}(4^n) \dots$
- A hierarchy of distinct language classes.

Next time...

- The Midterm!

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 16

*Lecturer: Scott Aaronson**Scribe: Jason Furtado*

Private-Key Cryptography

1 Recap

1.1 Derandomization

In the last six years, there have been some spectacular discoveries of deterministic algorithms, for problems for which the only similarly-efficient solutions that were known previously required randomness. The two most famous examples are

- the Agrawal-Kayal-Saxena (AKS) algorithm for determining if a number is prime or composite in deterministic polynomial time, and
- the algorithm of Reingold for getting out of a maze (that is, solving the undirected s-t connectivity problem) in deterministic LOGSPACE.

Beyond these specific examples, mounting evidence has convinced almost all theoretical computer scientists of the following

Conjecture: Every randomized algorithm can be simulated by a deterministic algorithm with at most polynomial slowdown. Formally, $P = BPP$.

1.2 Cryptographic Codes

1.2.1 Caesar Cipher

In this method, a plaintext message is converted to a ciphertext by simply adding 3 to each letter, wrapping around to A after you reach Z. This method is breakable by hand.

1.2.2 One-Time Pad

The “one-time pad” uses a random key that must be as long as the message we want to encrypt. The exclusive-or operation is performed on each bit of the message and key ($Msg \oplus Key = EncryptedMsg$) to end up with an encrypted message. The encrypted message can be decrypted by performing the same operation on the encrypted message and the key to retrieve the message ($EncryptedMsg \oplus Key = Msg$). An adversary that intercepts the encrypted message will be unable to decrypt it as long as the key is truly random.

The one-time pad was the first example of a cryptographic code that can *proven* to be secure, even if the adversary has all the computation time in the universe.

The main drawback of this method is that keys can never be reused, and the key must be the same size as the message to encrypt. If you were to use the same key twice, an eavesdropper could compute $(Enc \oplus Msg1) \oplus (Enc \oplus Msg2) = Msg1 \oplus Msg2$. This would leak information about $Msg1$ and $Msg2$.

Example. Suppose $Msg1$ and $Msg2$ were bitmaps and $Msg1$ had sections that were all the same (say, a plain white background). For simplicity, assume $Msg1$ is all zeros at bit positions 251-855. Then $Msg2$ will show through in those bit positions. During the Cold War, spies were actually caught using this sort of technique.

Also, note that the sender and the recipient must agree on the key in advance. Having shared random keys available for every possible message size is often not practical. Can we create encryption methods that are secure with smaller keys, by assuming our adversary doesn't have unlimited computing power (say, is restricted to running polynomial-time algorithms)?

2 Pseudorandom Generators

A pseudorandom generator (PRG) is a function that takes as input a short, truly random string (called the *seed*) and produces as output a long, seemingly random string.

2.1 Seed Generation

A seed is a “truly” random string used as input to a PRG. How do you get truly random numbers? Some seeds used are generated from the system time, typing on a keyboard randomly, the last digits of stock prices, or mouse movements. There are subtle correlations in these sources so they aren't completely random, but there are ways of extracting randomness from weak random sources. For example, according to some powerful recent results, nearly “pure” randomness can often be extracted from two or more weak random sources that are assumed to be uncorrelated with each other.

How do you prove that a sequence of numbers is random? Well, it's much easier to give overwhelming evidence that a sequence is *not* random! In general, one does this by finding a *pattern* in the sequence, i.e. a computable description with fewer bits than the sequence itself. (In other words, by showing that the sequence has less-than-maximal Kolmogorov complexity.)

In this lecture, we'll simply assume that we have a short random seed, and consider the problem of how to expand it into a long “random-looking” sequence.

2.2 How to Expand Random Numbers

2.2.1 Linear-Congruential Generator

In most programming languages, if you ask for random numbers what you get will be something like the following (starting from integers a , b , and N):

$$x_1 = ax_0 + b \bmod N \quad x_2 = ax_1 + b \bmod N$$

...

$$x_n = ax_{n-1} + b \bmod N$$

This process is good enough for many non-cryptographic applications, but an adversary could easily distinguish the sequence $x_0, x_1, \dots$ from random by solving a small system of equations mod N . For cryptography applications, it must not be possible for an adversary to figure out a pattern in the output of the generator in polynomial time. Otherwise, the system is not secure.

2.2.2 Cryptographic Pseudorandom Generator (CPRG)

Definition: (Yao 1982)

A cryptographic pseudorandom generator (CPRG) is a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ such that:

1. f is computable in polynomial time.
2. For all polynomial-time algorithms A (adversaries),

$$\left| \Pr_{y \in \{0,1\}^{n+1}}[A(y) \text{ accepts}] - \Pr_{x \in \{0,1\}^n}[A(f(x)) \text{ accepts}] \right|,$$

the “advantage”, is negligibly small.

In other words, the output of the CPRG must “look random” to any polynomial time algorithm.

In the above definition, “negligibly small” means less than $1/p(n)$ for all polynomials p . This is a minimal requirement, since if the advantage of the adversary were $1/p(n)$, then in polynomial time the adversary could amplify the advantage to a constant (see Lecture 14). Of course it’s even better if the adversary’s advantage decreases exponentially.

The definition above only requires f to stretch an n -bit seed into a random-looking $(n + 1)$ -bit string. Could we use such an f to stretch an n -bit seed into, say, a random-looking n^2 -bit string? It turns out that the answer is yes; basically we feed f its own output n^2 times. (See Lecture 17 for more details.)

2.2.3 Enhanced One-Time Pad

Using such a CPRG $f : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n)}$, we can make our one-time pad work for messages polynomially larger than the original key s :

$$k = f(s)$$

$$e = x \oplus k$$

$$x = e \oplus k$$

Claim. With this construction, no polynomial-time adversary can recover the plaintext from the ciphertext.

Proof. Assume for simplicity that the plaintext consists of just a single repeated random bit (i.e., is either $00 \cdots 0$ or $11 \cdots 1$, both with equal probability). Also, suppose by way of contradiction that a polynomial-time adversary could guess the plaintext given the ciphertext, with probability non-negligibly greater than $1/2$. We know that if the key k were truly random, then the adversary would *not* be able to guess the plaintext with probability greater than $1/2$. But this means that the adversary must be distinguishing a pseudorandom key from a truly random key with non-negligible bias – thereby violating the assumption that f was a CPRG!

The system above is not yet a secure cryptographic system (we still need to deal with the issue of repeated keys, etc.), but hopefully this gives some idea of how CPRG’s can be used to construct computationally-secure cryptosystems.

3 Blum-Blum-Shub CPRG

The Blum-Blum-Shub (BBS) CPRG is proven to breakable if and only if a fast (polynomial-time) algorithm exists for factoring. With this generator, the seed consists of integers x and $N = pq$, where p, q are large primes. The output consists of the last bit of $x^2 \bmod N$, the last bit of $(x^2)^2 \bmod N$, the last bit of $((x^2)^2)^2 \bmod N$, etc.

4 $P \neq NP$ -based CPRG

Ideally, we would like to construct a CPRG or cryptosystem whose security was based on an NP-complete problem. Unfortunately, NP-complete problems are always about the worst case. In cryptography, this would translate to a statement like “there *exists* a message that’s hard to decode”, which is not a good guarantee for a cryptographic system! A message should be hard to decrypt with overwhelming probability. Despite decades of effort, no way has yet been discovered to relate worst case to average case for NP-complete problems. And this is why, if we want computationally-secure cryptosystems, we need to make stronger assumptions than $P \neq NP$.

5 One-Way Functions

The existence of one-way functions (OWF’s) is such a stronger assumption.

Definition: A one-way function is a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n)}$ such that:

1. f is computable in polynomial time.
2. For all polynomial-time algorithms A ,

$$\Pr_{x \in \{0,1\}^n}[f(A(f(x))) = f(x)]$$

is negligible.

In other words, a polynomial-time algorithm should only be able to invert f with negligible probability. The reason we don’t require $A(f(x)) = x$ is to rule out trivial “one-way functions” like $f(x) = 1$.

$CPRG \Rightarrow OWF?$

True. Any CPRG is also an OWF by the following argument: if given the output of a pseudorandom generator we could efficiently find the seed, then we’d be distinguishing the output from true randomness – thereby violating the assumption that we had a CPRG in the first place.

$OWF \iff CPRG?$

Also true, but this direction took over 20 years to prove! In 1997, Håstad, Impagliazzo, Levin, and Luby showed how to construct a pseudorandom generator from any one-way function, by a complicated reduction with many steps.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 17

*Lecturer: Scott Aaronson**Scribe: Adam Rogal*

1 Recap

1.1 Pseudorandom Generators

We will begin with a recap of pseudorandom generators (PRGs). As we discussed before a pseudorandom generator is a function that takes as input a short truly random input string and produces an output of a seemingly random string. Formally, a PRG is a polytime-computable function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ such that for all deterministic polynomial-time algorithms A ,

$$\left| \Pr_{y \in \{0,1\}^{n+1}} [A(y) \text{ accepts}] - \Pr_{x \in \{0,1\}^n} [A(f(x)) \text{ accepts}] \right|$$

is negligible.

Given a PRG that stretches n bits to $n + 1$ bits, we can create a PRG that stretches n bits to $p(n)$ bits for any polynomial p . To do so, we repeatedly break off a single bit of the PRG's output, and feeding the remaining n bits back into the PRG to get another $n + 1$ pseudorandom bits. This process is shown in figure 1. To prove that it works, one needs to show that, could we distinguish the $p(n)$ -bit output from random, we could *also* distinguish the original $(n + 1)$ -bit output from random, thereby violating the assumption that we started with a PRG. Formalizing this intuition is somewhat tricky and will not be done here.

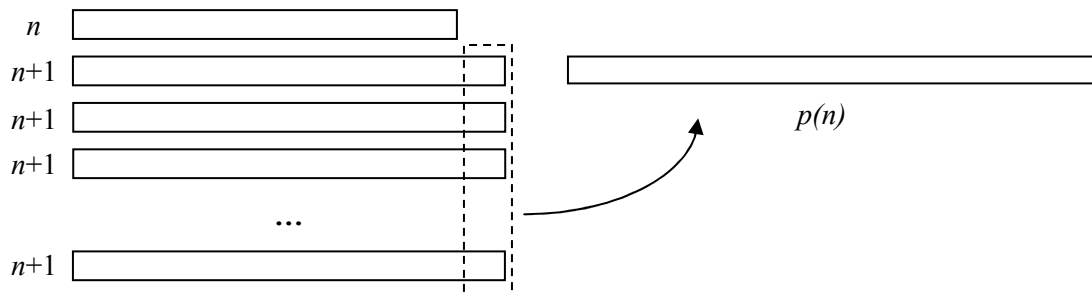


Figure 1: A seemingly random string of size $p(n)$ is generated from an n -bit seed using the feed and repeat method.

1.2 Cryptographic Codes

Using pseudorandom generators, it's possible to create secure cryptographic codes with small key sizes. The details of this are complicated if you want to protect against realistic

attacks (for example, so-called *chosen-message attacks*). But at the simplest level, the intuition is the following: we should be able to simulate a one-time pad (which is provably unbreakable when used correctly) by (1) taking a small random key, (2) stretching it to a longer key using a PRG, and then (3) treating that longer key as the one-time pad. If a polynomial-time adversary could break such a system, that would mean that the adversary was distinguishing the PRG's output from a truly random string, contrary to assumption.

1.3 One-Way Functions

In addition to PRGs, we'll be interested in a closely-related class of objects called OWFs, or *one-way functions*. An OWF is a polytime-computable function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n)}$ such that for all deterministic polynomial-time algorithms A ,

$$\Pr_{x \in \{0,1\}^n} [f(A(f(x))) = f(x)]$$

is negligible.

Or in plainer language, an OWF is a function that's easy to compute but hard to invert.

1.4 Yao's Minimax Principle

As a side note, you might wonder why we assumed the adversary A was deterministic rather than probabilistic. The answer is that it makes no difference! If you're playing rock-paper-scissors, and you *know* the probability distribution over your opponent's move, then there's always some *fixed* move you can make that does as well as any randomized strategy. Similarly, once you fix the probability distribution over inputs – as we do with PRGs and OWFs – there's always a deterministic algorithm whose success probability is as large as any randomized algorithm's. This is (the easy part of) *Yao's Minimax Principle*, one of the most useful facts in theoretical computer science.

1.5 Relation Between PRGs and OWFs

Claim: Every PRG is also an OWF. Why? Because if we could invert a PRG, then it wouldn't be pseudorandom! We'd learn that there was *some* seed that generated the output string, which would be true for a random string with probability at most $1/2$.

In 1997, Håstad et al. proved the opposite direction: if OWFs exist then so do PRGs. This direction was much, *much* harder (note that transformations of the OWF are necessary, since it's easy to give examples of OWFs that are not PRGs). Because of this result, we now know that the possibility of private-key encryption with small keys is essentially equivalent to the existence of OWFs.

2 Public-Key Cryptography

2.1 Abstract Problem

Suppose Alice is trying to send Bob a package, so that no third party can open it *en route*. We'll assume that boxes can be "locked," in such a way that you can only open a box if you have the right key.

If Alice and Bob share duplicates of the same key, then this problem is trivial: Alice locks the box with her key and sends it to Bob, who then opens it with his key. But what

if Alice and Bob *don't* share a key? Obviously, we don't want Alice to send the package in a locked box, and the key that opens the lock in an unlocked box! We seem to be faced with an infinite regress.

Fortunately, there's a simple solution. As shown in Figure 2, first Alice puts the package in a box, locks it, and sends it to Bob. Then Bob puts a *second* lock on the box and sends it back to Alice. Then Alice removes her lock and sends the box back to Bob. Finally Bob removes his lock and opens the box.

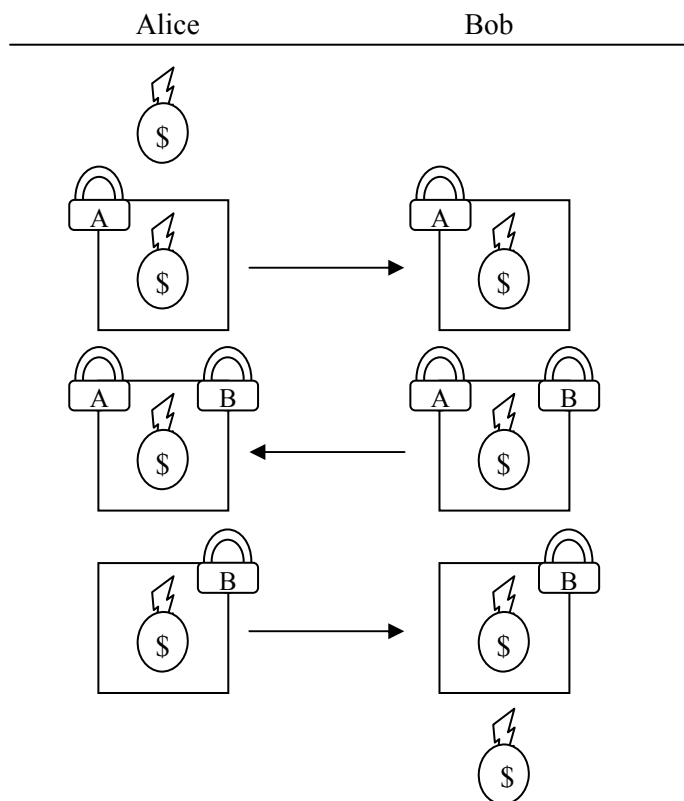


Figure 2: The smarter approach has Alice and Bob passing the package with at least one form of protection at all times. This ensures that only Alice and Bob will be able to open the package.

2.2 Diffie-Hellman

How could we simulate the above protocol, in the situation where Alice and Bob are sending bits of information rather than physical boxes? The first serious proposal in the open literature for how to do this was given by Diffie and Hellman in 1976.

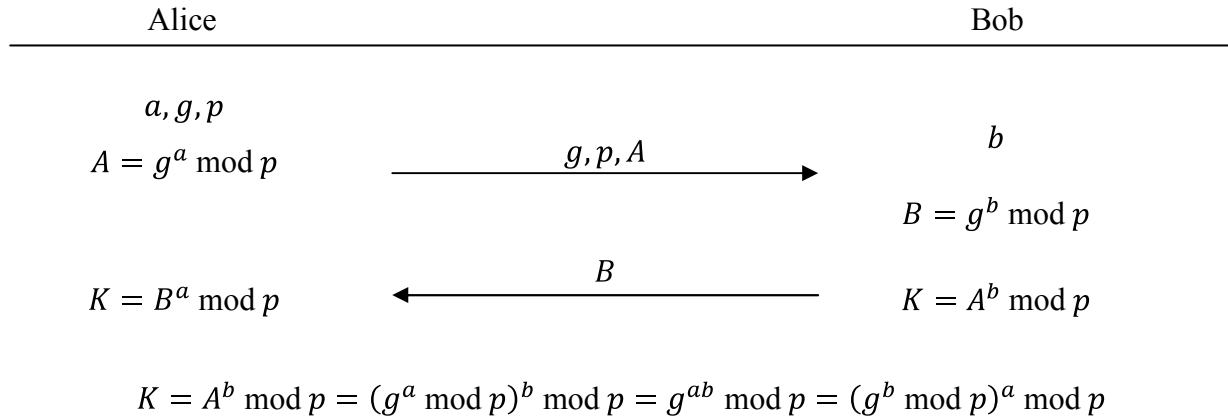


Figure 3: The Diffie-Hellman protocol for creating a shared secret key K between Alice and Bob.

The process, shown in figure 3, begins by Alice choosing a large prime number, p , a base, g , and a secret integer, a . Alice will calculate a public number $A = g^a \bmod p$. She will then send p , g , and A to Bob. Bob will then pick his own secret b , and send $B = g^b \bmod p$ back to Alice. Finally, Alice calculates the secret key K as $K = B^a \bmod p$, and Bob calculates it as $K = A^b \bmod p$. They both now have the same key with which to encode messages to each other.

We've seen that Diffie-Hellman is a simple way to exchange a key; yet, but it's a bit cumbersome in practice. What we'd really like is a public-key protocol that involves fewer messages back and forth—and in which only one person, not two, needs to create public and private keys.

3 RSA

RSA (together with its variants) is probably the most widely-used cryptographic protocol in modern electronic commerce. Much like Diffie-Hellman, it is built on modular arithmetic.

3.1 How It Works

As shown in Figure 4, the process is more direct than with Diffie-Hellman. Let's suppose you want to send your credit card number to Amazon.com. Then in the simplest variant, Amazon picks two large prime numbers, p and q , with the condition that neither $p - 1$ nor $q - 1$ is divisible by 3. It then multiplies them together to get $N = pq$ and sends N to you. On retrieving N , you calculate $y = x^3 \bmod N$, where x is your credit card number, and send y back to Amazon.

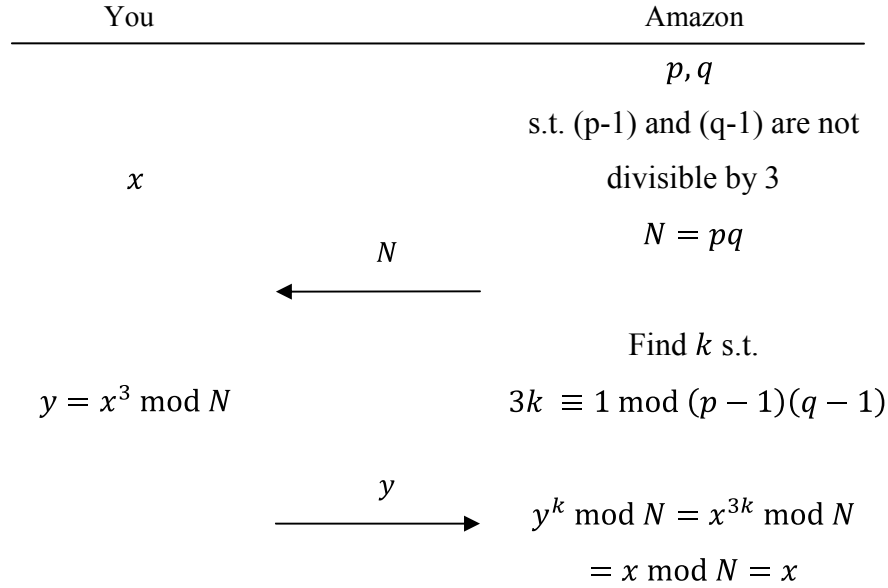


Figure 4: RSA uses modular arithmetic to retrieve x efficiently from an encoded message. An eavesdropper will only see N and $x^3 \pmod N$.

Amazon then faces the problem of how to recover x given y . In other words, how does it take a *cube root* modulo N ? Fortunately, it can do that given using its knowledge of the prime factors p and q , together with the following formula discovered by the mathematician Leonhard Euler in the 1700's:

$$x^{(p-1)(q-1)} = 1 \pmod N$$

(Why is this formula true? Basically, because $(p-1)(q-1)$ is the order of the *multiplicative group* $\pmod N$, consisting of all numbers from 1 to N that are relatively prime to N . We won't give a more detailed proof here.)

Euler's formula implies that, if Amazon can only find an integer k such that $3k = 1 \pmod{(p-1)(q-1)}$, then

$$y^k = x^{3k} = x^{c(p-1)(q-1)+1} = x \pmod N,$$

where c is some integer. But the fact that neither $p-1$ nor $q-1$ is divisible by 3 implies that such an integer k must exist – and furthermore k can be found in polynomial time given p and q , for example by using Euclid's algorithm. And once Amazon has k , it can also compute $y^k \pmod N = x$ in polynomial time using repeated squaring. It can thereby recover your credit card number x , as desired.

The obvious question is, how secure is this system? Well, any adversary who could factor N into pq could obviously decrypt the message x , by using the same algorithm that Amazon itself uses. Hence this whole system is predicated on the presumed intractability of factoring large integers (an assumption that would be violated if, for example, we built large-scale quantum computers). And of course, any proof that factoring is hard would also prove $P \neq NP$.

In the other direction, you might wonder: *assuming* the factoring problem is hard, is RSA secure? Alas, that's been an open problem for 30 years! Yet despite its uncertain theoretical foundations, the RSA system has withstood all attacks thus far (unlike many other proposed cryptosystems), and today millions of people rely on it.

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and Complexity (GITCS)

Class 15
Nancy Lynch

Today: More Complexity Theory

- Polynomial-time reducibility, NP-completeness, and the Satisfiability (SAT) problem
- **Topics:**
 - Introduction (Review and preview)
 - Polynomial-time reducibility, $\leq_p$
 - $\text{Clique} \leq_p \text{VertexCover}$ and vice versa
 - NP-completeness
 - SAT is NP-complete
- **Reading:**
 - Sipser Sections 7.4-7.5
- **Next:**
 - Sipser Sections 7.4-7.5

Introduction

Introduction

- $P = \{ L \mid \text{there is some polynomial-time deterministic Turing machine that decides } L \}$
- $NP = \{ L \mid \text{there is some polynomial-time nondeterministic Turing machine that decides } L \}$
- Alternatively, $L \in NP$ if and only if $(\exists V, \text{ a polynomial-time verifier }) (\exists p, \text{ a polynomial })$ such that:
$$x \in L \text{ iff } (\exists c, |c| \leq p(|x|)) [V(x, c) \text{ accepts}]$$

certificate



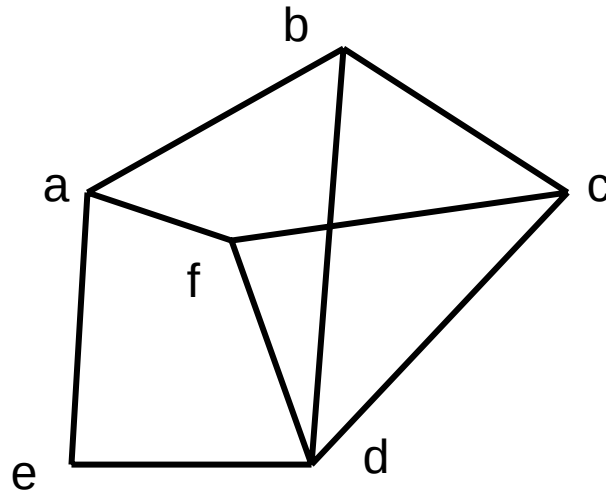
- To show that $L \in NP$, we need only exhibit a suitable verifier V and show that it works (which requires saying what the certificates are).
- $P \subseteq NP$, but it's not known whether $P = NP$.

Introduction

- $P = \{ L \mid \exists \text{ poly-time deterministic TM that decides } L \}$
- $NP = \{ L \mid \exists \text{ poly-time nondeterministic TM that decides } L \}$
- $L \in NP$ if and only if $(\exists V, \text{ poly-time verifier }) (\exists p, \text{ poly})$
 $x \in L \text{ iff } (\exists c, |c| \leq p(|x|)) [V(x, c) \text{ accepts}]$
- Some languages are in NP, but are not known to be in P (and are not known to not be in P):
 - $SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
 - $3COLOR = \{ \langle G \rangle \mid G \text{ is an (undirected) graph whose vertices can be colored with } \leq 3 \text{ colors with no 2 adjacent vertices colored the same} \}$
 - $CLIQUE = \{ \langle G, k \rangle \mid G \text{ is a graph with a } k\text{-clique} \}$
 - $VERTEX-COVER = \{ \langle G, k \rangle \mid G \text{ is a graph having a vertex cover of size } k \}$

CLIQUE

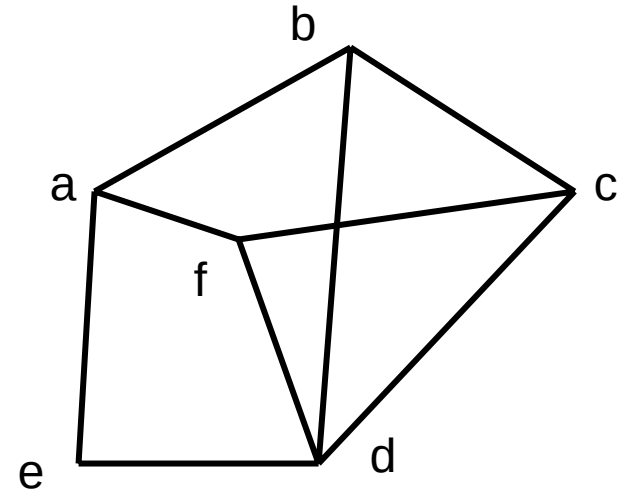
- **CLIQUE** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a } k\text{-clique} \}$
- **k-clique**: k vertices with edges between all pairs in the clique.
- In NP, not known to be in P, not known to not be in P.



- 3-cliques: $\{ b, c, d \}$, $\{ c, d, f \}$
- Cliques are easy to verify, but may be hard to find.

CLIQUE

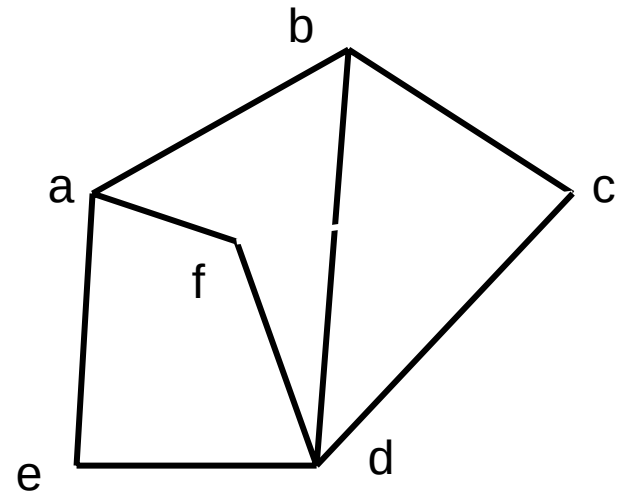
- **CLIQUE** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a } k\text{-clique} \}$



- Input to the VC problem: $\langle G, 3 \rangle$
- Certificate, to show that $\langle G, 3 \rangle \in \text{CLIQUE}$, is $\{ b, c, d \}$ (or $\{ c, d, f \}$).
- Polynomial-time verifier can check that $\{ b, c, d \}$ is a 3-clique.

VERTEX-COVER

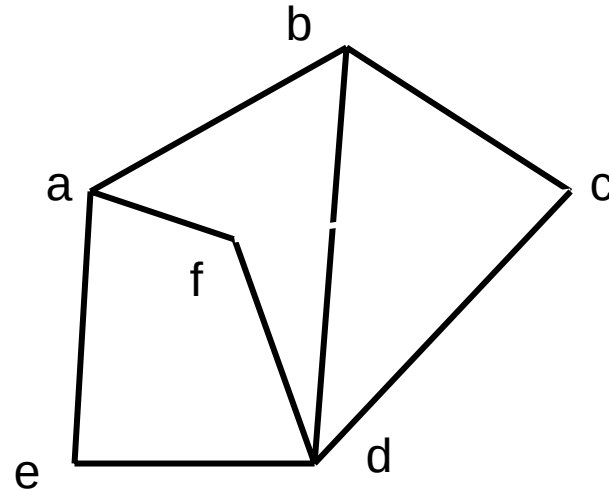
- **VERTEX-COVER** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a vertex cover of size } k \}$
- **Vertex cover of $G = (V, E)$:** A subset C of V such that, for every edge (u,v) in E , either $u \in C$ or $v \in C$.
 - A set of vertices that “covers” all the edges.
- In NP, not known to be in P, not known to not be in P.



- 3-vc: $\{ a, b, d \}$
- Vertex covers are easy to verify, may be hard to find.

VERTEX-COVER

- **VERTEX-COVER** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a vertex cover of size } k \}$



- Input to the VC problem: $\langle G, 3 \rangle$
- Certificate, to show that $\langle G, 3 \rangle \in VC$, is $\{ a, b, d \}$.
- Polynomial-time verifier can check that $\{ a, b, d \}$ is a 3-vertex-cover.

Introduction

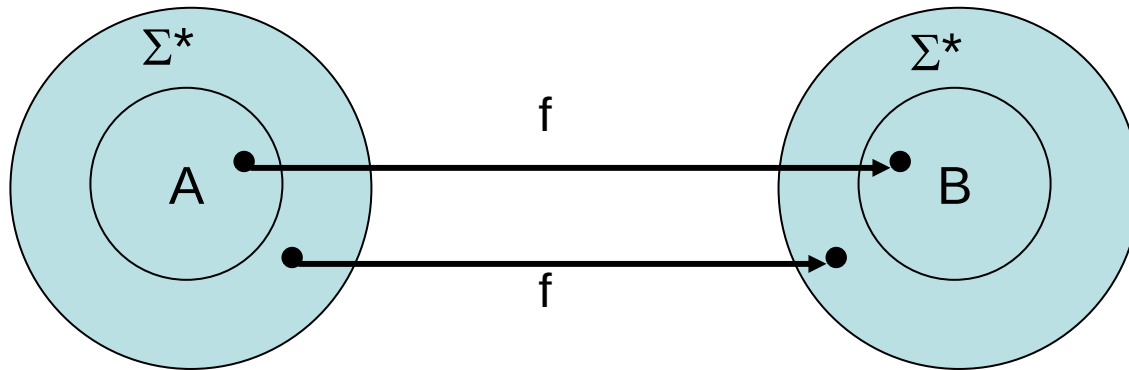
- Languages in NP, not known to be in P, not known to not be in P:
 - **SAT** = $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
 - **3COLOR** = $\{ \langle G \rangle \mid G \text{ is a graph whose vertices can be colored with } \leq 3 \text{ colors with no 2 adjacent vertices colored the same} \}$
 - **CLIQUE** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a } k\text{-clique} \}$
 - **VERTEX-COVER** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a vc of size } k \}$
- There are many problems like these, where some structure seems hard to find, but is easy to verify.
- **Q:** Are these easy (in P) or hard (not in P)?
- Not yet known. We don't yet have the math tools to answer this question.
- We can say something useful to reduce the apparent diversity of such problems---that many such problems are “reducible” to each other.
- So in a sense, they are the “same problem”.

Polynomial-Time Reducibility

Polynomial-Time Reducibility

- Definition:** $A \subseteq \Sigma^*$ is polynomial-time reducible to $B \subseteq \Sigma^*$, $A \leq_p B$, provided there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that:

$$(\forall w) [w \in A \text{ if and only if } f(w) \in B]$$



- Extends to different alphabets Σ_1 and Σ_2 .
- Same as mapping reducibility, $\leq_m$, but with a polynomial-time restriction.

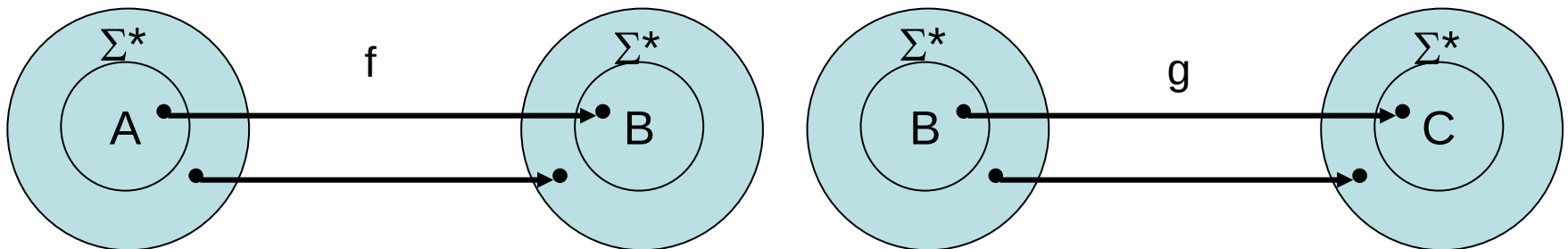
Polynomial-Time Reducibility

- Definition:** $A \subseteq \Sigma^*$ is polynomial-time reducible to $B \subseteq \Sigma^*$, $A \leq_p B$, provided there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that:

$$(\forall w) [w \in A \text{ if and only if } f(w) \in B]$$

- Theorem:** (Transitivity of $\leq_p$)
If $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.

- Proof:**
 - Let f be a polynomial-time reducibility function from A to B .
 - Let g be a polynomial-time reducibility function from B to C .



Polynomial-Time Reducibility

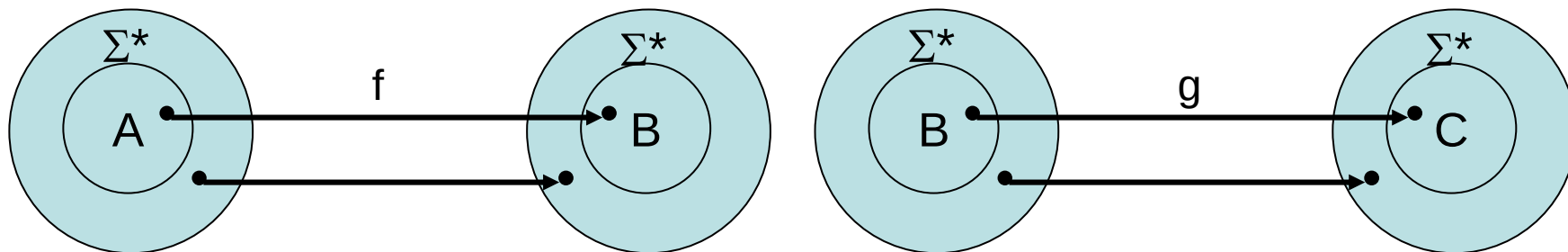
- **Definition:** $A \leq_p B$, provided there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that:

$$(\forall w) [w \in A \text{ if and only if } f(w) \in B]$$

- **Theorem:** If $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.

- **Proof:**

- Let f be a polynomial-time reducibility function from A to B .
- Let g be a polynomial-time reducibility function from B to C .

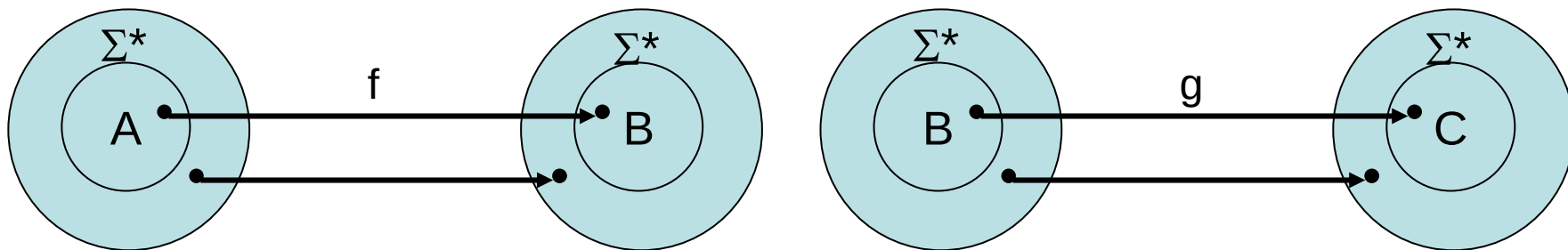


- Define $h(w) = g(f(w))$.
- Then $w \in A$ if and only if $f(w) \in B$ if and only if $\underbrace{g(f(w))}_{h(w)} \in C$.
- h is poly-time computable:

$h(w)$

Polynomial-Time Reducibility

- **Theorem:** If $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$.
- **Proof:**
 - Let f be a polynomial-time reducibility function from A to B .
 - Let g be a polynomial-time reducibility function from B to C .



- Define $h(w) = g(f(w))$.
- h is poly-time computable:
 - $|f(w)|$ is bounded by a polynomial in $|w|$.
 - Time to compute $g(f(w))$ is bounded by a polynomial in $|f(w)|$, and therefore by a polynomial in $|w|$.
 - Uses the fact that substituting one polynomial for the variable in another yields yet another polynomial.

Polynomial-Time Reducibility

- **Definition:** $A \leq_p B$, provided there is a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that:

$$(\forall w) [w \in A \text{ if and only if } f(w) \in B]$$

- **Theorem:** If $A \leq_p B$ and $B \in P$ then $A \in P$.
- **Proof:**
 - Let f be a polynomial-time reducibility function from A to B .
 - Let M be a polynomial-time decider for B .
 - To decide whether $w \in A$:
 - Compute $x = f(w)$.
 - Run M to decide whether $x \in B$, and accept / reject accordingly.
 - Polynomial time.
- **Corollary:** If $A \leq_p B$ and A is not in P then B is not in P .
- **Easiness propagates downward, hardness propagates upward.**

Polynomial-Time Reducibility

- Can use $\leq_p$ to relate the difficulty of two problems:
- **Theorem:** If $A \leq_p B$ and $B \leq_p A$ then either both A and B are in P or neither is.
- Also, for problems in NP :
- **Theorem:** If $A \leq_p B$ and $B \in NP$ then $A \in NP$.
- **Proof:**
 - Let f be a polynomial-time reducibility function from A to B .
 - Let M be a polynomial-time nondeterministic TM that decides B .
 - Poly-bounded on all branches.
 - Accepts on at least one branch iff and only if input string is in B .
 - NTM M' to decide membership in A :
 - **On input w :**
 - Compute $x = f(w)$; $|x|$ is bounded by a polynomial in $|w|$.
 - Run M on x and accept/reject (on each branch) if M does.
 - Polynomial time-bounded NTM.

Polynomial-Time Reducibility

- **Theorem:** If $A \leq_p B$ and $B \in \text{NP}$ then $A \in \text{NP}$.
- **Proof:**
 - Let f be a polynomial-time reducibility function from A to B .
 - Let M be a polynomial-time nondeterministic TM that decides B .
 - NTM M' to decide membership in A :
 - **On input w :**
 - Compute $x = f(w)$; $|x|$ is bounded by a polynomial in $|w|$.
 - Run M on x and accept/reject (on each branch) if M does.
 - Polynomial time-bounded NTM.
 - Decides membership in A :
 - M' has an accepting branch on input w
iff M has an accepting branch on $f(w)$, by definition of M' ,
iff $f(w) \in B$, since M decides B ,
iff $w \in A$, since $A \leq_p B$ using f .
 - So M' is a poly-time NTM that decides A , $A \in \text{NP}$.

Polynomial-Time Reducibility

- **Theorem:** If $A \leq_p B$ and $B \in \text{NP}$ then $A \in \text{NP}$.
- **Corollary:** If $A \leq_p B$ and A is not in NP, then B is not in NP.

Polynomial-Time Reducibility

- A technical result (curiosity):
- **Theorem:** If $A \in P$ and B is any nontrivial language (meaning not $\emptyset$, not Σ^*), then $A \leq_p B$.
- **Proof:**
 - Suppose $A \in P$.
 - Suppose B is a nontrivial language; pick $b_0 \in B$, $b_1 \in B^c$.
 - Define $f(w) = b_0$ if $w \in A$, b_1 if w is not in A .
 - f is polynomial-time computable; why?
 - Because A is polynomial time decidable.
 - Clearly $w \in A$ if and only if $f(w) \in B$.
 - So $A \leq_p B$.
- Trivial reduction: All the work is done by the decider for A , not by the reducibility and the decider for B .

CLIQUE and VERTEX-COVER

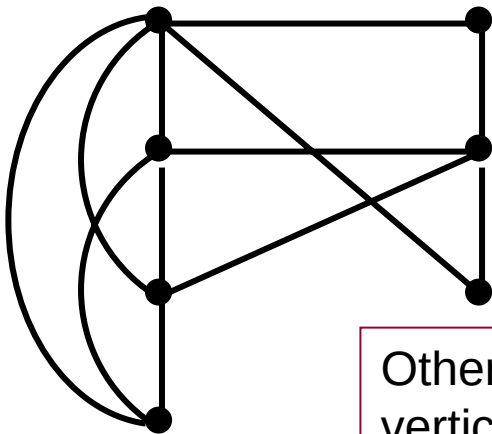
CLIQUE and VERTEX-COVER

- Two illustrations of $\leq_p$.
- Both CLIQUE and VC are in NP, not known to be in P, not known to not be in P.
- However, we can show that they are **essentially equivalent**: polynomial-time reducible to each other.
- So, although we don't know how hard they are, we know they are (approximately) equally hard.
 - E.g., if either is in P, then so is the other.
- **Theorem:** $\text{CLIQUE} \leq_p \text{VC}$.
- **Theorem:** $\text{VC} \leq_p \text{CLIQUE}$.

CLIQUE and VERTEX-COVER

- **Theorem:** $\text{CLIQUE} \leq_p \text{VC}$.
- **Proof:**
 - Given input $\langle G, k \rangle$ for CLIQUE, transform to input $\langle G', k' \rangle$ for VC, in poly time, so that:
 $\langle G, k \rangle \in \text{CLIQUE}$ if and only if $\langle G', k' \rangle \in \text{VC}$.
- **Example:**

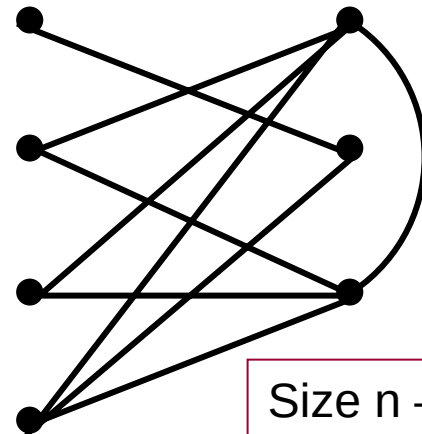
$G = (V, E), k = 4$



Clique of size $k = 4$

Other $n - k = 3$
vertices

$G' = (V, E'), k' = n - k = 3$

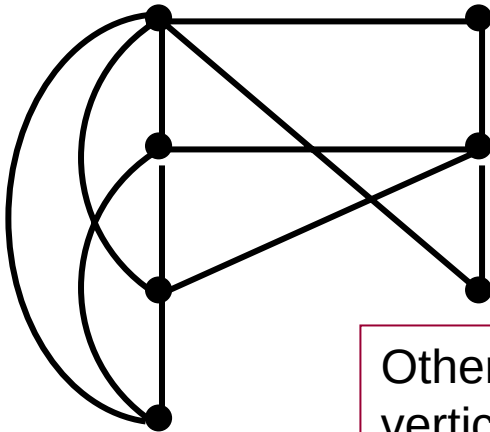


k vertices

Size $n - k$
Vertex cover

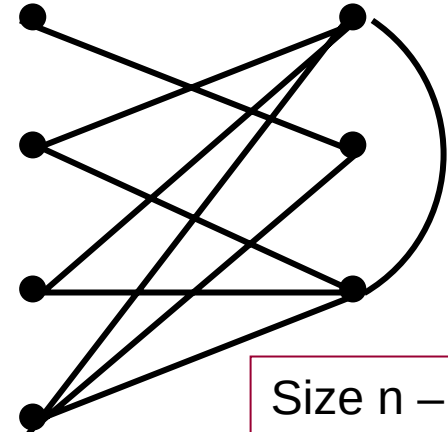
CLIQUE and VERTEX-COVER

- $\langle G, k \rangle \in \text{CLIQUE}$ if and only if $\langle G', k' \rangle \in \text{VC}$.
- **Example:** $G = (V, E)$, $k = 4$, $G' = (V, E')$, $k' = n - k = 3$



Clique of size $k = 4$

Other $n - k = 3$
vertices



k vertices

Size $n - k$
Vertex cover

- $E' = (V \times V) - E$, complement of edge set
- G has clique of size 4 (left nodes), G' has a vertex cover of size $7 - 4 = 3$ (right nodes).
- All edges between 2 nodes on left are in E , hence not in E' , so right nodes cover all edges in E' .

CLIQUE and VERTEX-COVER

- **Theorem:** $\text{CLIQUE} \leq_p \text{VC}$.
- **Proof:**
 - Given input $\langle G, k \rangle$ for CLIQUE, transform to input $\langle G', k' \rangle$ for VC, in poly time, so that $\langle G, k \rangle \in \text{CLIQUE}$ iff $\langle G', k' \rangle \in \text{VC}$.
 - **General transformation:** $f(\langle G, k \rangle)$, where $G = (V, E)$ and $|V| = n$,
 $= \langle G', n-k \rangle$, where $G' = (V, E')$ and $E' = (V \times V) - E$.
 - Transformation is obviously polynomial-time.
 - **Claim:** G has a k -clique iff G' has a size $(n-k)$ vertex cover.
 - **Proof of claim:** Two directions:
 - $\Rightarrow$ Suppose G has a k -clique, show G' has an $(n-k)$ -vc.
 - Suppose C is a k -clique in G .
 - $V - C$ is an $(n-k)$ -vc in G' :
 - Size is obviously right.
 - All edges between nodes in C appear in G , so all are missing in G' .
 - So nodes in $V-C$ cover all edges of G' .

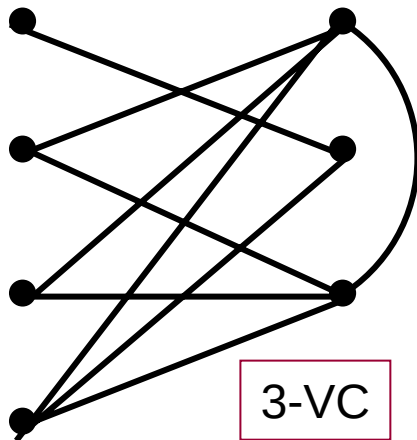
CLIQUE and VERTEX-COVER

- **Theorem:** $\text{CLIQUE} \leq_p \text{VC}$.
- **Proof:**
 - Given input $\langle G, k \rangle$ for CLIQUE, transform to input $\langle G', k' \rangle$ for VC, in poly time, so that $\langle G, k \rangle \in \text{CLIQUE}$ iff $\langle G', k' \rangle \in \text{VC}$.
 - **General transformation:** $f(\langle G, k \rangle)$, where $G = (V, E)$ and $|V| = n$,
 $= \langle G', n-k \rangle$, where $G' = (V, E')$ and $E' = (V \times V) - E$.
 - **Claim:** G has a k -clique iff G' has a size $(n-k)$ vertex cover.
 - **Proof of claim:** Two directions:
 - $\Leftarrow$ Suppose G' has an $(n-k)$ -vc, show G has a k -clique.
 - Suppose D is an $(n-k)$ -vc in G' .
 - $V - D$ is a k -clique in G :
 - Size is obviously right.
 - All edges between nodes in $V-D$ are missing in G' , so must appear in G .
 - So $V-D$ is a clique in G .

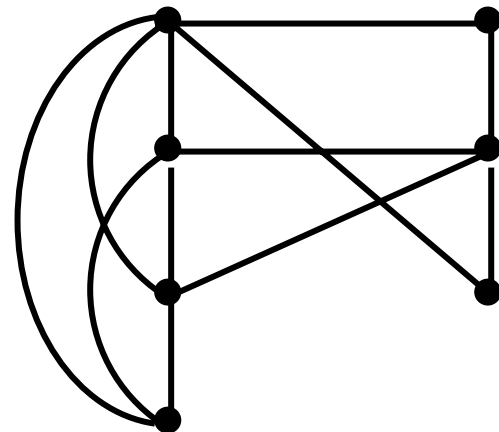
CLIQUE and VERTEX-COVER

- **Theorem:** $VC \leq_p CLIQUE$.
- **Proof:** Almost the same.
 - Given input $\langle G, k \rangle$ for VC, transform to input $\langle G', k' \rangle$ for CLIQUE, in poly time, so that:
 $\langle G, k \rangle \in VC$ if and only if $\langle G', k' \rangle \in CLIQUE$.
- **Example:**

$G = (V, E), k = 3$



$G' = (V, E'), k' = 4$

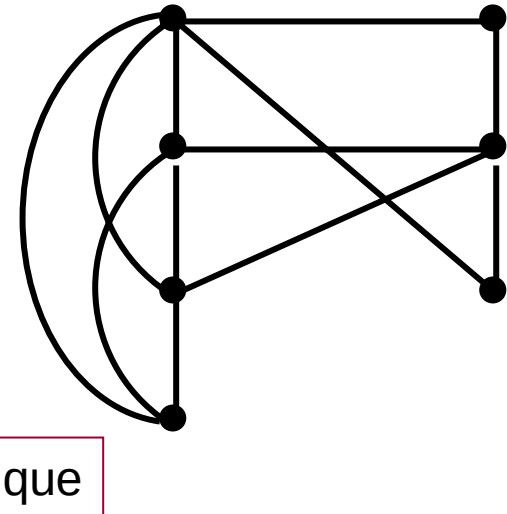
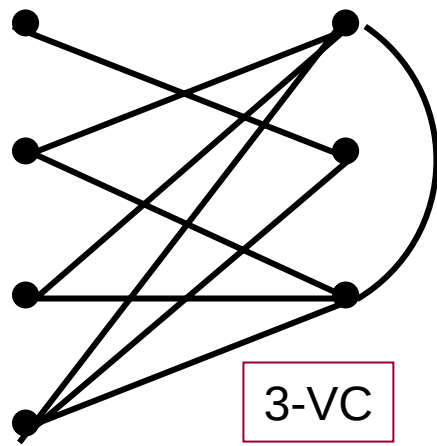


4-clique

CLIQUE and VERTEX-COVER

$\langle G, k \rangle \in \text{VC}$ if and only if $\langle G', k' \rangle \in \text{CLIQUE}$.

- **Example:** $G = (V, E)$, $k = 3$, $G' = (V, E')$, $k' = 4$



- $E' = (V \times V) - E$, complement of edge set
- G has a 3-vc (right nodes), G' has clique of size $7 - 3 = 4$ (left nodes).
- All edges between 2 nodes on left are missing from G , so are in G' , so left nodes form a clique in G' .

CLIQUE and VERTEX-COVER

- **Theorem:** $VC \leq_p \text{CLIQUE}$.
- **Proof:**
 - Given input $\langle G, k \rangle$ for VC, transform to input $\langle G', k' \rangle$ for CLIQUE, in poly time, so that $\langle G, k \rangle \in VC$ iff $\langle G', k' \rangle \in \text{CLIQUE}$.
 - **General transformation:** Same as before.
 $f(\langle G, k \rangle)$, where $G = (V, E)$ and $|V| = n$,
 $= \langle G', n-k \rangle$, where $G' = (V, E')$ and $E' = (V \times V) - E$.
 - **Claim:** G has a k -vc iff G' has an $(n-k)$ -clique.
 - **Proof of claim:** Similar to before, LTTR.

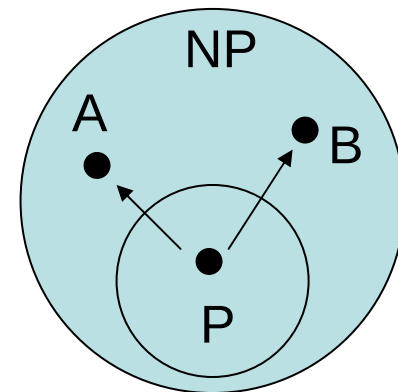
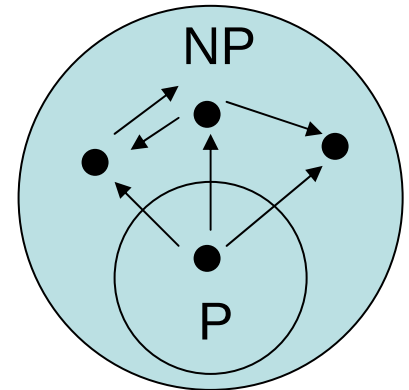
CLIQUE and VERTEX-COVER

- We have shown:
- **Theorem:** $\text{CLIQUE} \leq_p \text{VC}$.
- **Theorem:** $\text{VC} \leq_p \text{CLIQUE}$.
- So, they are essentially equivalent.
- Either both CLIQUE and VC are in P or neither is.

NP-Completeness

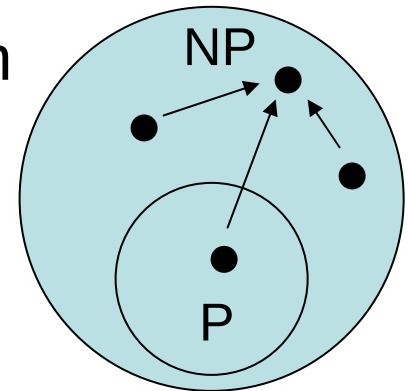
NP-Completeness

- $\leq_p$ allows us to relate problems in NP, saying which allow us to solve which others efficiently.
- Even though we don't know whether all of these problems are in P, we can use $\leq_p$ to impose some structure on the class NP:
- $A \rightarrow B$ here means $A \leq_p B$.
- Sets in $NP - P$ might not be totally ordered by $\leq_p$: we might have A, B with neither $A \leq_p B$ nor $B \leq_p A$:



NP-Completeness

- Some languages in NP are **hardest**, in the sense that every language in NP is $\leq_p$ -reducible to them.
- Call these **NP-complete**.
- **Definition:** Language B is **NP-complete** if both of the following hold:
 - (a) $B \in \text{NP}$, and
 - (b) For any language $A \in \text{NP}$, $A \leq_p B$.



- Sometimes, we consider languages that aren't, or might not be, in NP, but to which all NP languages are reducible.
- Call these **NP-hard**.
- **Definition:** Language B is **NP-hard** if, for any language $A \in \text{NP}$, $A \leq_p B$.

NP-Completeness

- Today, and next time, we'll:
 - Give examples of interesting problems that are NP-complete, and
 - Develop methods for showing NP-completeness.
- **Theorem:** $\exists B$, B is NP-complete.
 - There is at least one NP-complete problem.
 - We'll show this later.
- **Theorem:** If A , B , are NP-complete, then $A \leq_p B$.
 - Two NP-complete problems are essentially equivalent (up to $\leq_p$).
- **Proof:** $A \in \text{NP}$, B is NP-hard, so $A \leq_p B$ by definition.

NP-Completeness

- **Theorem:** If some NP-complete language is in P , then $P = NP$.
 - That is, if a polynomial-time algorithm exists for any NP-complete problem, then the entire class NP collapses into P .
 - Polynomial algorithms immediately arise for **all** problems in NP .
- **Proof:**
 - Suppose B is NP-complete and $B \in P$.
 - Let A be any language in NP ; show $A \in P$.
 - We know $A \leq_p B$ since B is NP-complete.
 - Then $A \in P$, since $B \in P$ and “easiness propagates downward”.
 - Since every A in NP is also in P , $NP \subseteq P$.
 - Since $P \subseteq NP$, it follows that $P = NP$.

NP-Completeness

- **Theorem:** The following are equivalent.

1. $P = NP$.
2. Every NP-complete language is in P .
3. Some NP-complete language is in P .

- **Proof:**

1 $\Rightarrow$ 2:

- Assume $P = NP$, and suppose that B is NP-complete.
- Then $B \in NP$, so $B \in P$, as needed.

2 $\Rightarrow$ 3:

- Immediate because there is at least NP-complete language.

3 $\Rightarrow$ 1:

- By the previous theorem.

Beliefs about P vs. NP

- Most theoretical computer scientists believe $P \neq NP$.
- Why?
- Many interesting NP-complete problems have been discovered over the years, and many smart people have tried to find fast algorithms; no one has succeeded.
- The problems have arisen in many different settings, including logic, graph theory, number theory, operations research, games and puzzles.
- Entire book devoted to them [Garey, Johnson].
- All these problems are essentially the same since all NP-complete problems are polynomial-reducible to each other.
- So essentially the same problem has been studied in many different contexts, by different groups of people, with different backgrounds, using different methods.

Beliefs about P vs. NP

- Most theoretical computer scientists believe $P \neq NP$.
- Because many smart people have tried to find fast algorithms and no one has succeeded.
- That doesn't mean $P \neq NP$; this is just some kind of empirical evidence.
- The essence of why NP-complete problems seem hard:
 - They have NP structure:
$$x \in L \text{ iff } (\exists c, |c| \leq p(|x|)) [V(x, c) \text{ accepts }],$$
where V is poly-time.
 - Guess and verify.
 - Seems to involve exploring a tree of possible choices, exponential blowup.
- However, no one has yet succeeded in proving that they actually are hard!
 - We don't have sharp enough methods.
 - So in the meantime, we just show problems are NP-complete.

Satisfiability is NP-Complete

Satisfiability is NP-Complete

- $SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
- **Definition:** (Boolean formula):
 - **Variables:** $x, x_1, x_2, \dots, y, \dots, z, \dots$
 - Can take on values 1 (true) or 0 (false).
 - **Literal:** A variable or its negated version: $x, \neg x, \neg x_1, \dots$
 - **Operations:** $\wedge \vee \neg$
 - **Boolean formula:** Constructed from literals using operations, e.g.:
$$\phi = x \wedge ((y \wedge z) \vee (\neg y \wedge \neg z)) \wedge \neg (x \wedge z)$$
- **Definition:** (Satisfiability):
 - A Boolean formula is satisfiable iff there is an assignment of 0s and 1s to the variables that makes the entire formula evaluate to 1 (true).

Satisfiability is NP-Complete

- $SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
- **Boolean formula:** Constructed from literals using operations, e.g.:
$$\phi = x \wedge ((y \wedge z) \vee (\neg y \wedge \neg z)) \wedge \neg(x \wedge z)$$
- A Boolean formula is satisfiable iff there is an assignment of 0s and 1s to the variables that makes the entire formula evaluate to 1 (true).
- **Example:** ϕ above
 - Satisfiable, using the assignment $x = 1, y = 0, z = 0$.
 - So $\phi \in SAT$.
- **Example:** $x \wedge ((y \wedge z) \vee (\neg y \wedge z)) \wedge \neg(x \wedge z)$
 - Not in SAT.
 - x must be set to 1, so z must = 0.

Satisfiability is NP-Complete

- $SAT = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
- **Theorem:** SAT is NP-complete.
- **Lemma 1:** $SAT \in NP$.
- **Lemma 2:** SAT is NP-hard.
- **Proof of Lemma 1:**
 - Recall: $L \in NP$ if and only if $(\exists V, \text{poly-time verifier}) (\exists p, \text{poly})$
 $x \in L$ iff $(\exists c, |c| \leq p(|x|)) [V(x, c) \text{ accepts}]$
 - So, to show $SAT \in NP$, it's enough to show $(\exists V) (\exists p)$
 $\phi \in SAT$ iff $(\exists c, |c| \leq p(|\phi|)) [V(\phi, c) \text{ accepts}]$
 - We know: $\phi \in SAT$ iff there is an assignment to the variables such that ϕ with this assignment evaluates to 1.
 - So, let certificate c be the assignment.
 - Let verifier V take a formula ϕ and an assignment c and accept exactly if ϕ with c evaluates to true.
 - Evaluate ϕ bottom-up, takes poly time.

Satisfiability is NP-Complete

- Lemma 2: SAT is NP-hard.
- Proof of Lemma 2:
 - Need to show that, for any $A \in \text{NP}$, $A \leq_p \text{SAT}$.
 - Fix $A \in \text{NP}$.
 - Construct a poly-time f such that
$$w \in A \text{ if and only if } f(w) \in \text{SAT}.$$

A formula, write it as ϕ_w .
 - By definition, since $A \in \text{NP}$, there is a nondeterministic TM M that decides A in polynomial time.
 - Fix polynomial p such that M on input w always halts, on all branches, in time $\leq p(|w|)$; assume $p(|w|) \geq |w|$.
 - $w \in A$ if and only if there is an accepting computation history (CH) of M on w .

Satisfiability is NP-Complete

- **Lemma 2:** SAT is NP-hard.
- **Proof, cont'd:**
 - Need $w \in A$ if and only if $f(w)$ ($= \phi_w$) $\in$ SAT.
 - $w \in A$ if and only if there is an accepting CH of M on w .
 - So we must construct formula ϕ_w to be satisfiable iff there is an accepting CH of M on w .
 - Recall definitions of **computation history** and **accepting computation history** from Post Correspondence Problem:
C_0 # C_1 # C_2 ...
 - Configurations include tape contents, state, head position.
 - We construct ϕ_w to describe an accepting CH.
 - Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$ as usual.
 - Instead of lining up configs in a row as before, arrange in $(p(|w|) + 1)$ row $\times$ $(p(|w|) + 3)$ column matrix:

Proof that SAT is NP-hard

- ϕ_w will be satisfiable iff there is an accepting CH of M on w .
- Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$.

- Arrange configs in $(p(|w|) + 1) \times (p(|w|) + 3)$ matrix:

#	q_0	w_1	w_2	w_3	...	w_n	--	--	...	--	#
#	...										#
#	...										#
⋮											⋮
#	...										#

- Successive configs, ending with accepting config.
- Assume WLOG that each computation takes exactly $p(|w|)$ steps, so we use $p(|w|) + 1$ rows.
- $p(|w|) + 3$ columns: $p(|w|)$ for the interesting portion of the tape, one for head and state, two for endmarkers.

Proof that SAT is NP-hard

- ϕ_w is satisfiable iff there is an accepting CH of M on w .
- Entries in the matrix are represented by Boolean variables:
 - Define $C = Q \cup \Gamma \cup \{ \# \}$, alphabet of possible matrix entries.
 - Variable $x_{i,j,c}$ represents “the entry in position (i, j) is c ”.
- Define ϕ_w as a formula over these $x_{i,j,c}$ variables, satisfiable if and only if there is an accepting computation history for w (in matrix form).
- Moreover, an assignment of values to the $x_{i,j,c}$ variables that satisfies ϕ_w will correspond to an encoding of an accepting computation.
- Specifically, $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$, where:
 - ϕ_{cell} : There is exactly one value in each matrix location.
 - ϕ_{start} : The first row represents the starting configuration.
 - ϕ_{accept} : The last row is an accepting configuration.
 - ϕ_{move} : Successive rows represent allowable moves of M .

ϕ_{cell}

- For each position (i,j) , write the conjunction of two formulas:

$\bigvee_{c \in C} x_{i,j,c}$: Some value appears in position (i,j) .

$\bigwedge_{c, d \in C, c \neq d} (\neg x_{i,j,c} \vee \neg x_{i,j,d})$: Position (i,j) doesn't contain two values.

- ϕ_{cell} : Conjoin formulas for all positions (i,j) .
- Easy to construct the entire formula ϕ_{cell} given w input.
- Construct it in polynomial time.
- Sanity check: Length of formula is polynomial in $|w|$:
 - $O(p(|w|)^2)$ subformulas, one for each (i,j) .
 - Length of each subformula depends on C , $O(|C|^2)$.

ϕ_{start}

- The right symbols appear in the first row:

q_0 w_1 w_2 w_3 ... w_n -- -- ... --

$$\begin{aligned} \phi_{\text{start}}: & \quad X_{1,1,\#} \wedge X_{1,2,q_0} \wedge X_{1,3,w_1} \wedge X_{1,4,w_2} \wedge \dots \\ & \quad \wedge X_{1,n+2,w_n} \wedge X_{1,n+3,--} \wedge \dots \\ & \quad \wedge X_{1,p(n)+2,--} \wedge X_{1,p(n)+3,\#} \end{aligned}$$

ϕ_{accept}

- For each j , $2 \leq j \leq p(|w|) + 2$, write the formula:

$$x_{p(|w|)+1,j,q_{\text{acc}}}$$

- q_{acc} appears in position j of the last row.
- ϕ_{accept} : Take disjunction (or) of all formulas for all j .
- That is, q_{acc} appears in some position of the last row.

ϕ_{move}

- As for PCP, correct moves depend on correct changes to local portions of configurations.
- It's enough to consider 2×3 rectangles:
- If every 2×3 rectangle is “good”, i.e., consistent with the transitions, then the entire matrix represents an accepting CH.
- For each position (i,j) , $1 \leq i \leq p(|w|)$, $1 \leq j \leq p(|w|)+1$, write a formula saying that the rectangle with upper left at (i,j) is “good”.
- Then conjoin all of these, $O(p(|w|)^2)$ clauses.
- Good tiles for (i,j) , for a, b, c in Γ :

a	b	c
a	b	c

#	a	b
#	a	b

a	b	#
a	b	#

ϕ_{move}

- Other good tiles are defined in terms of the nondeterministic transition function δ .
- E.g., if $\delta(q_1, a)$ includes tuple (q_2, b, L) , then the following are good:
 - Represents the move directly; for any c:
 - Head moves left out of the rectangle; for any c, d:
 - Head is just to the left of the rectangle; for any c, d:
 - Head at right; for any c, d, e:
 - And more, for #, etc.
- Analogously if $\delta(q_1, a)$ includes (q_2, b, R) .
- Since M is nondeterministic, $\delta(q_1, a)$ may contain several moves, so include all the tiles.

c	q_1	a
q_2	c	b

q_1	a	c
d	b	c

a	c	d
b	c	d

d	c	q_1
d	q_2	c

e	d	c
e	d	q_2

ϕ_{move}

- The good tiles give partial constraints on the computation.
- When taken together, they give enough constraints so that only a correct CH can satisfy them all.
- The part (conjunct) of ϕ_{move} for (i,j) should say that the rectangle with upper left at (i,j) is good:
- It is simply the disjunction (or), over all allowable tiles, of the subformula:

a1	a2	a3
b1	b2	b3

$$X_{i,j,a1} \wedge X_{i,j+1,a2} \wedge X_{i,j+2,a3} \wedge X_{i+1,j,b1} \wedge X_{i+1,j+1,b2} \wedge X_{i+1,j+2,b3}$$

- Thus, ϕ_{move} is the conjunction over all (i,j) , of the disjunction over all good tiles, of the formula just above.

ϕ_{move}

- ϕ_{move} is the conjunction over all (i,j) , of the disjunction over all good tiles, of the given six-term conjunctive formula.
- **Q:** How big is the formula ϕ_{move} ?
- $O(p(|w|)^2)$ clauses, one for each (i,j) pair.
- Each clause is only constant length, $O(1)$.
 - Because machine M yields only a constant number of good tiles.
 - And there are only 6 terms for each tile.
- Thus, length of ϕ_{move} is polynomial in $|w|$.
- $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$, length also poly in $|w|$.

ϕ_{move}

- $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$, length poly in $|w|$.
- More importantly, can produce ϕ_w from w in time that is polynomial in $|w|$.
- $w \in A$ if and only if M has an accepting CH for w if and only if ϕ_w is satisfiable.
- Thus, $A \leq_p \text{SAT}$.
- Since A was any language in NP, this proves that SAT is NP-hard.
- Since SAT is in NP and is NP-hard, SAT is NP-complete.

Next time...

- NP-completeness---more examples
- Reading:
 - Sipser Sections 7.4-7.5

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and Complexity (GITCS)

Class 16
Nancy Lynch

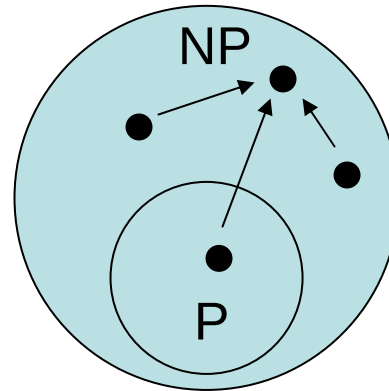
Today: More NP-Completeness

- **Topics:**
 - 3SAT is NP-complete
 - Clique and VertexCover are NP-complete
 - More examples, overview
 - Hamiltonian path and Hamiltonian circuit
 - Traveling Salesman problem
 - More examples, revisited
- **Reading:**
 - Sipser Sections 7.4-7.5
 - Garey and Johnson
- **Next:**
 - Sipser Section 10.2

3SAT is NP-Complete

NP-Completeness

- **Definition:** Language B is **NP-complete** if both of the following hold:
 - (a) $B \in \text{NP}$, and
 - (b) For any language $A \in \text{NP}$, $A \leq_p B$.



- **Definition:** Language B is **NP-hard** if, for any language $A \in \text{NP}$, $A \leq_p B$.

3SAT is NP-Complete

- **SAT** = $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula} \}$
- **Boolean formula**: Constructed from literals using operations, e.g.:
$$\phi = x \wedge ((y \wedge z) \vee (\neg y \wedge \neg z)) \wedge \neg(x \wedge z)$$
- A Boolean formula is satisfiable iff there is an assignment of 0s and 1s to the variables that makes the entire formula evaluate to 1 (true).
- **Theorem**: SAT is NP-complete.
- **3SAT**: Satisfiable Boolean formulas of a restricted kind---conjunctive normal form (CNF) with exactly 3 literals per clause.
- **Theorem**: 3SAT is NP-complete.
- **Proof**:
 - 3SAT $\in$ NP: Obvious.
 - 3SAT is NP-hard: ...

3SAT is NP-hard

- **Clause:** Disjunction of literals, e.g., $(\neg x_1 \vee x_2 \vee \neg x_3)$
- **CNF:** Conjunction of such clauses
- **Example:**
$$(\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge (x_3)$$
- **3-CNF:**
$$\{ \langle \phi \rangle \mid \phi \text{ is a CNF formula in which each clause has exactly 3 literals} \}$$
- **CNF-SAT:** $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable CNF formula} \}$
- **3-SAT:** $\{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 3-CNF formula} \}$
$$= \text{SAT} \cap \text{3-CNF}$$
- **Theorem:** 3SAT is NP-hard.
- **Proof:** Show CNF-SAT is NP-hard, and $\text{CNF-SAT} \leq_p \text{3SAT}$.

CNF-SAT is NP-hard

- Theorem: CNF-SAT is NP-hard.
- Proof:
 - We won't show $\text{SAT} \leq_p \text{CNF-SAT}$.
 - Instead, modify the proof that SAT is NP-hard, so that it shows $A \leq_p \text{CNF-SAT}$, for an arbitrary A in NP, instead of just $A \leq_p \text{SAT}$ as before.
 - We've almost done this: formula ϕ_w is almost in CNF.
 - It's a conjunction $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$.
 - And each of these is itself in CNF, except ϕ_{move} .
 - ϕ_{move} is:
 - a conjunction over all (i,j)
 - of disjunctions over all tiles
 - of conjunctions of 6 conditions on the 6 cells:

$$X_{i,j,a1} \wedge X_{i,j+1,a2} \wedge X_{i,j+2,a3} \wedge X_{i+1,j,b1} \wedge X_{i+1,j+1,b2} \wedge X_{i+1,j+2,b3}$$

CNF-SAT is NP-hard

- Show $A \leq_p \text{CNF-SAT}$.
- ϕ_w is a conjunction $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$, where each is in CNF, except ϕ_{move} .
- ϕ_{move} is:
 - a conjunction ($\wedge$) over all (i,j)
 - of disjunctions ($\vee$) over all tiles
 - of conjunctions ($\wedge$) of 6 conditions on the 6 cells:
$$X_{i,j,a1} \wedge X_{i,j+1,a2} \wedge X_{i,j+2,a3} \wedge X_{i+1,j,b1} \wedge X_{i+1,j+1,b2} \wedge X_{i+1,j+2,b3}$$
- We want just $\wedge$ of $\vee$.
- Can use distributive laws to replace ($\vee$ of $\wedge$) with ($\wedge$ of $\vee$), which would yield overall $\wedge$ of $\vee$, as needed.
- In general, transforming ($\vee$ of $\wedge$) to ($\wedge$ of $\vee$), could cause formula size to grow too much (exponentially).
- However, in this situation, the clauses for each (i,j) have total size that depends only on the TM M , and not on w .
- So the size of the transformed formula is still poly in $|w|$.

CNF-SAT is NP-hard

- Theorem: CNF-SAT is NP-hard.
- Proof:
 - Modify the proof that SAT is NP-hard.
 - $\phi_w = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}}$.
 - Can be put into CNF, while keeping the size of the transformed formula poly in $|w|$.
 - Shows that $A \leq_p \text{CNF-SAT}$.
 - Since A is any language in NP, CNF-SAT is NP-hard.

3SAT is NP-hard

- Proved: **Theorem:** CNF-SAT is NP-hard.
- Now: **Theorem:** 3SAT is NP-hard.
- **Proof:**
 - Use reduction, show $\text{CNF-SAT} \leq_p \text{3SAT}$.
 - Construct f , polynomial-time computable, such that $w \in \text{CNF-SAT}$ if and only if $f(w) \in \text{3SAT}$.
 - If w isn't a CNF formula, then $f(w)$ isn't either.
 - If w is a CNF formula, then $f(w)$ is another CNF formula, this one with 3 literals per clause, satisfiable iff w is satisfiable.
 - f works by converting each clause to a conjunction of clauses, each with ≤ 3 literals (add repeats to get 3).
 - Show by example: $(a \vee b \vee c \vee d \vee e)$ gets converted to
$$(a \vee r_1) \wedge (\neg r_1 \vee b \vee r_2) \wedge (\neg r_2 \vee c \vee r_3) \wedge (\neg r_3 \vee d \vee r_4) \wedge (\neg r_4 \vee e)$$
 - f is polynomial-time computable.

3SAT is NP-hard

- **Proof:**

- Show $\text{CNF-SAT} \leq_p \text{3SAT}$.
- Construct f such that $w \in \text{CNF-SAT}$ iff $f(w) \in \text{3SAT}$; converts each clause to a conjunction of clauses.
- f converts $w = (a \vee b \vee c \vee d \vee e)$ to $f(w) = (a \vee r_1) \wedge (\neg r_1 \vee b \vee r_2) \wedge (\neg r_2 \vee c \vee r_3) \wedge (\neg r_3 \vee d \vee r_4) \wedge (\neg r_4 \vee e)$
- **Claim w is satisfiable iff $f(w)$ is satisfiable.**

- $\Rightarrow$:

- Given a satisfying assignment for w , add values for $r_1, r_2, \dots$, to satisfy $f(w)$.
- Start from a clause containing a literal with value 1---there must be one---make the new literals in that clause 0 and propagate consequences left and right.
- Example: Above, if $c = 1$, $a = b = d = e = 0$ satisfy w , use:

$$(a \vee r_1) \wedge (\neg r_1 \vee b \vee r_2) \wedge (\neg r_2 \vee c \vee r_3) \wedge (\neg r_3 \vee d \vee r_4) \wedge (\neg r_4 \vee e)$$

0	1	0	0	1	0	1	0	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---

3SAT is NP-hard

- **Proof:**

- Show $\text{CNF-SAT} \leq_p \text{3SAT}$.
- Construct f such that $w \in \text{CNF-SAT}$ iff $f(w) \in \text{3SAT}$; converts each clause to a conjunction of clauses.
- f converts $w = (a \vee b \vee c \vee d \vee e)$ to $f(w) = (a \vee r_1) \wedge (\neg r_1 \vee b \vee r_2) \wedge (\neg r_2 \vee c \vee r_3) \wedge (\neg r_3 \vee d \vee r_4) \wedge (\neg r_4 \vee e)$
- **Claim w is satisfiable iff $f(w)$ is satisfiable.**

- $\Leftarrow$:

- Given satisfying assignment for $f(w)$, restrict to satisfy w .
- Each r_i can make only one clause true.
- There's one fewer r_i than clauses; so some clause must be made true by an original literal, i.e., some original literal must be true, satisfying w .

3SAT is NP-hard

- **Theorem:** CNF-SAT is NP-hard.
- **Theorem:** 3SAT is NP-hard.
- **Proof:**
 - Constructed polynomial-time-computable f such that $w \in \text{CNF-SAT}$ iff $f(w) \in \text{3SAT}$.
 - Thus, $\text{CNF-SAT} \leq_p \text{3SAT}$.
 - Since CNF-SAT is NP-hard, so is 3SAT.

CLIQUE and VERTEX-COVER are
NP-Complete

CLIQUE and VERTEX-COVER

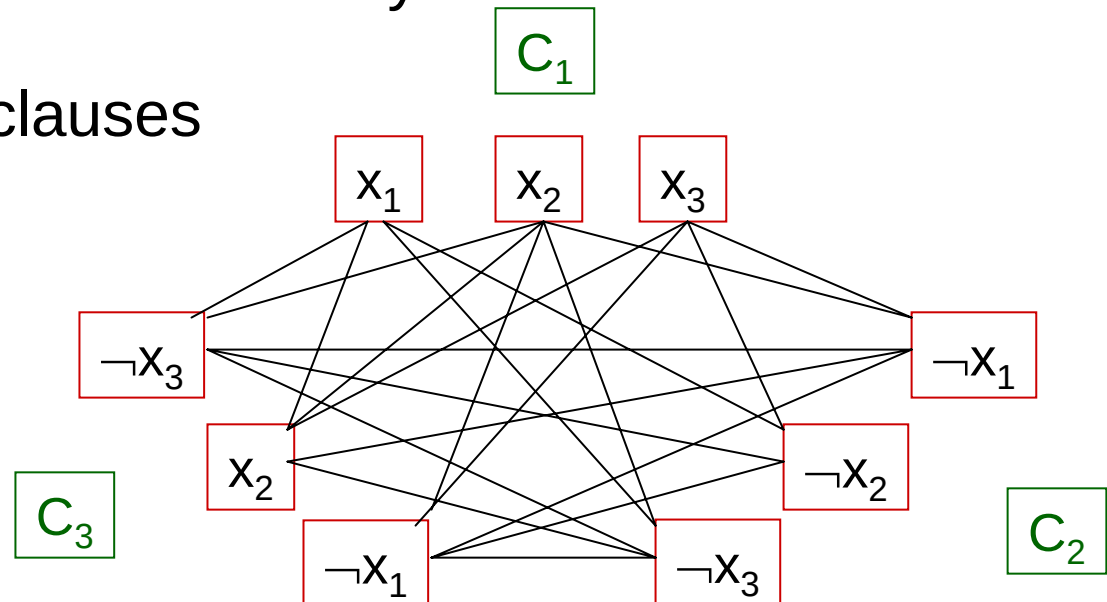
- **CLIQUE** = $\{ \langle G, k \rangle \mid G \text{ is a graph with a } k\text{-clique} \}$
- **k-clique**: k vertices with edges between all pairs in the clique.
- **Theorem**: CLIQUE is NP-complete.
- **Proof**:
 - CLIQUE $\in$ NP, already shown.
 - To show CLIQUE is NP-hard, show $3\text{SAT} \leq_p \text{CLIQUE}$.
 - Need poly-time-computable f , such that $w \in 3\text{SAT}$ iff $f(w) \in \text{CLIQUE}$.
 - f must map a formula w in 3-CNF to $\langle G, k \rangle$ such that w is satisfiable iff G has a k -clique.
 - Show by example:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

CLIQUE is NP-hard

- **Proof:**

- Show $3SAT \leq_p CLIQUE$; construct f such that $w \in 3SAT$ iff $f(w) \in CLIQUE$.
- f maps a formula w in 3-CNF to $\langle G, k \rangle$ such that w is satisfiable iff G has a k -clique.
- $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$
- **Graph G :** Nodes for all (clause, literal) pairs, edges between all non-contradictory nodes in different clauses.
- **k :** Number of clauses

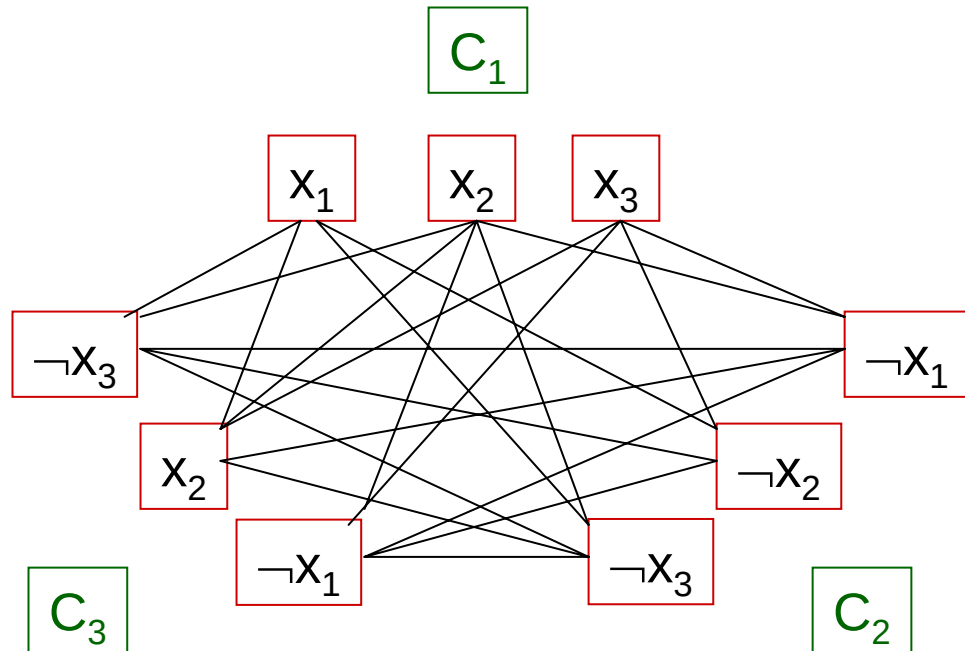


CLIQUE is NP-hard

- **Graph G:** Nodes for all (clause, literal) pairs, edges between all non-contradictory nodes in different clauses.
- **k:** Number of clauses
 $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$
- Claim (general): ϕ satisfiable iff G has a k-clique.

• $\Rightarrow$:

- Assume the formula is satisfiable.
- Satisfying assignment gives one literal in each clause, all with non-contradictory assignments.
- Yields a k-clique.



CLIQUE is NP-hard

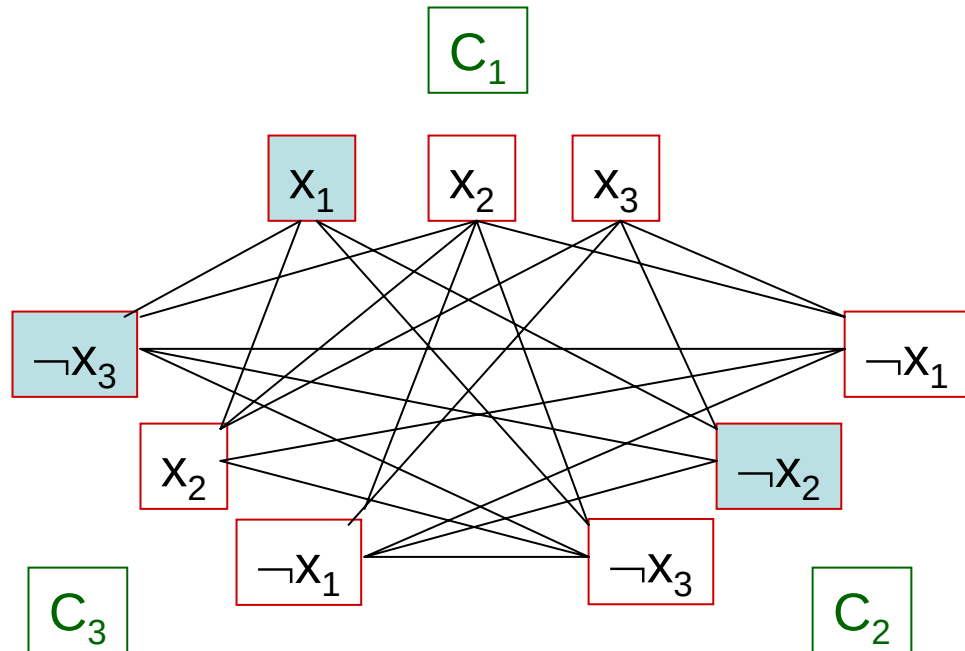
- Example:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$$

- Satisfiable, with satisfying assignment $x_1 = 1, x_2 = x_3 = 0$
- Yields 3-clique:

- $\Rightarrow$:

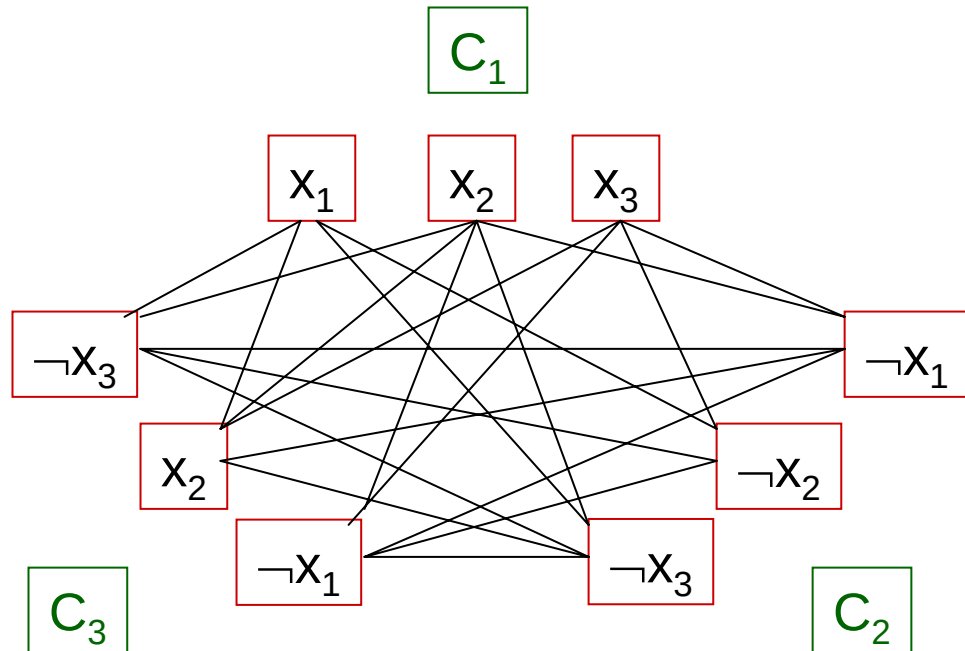
- Assume the formula is satisfiable.
- Satisfying assignment gives one literal in each clause, all with non-contradictory assignments.
- Yields a k-clique.



CLIQUE is NP-hard

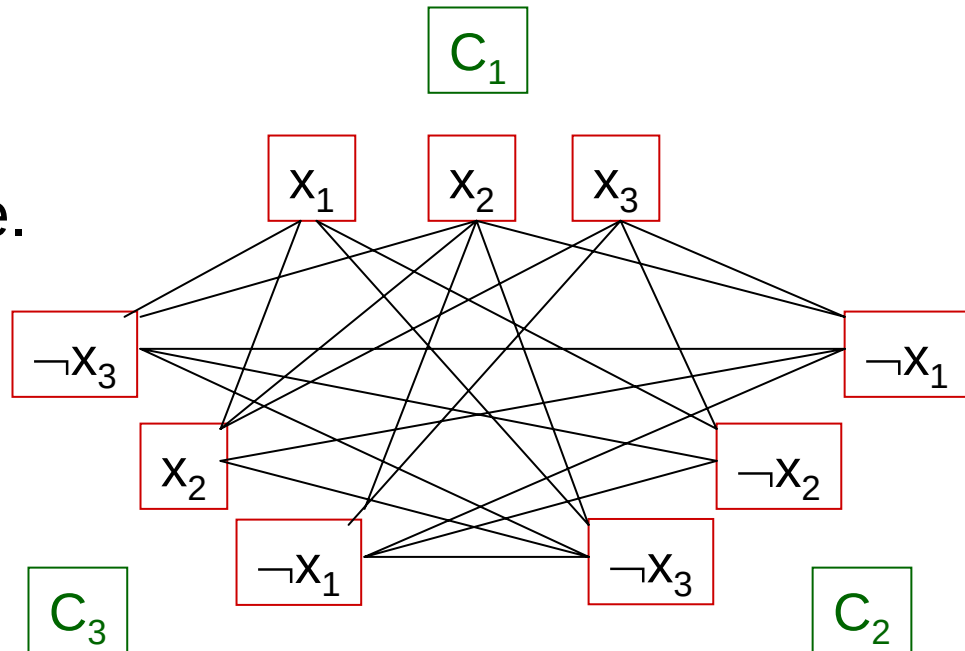
- **Graph G:** Nodes for all (clause, literal) pairs, edges between all non-contradictory nodes in different clauses.
- **k:** Number of clauses
 $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$
- Claim (general): w satisfiable iff G has a k -clique.
- $\Leftarrow$:

- Assume a k -clique.
- Yields one node per clause, none contradictory.
- Yields a consistent assignment satisfying all clauses of w .



CLIQUE is NP-hard

- **Graph G:** Nodes for all (clause, literal) pairs, edges between all non-contradictory nodes in different clauses.
- **k:** Number of clauses
- Claim (general): ϕ satisfiable iff G has a k-clique.
- So, $3SAT \leq_p CLIQUE$.
- Since 3SAT is NP-hard, so is CLIQUE.
- So CLIQUE is NP-complete.



VERTEX-COVER is NP-complete

- **VERTEX-COVER** =
 $\{ \langle G, k \rangle \mid G \text{ is a graph with a vertex cover of size } k \}$
- **Vertex cover of $G = (V, E)$:** A subset C of V such that, for every edge (u,v) in E , either u or $v \in C$.
- **Theorem:** VERTEX-COVER is NP-complete.
- **Proof:**
 - VERTEX-COVER $\in$ NP, already shown.
 - Show VERTEX-COVER is NP-hard.
 - That is, if $A \in$ NP, then $A \leq_p$ VERTEX-COVER.
 - We know $A \leq_p$ CLIQUE, since CLIQUE is NP-hard.
 - Recall $\text{CLIQUE} \leq_p \text{VERTEX-COVER}$.
 - By transitivity of $\leq_p$, $A \leq_p \text{VERTEX-COVER}$, as needed.

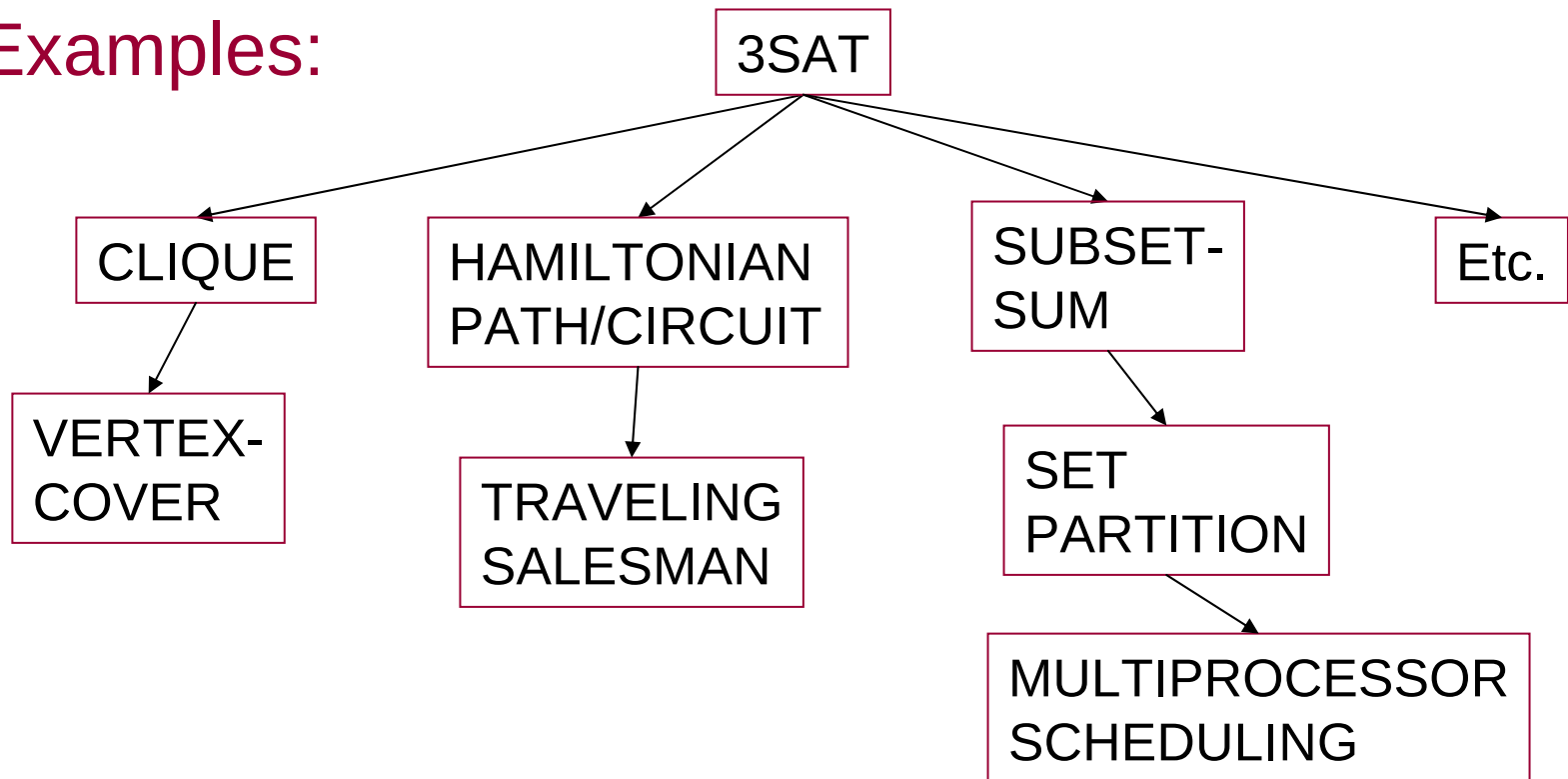
VERTEX-COVER is NP-complete

- **Theorem:** VERTEX-COVER is NP-complete.
- **More succinct proof:**
 - $VC \in NP$; show VC is NP-hard.
 - CLIQUE is NP-hard.
 - $CLIQUE \leq_p VC$.
 - So VC is NP-hard.
- In general, can show language B is NP-complete by:
 - Showing $B \in NP$, and
 - Showing $A \leq_p B$ for some known NP-hard problem A.

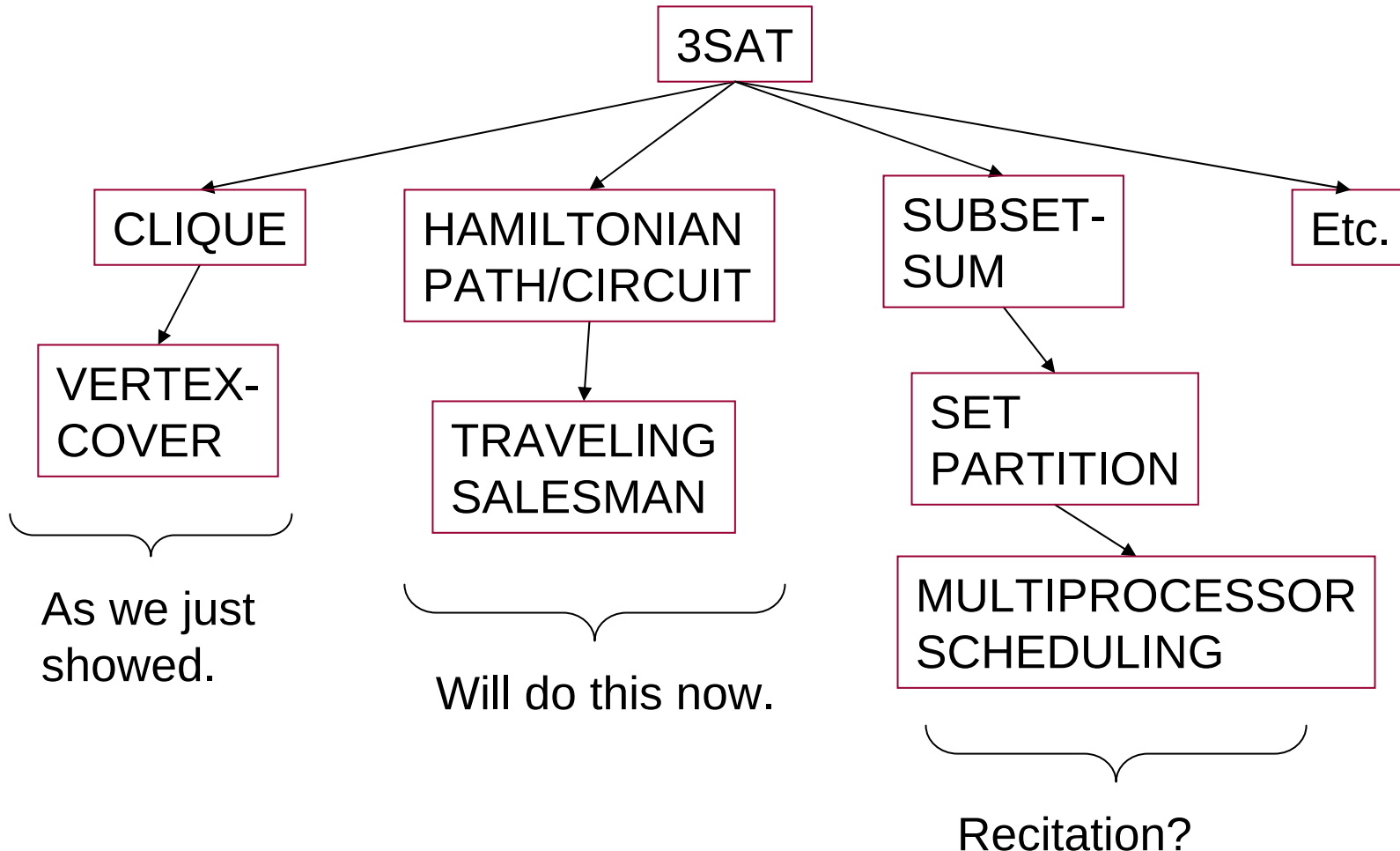
More Examples

More NP-Complete Problems

- [Garey, Johnson] show hundreds of problems are NP-complete.
- All but 3SAT use the polynomial-time reduction method.
- Examples:



More NP-Complete Problems

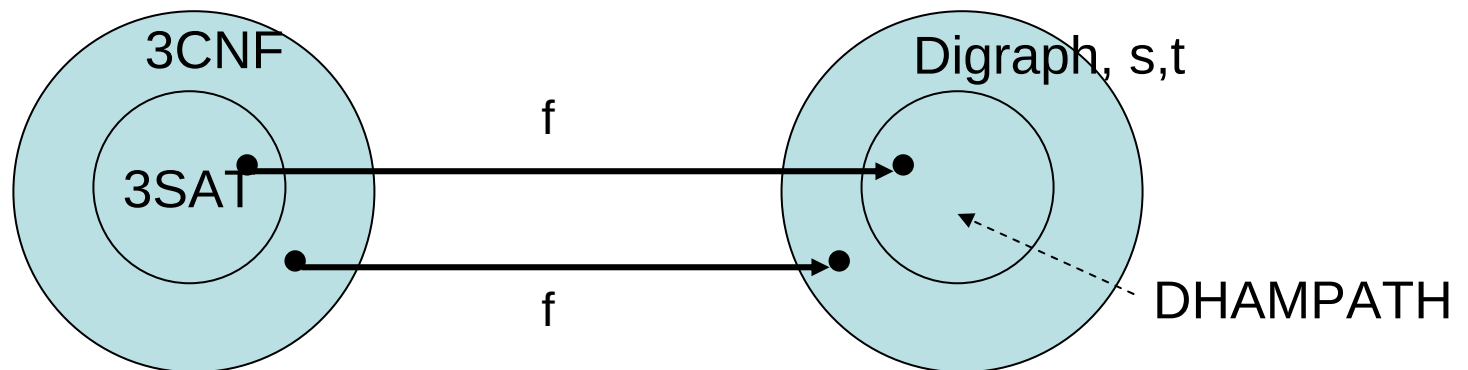


- $A \rightarrow B$ means $A \leq_p B$.
- Hardness propagates to the right in $\leq_p$, downward along tree branches.

3SAT $\leq_p$ HAMILTONIAN
PATH/CIRCUIT

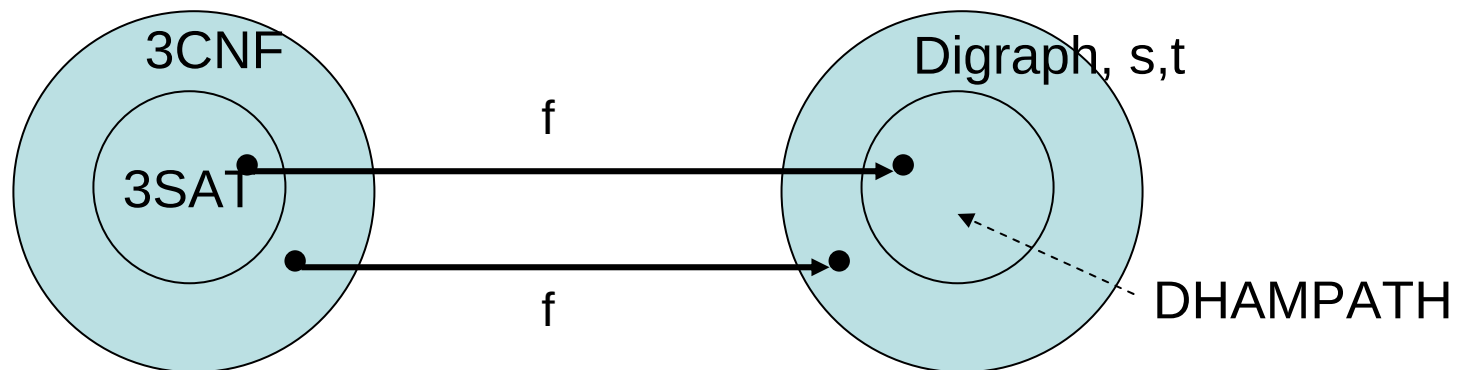
$3SAT \leq_p \text{HAMILTONIAN PATH/CIRCUIT}$

- Two versions of the problem, for directed and undirected graphs.
- Consider directed version; undirected shown by reduction from directed version.
- **DHAMPATH** = $\{ \langle G, s, t \rangle \mid G \text{ is a directed graph, } s \text{ and } t \text{ are two distinct vertices, and there is a path from } s \text{ to } t \text{ in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- **DHAMPATH** $\in$ NP: Guess path and verify.
- **$3SAT \leq_p \text{DHAMPATH}$** :



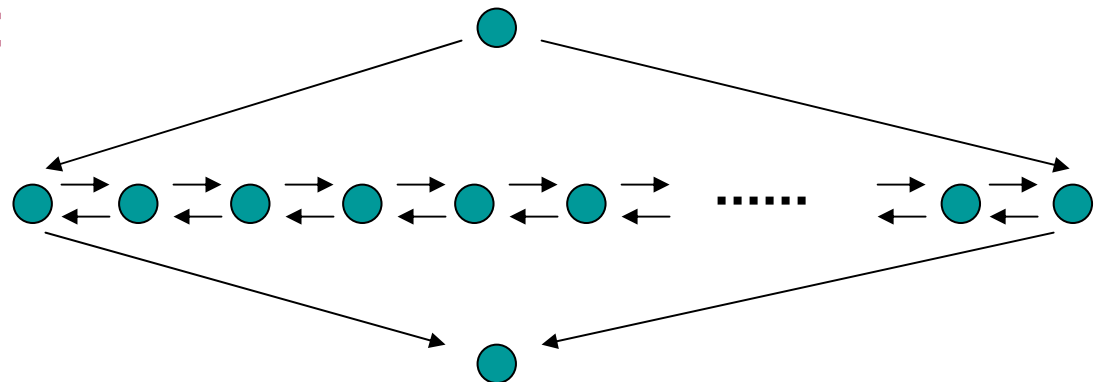
$3SAT \leq_p \text{HAMILTONIAN PATH/CIRCUIT}$

- **DHAMPATH** = $\{ \langle G, s, t \rangle \mid G \text{ is a directed graph, } s \text{ and } t \text{ are two distinct vertices, and there is a path from } s \text{ to } t \text{ in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- **$3SAT \leq_p \text{DHAMPATH}$:**
 - Map a 3CNF formula ϕ to $\langle G, s, t \rangle$ so that ϕ is satisfiable if and only if G has a Hamiltonian path from s to t .
 - In fact, there will be a direct correspondence between a satisfying assignment for ϕ and a Hamiltonian path in G .



$3SAT \leq_p DHAMPATH$

- Map a 3CNF formula ϕ to $\langle G, s, t \rangle$ so that ϕ is satisfiable if and only if G has a Hamiltonian path from s to t .
- Correspondence between satisfying assignment for ϕ and Hamiltonian path in G .
- **Notation:**
 - Write $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$
 - k clauses $C_1, C_2, \dots, C_k$
 - Variables: $x_1, x_2, \dots, x_l$
 - Each a_j, b_j , and c_j is either some x_i or some $\neg x_i$.
- Digraph is constructed from pieces (gadgets), one for each variable x_i and one for each clause C_j .
- **Gadget for variable x_i :**

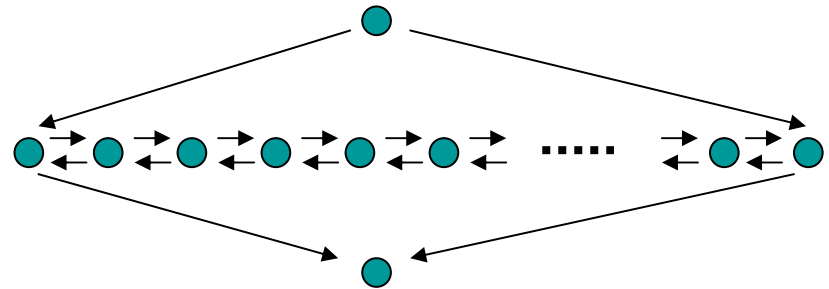


$3SAT \leq_p DHAMPATH$

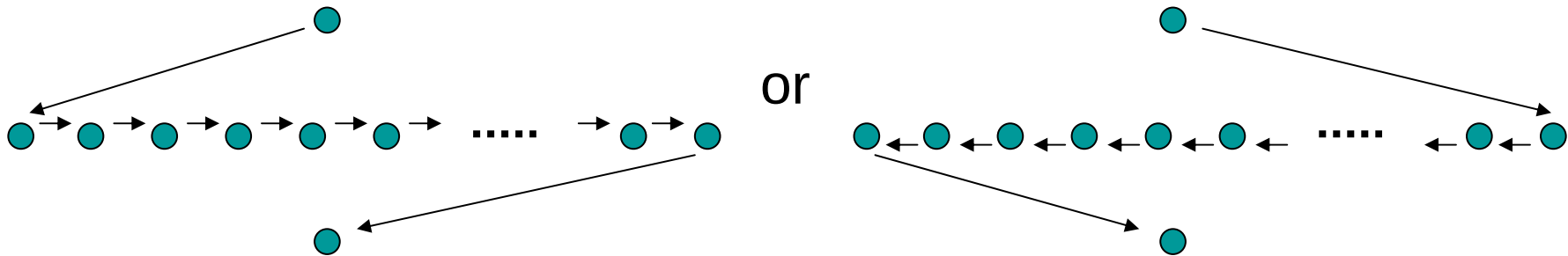
- **Notation:**

- $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$
- k clauses $C_1, C_2, \dots, C_k$
- Variables: $x_1, x_2, \dots, x_l$
- Each a_j, b_j , and c_j is either some x_i or some $\neg x_i$.

- **Gadget for variable x_i :**



- Can get from top node to bottom node in two ways:



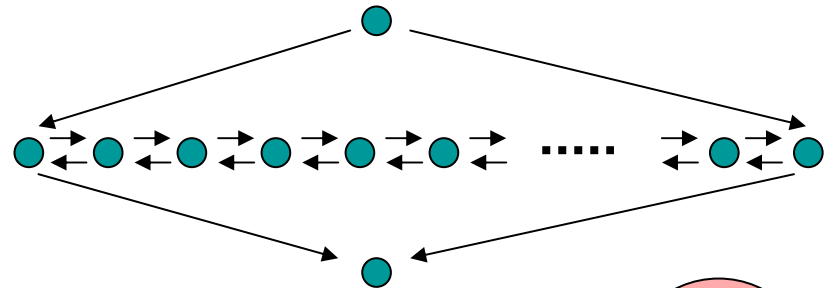
- Both ways visit all intermediate nodes.

$3SAT \leq_p DHAMPATH$

- **Notation:**

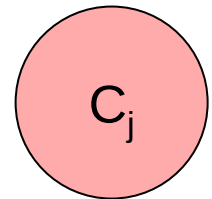
- $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$
- k clauses $C_1, C_2, \dots, C_k$
- Variables: $x_1, x_2, \dots, x_l$
- Each a_j, b_j , and c_j is either some x_i or some $\neg x_i$.

- **Gadget for variable x_i :**



- **Gadget for clause C_j :**

- Just a single node.

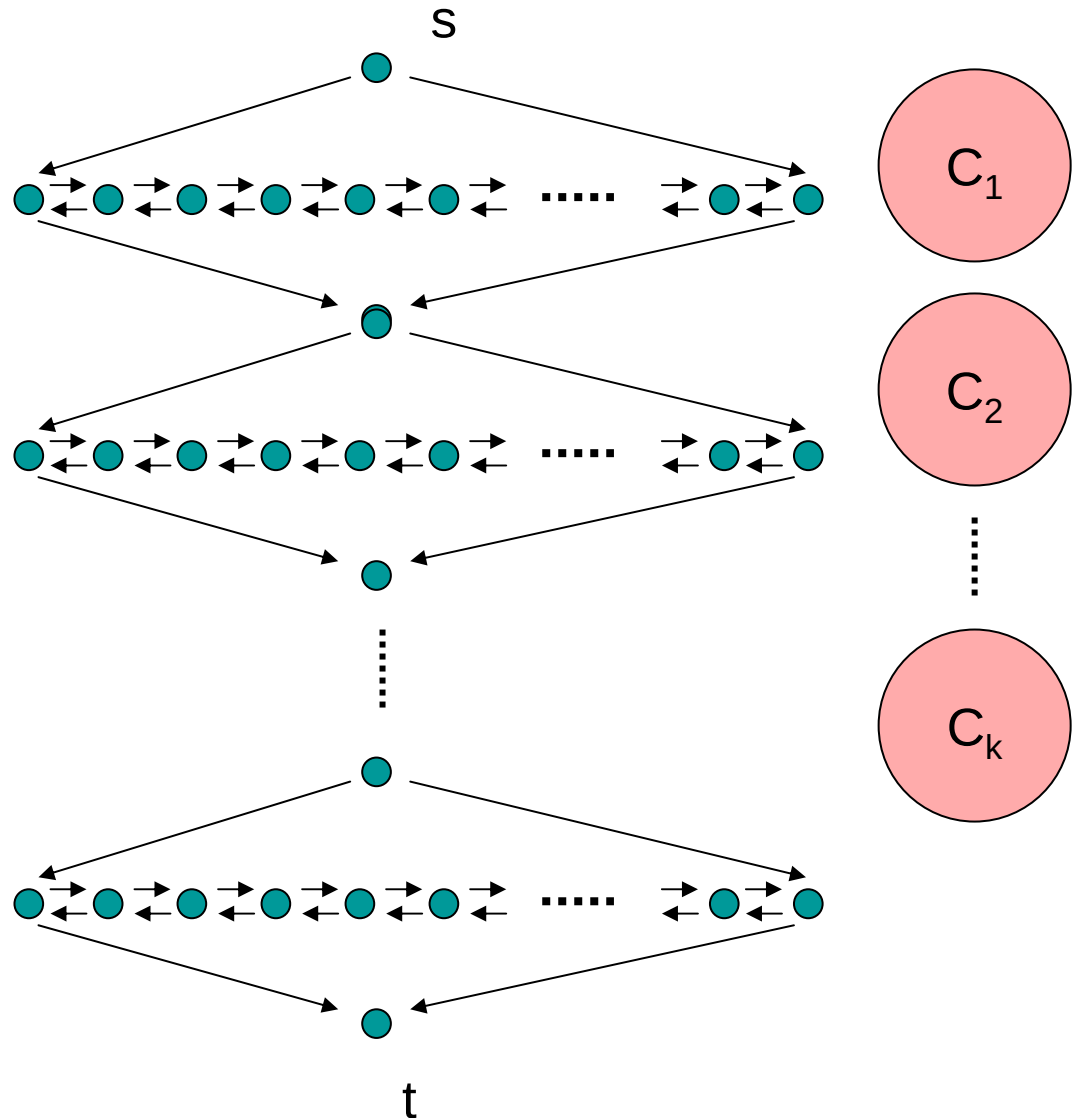


- **Putting the pieces together:**

- Put variables' gadgets in order $x_1, x_2, \dots, x_l$, top to bottom, identifying bottom node of each gadget with top node of the next.
- Make s and t the overall top and bottom node, respectively

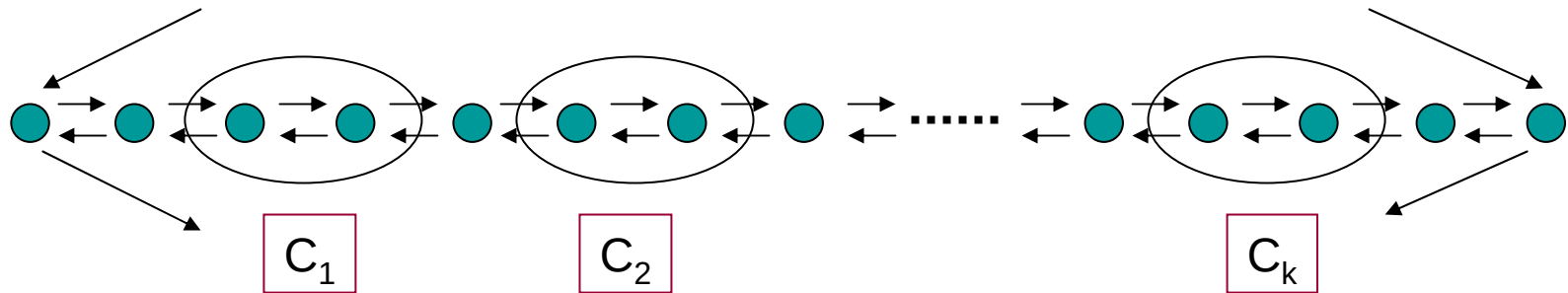
$3SAT \leq_p DHAMPATH$

- Putting the pieces together:
 - Put variables' gadgets in order $x_1, x_2, \dots, x_l$, identifying bottom node of each with top node of the next.
 - Make s and t the overall top and bottom node.
- We still must connect x -gadgets with C -gadgets.

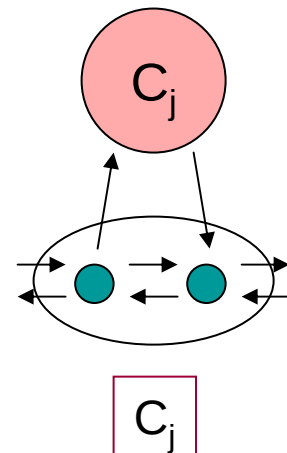


$3SAT \leq_p DHAMPATH$

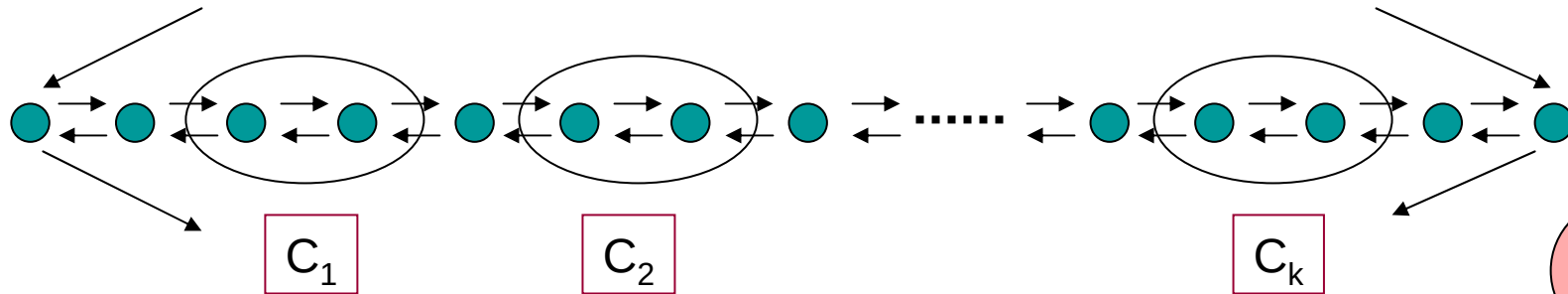
- We still must connect x-gadgets with C-gadgets.
- Divide the $3k+1$ nodes in the cross-bar of x_i 's gadget into k pairs, one per clause, separated by $k+1$ **separator nodes**:



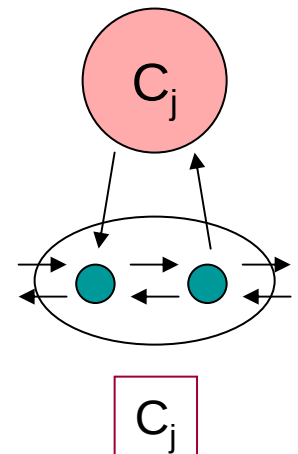
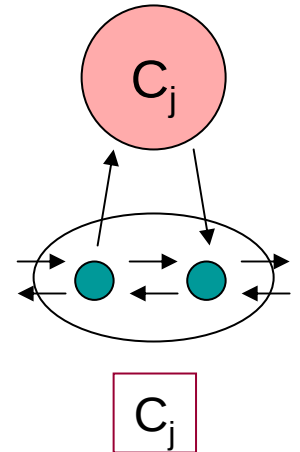
- If x_i appears in C_j , add edges between the C_j node and the nodes for C_j in the crossbar, going from left to right.
 - Allows detour to C_j while traversing crossbar left-to-right.



$3SAT \leq_p DHAMPATH$

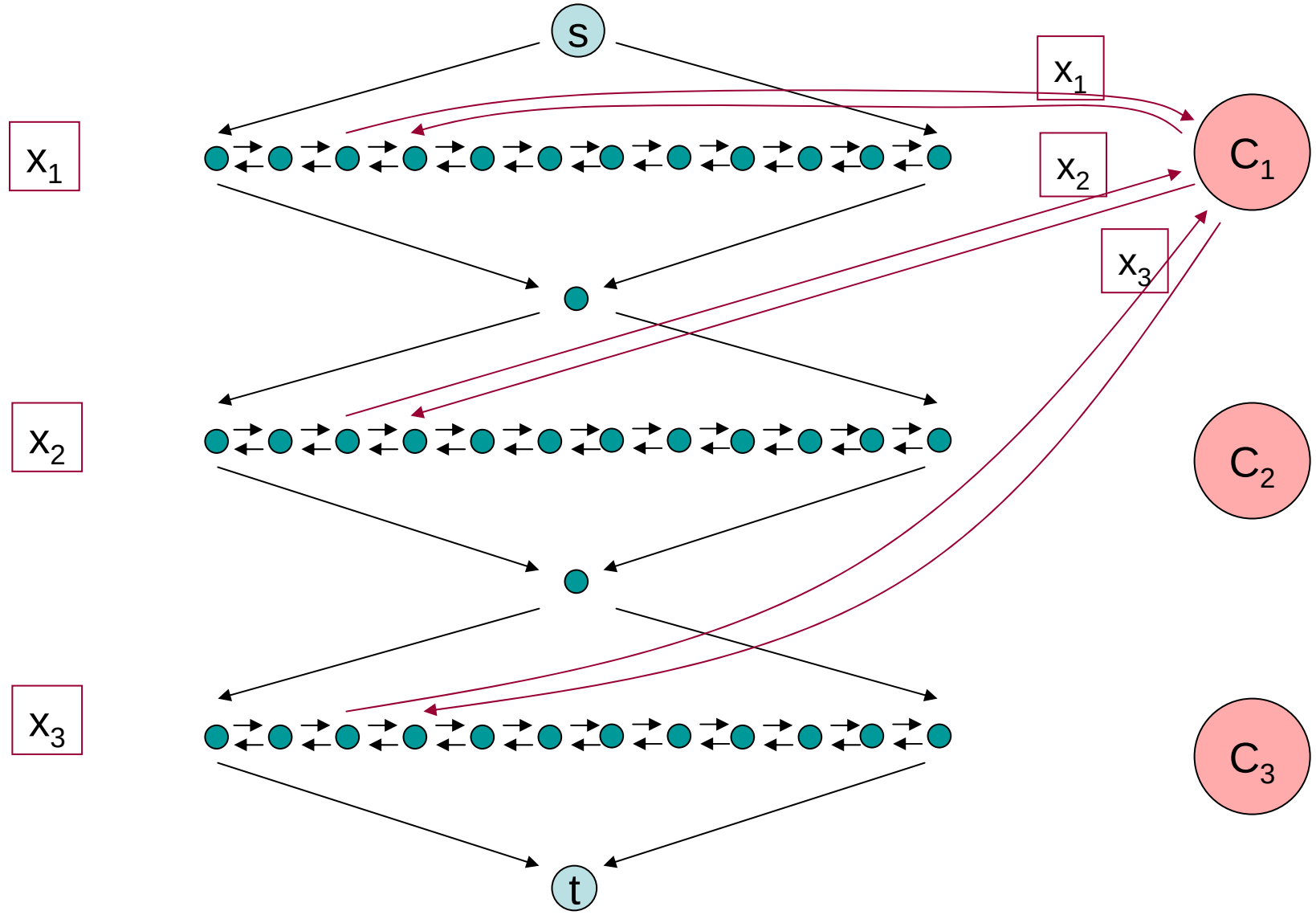


- If x_i appears in C_j , add edges L to R.
 - Allows detour to C_j while traversing crossbar L to R.
- If $\neg x_i$ appears in C_j , add edges R to L.
 - Allows detour to C_j while traversing crossbar R to L.
- If both x_i and $\neg x_i$ appear, add both sets of edges.
- This completes the construction of G , s , t .



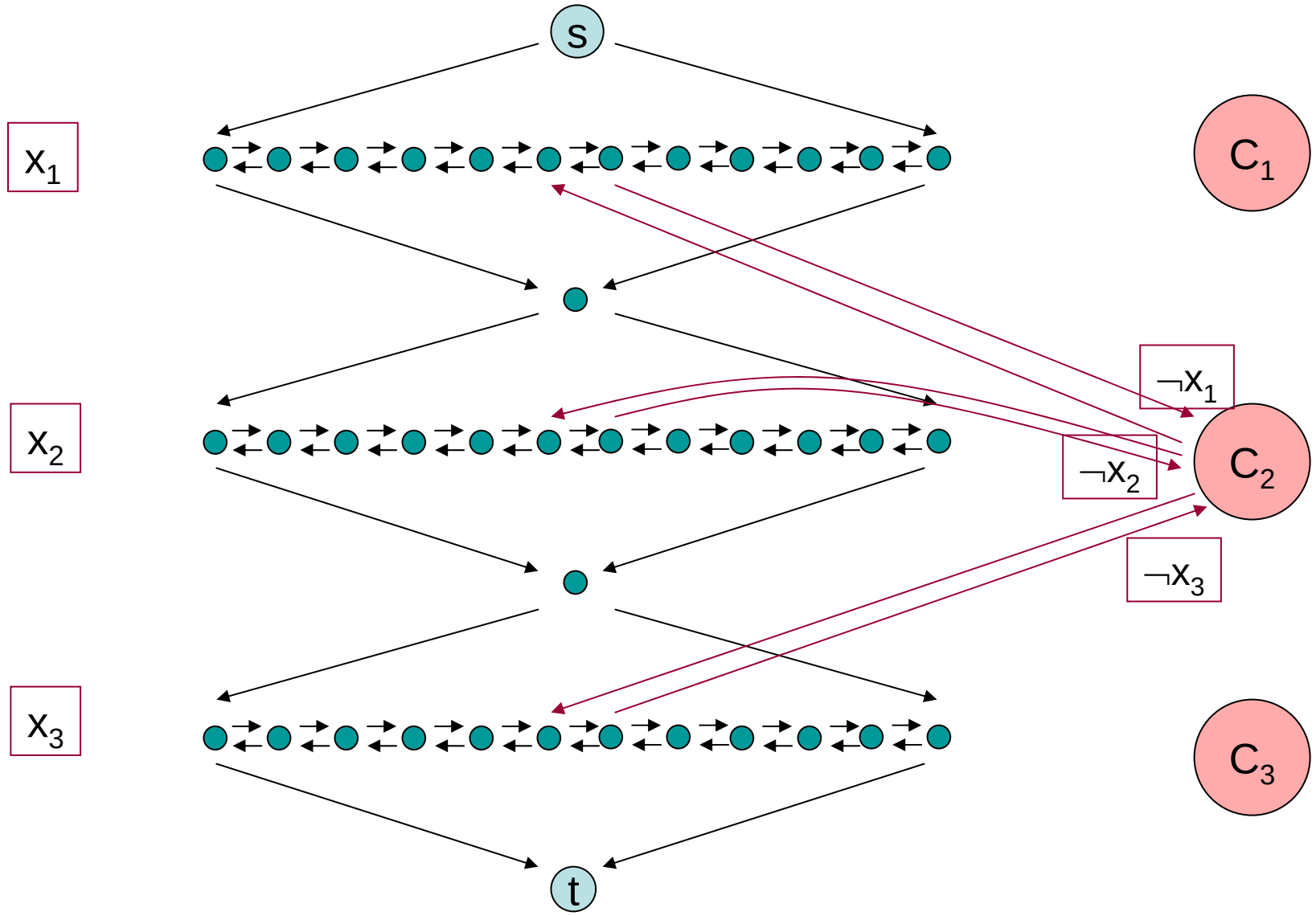
Example

- $\phi = (X_1 \vee X_2 \vee X_3) \wedge (\neg X_1 \vee \neg X_2 \vee \neg X_3) \wedge (\neg X_1 \vee X_2 \vee \neg X_3)$



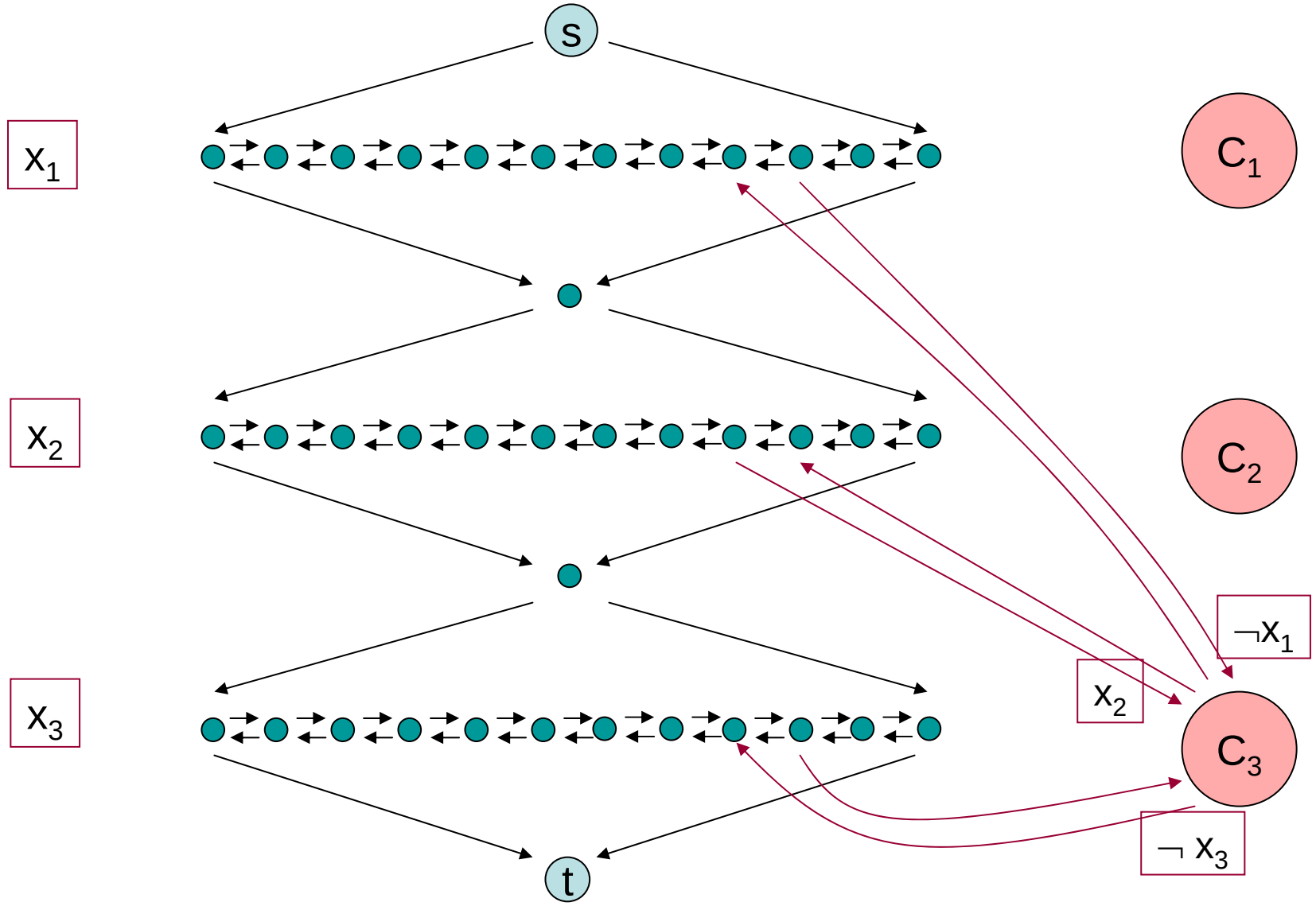
Example

- $\phi = (X_1 \vee X_2 \vee X_3) \wedge (\neg X_1 \vee \neg X_2 \vee \neg X_3) \wedge \dots \wedge (\neg X_1 \vee X_2 \vee \neg X_3)$



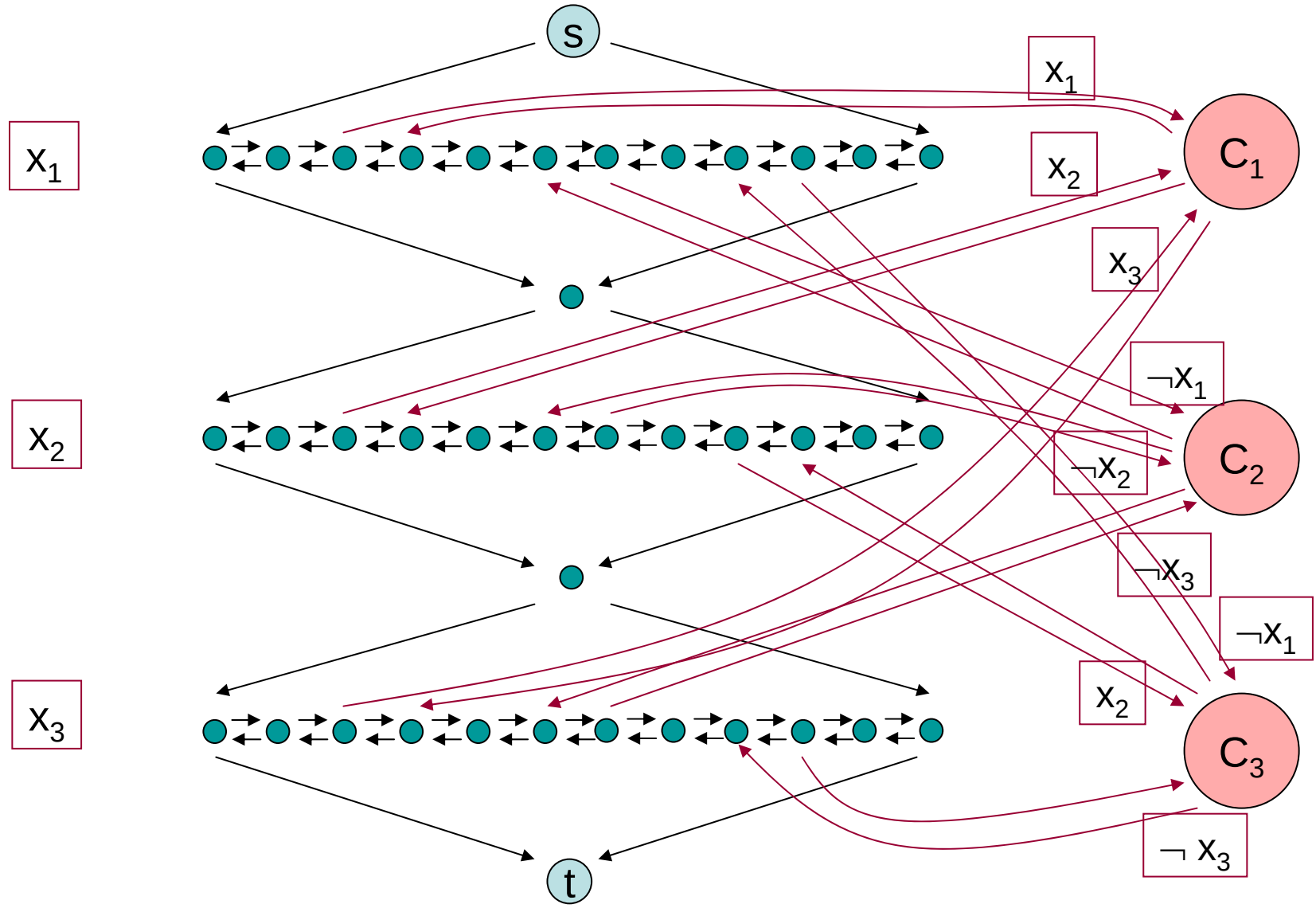
Example

- $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge \dots \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$



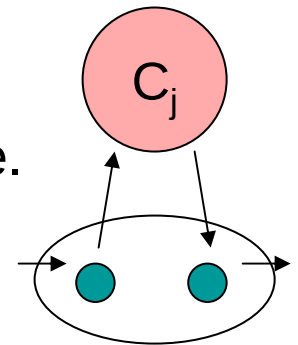
The entire graph G

- $\phi = (X_1 \vee X_2 \vee X_3) \wedge (\neg X_1 \vee \neg X_2 \vee \neg X_3) \wedge \dots \wedge (\neg X_1 \vee X_2 \vee \neg X_3)$



$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Rightarrow$
 - Assume ϕ is satisfiable; fix a particular satisfying assignment.
 - Follow path top-to-bottom, going
 - L to R through gadgets for x_i s that are set true.
 - R to L through gadgets for x_i s that are set false.
 - This visits all nodes of G except the C_j nodes.
 - For these, we must take detours.
 - For any particular clause C_j :
 - At least one of its literals must be set true; pick one.
 - If it's of the form x_i , then do:

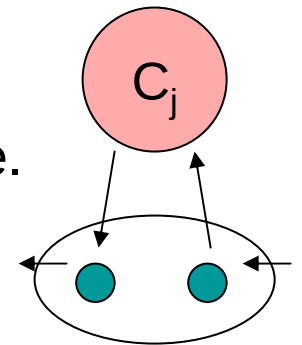


C_j pair in x_i row

- Works since $x_i = \text{true}$ means we traverse this crossbar L to R.

$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Rightarrow$
 - Assume ϕ is satisfiable; fix a particular satisfying assignment.
 - Follow path top-to-bottom, going
 - L to R through gadgets for x_i s that are set true.
 - R to L through gadgets for x_i s that are set false.
 - This visits all nodes of G except the C_j nodes.
 - For these, we must take detours.
 - For any particular clause C_j :
 - At least one of its literals must be set true; pick one.
 - If it's of the form $\neg x_i$, then do:

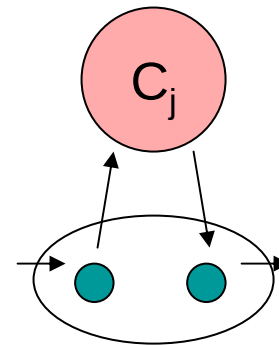


C_j pair in x_i row

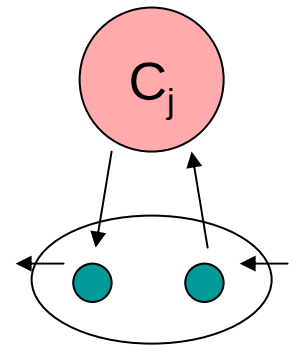
- Works since $x_i = \text{false}$ means we traverse this crossbar R to L.

$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Leftarrow$
 - Assume G has a Hamiltonian path from s to t , get a satisfying assignment for ϕ .
 - If the path is “normal” (goes in order through the gadgets, top to bottom, going one way or the other through each crossbar, and detouring to pick up the C_j nodes), then define the assignment by:
Set each x_i true if path goes L to R through x_i 's gadget, false if it goes R to L.
 - Why is this a satisfying assignment for ϕ ?
 - Consider any clause C_j .
 - The path goes through its node in one of two ways:



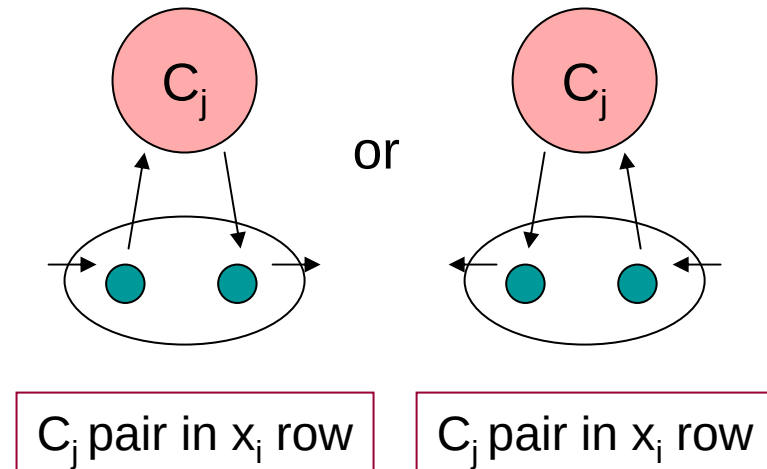
C_j pair in x_i row



C_j pair in x_i row

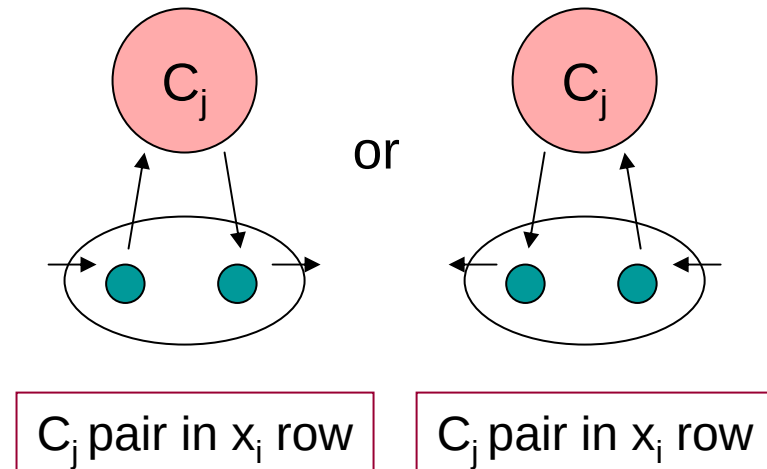
$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Leftarrow$
 - Assume G has a Hamiltonian path from s to t , get a satisfying assignment for ϕ .
 - If the path is “normal”, then define the assignment by:
Set each x_i true if path goes L to R through x_i 's gadget, false if it goes R to L.
 - To see that this satisfies ϕ , consider any clause C_j .
 - The path goes through C_j 's node by:
 - If the first, then:
 - x_i is true, since path goes L-R.
 - By the way the detour edges are set, C_j contains literal x_i .
 - So C_j is satisfied by x_i .



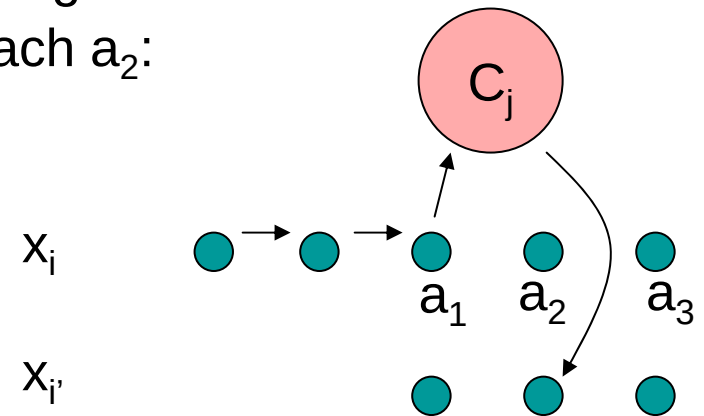
$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Leftarrow$
 - Assume G has a Hamiltonian path from s to t , get a satisfying assignment for ϕ .
 - If the path is “normal”, then define the assignment by:
Set each x_i true if path goes L to R through x_i 's gadget, false if it goes R to L.
 - To see that this satisfies ϕ , consider any clause C_j .
 - The path goes through C_j 's node by:
 - If the second, then:
 - x_i is false, since path goes R-L.
 - By the way the detour edges are set, C_j contains literal $\neg x_i$.
 - So C_j is satisfied by $\neg x_i$.



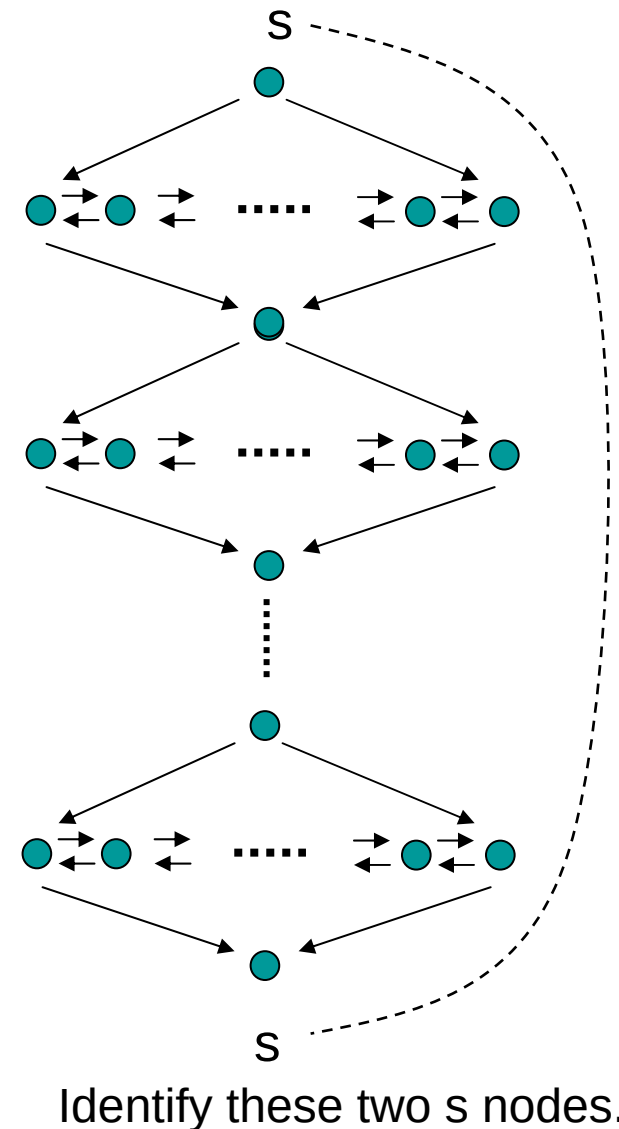
$3SAT \leq_p DHAMPATH$

- **Claim:** ϕ is satisfiable iff the graph G has a Hamiltonian path from s to t .
- **Proof:** $\Leftarrow$
 - Assume G has a Hamiltonian path from s to t .
 - If the path is normal, then it yields a satisfying assignment.
 - **It remains to show that the path is normal** (goes in order through the gadgets, top to bottom, going one way or the other through each crossbar, and detouring to pick up the C_j nodes),
 - The only problem (hand-waving) is if a detour doesn't work right, but jumps from one gadget to another, e.g.:
 - But then the Ham. path could never reach a_2 :
 - Can reach a_2 only from a_1 , a_3 , and (possibly) C_j .
 - But a_1 and C_j already lead elsewhere.
 - And reaching a_2 from a_3 leaves nowhere to go from a_2 , stuck.



Summary: DHAMPATH

- We have proved $3SAT \leq_p$ DHAMPATH.
- So DHAMPATH is NP-complete.
- Can prove similar result for **DHAMCIRCUIT** = $\{ \langle G \rangle \mid G \text{ is a directed graph, and there is a circuit in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- **Theorem:** $3SAT \leq_p$ DHAMCIRCUIT.
- **Proof:**
 - Same construction, but wrap around, identifying s and t nodes.
 - Now a satisfying assignment for ϕ corresponds to a Hamiltonian circuit.

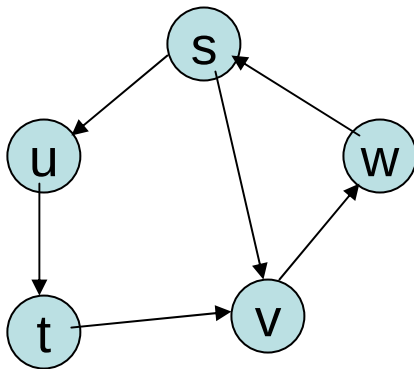


UHAMPATH and UHAMCIRCUIT

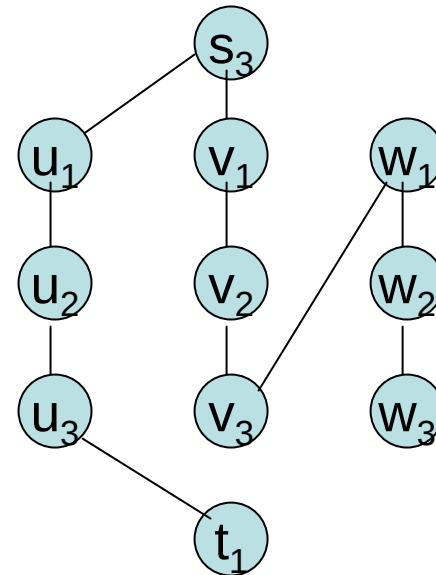
- Same questions about paths/circuits in undirected graphs.
- **UHAMPATH** = $\{ \langle G, s, t \rangle \mid G \text{ is an undirected graph, } s \text{ and } t \text{ are two distinct vertices, and there is a path from } s \text{ to } t \text{ in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- **UHAMCIRCUIT** = $\{ \langle G \rangle \mid G \text{ is an undirected graph, and there is a circuit in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- **Theorem:** Both are NP-complete.
- Obviously in NP.
- To show NP-hardness, reduce the digraph versions of the problems to the undirected versions---no need to consider Boolean formulas again.
 - $\text{DHAMPATH} \leq_p \text{UHAMPATH}$
 - $\text{DHAMCIRCUIT} \leq_p \text{UHAMCIRCUIT}$

DHAMPATH $\leq_p$ UHAMPATH

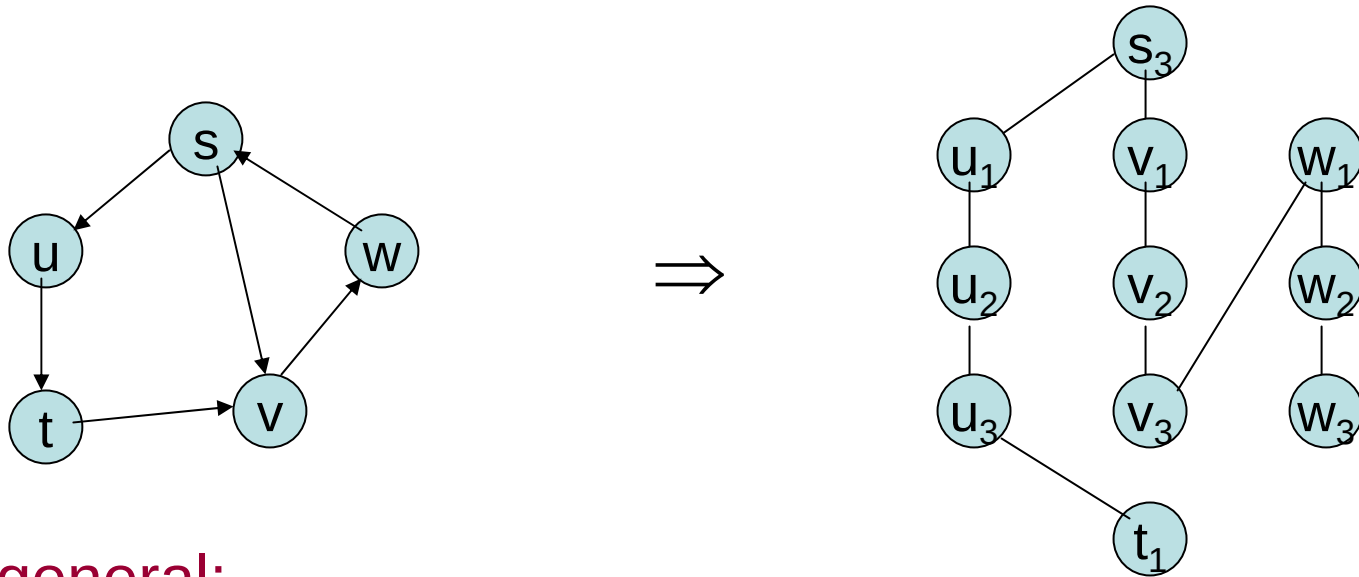
- **UHAMPATH** = $\{ \langle G, s, t \rangle \mid G \text{ is an undirected graph, } s \text{ and } t \text{ are two distinct vertices, and there is a path from } s \text{ to } t \text{ in } G \text{ that passes through each vertex of } G \text{ exactly once} \}$
- Map $\langle G, s, t \rangle$ (directed) to $\langle G', s', t' \rangle$ (undirected) so that $\langle G, s, t \rangle \in \text{DHAMPATH}$ iff $\langle G', s', t' \rangle \in \text{UHAMPATH}$.
- **Example:**



$\Rightarrow$



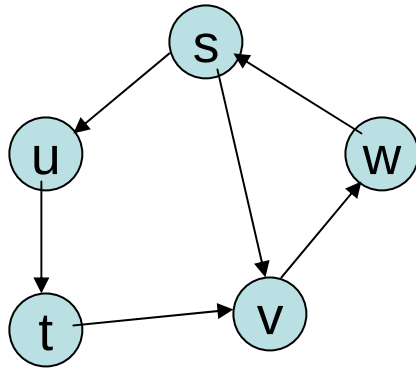
DHAMPATH $\leq_p$ UHAMPATH



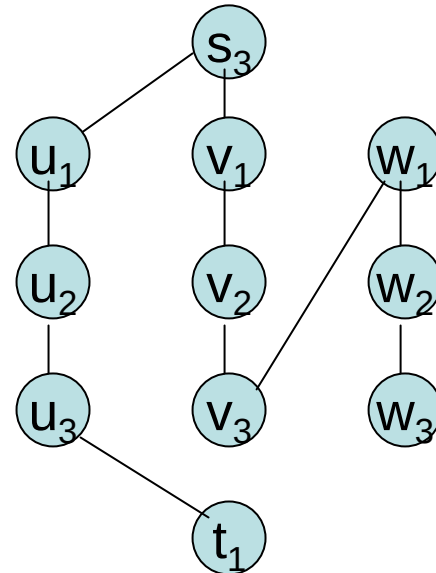
- In general:

- Replace each vertex x other than s, t with vertices x_1, x_2, x_3 , connected in a line.
- Replace s with just s_3 , t with just t_1 .
- For each directed edge from x to y in G , except incoming edges of s and outgoing edges of t , include undirected edge between x_3 and y_1 .
- Don't include anything for incoming edges of s or outgoing edges of t --- not needed since they can't be part of a Ham. path in G from s to t .

DHAMPATH $\leq_p$ UHAMPATH



$\Rightarrow$



- In general:
 - Replace each vertex x other than s, t with $x_1 \text{---} x_2 \text{---} x_3$.
 - Replace s with s_3 , t with t_1 .
 - For each directed edge from x to y in G , except incoming edges of s and outgoing edges of t , include $x_3 \text{---} y_1$.
- $G' =$ the resulting undirected graph; $s' = s_3$; $t' = t_1$
- Claim G has directed Hamiltonian path from s to t iff G' has an undirected Hamiltonian path from s' to t' .
- Idea: Indices 1,2,3 enforce consistent direction of traversal.
- Proof LTTR (in book).

Summary: UHAMPATH

- We have proved $\text{DHAMPATH} \leq_p \text{UHAMPATH}$.
- So UHAMPATH is NP-complete.
- Can prove similar result for
UHAMCIRCUIT = { $\langle G \rangle$ | G is an undirected graph, and there is a circuit in G that passes through each vertex of G exactly once }
- **Theorem:** $\text{DHAMCIRCUIT} \leq_p \text{UHAMCIRCUIT}$.
- **Proof:**
 - Similar construction.

The Traveling Salesman Problem

Traveling Salesman Problem (TSP)

- Variant of UHAMCIRCUIT.
- n cities = vertices, in a complete (undirected) graph.
- Each edge (u,v) has a cost, $c(u,v)$, a nonnegative integer.
- Salesman should visit all cities, each just once, at low cost.
- Express as a language:

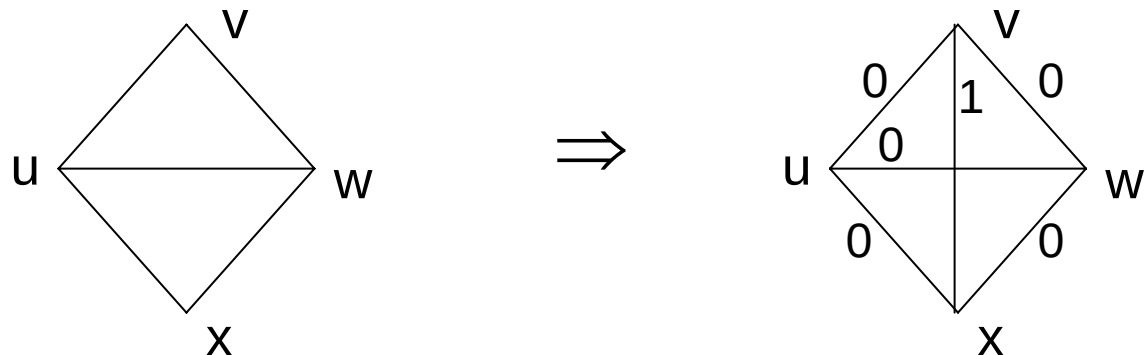
$\text{TSP} = \{ \langle G, c, k \rangle \mid G = (V, E) \text{ is a complete graph, } c: E \rightarrow \mathbb{N}, k \in \mathbb{N}, \text{ and } G \text{ has a cycle visiting each node exactly once, with total cost } \leq k \}$

- **Theorem:** TSP is NP-complete.
- **Proof:**
 - TSP $\in$ NP: Guess tour and verify.
 - TSP is NP-hard: Show UHAMCIRCUIT $\leq_p$ TSP.
 - Map $\langle G \rangle$ (undirected graph) to $\langle G', c', k' \rangle$ so that G has a Ham. circuit iff G' with cost function c' has a tour of total cost at most k' .

UHAMCIRCUIT $\leq_p$ TSP

- TSP = $\{ \langle G, c, k \rangle \mid G = (V, E) \text{ is a complete graph, } c: E \rightarrow \mathbb{N}, k \in \mathbb{N}, \text{ and } G \text{ has a cycle visiting each node exactly once, with total cost } \leq k \}$
- Map $\langle G \rangle$ (undirected graph) to $\langle G', c', k' \rangle$ so that G has a Ham. circuit iff G' with cost function c' has a tour of total cost $\leq k'$.
- Define mapping so that a Ham. circuit corresponds closely with a tour of cost $\leq k'$.
 - $G' = (V', E')$, where $V' = V$, all vertices of G , $E' =$ all edges (complete graph).
 - $c'(u, v) = 1$ if $(u, v) \notin E$, 0 if $(u, v) \in E$.
 - $k' = 0$.

- **Example:**



UHAMCIRCUIT $\leq_p$ TSP

- TSP = $\{ \langle G, c, k \rangle \mid G = (V, E) \text{ is a complete graph, } c: E \rightarrow \mathbb{N}, k \in \mathbb{N}, \text{ and } G \text{ has a cycle visiting each node exactly once, with total cost } \leq k \}$
- Map $\langle G \rangle$ (undirected graph) to $\langle G', c', k' \rangle$:
 - $G' = (V', E')$, where $V' = V$, all vertices of G , $E' =$ all edges (complete graph).
 - $c'(u, v) = 1$ if $(u, v) \notin E$, 0 if $(u, v) \in E$.
 - $k' = 0$.
- Claim: G has a Ham. circuit iff G' with cost function c' has a tour of total cost $\leq k'$.
- Proof:
 - $\Rightarrow$ If G has a Ham. circuit, all its edges have cost 0 in G' with c' , so we have a circuit of cost 0 in G' .
 - $\Leftarrow$ Tour of cost 0 in G' must consist of edges of cost 0, which are edges in G .

More Examples, Revisited

SUBSET-SUM

- $\text{SUBSET-SUM} = \{ \langle S, t \rangle \mid S \text{ is a multiset of } \mathbb{N}, t \in \mathbb{N}, \text{ and } t \text{ is expressible as the sum of some of the elements of } S \}$
- **Example:** $S = \{ 2, 2, 4, 5, 5, 7 \}, t = 13$
 $\langle S, t \rangle \in \text{SUBSET-SUM}$, because $7 + 4 + 2 = 13$
- **Theorem:** SUBSET-SUM is NP-complete.
- **Proof:**
 - Show $3\text{SAT} \leq_p \text{SUBSET-SUM}$.
 - Tricky, detailed, see book.

PARTITION

- $\text{PARTITION} = \{ \langle S \rangle \mid S \text{ is a multiset of } \mathbb{N} \text{ and } S \text{ can be split into multisets } S_1 \text{ and } S_2 \text{ having equal sums} \}$
- **Example:** $S = \{ 2, 2, 4, 5, 5, 7 \}$
 $S \notin \text{PARTITION}$, since the sum is odd
- **Example:** $T = \{ 2, 2, 5, 6, 9, 12 \}$
 $T \in \text{PARTITION}$, since $2 + 2 + 5 + 9 = 6 + 12$.
- **Theorem:** PARTITION is NP-complete.
- **Proof:**
 - Show $\text{SUBSET-SUM} \leq_p \text{PARTITION}$.
 - Simple...in recitation?

MULTIPROCESSOR SCHEDULING

- $MPS = \{ \langle S, m, D \rangle \mid$
 - S is a multiset of N (represents durations for tasks),
 - $m \in N$ (number of processors), and
 - $D \in N$ (deadline),and S can be written as $S_1 \cup S_2 \cup \dots \cup S_m$ such that, for every i , $\text{sum}(S_i) \leq D$ }
- **Theorem:** MPS is NP-complete.
- **Proof:**
 - Show $\text{PARTITION} \leq_p \text{MPS}$.
 - Simple...in recitation?

Next time...

- Probabilistic Turing Machines and Probabilistic Time Complexity Classes
- Reading:
 - Sipser Section 10.2

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

6.045: Automata, Computability, and Complexity (GITCS)

Class 17
Nancy Lynch

Today

- Probabilistic Turing Machines and Probabilistic Time Complexity Classes
- Now add a new capability to standard TMs: random choice of moves.
- Gives rise to new complexity classes: BPP and RP
- Topics:
 - Probabilistic polynomial-time TMs, BPP and RP
 - Amplification lemmas
 - Example 1: Primality testing
 - Example 2: Branching-program equivalence
 - Relationships between classes
- Reading:
 - Sipser Section 10.2

Probabilistic Polynomial-Time Turing Machines, BPP and RP

Probabilistic Polynomial-Time TM

- New kind of NTM, in which each nondeterministic step is a coin flip: has exactly 2 next moves, to each of which we assign probability $\frac{1}{2}$.

- **Example:**

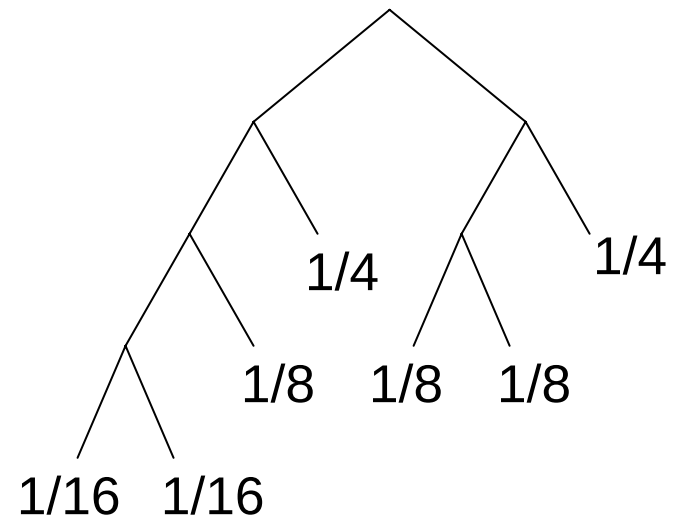
- To each maximal branch, we assign a probability:

$$\underbrace{\frac{1}{2} \times \frac{1}{2} \times \dots \times \frac{1}{2}}$$

number of coin flips
on the branch

- Has accept and reject states, as for NTMs.
- Now we can talk about probability of acceptance or rejection, on input w .

Computation on input w



Probabilistic Poly-Time TMs

- Probability of acceptance =

$$\sum_{b \text{ an accepting branch}} \Pr(b)$$

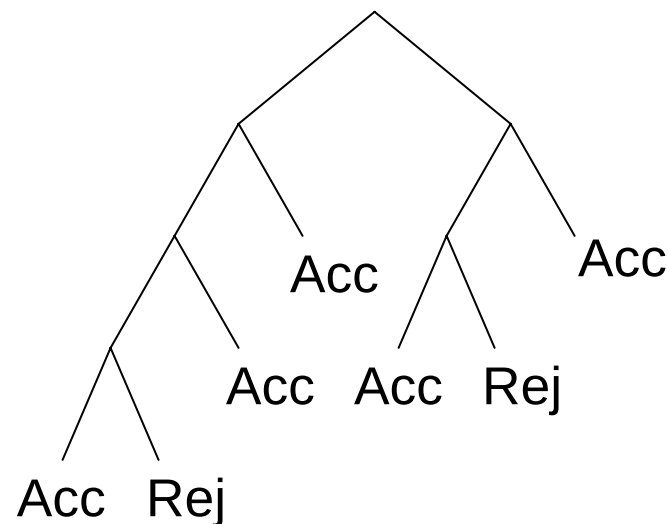
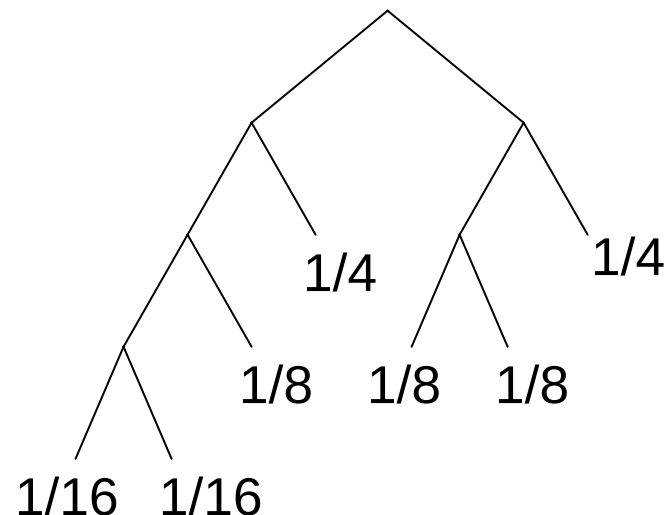
- Probability of rejection =

$$\sum_{b \text{ a rejecting branch}} \Pr(b)$$

- **Example:**

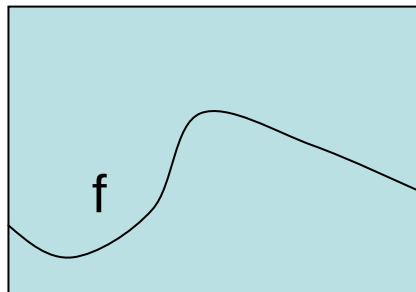
- Add accept/reject information
- Probability of acceptance = $1/16 + 1/8 + 1/4 + 1/8 + 1/4 = 13/16$
- Probability of rejection = $1/16 + 1/8 = 3/16$
- We consider TMs that halt (either accept or reject) on every branch--**deciders**.
- So the two probabilities total 1.

Computation on input w



Probabilistic Poly-Time TMs

- **Time complexity:**
 - Worst case over all branches, as usual.
- **Q:** What good are probabilistic TMs?
- Random choices can help solve some problems efficiently.
- Good for getting estimates---arbitrarily accurate, based on the number of choices.
- **Example:** Monte Carlo estimation of areas
 - E.g, integral of a function f .
 - Repeatedly choose a random point (x,y) in the rectangle.
 - Compare y with $f(x)$.
 - Fraction of trials in which $y \leq f(x)$ can be used to estimate the integral of f .



Probabilistic Poly-Time TMs

- Random choices can help solve some problems efficiently.
- We'll see 2 languages that have efficient probabilistic estimation algorithms.
- **Q:** What does it mean to estimate a language?
- Each w is either in the language or not; what does it mean to “approximate” a binary decision?
- **Possible answer:** For “most” inputs w , we always get the right answer, on all branches of the probabilistic computation tree.
- **Or:** For “most” w , we get the right answer with high probability.
- **Better answer:** For **every** input w , we get the right answer with high probability.

Probabilistic Poly-Time TMs

- **Better answer:** For **every** input w , we get the right answer with high probability.
- **Definition:** A probabilistic TM decider **M decides language L with error probability ε** if
 - $w \in L$ implies that $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$, and
 - $w \notin L$ implies that $\Pr[M \text{ rejects } w] \geq 1 - \varepsilon$.
- **Definition:** Language L is in **BPP** (Bounded-error Probabilistic Polynomial time) if there is a probabilistic polynomial-time TM that decides L with error probability $1/3$.
- **Q:** What's so special about $1/3$?
- Nothing. We would get an equivalent definition (same language class) if we chose ε to be any value with $0 < \varepsilon < 1/2$.
- We'll see this soon---**Amplification Theorem**

Probabilistic Poly-Time TMs

- Another class, RP, where the error is 1-sided:
- **Definition:** Language L is in **RP** (Random Polynomial time) if there is a probabilistic polynomial-time TM that decides L , where:
 - $w \in L$ implies that $\Pr[M \text{ accepts } w] \geq 1/2$, and
 - $w \notin L$ implies that $\Pr[M \text{ rejects } w] = 1$.
- Thus, absolutely guaranteed to be correct for words not in L ---always rejects them.
- But, might be incorrect for words in L ---might mistakenly reject these, in fact, with probability up to $1/2$.
- We can improve the $1/2$ to any larger constant < 1 , using another **Amplification Theorem**.

RP

- **Definition:** Language L is in **RP** (Random Polynomial time) if there is a probabilistic polynomial-time TM that decides L , where:
 - $w \in L$ implies that $\Pr[M \text{ accepts } w] \geq 1/2$, and
 - $w \notin L$ implies that $\Pr[M \text{ rejects } w] = 1$.
- Always correct for words not in L .
- Might be incorrect for words in L ---can reject these with probability up to $1/2$.
- Compare with nondeterministic TM acceptance:
 - $w \in L$ implies that there is some accepting path, and
 - $w \notin L$ implies that there is no accepting path.

Amplification Lemmas

Amplification Lemmas

- **Lemma:** Suppose that M is a PPT-TM that decides L with error probability ε , where $0 \leq \varepsilon < 1/2$.

Then for any ε' , $0 \leq \varepsilon' < 1/2$, there exists M' , another PPT-TM, that decides L with error probability ε' .

- **Proof idea:**
 - M' simulates M many times and takes the majority value for the decision.
 - Why does this improve the probability of getting the right answer?
 - E.g., suppose $\varepsilon = 1/3$; then each trial gives the right answer at least $2/3$ of the time (with $2/3$ probability).
 - If we repeat the experiment many times, then with very high probability, we'll get the right answer a majority of the times.
 - How many times? Depends on ε and ε' .

Amplification Lemmas

- **Lemma:** Suppose that M is a PPT-TM that decides L with error probability ε , where $0 \leq \varepsilon < 1/2$.

Then for any ε' , $0 \leq \varepsilon' < 1/2$, there exists M' , another PPT-TM, that decides L with error probability ε' .

- **Proof idea:**
 - M' simulates M many times, takes the majority value.
 - E.g., suppose $\varepsilon = 1/3$; then each trial gives the right answer at least $2/3$ of the time (with $2/3$ probability).
 - If we repeat the experiment many times, then with very high probability, we'll get the right answer a majority of the times.
 - How many times? Depends on ε and ε' .
 - $2k$, where $(4\varepsilon(1-\varepsilon))^k \leq \varepsilon'$, suffices.
 - In other words $k \geq (\log_2 \varepsilon') / (\log_2 (4\varepsilon(1-\varepsilon)))$.
 - See book for calculations.

Characterization of BPP

- **Theorem:** $L \in \text{BPP}$ if and only for, for some ε , $0 \leq \varepsilon < 1/2$, there is a PPT-TM that decides L with error probability ε .
- **Proof:**
 - $\Rightarrow$ If $L \in \text{BPP}$, then there is some PPT-TM that decides L with error probability $\varepsilon = 1/3$, which suffices.
 - $\Leftarrow$ If for some ε , a PPT-TM decides L with error probability ε , then by the Lemma, there is a PPT-TM that decides L with error probability $1/3$; this means that $L \in \text{BPP}$.

Amplification Lemmas

- For **RP**, the situation is a little different:
 - If $w \in L$, then $\Pr[M \text{ accepts } w]$ could be equal to $\frac{1}{2}$.
 - So after many trials, the majority would be just as likely to be correct or incorrect.
- But this isn't useless, because when $w \notin L$, the machine always answers correctly.
- **Lemma:** Suppose M is a PPT-TM that decides L , $0 \leq \varepsilon < 1$, and
 - $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$.
 - $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.Then for any ε' , $0 \leq \varepsilon' < 1$, there exists M' , another PPT-TM, that decides L with:
 - $w \in L$ implies $\Pr[M' \text{ accepts } w] \geq 1 - \varepsilon'$.
 - $w \notin L$ implies $\Pr[M' \text{ rejects } w] = 1$.

Amplification Lemmas

- **Lemma:** Suppose M is a PPT-TM that decides L , $0 \leq \varepsilon < 1$,
 $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$.
 $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.
Then for any ε' , $0 \leq \varepsilon' < 1$, there exists M' , another PPT-TM, that decides L with:
 $w \in L$ implies $\Pr[M' \text{ accepts } w] \geq 1 - \varepsilon'$.
 $w \notin L$ implies $\Pr[M' \text{ rejects } w] = 1$.
- **Proof idea:**
 - M' : On input w :
 - Run k independent trials of M on w .
 - If any accept, then accept; else reject.
 - Here, choose k such that $\varepsilon^k \leq \varepsilon'$.
 - If $w \notin L$ then all trials reject, so M' rejects, as needed.
 - If $w \in L$ then each trial accepts with probability $\geq 1 - \varepsilon$, so
 $\text{Prob(at least one of the } k \text{ trials accepts)}$
 $= 1 - \text{Prob(all } k \text{ reject)} \geq 1 - \varepsilon^k \geq 1 - \varepsilon'$.

Characterization of RP

- **Lemma:** Suppose M is a PPT-TM that decides L , $0 \leq \varepsilon < 1$,
 $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$.
 $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.

Then for any ε' , $0 \leq \varepsilon' < 1$, there exists M' , another PPT-TM, that decides L with:

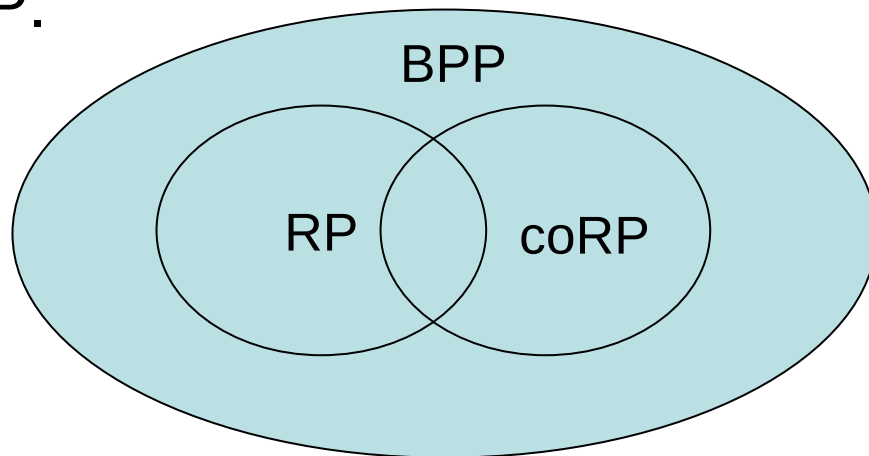
- $w \in L$ implies $\Pr[M' \text{ accepts } w] \geq 1 - \varepsilon'$.
 $w \notin L$ implies $\Pr[M' \text{ rejects } w] = 1$.
- **Theorem:** $L \in \text{RP}$ iff for some ε , $0 \leq \varepsilon < 1$, there is a PPT-TM that decides L with:
 $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$.
 $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.

RP vs. BPP

- **Lemma:** Suppose M is a PPT-TM that decides L , $0 \leq \varepsilon < 1$,
 - $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1 - \varepsilon$.
 - $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.Then for any ε' , $0 \leq \varepsilon' < 1$, there exists M' , another PPT-TM, that decides L with:
 - $w \in L$ implies $\Pr[M' \text{ accepts } w] \geq 1 - \varepsilon'$.
 - $w \notin L$ implies $\Pr[M' \text{ rejects } w] = 1$.
- **Theorem:** $RP \subseteq BPP$.
- **Proof:**
 - Given $A \in RP$, get (by def. of RP) a PPT-TM M with:
 - $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 1/2$.
 - $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.
 - By Lemma, get another PPT-TM for A , with:
 - $w \in L$ implies $\Pr[M \text{ accepts } w] \geq 2/3$.
 - $w \notin L$ implies $\Pr[M \text{ rejects } w] = 1$.
 - Implies $A \in BPP$, by definition of BPP .

RP, co-RP, and BPP

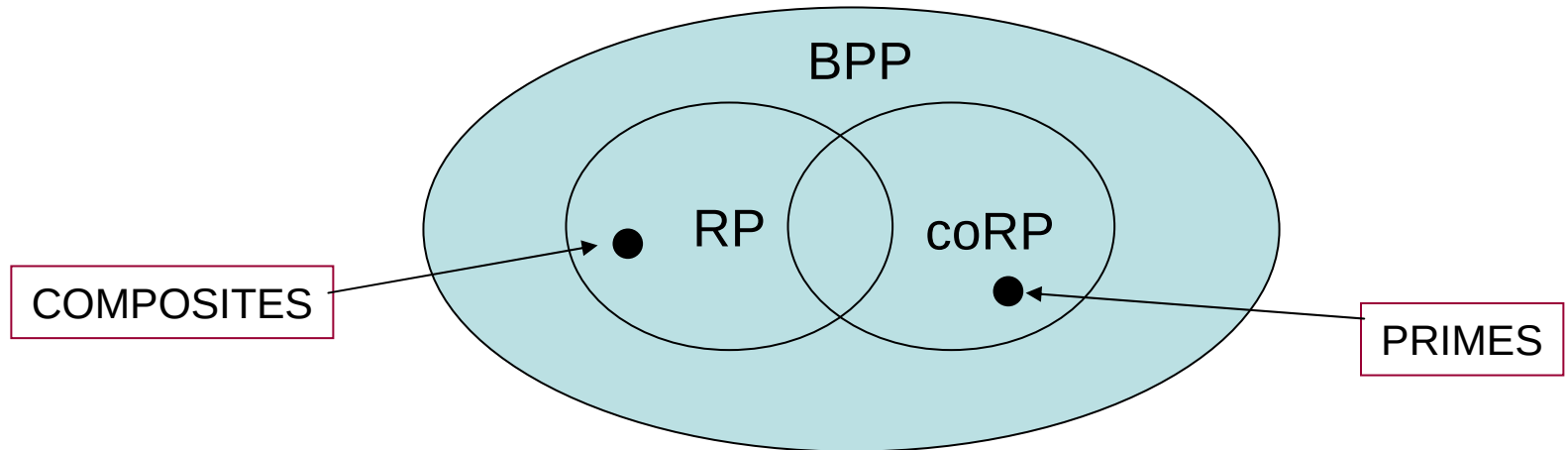
- **Definition:** $\text{coRP} = \{ L \mid L^c \in \text{RP} \}$
- coRP contains the languages L that can be decided by a PPT-TM that is always correct for $w \in L$ and has error probability at most $1/2$ for $w \notin L$.
- That is, L is in coRP if there is a PPT-TM that decides L , where:
 - $w \in L$ implies that $\Pr[M \text{ accepts } w] = 1$, and
 - $w \notin L$ implies that $\Pr[M \text{ rejects } w] \geq 1/2$.
- **Theorem:** $\text{coRP} \subseteq \text{BPP}$.
- So we have:



Example 1: Primality Testing

Primality Testing

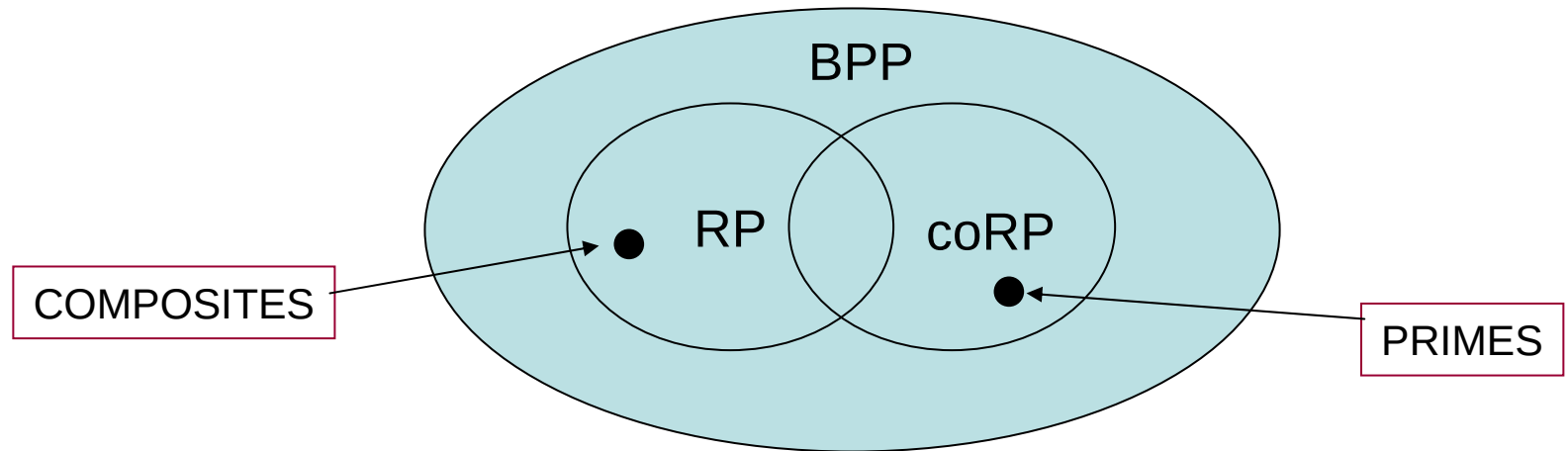
- **PRIMES** = $\{ \langle n \rangle \mid n \text{ is a natural number } > 1 \text{ and } n \text{ cannot be factored as } q r, \text{ where } 1 < q, r < n \}$
- **COMPOSITES** = $\{ \langle n \rangle \mid n > 1 \text{ and } n \text{ can be factored...} \}$
- We will show an algorithm demonstrating that $\text{PRIMES} \in \text{coRP}$.
- So $\text{COMPOSITES} \in \text{RP}$, and both $\in \text{BPP}$.



- This is not exciting, because it is now known that both are in P. [\[Agrawal, Kayal, Saxena 2002\]](#)
- But their poly-time algorithm is hard, whereas the probabilistic algorithm is easy.
- And anyway, this illustrates some nice probabilistic methods.

Primality Testing

- **PRIMES** = $\{ \langle n \rangle \mid n \text{ is a natural number } > 1 \text{ and } n \text{ cannot be factored as } q r, \text{ where } 1 < q, r < n \}$
- **COMPOSITES** = $\{ \langle n \rangle \mid n > 1 \text{ and } n \text{ can be factored...} \}$



- **Note:**
 - Deciding whether n is prime/composite isn't the same as factoring.
 - Factoring seems to be a much harder problem; it's at the heart of modern cryptography.

Primality Testing

- **PRIMES** = $\{ \langle n \rangle \mid n \text{ is a natural number } > 1 \text{ and } n \text{ cannot be factored as } q r, \text{ where } 1 < q, r < n \}$
- Show $\text{PRIMES} \in \text{coRP}$.
- Design PPT-TM (algorithm) M for PRIMES that satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] \leq 2^{-k}.$
- Here, k depends on the number of “trials” M makes.
- M always accepts primes, and almost always correctly identifies composites.
- Algorithm rests on some number-theoretic facts about primes (just state them here):

Fermat's Little Theorem

- **PRIMES** = $\{ \langle n \rangle \mid n \text{ is a natural number } > 1 \text{ and } n \text{ cannot be factored as } q r, \text{ where } 1 < q, r < n \}$
- Design PPT-TM (algorithm) M for PRIMES that satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] \leq 2^{-k}.$
- **Fact 1: Fermat's Little Theorem:** If n is prime and $a \in \mathbb{Z}_n^+$ then $a^{n-1} \equiv 1 \pmod n$.

Integers mod n except for 0, that is, $\{1, 2, \dots, n-1\}$



- **Example:** $n = 5$, $\mathbb{Z}_n^+ = \{1, 2, 3, 4\}.$
 - $a = 1$: $1^{5-1} = 1^4 = 1 \equiv 1 \pmod 5.$
 - $a = 2$: $2^{5-1} = 2^4 = 16 \equiv 1 \pmod 5.$
 - $a = 3$: $3^{5-1} = 3^4 = 81 \equiv 1 \pmod 5.$
 - $a = 4$: $4^{5-1} = 4^4 = 256 \equiv 1 \pmod 5.$

Fermat's test

- Design PPT-TM (algorithm) M for PRIMES that satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] \leq 2^{-k}.$
- **Fermat:** If n is prime and $a \in \mathbb{Z}_n^+$ then $a^{n-1} \equiv 1 \pmod n$.
- We can use this fact to identify some composites without factoring them:
- **Example:** $n = 8, a = 3.$
 - $3^{8-1} = 3^7 \equiv 3 \pmod 8$, not $1 \pmod 8$.
 - So 8 is composite.
- **Algorithm attempt 1:**
 - On input n :
 - Choose a number a randomly from $\mathbb{Z}_n^+ = \{ 1, \dots, n-1 \}.$
 - If $a^{n-1} \equiv 1 \pmod n$ then accept (passes Fermat test).
 - Else reject (known not to be prime).

Algorithm attempt 1

- Design PPT-TM (algorithm) M for PRIMES that satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] \leq 2^{-k}.$
- **Fermat:** If n is prime and $a \in \mathbb{Z}_n^+$ then $a^{n-1} \equiv 1 \pmod n$.
- **First try: On input n :**
 - Choose number a randomly from $\mathbb{Z}_n^+ = \{ 1, \dots, n-1 \}.$
 - If $a^{n-1} \equiv 1 \pmod n$ then accept (passes Fermat test).
 - Else reject (known not to be prime).
- This guarantees:
 - $n \in \text{PRIMES} \Rightarrow \Pr[M \text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow ??$
 - Don't know. It could pass the test, and be accepted erroneously.
- The problem isn't helped by repeating the test many times, for many values of a ---because there are some non-prime n 's that pass the test for all values of a .

Carmichael numbers

- **Fermat:** If n is prime and $a \in \mathbb{Z}_n^+$ then $a^{n-1} \equiv 1 \pmod{n}$.
- **On input n :**
 - Choose a randomly from $\mathbb{Z}_n^+ = \{1, \dots, n-1\}$.
 - If $a^{n-1} \equiv 1 \pmod{n}$ then accept (passes Fermat test).
 - Else reject (known not to be prime).
- **Carmichael numbers:** Non-primes that pass all Fermat tests, for all values of a .
- **Fact 2:** Any non-Carmichael composite number fails at least half of all Fermat tests (for at least half of all values of a).
- So for any non-Carmichael composite, the algorithm correctly identifies it as composite, with probability $\geq 1/2$.
- So, we can repeat k times to get more assurance.
- **Guarantees:**
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{M accepts } n] = 1$.
 - n a non-Carmichael composite number $\Rightarrow \Pr[\text{M accepts } n] \leq 2^{-k}$.
 - n a Carmichael composite number $\Rightarrow \Pr[\text{M accepts } n] = 1$ (wrong)

Carmichael numbers

- **Fermat:** If n is prime and $a \in \mathbb{Z}_n^+$ then $a^{n-1} \equiv 1 \pmod{n}$.
- **On input n :**
 - Choose a randomly from $\mathbb{Z}_n^+ = \{1, \dots, n-1\}$.
 - If $a^{n-1} \equiv 1 \pmod{n}$ then accept (passes Fermat test).
 - Else reject (known not to be prime).
- **Carmichael numbers:** Non-primes that pass all Fermat tests.
- **Algorithm guarantees:**
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{M accepts } n] = 1$.
 - n a non-Carmichael composite number $\Rightarrow \Pr[\text{M accepts } n] \leq 2^{-k}$.
 - n a Carmichael composite number $\Rightarrow \Pr[\text{M accepts } n] = 1$.
- **We must do something about the Carmichael numbers.**
- Use another test, based on:
- **Fact 3:** For every Carmichael composite n , there is some $b \neq 1, -1$ such that $b^2 \equiv 1 \pmod{n}$ (that is, 1 has a nontrivial square root, mod n). No prime has such a square root.

Primality-testing algorithm

- **Fact 3:** For every Carmichael composite n , there is some $b \neq 1, -1$ such that $b^2 \equiv 1 \pmod n$. No prime has such a square root.
- **Primality-testing algorithm: On input n :**
 - If $n = 1$ or n is even: Give the obvious answer (easy).
 - If n is odd and > 1 : Choose a randomly from Z_n^+ .
 - (Fermat test) If a^{n-1} is not congruent to 1 mod n then reject.
 - (Carmichael test) Write $n - 1 = 2^h s$, where s is odd (factor out twos).
 - Consider successive squares, $a^s, a^{2s}, a^{4s}, a^{8s} \dots, a^{2^{h-1}s} = a^{n-1}$.
 - If all terms are $\equiv 1 \pmod n$, then accept.
 - If not, then find the last one that isn't congruent to 1.
 - If it's $\equiv -1 \pmod n$ then accept else reject.

Primality-testing algorithm

- If n is odd and > 1 :
 - Choose a randomly from $\mathbb{Z}_n^+$.
 - (Fermat test) If a^{n-1} is not congruent to 1 mod n then reject.
 - (Carmichael test) Write $n - 1 = 2^h s$, where s is odd.
 - Consider successive squares, $a^s, a^{2s}, a^{4s}, a^{8s} \dots, a^{2^{h-1}s} = a^{n-1}$.
 - If all terms are $\equiv 1 \pmod n$, then accept.
 - If not, then find the last one that isn't congruent to 1.
 - If it's $\equiv -1 \pmod n$ then accept else reject.
- Theorem: This algorithm satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] \leq \frac{1}{2}.$
- By repeating it k times, we get:
 - $n \notin \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] \leq (\frac{1}{2})^k.$

Primality-testing algorithm

- If n is odd and > 1 :
 - Choose a randomly from Z_n^+ .
 - (Fermat test) If a^{n-1} is not congruent to 1 mod n then reject.
 - (Carmichael test) Write $n - 1 = 2^h s$, where s is odd.
 - Consider successive squares, $a^s, a^{2s}, a^{4s}, a^{8s} \dots, a^{2^h s} = a^{n-1}$.
 - If all terms are $\equiv 1 \pmod n$, then accept.
 - If not, then find the last one that isn't congruent to 1.
 - If it's $\equiv -1 \pmod n$ then accept else reject.
- Theorem: This algorithm satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] \leq \frac{1}{2}.$
- Proof: Suppose n is odd and > 1 .

Proof

- If n is odd and > 1 :
 - Choose a randomly from Z_n^+ .
 - (Fermat test) If a^{n-1} is not congruent to 1 mod n then reject.
 - (Carmichael test) Write $n - 1 = 2^h s$, where s is odd.
 - Consider successive squares, $a^s, a^{2s}, a^{4s}, a^{8s} \dots, a^{2^{h-1}s} = a^{n-1}$.
 - If all terms are $\equiv 1 \pmod n$, then accept.
 - If not, then find the last one that isn't congruent to 1.
 - If it's $\equiv -1 \pmod n$ then accept else reject.
- **Proof that $n \in \text{PRIMES} \Rightarrow \Pr[\text{accepts } n] = 1$.**
 - Show that, if the algorithm rejects, then n must be composite.
 - Reject because of Fermat: Then not prime, by Fact 1 (primes pass).
 - Reject because of Carmichael: Then 1 has a nontrivial square root b , mod n , so n isn't prime, by Fact 3.
 - Let b be the last term in the sequence that isn't congruent to 1 mod n .
 - b^2 is the next one, and is $\equiv 1 \pmod n$, so b is a square root of 1, mod n .

Proof

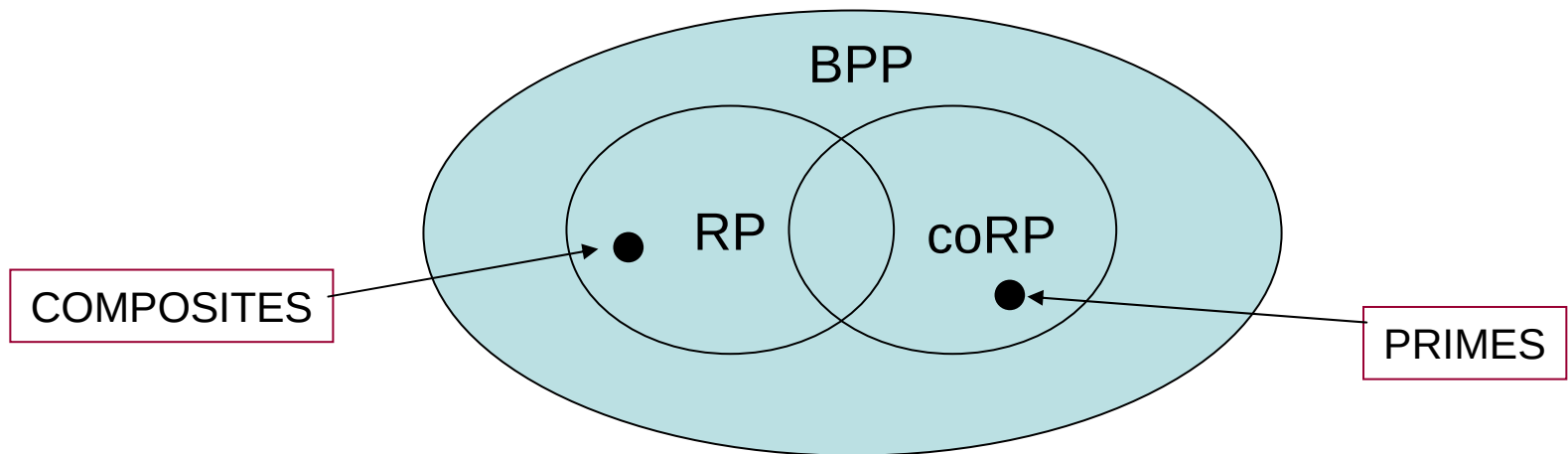
- If n is odd and > 1 :
 - Choose a randomly from Z_n^+ .
 - (Fermat test) If a^{n-1} is not congruent to 1 mod n then reject.
 - (Carmichael test) Write $n - 1 = 2^h s$, where s is odd.
 - Consider successive squares, $a^s, a^{2s}, a^{4s}, a^{8s} \dots, a^{2^h s} = a^{n-1}$.
 - If all terms are $\equiv 1 \pmod n$, then accept.
 - If not, then find the last one that isn't congruent to 1.
 - If it's $\equiv -1 \pmod n$ then accept else reject.
- **Proof that $n \notin \text{PRIMES} \Rightarrow \Pr[\text{accepts } n] \leq 1/2$.**
 - Suppose n is a composite.
 - If n is not a Carmichael number, then at least half of the possible choices of a fail the Fermat test (by Fact 2).
 - If n is a Carmichael number, then Fact 3 says that **some b fails the Carmichael test (is a nontrivial square root)**.
 - Actually, when we generate b using a as above, **at least half of the possible choices of a generate bs that fail the Carmichael test**.
 - Why: Technical argument, in Sipser, p. 374-375.

Primality-testing algorithm

- So we have proved:
- Theorem: This algorithm satisfies:
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] \leq \frac{1}{2}.$
- This implies:
- Theorem: $\text{PRIMES} \in \text{coRP}.$
- Repeating k times, or using an amplification lemma, we get:
 - $n \in \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] = 1.$
 - $n \notin \text{PRIMES} \Rightarrow \Pr[\text{ accepts } n] \leq (\frac{1}{2})^k.$
- Thus, the algorithm might sometimes make mistakes and classify a composite as a prime, but the probability of doing this can be made arbitrarily low.
- Corollary: $\text{COMPOSITES} \in \text{RP}.$

Primality-testing algorithm

- Theorem: $\text{PRIMES} \in \text{coRP}$.
- Corollary: $\text{COMPOSITES} \in \text{RP}$.
- Corollary: Both PRIMES and $\text{COMPOSITES} \in \text{BPP}$.



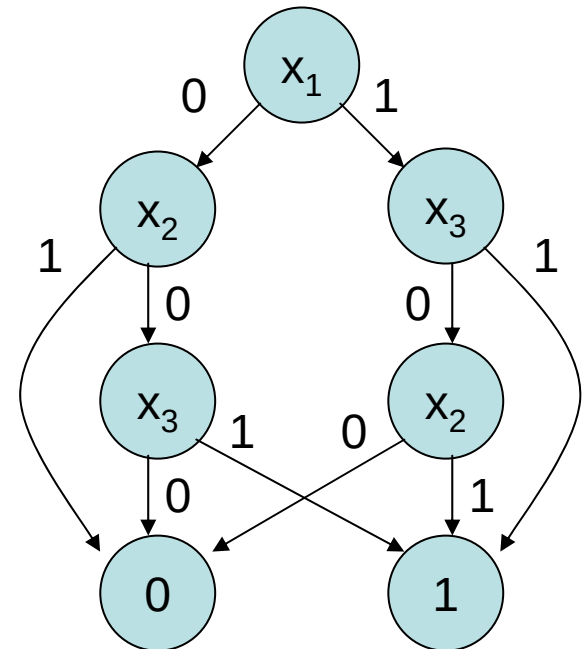
Example 2: Branching-Program Equivalence

Branching Programs

- **Branching program:** A variant of a decision tree. Can be a DAG, not just a tree:
- Describes a Boolean function of a set $\{x_1, x_2, x_3, \dots\}$ of Boolean variables.
- Restriction: Each variable appears at most once on each path.

- **Example:** x_1 x_2 x_3 **result**

0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Branching Programs

- Branching program representation for Boolean functions is used by system modeling and analysis tools, for systems in which the state can be represented using just Boolean variables.
- Programs called **Binary Decision Diagrams (BDDs)**.
- Analyzing a model involves exploring all the states, which in turn involves exploring all the paths in the diagram.
- Choosing the “right” order of evaluating the variables can make a big difference in cost (running time).
- **Q:** Given two branching programs, B_1 and B_2 , do they compute the same Boolean function?
- That is, do the same values for all the variables always lead to the same result in both programs?

Branching-Program Equivalence

- **Q:** Given two branching programs, B_1 and B_2 , do they compute the same Boolean function?
- Express as a language problem:
 $EQ_{BP} = \{ \langle B_1, B_2 \rangle \mid B_1 \text{ and } B_2 \text{ are BPs that compute the same Boolean function} \}.$
- **Theorem:** EQ_{BP} is in $coRP \subseteq BPP$.
- **Note:** Need the restriction that a variable appears at most once on each path. Otherwise, the problem is $coNP$ -complete.
- **Proof idea:**
 - Pick random values for $x_1, x_2, \dots$ and see if they lead to the same answer in B_1 and B_2 .
 - If so, accept; if not, reject.
 - Repeat several times for extra assurance.

Branching-Program Equivalence

$EQ_{BP} = \{ \langle B_1, B_2 \rangle \mid B_1 \text{ and } B_2 \text{ are BPs that compute the same Boolean function} \}$

- **Theorem:** EQ_{BP} is in $coRP \subseteq BPP$.
- **Proof idea:**
 - Pick random values for $x_1, x_2, \dots$ and see if they lead to the same answer in B_1 and B_2 .
 - If so, accept; if not, reject.
 - Repeat several times for extra assurance.
- This is not quite good enough:
 - Some inequivalent BPs differ on only one assignment to the vars.
 - Unlikely that the algorithm would guess this assignment.
- **Better proof idea:**
 - Consider the same BPs but now pretend the domain of values for the variables is Z_p , the integers mod p , for a large prime p , rather than just $\{0,1\}$.
 - This will let us make more distinctions, making it less likely that we would think B_1 and B_2 are equivalent if they aren't.

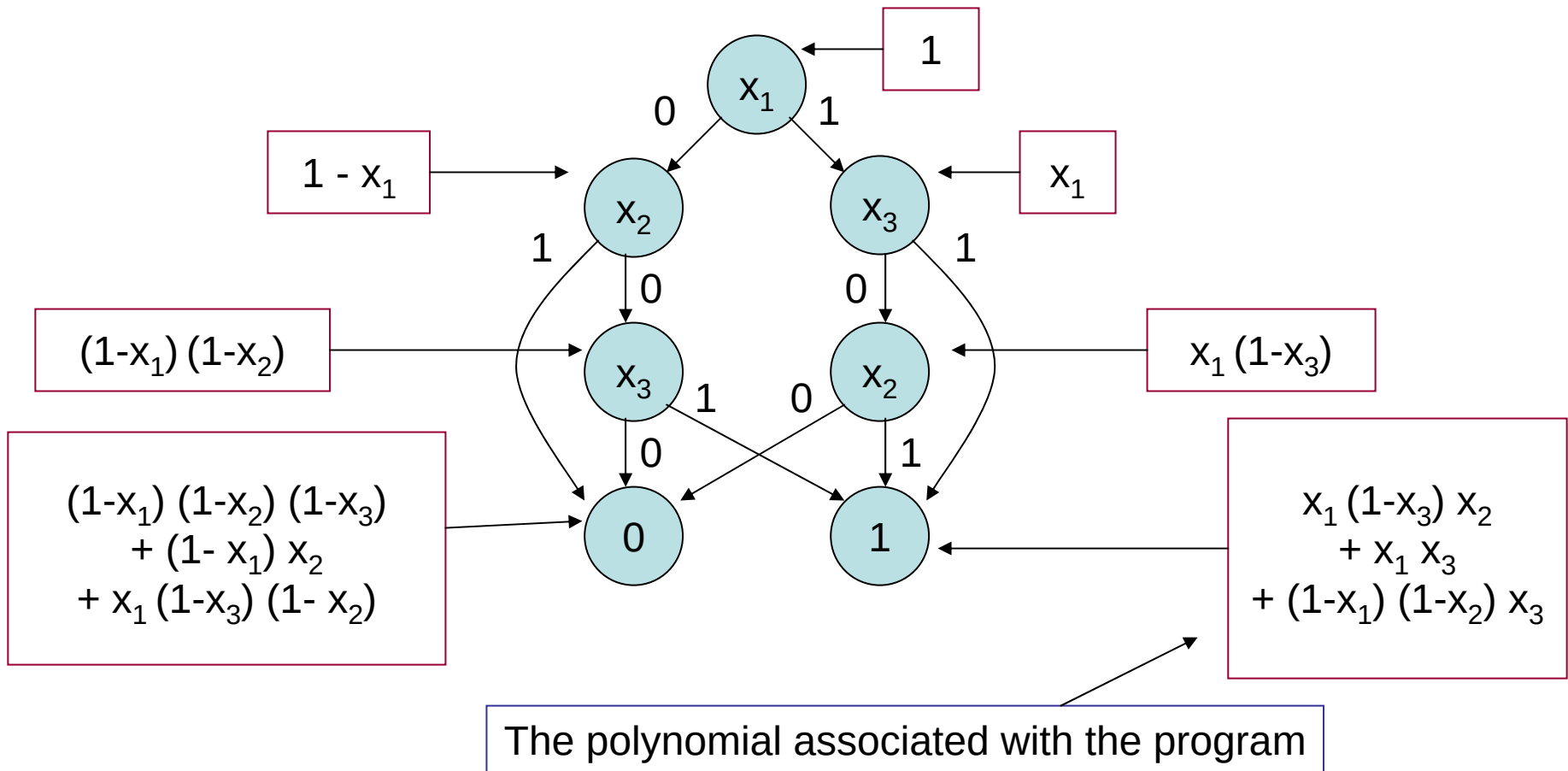
Branching-Program Equivalence

$EQ_{BP} = \{ \langle B_1, B_2 \rangle \mid B_1 \text{ and } B_2 \text{ are BPs that compute the same Boolean function} \}$

- **Theorem:** EQ_{BP} is in $coRP \subseteq BPP$.
- **Proof idea:**
 - Pick random values for $x_1, x_2, \dots$ and see if they lead to the same answer in B_1 and B_2 .
 - If so, accept; if not, reject.
 - Repeat several times for extra assurance.
- **Better proof idea:**
 - Pretend that the domain of values for the variables is Z_p , the integers mod p , for a large prime p , rather than just $\{0,1\}$.
 - This lets us make more distinctions, making it less likely that we would think B_1 and B_2 are equivalent if they aren't.
 - But how do we apply the programs to integers mod p ?
 - By associating a multi-variable polynomial with each program:

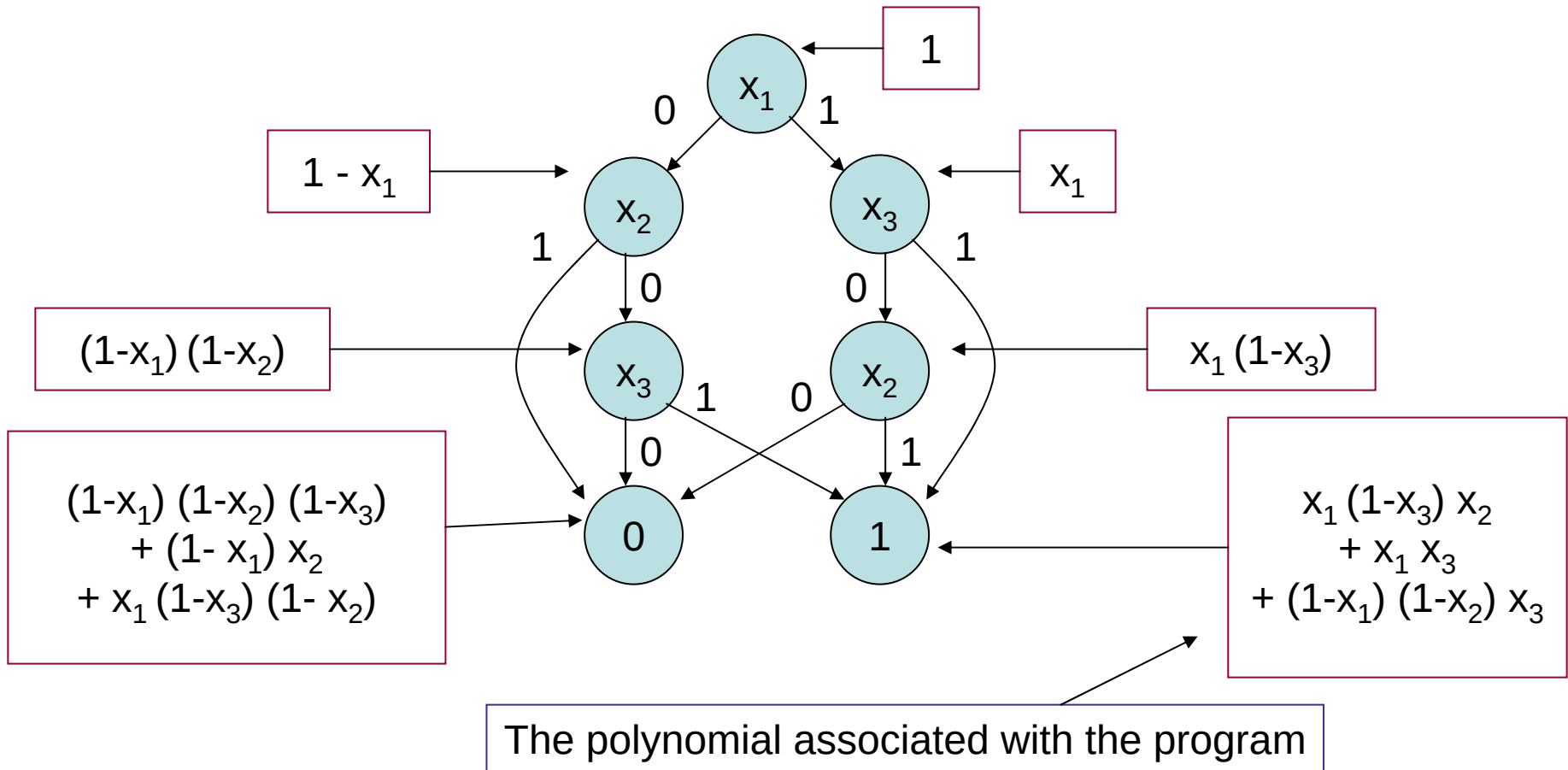
Associating a polynomial with a BP

- Associate a polynomial with each node in the BP, and use the poly associated with the 1-result node as the poly for the entire BP.



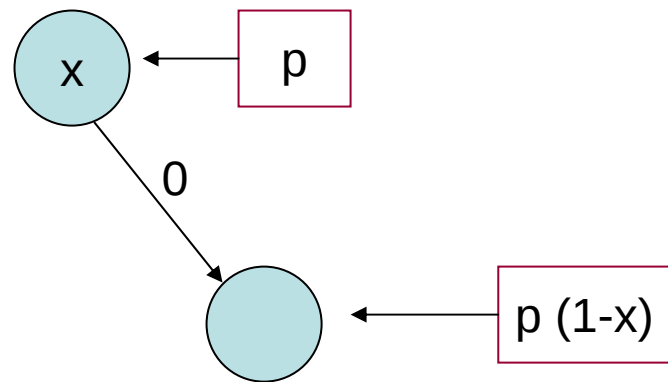
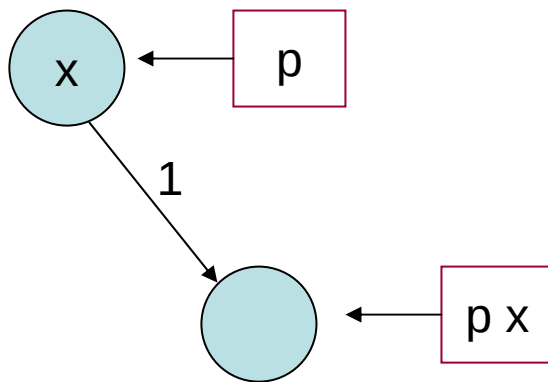
Labeling rules

- Top node: Label with polynomial 1.
- Non-top node: Label with sum of polys, one for each incoming edge:
 - Edge labeled with 1, from x , labeled with p , contributes $p x$.
 - Edge labeled with 0, from x , labeled with p , contributes $p (1-x)$.



Labeling rules

- Top node: Label with polynomial 1.
- Non-top node: Label with sum of polys, one for each incoming edge:
 - Edge labeled with 1, from x labeled with p , contributes $p x$.
 - Edge labeled with 0, from x labeled with p , contributes $p (1-x)$.



Associating a polynomial with a BP

- What do these polynomials mean for Boolean values?
- For any particular assignment of $\{0, 1\}$ to the variables, each polynomial at each node evaluates to either 0 or 1 (because of their special form).
- The polynomials on the path followed by that assignment all evaluate to 1, and all others evaluate to 0.
- The polynomial associated with the entire program evaluates to 1 exactly for the assignments that lead there = those that are assigned value 1 by the program.

- **Example:** Above.

- The assignments leading to result 1 are:
- Which are exactly the assignments for which the program's polynomial evaluates to 1.

x_1	x_2	x_3
0	0	1
1	0	1
1	1	0
1	1	1

$$\begin{aligned} & x_1 (1-x_3) x_2 \\ & + x_1 x_3 \\ & + (1-x_1) (1-x_2) x_3 \end{aligned}$$

Branching-Program Equivalence

- Now consider Z_p , integers mod p , for a large prime p (much bigger than the number of variables).
- **Equivalence algorithm:** On input $\langle B_1, B_2 \rangle$, where both programs use m variables:
 - Choose elements $a_1, a_2, \dots, a_m$ from Z_p at random.
 - Evaluate the polynomials p_1 associated with B_1 and p_2 associated with B_2 for $x_1 = a_1, x_2 = a_2, \dots, x_m = a_m$.
 - Evaluate them node-by-node, without actually constructing all the polynomials for both programs.
 - Do this in polynomial time in the size of $\langle B_1, B_2 \rangle$, LTTR.
 - If the results are equal (mod p) then accept; else reject.
- **Theorem:** The equivalence algorithm guarantees:
 - If B_1 and B_2 are equivalent BPs (for Boolean values) then $\Pr[\text{algorithm accepts } n] = 1$.
 - If B_1 and B_2 are not equivalent, then $\Pr[\text{algorithm rejects } n] \geq 2/3$.

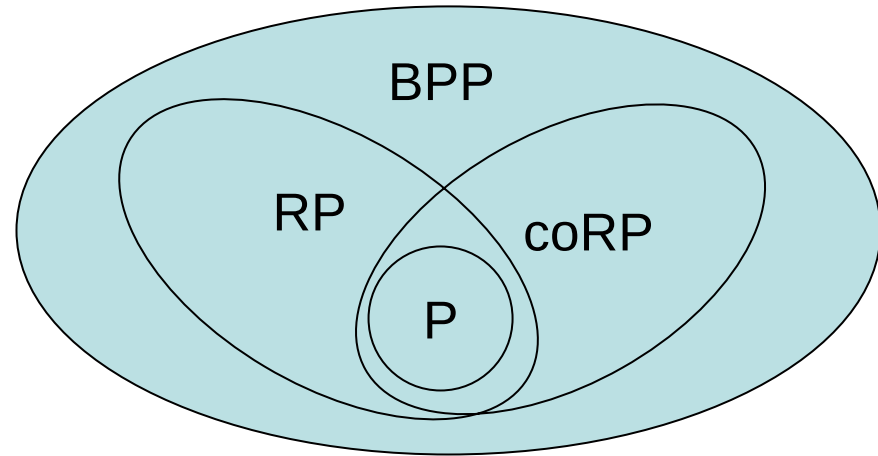
Branching-Program Equivalence

- **Equivalence algorithm:** On input $\langle B_1, B_2 \rangle$:
 - Choose elements $a_1, a_2, \dots, a_m$ from Z_p at random.
 - Evaluate the polynomials p_1 associated with B_1 and p_2 associated with B_2 for $x_1 = a_1, x_2 = a_2, \dots, x_m = a_m$.
 - If the results are equal (mod p) then accept; else reject.
- **Theorem:** The equivalence algorithm guarantees:
 - If B_1 and B_2 are equivalent BPs then $\Pr[\text{ accepts } n] = 1$.
 - If B_1 and B_2 are not equivalent, then $\Pr[\text{ rejects } n] \geq 2/3$.
- **Proof idea:** (See Sipser, p. 379)
 - If B_1 and B_2 are equivalent BPs (for Boolean values), then p_1 and p_2 are equivalent polynomials over Z_p , so always accepts.
 - If B_1 and B_2 are not equivalent (for Boolean values), then at least $2/3$ of the possible sets of choices from Z_p yield different values, so $\Pr[\text{ rejects } n] \geq 2/3$.
- **Corollary:** $EQ_{BP} \in \text{coRP} \subseteq \text{BPP}$.

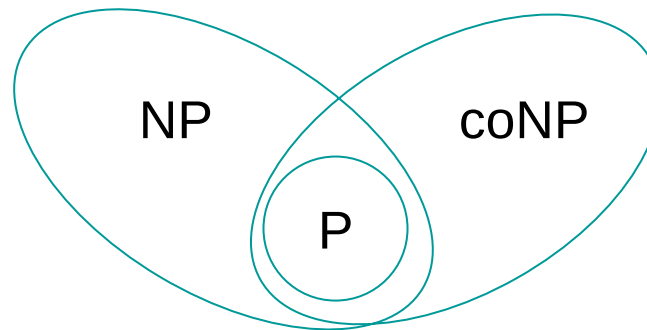
Relationships Between Complexity Classes

Relationships between complexity classes

- We know:



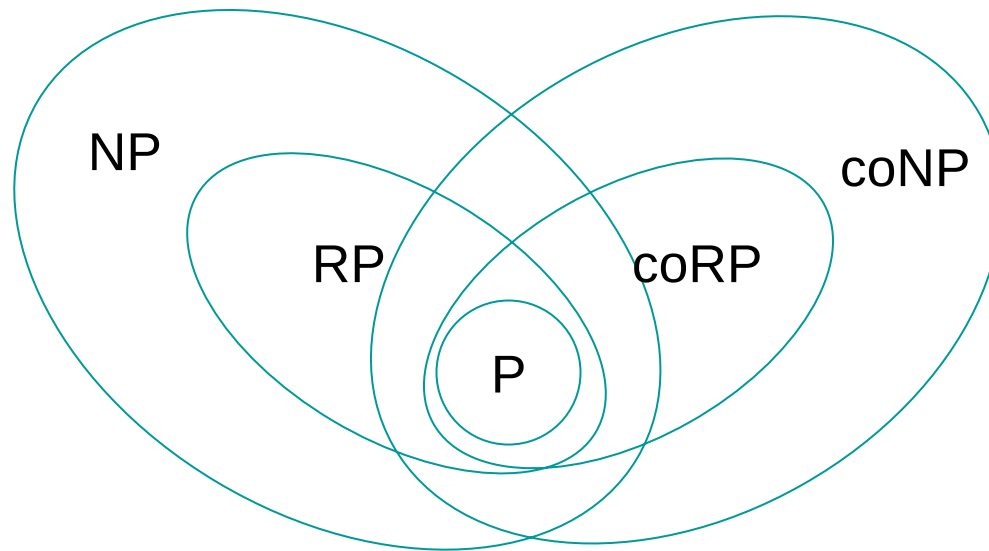
- Also recall:



- From the definitions, $RP \subseteq NP$ and $coRP \subseteq coNP$.
- So we have:

Relationships between classes

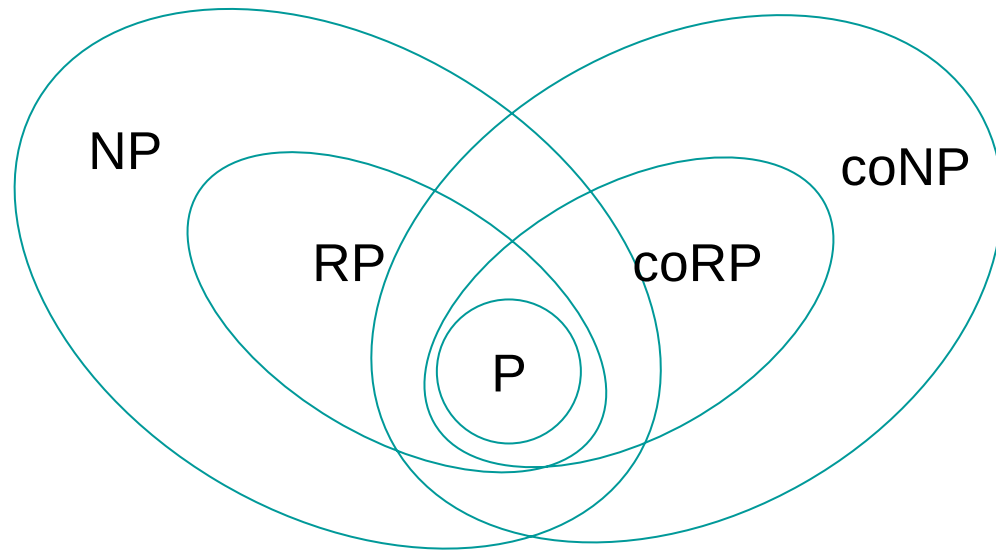
- So we have:



- **Q:** Where does BPP fit in?

Relationships between classes

- Where does BPP fit?
 - $NP \cup coNP \subseteq BPP$?
 - $BPP = P$?
 - Something in between ?
- Many people believe $BPP = RP = coRP = P$, that is, that randomness doesn't help.
- How could this be?
- Perhaps we can emulate randomness with pseudo-random generators---deterministic algorithms whose output “looks random”.
- What does it mean to “look random”?
- A polynomial-time TM can't distinguish them from random.
- Current research!



Next time...

- Cryptography!

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

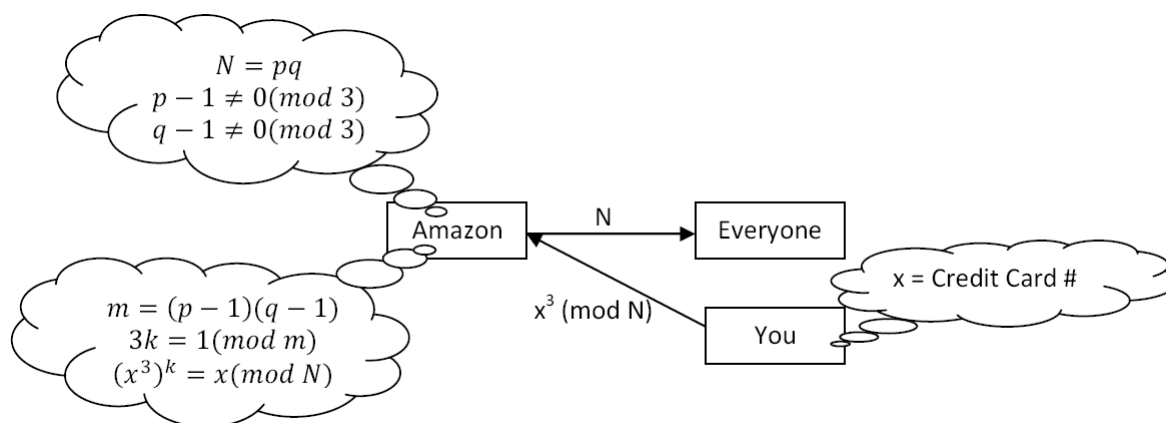
For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 18

*Lecturer: Scott Aaronson**Scribe: Hristo Paskov*

1 Recap

Last time we talked about public key cryptography which falls in the realm of accomplishing bizarre social goals using number theory. Our first example of a public-key cryptosystem, in which two people exchanging messages did not have to meet beforehand, was Diffie-Hellman. We then talked about the RSA cryptosystem, which is probably the most widely used today. Here are the basics of how it works:



The first step is taken by the recipient of the message, by generating two giant prime numbers p and q and setting $N = pq$. Note that p and q must be chosen such that $p - 1$ and $q - 1$ are not divisible by 3. The recipient keeps p and q a closely-guarded secret, but gives out N to anyone who asks. Suppose a sender has a secret message x that she wants to send to the recipient. The sender calculates $x^3 \pmod{N}$ and sends it to the recipient. Now it's the recipient's turn to recover the message. He can use some number theory together with the fact that he knows p and q , the factors of N . The recipient first finds an integer k such that $3k \equiv 1 \pmod{(p - 1)(q - 1)}$, which can be done in polynomial time via Euclid's algorithm, and then takes $(x^3)^k \pmod{N} = x^{3k} \pmod{N} = x$. The exponentiation can be done in polynomial time by using the trick of repeated squaring. Voila!

When you look at this procedure, you might wonder why are we cubing as opposed to raising to another power; is there anything special about 3? As it turns out, 3 is just the first choice that's convenient. Squaring would lead to a ciphertext that had multiple decryptions (corresponding to the multiple square roots $\pmod{N}$), while we want the decryption to be unique. Indeed, if we wanted the square root to be unique, then we'd need $p - 1$ and $q - 1$ to not be divisible by 2, which is a problem since p and q (being large prime numbers) are odd!

You could, however, raise to a power higher than 3, and in fact that's what people usually do. If the other components of the cryptosystem—such as the padding out of messages with random garbage—aren't implemented properly, then there's a class of attacks called “small-exponent attacks” which break RSA with small exponents though not with large ones. On the other hand, if everything else is implemented properly, then as far as we know $x^3 \pmod{N}$ is already secure.

(Just like in biology, everything in cryptography is always more complicated than what you said, *whatever* you said. In particular, as soon as you leave a clean mathematical model and enter the real world, where code is buggy, hardware inadvertently leaks information, etc. etc., there's always further scope for paranoia. And cryptographers are extremely paranoid people.)

As mentioned in the last lecture, we know that a fast factoring algorithm would lead to a break of RSA. However, we don't know the opposite direction: could you break RSA without factoring? In 1979, Rabin showed that if you squared the plaintext x instead of cubing it, then recovering x *would* be as hard as factoring. But as discussed earlier, in that case you'd lose the property that every decryption is unique. This problem is what's prevented widespread adoption of Rabin's variant of RSA.

2 Trapdoor One-Way Functions

The operation $x^3 \bmod N$ in RSA is an example of what's called *trapdoor one way function*, or TDOWF. A trapdoor one-way function is a one-way function with the additional property that if you know some secret "trapdoor" information then you can efficiently invert it. So for example, the function $f(x) = x^3 \bmod N$ is believed to be a one-way function, yet is easy to invert by someone who knows the prime factors of N .

2.1 Different Classes of TDOWF's

Question from the floor: Are there any candidate TDOWF's *not* based on modular arithmetic (like RSA is)?

Answer: One class that's been the subject of much recent research is based on lattices. (Strictly speaking, the objects in this class are not TDOWF's, but something called *lossy TDOWF's*, but they still suffice for public-key encryption.) Part of the motivation for studying this class is that the cryptosystems based on modular arithmetic could all be broken by a quantum computer, if we had one. By contrast, even with a quantum computer we don't yet know how to break lattice cryptosystems. Right now, however, lattice cryptosystems are not used much. Part of the problem is that, while the message and key lengths are polynomial in n , there are large polynomial blowups. Thus, these cryptosystems aren't considered to be as practical as RSA. On the other hand, in recent years people have come up with better constructions, so it's becoming more practical.

There's also a third class of public-key cryptosystems based on elliptic curves, and elliptic-curve cryptography *is* currently practical. Like RSA, elliptic-curve cryptography is based on abelian groups, and like RSA it can be broken by a quantum computer. However, elliptic-curve cryptography has certain nice properties that are not known to be shared by RSA.

In summary, we only know of a few classes of candidate TDOWF's, and all of them are based on *some* sort of interesting math. When you ask for a trapdoor that makes your one-way function easy to invert again, you're really asking for something mathematically special. It almost seems like an accident that plausible candidates exist at all! By contrast, if you just want an ordinary, *non*-trapdoor OWF, then as far as we know, all sorts of "generic" computational processes that scramble up the input will work.

3 NP-completeness and Cryptography

An open problem for decades has been to base cryptography on an NP-complete problem. There are strong heuristic arguments, however, that suggest that if this is possible, it'll require very

different ideas from what we know today. One reason (discussed last time) is that cryptography requires average-case hardness rather than worst-case. A second reason is that many problems in cryptography actually belong to the class $NP \cap coNP$. For example, given an encrypted message, we could ask if the first bit of the plaintext is 1. If it is, then a short proof is to decrypt the message. If it's not, then a short proof is *also* to decrypt the message. However, problems in $NP \cap coNP$ can't be NP-complete under the most common reductions unless $NP = coNP$.

3.1 Impagliazzo's Five Worlds

A famous paper by Impagliazzo discusses five possible worlds of computational complexity and cryptography, corresponding to five different assumptions you can make. You don't need to remember the names of the worlds, but I thought you might enjoy seeing them.

1. Algorithmica - $P = NP$ or at the least fast probabilistic algorithms exist to solve all NP problems.
2. Heuristica - $P \neq NP$, but while NP problems are hard in the worst case, they are easy on average.
3. Pessiland - NP-complete problems are hard on average *but* one-way functions don't exist, hence no cryptography
4. Minicrypt - One-way functions exist (hence private-key cryptography, pseudorandom number generators, etc.), but there's no public-key cryptography
5. Cryptomania - Public-key cryptography exists; there are TDOWF's

The reigning belief is that we live in Cryptomania, or at the very least in Minicrypt.

4 Fun with Encryption

4.1 Message Authentication

Besides encrypting a message, can you prove that a message actually came from you? Think back to the one-time pad, the first decent cryptosystem we saw. On its face, the one-time pad seems to provide authentication as a side benefit. Recall that this system involves you and a friend sharing a secret key k , you transmitting a message x securely by sending $y = x \oplus k$, and your friend decoding the message by computing $x = y \oplus k$. Your friend might reason as follows: if it was anyone other than you who sent the message, then why would $y \oplus k$ yield an intelligible message as opposed to gobbledygook?

There are some holes in this argument (see if you can spot any), but the basic idea is sound. However, to accomplish this sort of authentication, you do need the other person to share a secret with you, in this case the key. It's like a secret handshake of fraternity brothers.

Going with the analogy of private vs. public key cryptography, we can ask whether there's such a thing as public-key authentication. That is, if a person trusts that some public key N came from you, he or she should be able to trust any further message that you send as *also* coming from you. As a side benefit, RSA gives you this ability to authenticate yourself, but we won't go into the details.

4.2 Computer Scientists and Dating

Once you have cryptographic primitives like the RSA function, there are all sorts of games you can play. Take, for instance, the problem of Alice and Bob wanting to find out if they're both interested in dating each other. Being shy computer scientists, however, they should only find out they like each other if they're both interested; if one of them is *not* interested, then that one shouldn't be able to find out the other is interested.

An obvious solution (sometimes used in practice) would be to bring in a trusted mutual friend, Carl, but then Alice and Bob would have to trust Carl not to spill the beans. Apparently there are websites out there that give this sort of functionality. However, ideally we would like to not have to rely on a third party.

Suggestion from the floor: Alice and Bob could face each other with their eyes closed, and each open their eyes only if they're interested.

Response: If neither one is interested, then there seems to be a termination problem! Also, we'd like a protocol that doesn't require physical proximity – remember that they're shy computer scientists!

4.2.1 The Dating Protocol

So let's suppose Alice and Bob are at their computers, just sending messages back and forth. If we make no assumptions about computational complexity, then the dating task is clearly impossible. Why? Intuitively it's "obvious": because eventually one of them will have to say something, without yet knowing whether his or her interest will be reciprocated or not! And indeed one can make this intuitive argument more formal.

So we're going to need a cryptographic assumption. In particular, let's assume RSA is secure. Let's also assume, for the time being, that Alice and Bob are what the cryptographers call *honest but curious*. In other words, we'll assume that they can both be trusted to follow the protocol correctly, but that they'll also try to gain as much information as possible from whatever messages they see. Later we'll see how to remove the honest-but-curious assumption, to get a protocol that's valid even if one player is trying to cheat.

Before we give the protocol, three further remarks might be in order. First, the very fact that Alice and Bob are carrying out a dating protocol in the first place, might be seen as *prima facie* evidence that they're interested! So you should imagine, if it helps, that Alice and Bob are at a singles party where *every* pair of people has to carry out the protocol. Second, it's an unavoidable feature of any protocol that if one player is interested and the other one isn't, then the one who's interested will learn that the other one isn't. (Why?) Third, it's also unavoidable that one player could *pretend* to be interested, and then after learning of the other player's interest, say "ha ha! I wasn't serious. Just wanted to know if you were interested."

In other words, we can't ask cryptography to solve the problem of heartbreak, or of people being jerks. All we can ask it to do is ensure that each player can't learn whether the other player has stated an interest in them, without stating interest themselves.

Without further ado, then, here's how Alice and Bob can solve the dating problem:

1. Alice goes through the standard procedure of picking two huge primes, p and q , such that $p - 1$ and $q - 1$ are not divisible by 3, and then taking $N = pq$. She keeps p and q secret, but sends Bob N together with $x^3 \bmod N$ and $y^3 \bmod N$ for some x and y . If she's not interested, then x and y are both 0 with random garbage padded onto them. If she *is* interested, then x is again 0 with random garbage, but y is 1 with random garbage.

2. Assuming RSA is secure, Bob (not knowing the prime factors of N) doesn't know how to take cube roots mod N efficiently, so $x^3 \bmod N$ and $y^3 \bmod N$ both look completely random to him. Bob does the following: he first picks a random integer r from 0 to $N - 1$. Then, if he's not interested in Alice, he sends her $x^3 r^3 \bmod N$. If he *is* interested, he sends her $y^3 r^3 \bmod N$.
3. Alice takes the cube root of whatever number Bob sent. If Bob wasn't interested, this cube root will be $xr \bmod N$, while if he was interested it will be $yr \bmod N$. Either way, the outcome will look completely random to Alice, since she doesn't know r (which was chosen randomly). She then sends the cube root back to Bob.
4. Since Bob knows r , he can divide out r . We see that if Bob was not interested, he simply gets x which reveals nothing about Alice's interest. Otherwise he gets y which is 1 if and only if Alice is interested.

So there we have it. It seems that, at least in principle, computer scientists have solved the problem of flirting for shy people (assuming RSA is secure). This is truly nontrivial for computer scientists. However, this is just one example of what's called *secure multiparty computation*; a general theory to solve essentially all such problems was developed in the 1980's. So for example: suppose two people want to find out who makes more money, but without either of them learning anything else about the other's wealth. Or a group of friends want to know how much money they have *in total*, without any individual revealing her own amount. All of these problems, and many more, are known to be solvable cryptographically.

5 Zero-Knowledge Proofs

5.1 Motivation

In our dating protocol, we made the critical assumption that Alice and Bob were “honest but curious,” i.e. they both followed the protocol correctly. We'd now like to move away from this assumption, and have the protocol work even if one of the players is cheating. (Naturally, if they're *both* cheating then there's nothing we can do.)

As discussed earlier, we're not concerned with the case where Bob pretends that he likes Alice just to find out whether she likes him. There's no cryptographic protocol that helps with Bob being a jerk, and we can only hope he'll get caught being one. Rather, the situation we're concerned with is when one of the players *looks* like they're following the protocol, but are actually just trying to find out the other player's interest.

What we need is for Alice and Bob to *prove* to each other at each step of the protocol that they're correctly following the protocol—i.e., sending whatever message they're supposed to send, given whether they're interested or not. The trouble is, they have to do this without *revealing* whether they're interested or not! Abstractly, then, the question is how it's possible to prove something to someone without revealing a crucial piece of information on which the proof is based.

5.2 History

Zero-knowledge proofs have been a major idea in cryptography since the 1980's. They were introduced by Goldwasser, Micali, and Rackoff in 1985. Interestingly, their paper was rejected multiple times before publication but is now one of the foundational papers of theoretical computer science.

5.3 Interactive Proofs

For thousands of years, the definition of a proof accepted by mathematicians was a sequence of logical deductions that could be shared with anyone to convince them of a mathematical truth. But developments in theoretical computer science over the last few decades have required generalizing the concept of proof, to *any sort of computational process or interaction* that can terminate a certain way only if the statement to be proven is true. Zero-knowledge proofs fall into the latter category, as we'll see next.

5.4 Simple Example: Graph Nonisomorphism

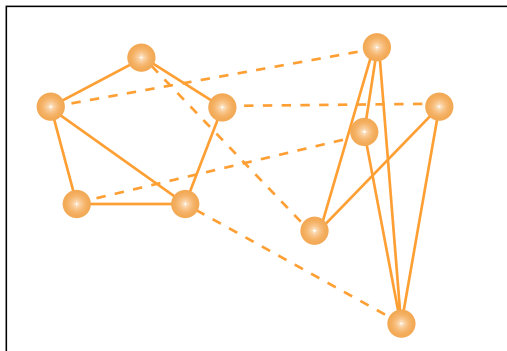


Figure by MIT OpenCourseWare.

To explain the concept of zero-knowledge proofs, it's easiest to start with a concrete example. The simplest example concerns the Graph Isomorphism problem. Here we're given two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, which are defined by lists of their edges and vertices. The graphs are called *isomorphic* if there's a way to permute their vertices so that they are the same graph.

5.4.1 Complexity

It's clear that the Graph Isomorphism problem is in NP, since a short proof that G_1 and G_2 are isomorphic is just to specify the isomorphism (i.e., a mapping between the vertices of G_1 and G_2).

Is Graph Isomorphism in P? Is it NP-complete? We don't yet know the answer to either question, though we do have strong evidence that it isn't NP-complete. Specifically, we know that if Graph Isomorphism is NP-complete then $NP^{NP} = coNP^{NP}$, or "the polynomial hierarchy collapses" (proving this statement is beyond the scope of the course). Some computer scientists conjecture that Graph Isomorphism is intermediate between P and NP-complete, just as we believe Factoring to be. Others conjecture that Graph Isomorphism is in P, and we simply don't know enough about graphs yet to give an algorithm. (Note that we have efficient algorithms for Graph Isomorphism that work extremely well *in practice* – just not any that we can prove will work in all cases.)

As an amusing side note, it's said that the reason Levin wasn't the first to publish on NP-completeness is that he got stuck trying to show the Graph Isomorphism problem was NP-complete.

5.4.2 Proving No Isomorphism Exists

We said before that Graph Isomorphism is in NP. But is it in coNP? That is, can you give a short proof that two graphs are *not* isomorphic? Enumerating all the possibilities obviously won't work, since it's exponentially inefficient (there are $n!$ possible mappings). To this day, we don't know whether Graph Isomorphism is in coNP (though there are some deep recent results suggesting that it is).

Still, let's see an incredibly simple way that an all-knowing prover could convince a polynomial-time verifier that two graphs are not isomorphic. To illustrate, consider Coke and Pepsi. Suppose you claim that the two drinks are different but I maintain they're the same. How can you convince me you're right, short of giving me the chemical formula for both? By doing a blind taste test! If I blindfold you and you can reliably tell which is which, then you'll have convinced me that they must be different, even if I don't understand how.

The same idea applies to proving that G_1 and G_2 are not isomorphic. Suppose you're some wizard who has unlimited computational power, but the person you are trying to convince does not. The person can pick one of the two graphs at random and permute the vertices in a random way to form a new graph G' , then send you G' and ask which graph she started with. If the graphs are indeed not isomorphic, then you'll be able to answer correctly every time, whereas if G_1 and G_2 are isomorphic, then your chance of guessing correctly will be at most $1/2$ (since a random permutation of G_1 is the same as a random permutation of G_2). If the verifier repeats this test 100 times and you answer correctly every time, then she can be sure to an extremely high confidence (namely $1 - 2^{-100}$) that the graphs are not isomorphic.

But notice something interesting: even though the verifier became convinced, she did so without gaining any new knowledge about G_1 and G_2 (by which, for example, she could convince someone else that they're not isomorphic)! In other words, if she'd merely trusted you, then she could have simulated her entire interaction with you on her own, without ever involving you at all. Any interactive proof system that has this property – that the prover only tells the verifier things that the latter “already knew” – is called a *zero-knowledge proof system*.

(Admittedly, it's only obvious that the verifier doesn't learn anything if she's “honest” – that is, if she follows the protocol correctly. Conceivably a *dishonest* verifier who violated the protocol could learn something she didn't know at the start. This is a distinction we'll see again later.)

5.5 The General Case

How can we extend this notion of a zero-knowledge proof to arbitrary problems, besides Graph Isomorphism? For example, suppose that you've proven the Riemann Hypothesis, but are paranoid and do not want anyone else to know your proof just yet. That might sound silly, but it's essentially how mathematicians worked in the Middle Ages: each knew how to solve some equation but didn't want to divulge the general method for solving it to competitors.

So suppose you have a proof of some arbitrary statement, and you want to convince people you have a proof without divulging any of the details. It turns out that there's a way to convert *any* mathematical proof into zero-knowledge form; what's more, the conversion can even be done in polynomial time. However, we'll need to make cryptographic assumptions.

5.6 Goldreich-Micali-Wigderson

In what follows, we'll assume that your proof is written out in machine-checkable form, in some formal system like Zermelo-Fraenkel set theory. We know that THEOREM, the problem of proving a theorem in at most n symbols, is an NP-complete problem, and is therefore efficiently reducible to any other NP-complete problem. Thus, we just need to find *some* NP-complete problem for which we can prove that we have a solution, without divulging the solution. Out of the thousands of known NP-complete problems, it turns out that the most convenient for our purpose will be the problem of 3-coloring a graph.

5.6.1 3-Coloring Proof

Suppose we have a 3-coloring of a graph and we want to prove that we have this 3-coloring without divulging it. Also, suppose that for each vertex of the graph, there's a magical box in which we can store the color of that vertex. What makes these boxes magical is that we can open them but the verifier can't. The key point is that, by storing colors in the boxes, we can “commit” to them: that is, we can assure the verifier that we've picked the color of each vertex beforehand, and are not just making them up based on which questions she asks.

Using these boxes, we can run the following protocol:

1. Start with a 3-coloring of the graph; then randomly permute the colors of the vertices. There are $3! = 6$ ways to permute the colors. For example, $\text{red} \Rightarrow \text{green}$, $\text{green} \Rightarrow \text{red}$, blue stays the same.
2. Write the color of each vertex on a slip of paper and place it in the magic box that's labeled with that vertex's number. Give all of the magic boxes to the verifier.
3. Let the challenger pick any two neighboring vertices, and open the boxes corresponding to those vertices.
4. Throw away the boxes and repeat the whole protocol as many times as desired.

If we really have a 3-coloring of the graph, then the verifier will see two different colors every time she chooses two neighboring vertices. On the other hand, suppose we were lying and didn't have a 3-coloring. Then eventually the verifier will find a conflict. Note that there are $O(n^2)$ edges, where n is the number of vertices of the graph. Therefore, since we commit to the colors in advance, there's a $\Omega(1/n^2)$ chance of catching us if we were lying. By repeating the whole protocol, say, n^3 times, the verifier can boost the probability of catching a lie exponentially close to 1, and can therefore (assuming everything checks out) become exponentially confident that we were telling the truth.

On the other hand, since we permute the colors randomly and reshuffle every time, the verifier learns nothing about the actual 3-coloring; she just sees two different random colors every time and thereby gains no knowledge!

Of course, the whole protocol relied on the existence of “magic boxes.” So what if we don't have the magic boxes available? Is there any way we could *simulate* their functionality, if we were just sending messages back and forth over the Internet?

Yes: using cryptography! Instead of locking each vertex's color in a box, we can *encrypt* each color and send the verifier the encrypted messages. Then, when the verifier picks two adjacent vertices and asks us for their colors, we can decrypt the corresponding messages (though not the encrypted messages for any other vertices). For this to work, we just need to ensure two things:

1. A polynomial-time verifier shouldn't be able to learn *anything* from the encrypted messages. In particular, this means that even if two vertices are colored the same, the corresponding encrypted messages should look completely different. Fortunately, this is easy to arrange, for example by padding out the color data with random garbage prior to encrypting it.
2. When, in the last step, we decrypt two chosen messages, we should be able to *prove* to the verifier that the messages were decrypted correctly. In other words, every encrypted message should have one and only one decryption. As discussed earlier, the most popular public-key cryptosystems, like RSA, satisfy this property by construction. But even with more

“generic” cryptosystems (based on arbitrary one-way functions), it’s known how to simulate the unique-decryption property by adding in more rounds of communication.

5.6.2 Back to Dating

Recall our original goal in discussing zero-knowledge: we wanted to make the dating protocol work correctly, even if Alice or Bob might be cheating. How can we do that? Well, first have Alice and Bob send each other encrypted messages that encode whether or not they’re interested in each other, as well as their secret numbers p , q , and r . Then have them follow the dating protocol exactly as before, but with one addition: *at each step, a player not only sends the message that’s called for in the protocol, but also provides a zero-knowledge proof that that’s exactly the message they were supposed to send—given the sequence of previous messages, whether or not they’re interested, and p, q, r .* Note that this is possible, since decrypting all the encrypted messages and verifying that the protocol is being followed correctly is an NP problem, which is therefore reducible to SAT and thence to 3-Coloring. And by definition, a zero-knowledge proof leaks no information about Alice and Bob’s private information, so the protocol remains secure.

To clarify one point, it’s not known how to implement this dating protocol using an arbitrary OWF—only how to implement the GMW part of it (the part that makes the protocol secure against a cheating Alice or Bob). To implement the protocol itself, we seem to need a stronger assumption, like the security of RSA or something similar. (Indeed, it’s not even known how to implement the dating protocol using an arbitrary *trapdoor* OWF, although if we know further that the trapdoor OWF is a permutation, then it’s possible.)

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 19

*Lecturer: Scott Aaronson**Scribe: Michael Fitzgerald*

1 Recap And Discussion Of Previous Lecture

In the previous lecture, we discussed different cryptographic protocols. People asked: “In the RSA cryptosystem, why do people raise to a power greater than three?” Raising to a power greater than three is an extra precaution; it’s like adding a second lock on your door. If everything has been implemented correctly, the RSA we discussed (cubing the message $(\text{mod } n)$) should be fine. This assumes that your message has been padded appropriately, however. If your message hasn’t been padded, “small-exponent attacks” can be successful at breaking RSA; sending the message to a bunch of different recipients with different public keys can let the attacker take advantage of the small exponent. Raising to a power greater than three mitigates this risk.

There are a couple of other attacks that can be successful. “Timing attacks” look at the length of time the computer takes to generate numbers to get hints as to what those numbers are. Other attacks can look at the electromagnetic waves coming from the computer to try and get hints about the number. Then there are attacks that abuse the cryptosystem with constructed inputs and try to determine some information about the system based on the error messages they receive. In general, modern cryptosystems are most often defeated when attackers find bugs in the *implementations* of the systems, not in the systems themselves. Social engineering remains the most successful way of breaching security; often just calling someone on the phone, pretending to be a company tech support person, and asking for their password will get you a response.

We also talked about zero-knowledge proofs and general interactive protocols in the last lecture. Twenty years ago, a revolution in the notion of “proof” drove home the point that a proof doesn’t have to be just a static set of symbols that someone checks for accuracy. For example, a proof can be an interactive process that ends with you being convinced of a statement’s truth, without learning much of anything else. We gave two examples of so-called *zero-knowledge protocols*: one that convinces a verifier that two graphs are not isomorphic, and another that proves *any* statement with a short conventional proof, assuming the existence of one-way functions.

2 More Interactive Proofs

It turns out that this notion of an interactive proof is good for more than just cryptography. It was discovered in the early 1990s that interactive proofs can convince you of solutions to problems that we think are much harder than *NP*-complete ones. As an analogy, it’s hard to tell that an author of a research paper knows what he’s talking about just from reading his paper. If you get a chance to ask him questions off the cuff and he’s able to respond correctly, it’s much more convincing. Similarly, if you can send messages back and forth with a prover, can you use that to convince yourself of more than just the solution to an *NP*-complete problem? To study this in the 1980s, people defined a complexity class called *IP*, which stands for “interactive proof.” The details of this story are beyond the scope of the class, but it’s being mentioned because it’s important.

Consider the following scenario. Merlin and Arthur are communicating. Merlin has infinite computational power, but he is not trustworthy. Arthur is a *PPT* (probabilistic polynomial time)

king; he can flip coins, generate random numbers, send messages back and forth with Merlin, etc. What we want from a good protocol is this: if Merlin is telling the truth, then there should be some strategy for Merlin that causes Arthur to accept with probability 1. On the other hand, if Merlin is lying, then Arthur should reject with probability greater than $1/2$, *regardless* of Merlin's strategy. These correspond to the properties of completeness and soundness that we discussed a while ago.

How big is the class IP , then? It certainly contains NP , as Merlin's strategy could just be to send a solution over to Arthur for the latter to check and approve. Is IP bigger than NP , though? Does interaction let you verify more statements than just a normal proof would? In 1990, Lund, Fortnow, Karloff, and Nisan showed that IP contains $coNP$ as well. This isn't obvious; the key idea in the proof involves how polynomials over finite fields can be judged as equal by testing them at random points. This theorem takes advantage of that fact, along with the fact that you can reinterpret a Boolean formula as a polynomial over a finite field. An even bigger bombshell came a month later, when Shamir showed that IP contains the entire class $PSPACE$, of problems solvable with polynomial memory. Since it was known that IP is *contained* in $PSPACE$, this yields the famous result $IP = PSPACE$.

What does this result mean, in intuitive terms? Suppose an alien comes to earth and says, "I can play perfect chess." You play the alien and it wins. But this isn't too surprising, since you're not very good at chess (for the purposes of this example, at least). The alien then plays against your local champion, then Kasparov, then Deep Blue, etc., and it beats them all. But just because the alien can beat anyone on earth, doesn't mean that it can beat anything in the universe! Is there any way for the alien to prove the stronger claim?

Well, remember that earlier we mentioned that a generalized $n \times n$ version of chess is a $PSPACE$ problem. Because of that, we can transform chess to a game about polynomials over finite fields. In this transformed game, the best strategy for one of the players is going to be to move randomly. Thus, if you play randomly against the alien in this transformed game and it wins, you can be certain (with only an exponentially small probability of error) that it has an optimal strategy, and could beat anyone.

You should be aware of this result, as well as the zero-knowledge protocol for the 3-Coloring, since they're two of the only examples we have in computational complexity theory where you take an NP -complete or $PSPACE$ -complete problem, and do something with it that actually exploits its *structure* (as opposed to just treating it as a generic search problem). And it's known that exploiting structure in this sort of way—no doubt, at an astronomically more advanced level—will someday be needed to solve the $P = NP$ problem.

3 Machine Learning

Up to this point, we've only talked about problems where all the information is explicitly given to you, and you just have to do something with it. It's like being handed a grammar textbook and asked if a sentence is grammatically correct. Give that textbook to a baby, however, and it will just drool on it; humans learn to speak and walk and other incredibly hard things (harder than anything taught at MIT) without ever being explicitly told how to do them. This is obviously something we'll need to grapple with if we ever want to understand the human brain. We talked before about how computer science grew out of this dream people had of eventually understanding the process of thought: can you reduce it to something mechanical, or automate it? At some point, then, we'll have to confront the problem of learning: inferring a general rule from specific examples when the rule is never explicitly given to you.

3.1 Philosophy Of Learning

As soon as we try to think about learning, we run into some profound philosophical problems. The most famous of these is the *Problem of Induction*, proposed by 18th-century Scottish philosopher David Hume. Consider two hypotheses:

1. The sun rises every morning.
2. The sun rises every morning until tomorrow, when it will turn into the Death Star and crash into Jupiter.

Hume makes the point that both of these hypotheses are completely compatible with all the data we have up until this point. They both explain the data we have equally well. We clearly believe the first over the second, but what grounds do we have for favoring one over the other? Some people say they believe the sun will rise because they believe in the laws of physics, but then the question becomes why they believe the laws of physics will continue.

To give another example, here's a "proof" of why it's not possible to learn a language, due to Quine. Suppose you're an anthropologist visiting a native tribe and trying to learn their language. One of the tribesmen points to a rabbit and says "gavagai." Can you infer that "gavagai" is their word for rabbit? Maybe gavagai is their word for food or dinner, or "little brown thing." By talking to them longer you could rule those out, but there are other possibilities that you haven't ruled out, and there will always be more. Maybe it means "rabbit" on weekdays but "deer" on weekends, etc.

Is there any way out of this? Right, we can go by Occam's Razor: if there are different hypotheses that explain the data equally well, we choose the simplest one.

Here's a slightly different way of saying it. What the above thought experiments really show is not the impossibility of learning, but rather the impossibility of learning in a theoretical vacuum. Whenever we try to learn something, we have some set of hypotheses in mind which is vastly smaller than the set of all logically conceivable hypotheses. That "gavagai" would mean "rabbit" is a plausible hypothesis; the weekday/weekend hypothesis does *not* seem plausible, so we can ignore it until such time as the evidence forces us to.

How, then, do we separate plausible hypotheses from hypotheses that aren't plausible? Occam's Razor seems related to this question. In particular, what we want are hypotheses that are *simpler than the data they explain*, ones that take fewer bits to write down than just the raw data. If your hypothesis is extremely complicated, and if you have to revise your hypothesis for every new data point that comes along, then you're probably doing something wrong.

Of course, it would be nice to have a theory that makes all of this precise and quantitative.

3.2 From Philosophy To Computer Science

It's a point that's not entirely obvious, but the problem of learning and prediction is related to the problem of data compression. Part of predicting the future is coming up with a succinct description of what has happened in the past. A philosophical person will ask why that should be so, but there might not be an answer. The belief that there are simple laws governing the universe has been a pretty successful assumption, so far at least. As an example, if you've been banging on a door for five minutes and it hasn't opened, a sane person isn't going to expect it to open on the next knock. This could almost be considered the definition of sanity.

If we want to build a machine that can make reasonable decisions and learn and all that good stuff, what we're really looking for is a machine that can create simple, succinct descriptions and

hypotheses to explain the data it has. What exactly is a “simple” description, then? One good way to define this is by Kolmogorov complexity; a simple description is one that corresponds to a Turing machine with few states. This is an approach that many people take. The fundamental problem with this is that Kolmogorov complexity is not computable, so we can’t really use this in practice. What we want is a quantitative theory that will let us deal with *any* definition of “simple” we might come up with. The question will then be: “given some class of hypotheses, if we want to be able to predict 90% of future data, how much data will we need to have seen?” This is where theoretical computer science really comes in, and in particular the field of *computational learning theory*. Within this field, we’re going to talk about a model of learning due to Valiant from 1984: the PAC (Probably Approximately Correct) model.

3.3 PAC Learning

To understand what this model is all about, it’s probably easiest just to give an example. Say there’s a hidden line on the chalk board. Given a point on the board, we need to classify whether it’s above or below the line. To help, we’ll get some sample data, which consists of random points on the board and whether each point is above or below the line. After seeing, say, twenty points, you won’t know *exactly* where the line is, but you’ll probably know roughly where it is. And using that knowledge, you’ll be able to predict whether most future points lie above or below the line.

Suppose we’ve agreed that predicting the right answer “most of the time” is okay. Is any random choice of twenty points going to give you that ability? No, because you could get really unlucky with the sample data, and it could tell you almost nothing about where the line is. Hence the “Probably” in PAC.

As another example, you can speak a language for your whole life, and there will still be edge cases of grammar that you’re not familiar with, or sentences you construct incorrectly. That’s the “Approximately” in PAC. To continue with that example, if as a baby you’re really unlucky and you only ever hear one sentence, you’re not going to learn much grammar at all (that’s the “Probably” again).

Let’s suppose that instead of a hidden line, there’s a hidden squiggle, with a side 1 and a side 2. It’s really hard to predict where the squiggle goes, just from existing data. If your class of hypotheses is arbitrary squiggles, it seems impossible to find a hypothesis that’s even probably approximately correct. But what is the difference between lines and squiggles, that makes one of them learnable and the other one not learnable?

Well, no matter how many points there are, you can always cook up a squiggle that works for those points, whereas the same is not true for lines. That seems related to the question somehow, but why?

What computational learning theory lets you do is delineate mathematically what it is about a class of hypotheses that makes it learnable or not learnable (we’ll get to that later).

3.4 Framework

Here’s the basic framework of Valiant’s PAC Learning theory, in the context of our line-on-the-chalkboard example:

S: Sample Space - The set of all the points on the blackboard.

D: Sample Distribution - The probability distribution from which the points are drawn (the uniform distribution in our case).

Concept - A function $h : S \rightarrow \{0, 1\}$ that maps each point to either 0 or 1. In our example, each concept corresponds to a line.

C : Concept Class - The set of all the possible lines.

“True Concept” $c \in C$: The actual hidden line; the thing you’re trying to learn.

In this model, you’re given a bunch of sample points drawn from S according to D , and each point comes with its classification. Your goal is to find a hypothesis $h \in C$ that classifies future points correctly almost all of the time:

$$\Pr_{x \in D}[h(x) = c(x)] \geq 1 - \epsilon$$

Note that the future points that you test on should be drawn from the same probability distribution D as the sample points. This is the mathematical encoding of the “future should follow from the past” declaration in the philosophy; it also encodes the well-known maxim that “nothing should be on the test that wasn’t covered in class.”

As discussed earlier, we won’t be able to achieve our goal with certainty, which is why it’s called *Probably Approximate Correct* learning. Instead, we only ask to succeed in finding a good classifier with probability at least $1 - \delta$ over the choice of sample points.

One other question: does the hypothesis h have to belong to the concept class C ? There are actually two notions, both of which we’ll discuss: *proper learning* (h must belong to C) and *improper learning* (h can be arbitrary).

These are the basic definitions for this theory.

Question from the floor: Don’t some people design learning algorithms that output confidence probabilities along with their classifications?

Sure! You can also consider learning algorithms that try to predict the output of a real-valued function, etc. Binary classification is just the simplest learning scenario – and for that reason, it’s a nice scenario to focus on to build our intuition.

3.5 Sample Complexity

One of the key issues in computational learning theory is *sample complexity*. Given a concept class C and a learning goal (the accuracy and confidence parameters ϵ and δ), how much sample data will you need to achieve the goal? Hopefully the number of samples m will be a finite number, but even more hopefully, it’ll a *small* finite number, too.

Valiant proposed the following theorem, for use with finite concept classes, which gives an upper bound on how many samples will suffice:

$$m \geq \frac{1}{\epsilon} \log \frac{|C|}{\delta}$$

As ϵ gets smaller (i.e., as we want a more accurate hypothesis), we need to see more and more data. As there are more concepts in our concept class, we also need to see more data.

A learning method that achieves Valiant’s bound is simply the following: find any hypothesis that fits all the sample data, and output it!

As long as you’ve seen m data points, the theorem says that with probability at least $1 - \delta$, you’ll have a classifier that predicts at least a $1 - \epsilon$ fraction of future data. There’s only a logarithmic dependency on $\frac{1}{\delta}$, which means we can learn within an exponentially small probability of error using only a polynomial number of samples. There’s also a log dependence on the number of concepts $|C|$, which means that even if there’s an exponential number of concepts in our concept class, we

can still do the learning with a polynomial amount of data. If that weren't true we'd really be in trouble.

Next time: proof of Valiant's bound, VC-dimension, and more...

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 20

*Lecturer: Scott Aaronson**Scribe: Geoffrey Thomas*

Probably Approximately Correct Learning

In the last lecture, we covered Valiant’s model of “Probably Approximately Correct” (PAC) learning. This involves:

- S : A *sample space* (e.g., the set of all points)
- D : A *sample distribution* (a probability distribution over points in the sample space)
- $c : S \rightarrow \{0, 1\}$: A *concept*, which accepts or rejects each point in the sample space
- C : A *concept class*, or collection of concepts

For example, we can take our sample space to be the set of all points on the blackboard, our sample distribution to be uniform, and our concept class to have one concept corresponding to each line (where a point is accepted if it’s above the line and rejected if it’s below it). Given a set of points, as well as which points are accepted or rejected, our goal is to *output a hypothesis* that explains the data: e.g., draw a line that will correctly classify most of the future points.

A bit more formally, there’s some “true concept” $c \in C$ that we’re trying to learn. Given sample points $x_1, \dots, x_m$, which are drawn independently from D , together with their classifications $c(x_1), \dots, c(x_m)$, our goal is to find a hypothesis $h \in C$ such that

$$\Pr[h(x) = c(x)] \geq 1 - \epsilon$$

. Furthermore, we want to succeed at this goal with probability at least $1 - \delta$ over the choice of x_i ’s. In other words, with high probability we want to output a hypothesis that’s approximately correct (hence “Probably Approximately Correct”).

How many samples to learn a finite class?

The first question we can ask concerns *sample complexity*: how many samples do we need to have seen to learn a concept effectively? It’s not hard to prove the following theorem: after we see

$$m = O\left(\frac{1}{\epsilon} \log \frac{|C|}{\delta}\right)$$

samples drawn from D , *any* hypothesis $h \in C$ we can find that agrees with all of these samples (i.e., such that $h(x_i) = c(x_i)$ for all i) will satisfy

$$\Pr[h(x) = c(x)] \geq 1 - \epsilon$$

with probability at least $1 - \delta$ over the choice of $x_1, \dots, x_m$.

We can prove this theorem by the contrapositive. Let $h \in C$ be any “bad” hypothesis: that is, such that $\Pr[h(x) = c(x)] < 1 - \epsilon$. Then if we independently pick m points from the sample distribution D , the hypothesis h will be correct on all of these points with probability at most $(1 - \epsilon)^m$. So by the union bound, the probability that there *exists* a bad hypothesis in C that nevertheless agrees with all our sample data is at most $|C|(1 - \epsilon)^m$ (the number of hypotheses,

good or bad, times the maximum probability of each bad hypothesis agreeing with the sample data). Now we just do algebra:

$$\begin{aligned}\delta &= |C| (1 - \epsilon)^m \\ m &= \log_{1-\epsilon} \frac{\delta}{|C|} \\ &= \frac{\log \delta / |C|}{\log 1 - \epsilon} \\ &\approx \frac{1}{\epsilon} \log \frac{|C|}{\delta}.\end{aligned}$$

Note that there always *exists* a hypothesis in C that agrees with c on all the sample points: namely, c itself (i.e. the truth)! So as our learning algorithm, we can simply do the following:

1. Find any hypothesis in $h \in C$ that agrees with all the sample data (i.e., such that $h(x_i) = c(x_i)$ for all $x_1, \dots, x_m$).
2. Output h .

Such an h will always exist, and by the theorem above it will probably be a good hypothesis. All we need is to see enough sample points.

How many samples to learn an infinite class?

The formula

$$m \approx \frac{1}{\epsilon} \log \frac{|C|}{\delta}$$

works so long as $|C|$ is finite, but it breaks down when $|C|$ is infinite. How can we formalize the intuition that the concept class of lines is learnable, but the concept class of arbitrary squiggles is not? A line seems easy to guess (at least approximately), if I give you a small number of random points and tell you whether each point is above or below the line. But if I tell you that *these* points are on one side of a squiggle, and *those* points are on the other side, then no matter how many points I give you, it seems impossible to predict which side the next point will be on.

So what's the difference between the two cases? It can't be the number of lines versus the number of squiggles, since they're both infinite (and be taken to have the same infinite cardinality).

From the floor: Isn't the difference just that you need two parameters to specify a line, but infinitely many parameters to specify a squiggle?

That's getting closer! The trouble is that the notion of a "parameter" doesn't occur anywhere in the theory; it's something we have to insert ourselves. To put it another way, it's possible to come up with silly parameterizations where even a line takes infinitely many parameters to specify, as well as clever parameterizations where a squiggle can be specified with just one parameter.

Well, the answer isn't obvious! The idea that finally answered the question is called *VC-dimension* (after two of its inventors, Vapnik and Chervonenkis). We say the set of points $x_1, \dots, x_m$ is *shattered* by a concept class C if for all 2^m possible settings of $c(x_1), \dots, c(x_m)$ to 0 or 1 (reject or accept), there is some concept $c \in C$ that agrees with those values. Then the VC-dimension of C , denoted $\text{VCdim}(C)$, is the size of the largest set of points shattered by C . If we can find arbitrarily large (finite) sets of points that can be shattered, then $\text{VCdim}(C) = \infty$.

If we let C be the concept class of lines in the plane, then $\text{VCdim}(C) = 3$. Why? Well, we can put three points in a triangle, and each of the eight possible classifications of those points can be realized by a single line. On the other hand, there's no set of four points such that all sixteen possible classifications of those points can be realized by a line. Either the points form a quadrilateral, in which case we can't make opposite corners have the same classification; or they form a triangle and an interior point, in which case we can't make the interior point have a different classification from the other three points; or three of the points are collinear, in which case we certainly can't classify those points with a line.

Blumer et al.¹ proved that a concept class is PAC-learnable if and only if its VC-dimension is finite, and that

$$m = O\left(\frac{\text{VCdim}(C)}{\epsilon} \log \frac{1}{\delta\epsilon}\right)$$

samples suffice. Once again, a learning algorithm that works is just to output *any* hypothesis h in the concept class that agrees with all the data. Unfortunately we don't have time to prove that here.

A useful intuition is provided by a corollary of Blumer et al.'s result called the *Occam's Razor Theorem*: whenever your hypothesis has sufficiently fewer bits of information than the original data, it will probably correctly predict most future data drawn from the same distribution.

Computational Complexity of Learning

We've seen that given a finite concept class—or even an infinite class with finite VC-dimension—after seeing enough sample points, you can predict the future just by finding any hypothesis in the class that fits the data. But how hard is it *as a computational problem* to find a hypothesis that fits the data? This has the general feel of something that might be NP-complete! In particular, it feels similar to satisfiability—find some hypothesis that satisfies certain fixed outputs—though it's not quite the same.

Here we need to make a subtle distinction. For *proper* learning—where the goal is to output a hypothesis in some fixed format (like a DNF expression), it's indeed possible to prove in some cases that finding a hypothesis that fits the data is NP-complete. For *improper* learning—where the hypothesis can be any polynomial-time algorithm so long as it predicts the data—to this day we don't know whether finding a hypothesis is NP-complete.

On the other hand, the learning problem is certainly *in NP*, since given a hypothesis it's easy to check whether it fits the data or not. This means that if $P = NP$, then all learning problems are in P and are computationally tractable. Think about what that means: we could ask our computer to find the shortest efficient description of the stock market, the patterns of neural firings in a human brain, etc., and thereby solve many of the hardest problems of AI! This is yet another reason to believe $P \neq NP$.

¹Blumer, Ehrenfeucht, Haussler, Warmuth, “Learnability and the Vapnik-Chervonenkis dimension”, JACM, 1989

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 21

*Lecturer: Scott Aaronson**Scribe: Scott Aaronson / Chris Granade*

1 Recap and Discussion of Previous Lecture

Theorem 1 (Valiant) $m = O\left(\frac{1}{\epsilon} \log(|C|/\delta)\right)$ samples suffice for (ϵ, δ) -learning.

Theorem 2 (Blumer et al.) $m = O\left(\frac{1}{\epsilon} \text{VCdim}(C) \log \frac{1}{\delta\epsilon}\right)$ samples suffice.

In both cases, the learning algorithm that achieves the bound is just “find any hypothesis h compatible with all the sample data, and output it.”

You asked great, probing questions last time, about what these theorems really mean. For example, “why can’t I just draw little circles around the ‘yes’ points, and expect that I can therefore predict the future?” It’s unfortunately a bit hidden in the formalism, but what these theorems are “really” saying is that to predict the future, it suffices to find a succinct description of the past—a description that takes many fewer bits to write down than the past data itself. Hence the dependence on $|C|$ or $\text{VCdim}(C)$: the size or dimension of the concept class from which our hypothesis is drawn.

We also talked about the computational problem of finding a small hypothesis that agrees with the data. Certainly we can always solve this problem in polynomial time if $P = NP$. But what if $P \neq NP$? Can we show that “learning is NP-hard”? Here we saw that we need to distinguish two cases:

Proper learning problems (where the hypothesis has to have a certain form): Sometimes we can show these are NP-hard. Example: Finding a DNF expression that agrees with the data.

Improper learning problems (where the hypothesis can be any Boolean circuit): It’s an open problem whether any of these are NP-hard. (Incidentally, why do we restrict the hypothesis to be a Boolean circuit? It’s equivalent to saying, we should be able to compute in polynomial time what a given hypothesis predicts.)

So, if we can’t show that improper (or “representation-independent”) learning is NP-complete, what other evidence might there be for its hardness? The teaser from last time: we could try to show that finding a hypothesis that explains past data is at least as hard as breaking some cryptographic code!

But how would we actually do this? How would we reduce a cryptanalysis problem to a learning problem? To be concrete, let’s just consider the RSA cryptosystem. Can any of you give me a PAC-learning problem, such that if you could solve it, then you could also break RSA?

How about this: our concept class C will have one concept c for each product of prime numbers $N = pq$, with $p - 1$ and $q - 1$ not divisible by 3. (Technically, for each N expressible with at most n bits.)

Our sample space S will consist of pairs of the form (y, i) , where $1 \leq y \leq N-1$ and $1 \leq i \leq \log N$. Here’s the trick: (y, i) will be in c if and only if the i^{th} bit of $y^{1/3} \bmod N$ is a 1. The sample distribution D will be uniform over S .

So basically, you (the learner) will be given a bunch of encrypted messages of the form $x^3 \bmod N$, and for each one, you'll also be told some bit of the plaintext x . Based on this “training” data, you need to infer the general rule for going from $x^3 \bmod N$ to some given bit of x .

First question: is there such a rule, which is expressible by a polynomial-size circuit? Sure there is! Namely, the rule that someone who knew the trapdoor information, who knew p and q , would use to decrypt the messages!

On the other hand, if you don't already know this rule, is there an efficient algorithm to infer it from sample data? Well, not if RSA is secure! The sample data—the set of (y, i) pairs—is stuff that an eavesdropper could not only plausibly have access to, but could actually generate itself! So if, by examining that data, the adversary could gain the ability to go from $x^3 \bmod N$ to a desired bit of x —well, then RSA is toast. (Today, it's actually known how to base the hardness of improper learning on the existence of any one-way function, not just the RSA function.) A beautiful connection between learning theory and cryptography—typical of the connections that abound in theoretical computer science.

1.1 RSA and Language Learning In Infants: The Argument Chomsky Should've Made

What is Noam Chomsky famous for, besides hating America? Linguistics, of course—among other things, what's called the “Poverty of the Stimulus Argument.” This is an argument that tries to show, more or less from first principles, that many of the basic ingredients of grammar (nouns, verbs, verb conjugation, etc.) must be hardwired into the human brain. They're not just taught to children by their parents: the children are “pre-configured” to learn grammar.

The argument says, suppose that weren't the case; suppose instead that babies started out as blank slates. Before it has to start speaking, a baby will hear, what, a hundred thousand sentences? Chomsky claims, with a bit of handwaving, that isn't nearly enough sentences to infer the general rules of grammar, the rules separating grammatical sentences from ungrammatical ones. The sample data available to the baby are just too impoverished for it to learn a language from scratch.

But here's the problem: the sample complexity bounds we saw earlier today should make us skeptical of any such argument! These bounds suggested that, in principle, it really *is* possible to predict the future given a surprisingly small amount of sample data. As long as the VC-dimension of your concept class is small—and I know I haven't convinced you of this, but in “most” practical cases it is—the amount of data needed for learning will be quite reasonable.

So the real stumbling block would seem to be not sample complexity, but computational complexity! In other words, if the basic ingredients of language weren't hardwired into every human baby, then even if in principle a baby has heard enough sentences spoken by its parents to infer the rules of the English language, how would the baby actually do the computation? It's just a baby!

More concretely, let's say I give you a list of n -bit strings, and I tell you that there's some nondeterministic finite automaton M , with much fewer than n states, such that each string was produced by following a path in M . Given that information, can you reconstruct M (probably and approximately)? It's been proven that if you can, then you can also break RSA! Now, finite automata are often considered the very simplest models of human languages. The grammar of any real human language is much too rich and complicated to be captured by a finite automaton. So this result is saying that even learning the least expressive, unrealistically simple languages is already as hard as breaking RSA!

So, using what we've learned about cryptography and computational learning theory, I submit

that we can now make the argument that Chomsky should have made but didn't. Namely: grammar must be hard-wired, since if a baby were able to pick up grammar from scratch, that baby could also break the RSA cryptosystem.¹

2 Quantum Computing

Up to now in this course, you might have gotten the impression that we're just doing pure math—and in some sense, we are! But for most of us, the real motivation comes from the fact that computation is not just some intellectual abstraction: it's something that actually takes place in our laptops, our brains, our cell nuclei, and maybe all over the physical universe. So given any of the models we've been talking about in this course, you can ask: does this mesh with our best understanding of the laws of physics?

Consider the Turing machine or the circuit models. For at least a couple of decades, there was a serious question: will it be possible to build a general-purpose computer that will scale beyond a certain size? With vacuum tubes, the answer really wasn't obvious. Vacuum tubes fail so often that some people guessed there was a fundamental physical limit on how complex (say) a circuit or a Turing machine tape head could be before it would inevitably fail. In the 1950s, John von Neumann (who we met earlier) became interested in this question, and he proved a powerful theorem: it's possible to build a reliable computer out of unreliable components (e.g., noisy AND, OR, and NOT gates), provided the failure probability of each component is below some critical threshold, and provided the failures are uncorrelated with each other.

But who knows if those assumptions are satisfied in the physical universe? What really settled the question was the invention of the transistor in 1947—which depended on understanding semiconductors (like silicon and germanium), which in turn depended on the quantum revolution in physics eighty years ago. In that sense, every computer we use today is a quantum computer.

But you might say, this is all for the EE people. Once you've got the physical substrate, then we theorists can work out everything else sitting in our armchairs. But can we really?

Consider: what do we mean by efficiently solvable problem? Already in this course, you've seen two plausible definitions: P (the class of problems solvable by polytime deterministic algorithms) and BPP (the class solvable by polytime randomized algorithms with bounded error probability). The fact that we already had to change models once—from P to BPP —should make you suspicious, and this despite the fact that nowadays we conjecture that $P = BPP$. Could nature have *another* surprise in store for us, besides randomness?

¹The ideas in this section are developed in much more detail in Ronald de Wolf's Masters thesis, "Philosophical Applications of Computational Learning Theory": <http://homepages.cwi.nl/~rdewolf/publ/philosophy/phthesis.pdf>

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 22/23

*Lecturer: Scott Aaronson**Scribe: Chris Granade*

1 Quantum Mechanics

1.1 Quantum states of n qubits

If you have an object that can be in two perfectly distinguishable states $|0\rangle$ or $|1\rangle$, then it can also be in a superposition of the $|0\rangle$ and $|1\rangle$ states:

$$\alpha|0\rangle + \beta|1\rangle$$

where α and β are complex numbers such that:

$$|\alpha|^2 + |\beta|^2 = 1$$

For simplicity, let's restrict to real amplitudes only. Then, the possible states of this object—which we call a quantum bit, or qubit—lie along a circle.

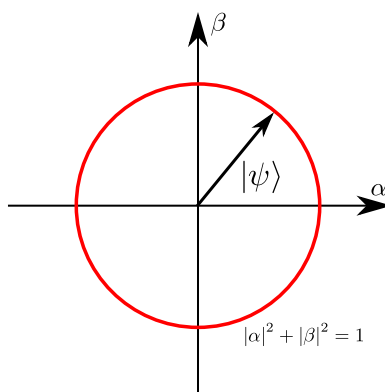


Figure 1: An arbitrary single-qubit state $|\psi\rangle$ drawn as a vector.

If you measure this object in the “standard basis,” you see $|0\rangle$ with probability $|\alpha|^2$ and $|1\rangle$ with probability $|\beta|^2$. Furthermore, the object “collapses” to whichever outcome you see.

1.2 Quantum Measurements

Measurements (yielding $|x\rangle$ with probability $|\alpha_x|^2$) are *irreversible, probabilistic, and discontinuous*.

As long as you don’t ask specifically what a measurement *is*—how the universe knows what constitutes a measurement and what doesn’t—but just assume it as an axiom, everything is well-defined mathematically. If you do ask, you enter a no-man’s land. Recently there’s been an important set of ideas, known as decoherence theory, about how to explain measurement as ordinary unitary interaction, but they still don’t explain where the probabilities come from.

1.3 Unitary transformations

But this is not yet interesting! The interesting part is what else we can do the qubit, besides measure it right away. It turns out that, by acting on a qubit in a suitable way—in the case of an electron, maybe shining a laser on it—we can effectively multiply the vector of amplitudes by any matrix that preserves the property that the probabilities sum to 1. By which I mean, any matrix that always maps unit vectors to other unit vectors. We call such a matrix a unitary matrix. Unitary transformations are *reversible, deterministic, and continuous*.

Examples of unitary matrices:

- The identity I .
- The NOT gate $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.
- The phase- i gate $\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$.
- 45-degree counterclockwise rotation.

Physicists think of quantum states in terms of the Schrödinger equation, $\frac{d|\psi\rangle}{dt} = iH|\psi\rangle$ (perhaps the third most famous equation in physics after $E = mc^2$ and $F = ma$). A unitary is just the result of leaving the Schrödinger equation “on” for a while.

Q: Why do we use complex numbers?

Scott: The short answer is that it works! A “deeper” answer is that if we used real numbers only, it would not be possible to divide a unitary into arbitrarily small pieces. For example, the NOT gate we saw earlier can’t be written as the square of a real-valued unitary matrix. We’ll see in a moment that you can do this if you have complex numbers.

For each of these matrices, what does it do? Why is it unitary? How about this one?

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Is it unitary? Given a matrix, how do you decide if it’s unitary or not?

Theorem 1 U is unitary if and only if $UU^* = I$, where U^* means you transpose the matrix and replace every entry by its complex conjugate. (A nice exercise if you’ve seen linear algebra.) Equivalently, $U^{-1} = U^*$. One corollary is that every unitary operation is reversible.

As an exercise for the reader, you can apply this theorem to find which of the matrices we’ve already seen are unitary.

Now, let's see what happens when we take the 45-degree rotation matrix, and apply it twice to the same state.

$$\begin{aligned} |0\rangle &\rightarrow (|0\rangle + |1\rangle) / \sqrt{2} \\ |1\rangle &\rightarrow (-|0\rangle + |1\rangle) / \sqrt{2} \\ (|0\rangle + |1\rangle) / \sqrt{2} &\rightarrow \left[\frac{|0\rangle + |1\rangle}{\sqrt{2}} + \frac{-|0\rangle + |1\rangle}{\sqrt{2}} \right] / \sqrt{2} \\ &= |1\rangle \end{aligned}$$

This matrix acts as the “square root of NOT”! Another way to see that is by squaring the matrix.

$$\begin{bmatrix} \cos(45^\circ) & -\sin(45^\circ) \\ \sin(45^\circ) & \cos(45^\circ) \end{bmatrix}^2 |\psi\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} |\psi\rangle$$

Already, we have something that doesn't exist in the classical world.

We can also understand the action of this matrix in terms of interference of amplitudes.

2 Two Qubits

To describe two qubits, how many amplitudes do we need? Right, four – one for each possible two-bit string.

$$\begin{aligned} \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle \\ |\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1 \end{aligned}$$

If you measure both qubits, you'll get $|00\rangle$ with probability $|\alpha|^2$, $|01\rangle$ with probability $|\beta|^2$, etc. And the state will collapse to whichever 2-bit string you see.

But what happens if you measure only the first qubit, not the second? With probability $|\alpha|^2 + |\beta|^2$, you get $|0\rangle$, and the state collapses to $\frac{\alpha|00\rangle + \beta|01\rangle}{\sqrt{|\alpha|^2 + |\beta|^2}}$. With probability $|\gamma|^2 + |\delta|^2$, you get $|1\rangle$, and the state collapses to $\frac{\gamma|10\rangle + \delta|11\rangle}{\sqrt{|\gamma|^2 + |\delta|^2}}$. Any time you ask the universe a question, it makes up its mind; any time you don't it ask a question, it puts off making up its mind for as long as it can.

What happens if you apply a NOT gate to the second qubit? Answer: You get $\beta|00\rangle + \alpha|01\rangle + \delta|10\rangle + \gamma|11\rangle$. “For every possible configuration of the other qubits, what happens if I apply the gate to this qubit?” If we consider $(\alpha, \beta, \gamma, \delta)$ as a vector of four complex numbers, what does this transformation look like as a 4×4 matrix?

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Can we always factor a two-qubit state: “here's the state of the first qubit, here's the state of the second qubit?” Sometimes we can:

- $|01\rangle = |0\rangle |1\rangle = |0\rangle \otimes |1\rangle$ (read $|0\rangle$ “tensor” $|1\rangle$).
- $|00\rangle + |01\rangle + |10\rangle + |11\rangle = \frac{1}{2} (|0\rangle + |1\rangle) (|0\rangle + |1\rangle)$.

In these cases, we say the state is **separable**. But what about $|00\rangle + |11\rangle$? This is a state that *can't* be factored. We therefore call it an **entangled** state. You might have heard about entanglement as one of the central features of quantum mechanics. Well, here it is.

Just as there are quantum states that can't be decomposed, there are also *operations* that can't be decomposed. Perhaps the simplest is the **Controlled-NOT**, which maps $|x\rangle|y\rangle$ to $|x\rangle|x \oplus y\rangle$ (i.e., flips the second bit iff the first bit is 1).

$$\begin{aligned} |00\rangle &\rightarrow |00\rangle \\ |01\rangle &\rightarrow |01\rangle \\ |10\rangle &\rightarrow |11\rangle \\ |11\rangle &\rightarrow |10\rangle \end{aligned}$$

What does this look like as a 4×4 matrix?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Incidentally, could we have a 2-qubit operation that mapped $|x\rangle|y\rangle$ to $|x\rangle|x \text{ AND } y\rangle$? Why not?

$$\begin{aligned} |0\rangle|0\rangle &\rightarrow |0\rangle|0\rangle \\ |0\rangle|1\rangle &\rightarrow |0\rangle|0\rangle \end{aligned}$$

This is not reversible!

2.1 Obtaining Entanglement

Before we can create a quantum computer, we need some way to entangle the qubits so they're not just a bunch of particles laying around. Perhaps the simplest such operation is the CNOT gate that we saw earlier.

So how do we use CNOT to produce entanglement? We can use a Hadamard followed by a CNOT, where the Hadamard matrix $\boxed{\text{H}}$ puts a qubit into superposition by switching between the $\{|0\rangle, |1\rangle\}$ basis and the $\{|+\rangle, |-\rangle\}$ basis.

$$\begin{aligned} |+\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ |-\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ \boxed{\text{H}} &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \end{aligned}$$

Applying $\boxed{\text{H}}$ to $|0\rangle$ and $|1\rangle$ results in:

$$\begin{aligned} |0\rangle &\rightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle \\ |1\rangle &\rightarrow \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle \end{aligned}$$

Already with two qubits, we're in a position to see some profound facts about quantum mechanics that took people decades to understand.

Think again about the state $|00\rangle + |11\rangle$. What happens if you measure just the first qubit? Right, with probability $1/2$ you get $|00\rangle$, with probability $1/2$ you get $|11\rangle$. Now, why might that be disturbing? Right: because the second qubit might be light-years away from the first one! For a measurement of the first qubit to *affect the second qubit* would seem to require faster-than-light communication! This is what Einstein called “spooky action at a distance.”

But think about it more carefully. Can you actually use this effect to send a message faster than light? What would happen if you tried? Right, the result would be random! In fact, we're not going to prove it here, but there's something called the *no-communication theorem*, which says *nothing* you do to the first qubit only can affect the probability of any measurement outcome on the second qubit only.

But in that case, why can't we just imagine that at the moment these two qubits were created, they flipped a coin, and said, “OK, if anyone asks, we'll both be 1.” Well, because in 1964, John Bell proved there are certain experiments where no explanation of that kind can possibly agree with quantum mechanics. And in the 1980s, the experiments were actually done, and they vindicated quantum mechanics and in most physicists' view, dashed Einstein's hope for a “completion” of quantum mechanics. That's on your problem set.

2.2 No-Cloning Theorem

Is it possible to duplicate a quantum state? This would be very nice, since we know we only have one chance to measure a quantum state. Here is what such a duplication would look like:

$$\alpha|0\rangle + \beta|1\rangle \rightarrow (\alpha|0\rangle + \beta|1\rangle)(\alpha|0\rangle + \beta|1\rangle) = \alpha^2|00\rangle + \alpha\beta|01\rangle + \alpha\beta|10\rangle + \beta^2|11\rangle$$

This operation is not possible because it is not linear. The final amplitudes α^2 , β^2 and $\alpha\beta$ don't depend linearly on α and β . That's the **no-cloning theorem**, and it's really as simple as it looks.

3 n Qubits

For 60 years, *these* were the sorts of examples that drove people's intuitions about quantum mechanics: one particle, occasionally two particles. Rarely did people think abstractly about hundreds or thousands of particles all entangled with one another. But within the last 15 years, we've realized that's where things get *really* crazy. And that brings us to quantum computing. It goes without saying that I'm going to present just the theory at first. Later we can discuss where current experiments are.

How many amplitudes would we need to describe the state of 1000 qubits? Right, 2^{1000} . One for every possible string of 1000 bits:

$$\sum_{x \in \{0,1\}^{1000}} \alpha_x |x\rangle$$

Think about what this *means*. To keep track of the state of 1000 numbers, Nature, off to the side somewhere, apparently has to write down this list of 2^{1000} complex numbers. That's more numbers than there are atoms in the visible universe. Think about how much *computing power* Nature must be expending for that. What a colossal waste! The next thought: we might as well try and take advantage of it!

Q: Doesn't a single qubit already require an infinite amount of information to specify?

Scott: The answer is yes, but there is always noise and error in the real world, so we only care about approximating the amplitudes to some finite precision. In some sense, the “infinite amount of information” is just an artifact of our mathematical description of the qubit's state. By contrast, the exponent in the description of n entangled particles is not an artifact; it's real (if quantum mechanics is the right description of Nature).

3.1 Exploiting Interference

What's an immediate difficulty with taking advantage of this computational power? Well, if we simply measure n qubits, all we get is a classical n -bit string; everything else disappears. It's like the instant we look, nature tries to “hide” the fact that it's doing an exponential amount of computation.

But luckily for us, Nature doesn't always do a good job of hiding. A good example of this is the double-slit experiment: we don't measure which of the two slits the photon passed through, but rather the resulting interference pattern. In particular, we saw that the different paths taken by a quantum system can *interfere destructively* and cancel each other out.

So *that's* what we want to exploit in quantum computing. The goal is to choreograph things so that the different computational paths leading to a given wrong answer interfere destructively and cancel each other out, while the different paths leading to a given right answer interfere constructively, hence the right answers are observed with high probability when we measure. You can see how this is gonna be tricky, if it's possible at all.

A key point about interference is that for two computation paths to destructively interfere with each other, they must lead to outcomes that are identical in *every respect*. To calculate the amplitude of a given outcome, you add up the amplitudes for all of the paths leading to that outcome; destructive interference is when the amplitudes cancel each other out.

3.2 Universal Set of Quantum Gates

Concretely, in a quantum computer we have n qubits, which we assume for simplicity start out all in the $|0\rangle$ state. Given these qubits, we apply a sequence of unitary transformation called “quantum gates.” These gates form what's called a *quantum circuit*.

An example of such a circuit is shown below, where we apply the Hadamard to the first qubit, then do a CNOT with the second qubit acting as the control bit. Written out, the effect is $\left(\frac{|0\rangle+|1\rangle}{\sqrt{2}}\right) |0\rangle \xrightarrow{\text{CNOT}} \frac{|00\rangle+|11\rangle}{\sqrt{2}}$, the result being entangled qubits, as we discussed before. A crucial

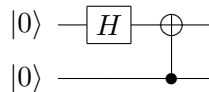


Figure 2: Entangling two qubits

point: each individual gate in a quantum circuit has to be extremely “simple”, just like a classical circuit is built of AND, OR, NOT gates, the simplest imaginable building blocks. What does “simple” mean in the quantum case? Basically, that each quantum gate acts on at most (say) 2

or 3 qubits, and is the identity on all the other qubits. Why do we need to assume this? *Because physical interactions are local.*

To work with this constraint, we want a *universal set of quantum gates* that we can use to build more complex circuits, just like AND, OR, and NOT in classical computers. This universal set must contain 1-, 2-, and 3-qubit gates that can be combined to produce any unitary matrix.

We have to be careful when we say *any* unitary matrix, since there are uncountably infinitely many unitary matrices (you can rotate by any real-number angle, for instance). However, there are small sets of quantum gates that can be used to *approximate* any unitary matrix to arbitrary precision. As a technical note, the word “universal” has different meanings; for example, we usually call a set of gates universal if it can be used to approximate any unitary matrix *involving real numbers only*; this certainly suffices for quantum computation.

We’ve already seen the Hadamard and CNOT gates, but unfortunately these aren’t sufficient to be a universal set of quantum gates. According to the Gottesman-Knill Theorem, any circuit constructed with just Hadamard and CNOT gates can be simulated efficiently with a classical computer. However, the Hadamard matrix paired with another gate called the **Toffoli gate** (also called controlled-controlled-NOT, or CCNOT) *is* sufficient to be used as a universal set of gates (for real-valued matrices).

The Toffoli gate will act similarly to the CNOT gate, except that we will control based on the first *two* qubits:

$$|x\rangle |y\rangle |z\rangle \rightarrow |x\rangle |y\rangle |z \oplus xy\rangle$$

where xy indicates the Boolean AND of x and y .

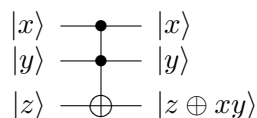


Figure 3: The Toffoli Gate diagram

Note, however, that these are not the only two gates whose combination allows for universal quantum computation. Another example of a universal pair of gates is the CNOT gate taken with the $\pi/8$ gate. We represent the $\pi/8$ gate using the following unitary:

$$T = \begin{bmatrix} \cos(\pi/8) & \sin(\pi/8) \\ -\sin(\pi/8) & \cos(\pi/8) \end{bmatrix}$$

But how many of these gates would be needed to approximate a random n -qubit unitary? Well, you remember Shannon’s counting argument? What if we tried something similar in the quantum world? An n -qubit unitary has roughly $2^n \times 2^n$ degrees of freedom. On the other hand, the number of quantum circuits of size T is “merely” exponential in T . Hence, we need $T = \exp(n)$.

We, on the other hand, are only interested in the tiny subset of unitaries that can be built up out of a *polynomial* number of gates. Polynomial time is still our gold standard.

So, a quantum circuit has this polynomial number of gates, and then, at the end, *something* has to be measured. For simplicity, we assume a single qubit is measured. (Would it make a difference if there were intermediate measurements? No? Why not? Because we can simulate measurements using CNOTs.) Just like with BPP, we stipulate that if $x \in L$ (the answer is “yes”), then the

measurement outcome should be $|1\rangle$ with probability at least $2/3$, while if $x \notin L$ (the answer is “no”), then the measurement outcome should be $|1\rangle$ with probability at most $1/3$.

There’s a final, technical requirement. We have to assume there’s a classical polynomial-time algorithm to *produce* the quantum circuit in the first place. Otherwise, how do we find the circuit?

The class of all decision problems L that can be solved by such a family of quantum circuits is called BQP (Bounded-Error Quantum Polynomial Time).

4 Bounded-Error Quantum Polynomial Time (BQP)

Bounded-Error Quantum Polynomial Time (BQP) is, informally, the class of problems that can be efficiently solved by a quantum computer.

Incidentally: the idea of quantum computing occurred independently to a bunch of people in the 70s and 80s, but is usually credited to Richard Feynman and David Deutsch. BQP was defined by Bernstein and Vazirani in 1993.

4.1 Requirements for a BQP circuit

To be in BQP, a problem has to satisfy a few requirements:

Polynomial Size. How many of our building-block circuits (e.g., Hadamard and Toffoli) do we need to approximate an arbitrary n -qubit unitary? The answer is the quantum analogue to Shannon’s counting argument. An n -qubit unitary has $2^n \times 2^n$ degrees of freedom, and there are doubly-exponentially many of them. On the other hand, the number of quantum circuits of size T is “merely” exponential in T . Hence, “almost all” unitaries will require an exponential number of quantum gates.

However, we are only interested in the small subset of unitaries that can be built using a *polynomial* number of gates. Polynomial time is still the gold standard.

Output. For simplicity, we assume that we measure a single qubit at the end of a quantum circuit. Just like with BPP, we stipulate that:

$$\text{Output} = \begin{cases} \text{if } x \in L : & |1\rangle \text{ with probability } \geq \frac{2}{3} \\ \text{if } x \notin L : & |1\rangle \text{ with probability } \leq \frac{1}{3} \end{cases}$$

Circuit Construction. There is a final technical requirement to constructing quantum circuits. We have to assume that there is a classical polynomial-time algorithm to *produce* the quantum circuit in the first place. Otherwise, how do we find the circuit?

4.2 BQP’s Relation to Other Algorithm Families

$P \subseteq BQP$: A quantum computer can always simulate a classical one (like using an airplane to drive down the highway). We can use the CNOT gate to simulate the NOT gate, and the Toffoli gate to simulate the AND gate.

$BPP \subseteq BQP$: Loosely speaking, in quantum mechanics we “get randomness for free.” More precisely, any time we need to make a random decision, all we need to do is apply a Hadamard

to some $|0\rangle$ qubit, putting it into an equal superposition of $|0\rangle$ and $|1\rangle$ states. Then we can CNOT that bit wherever we needed a random bit. We're not exploiting interference here; we're just using quantum mechanics as a source of random numbers.

BQP $\subseteq$ EXP: In exponential time, we can always write out a quantum state as an exponentially long vector of amplitudes, then explicitly calculate the effect of each gate in a quantum circuit.

BQP $\subseteq$ PSPACE: We can calculate the probability of each measurement outcome $|x\rangle$ by summing the amplitudes of all paths that lead to $|x\rangle$, which only takes polynomial space, as was shown by Bernstein and Vazirani. We won't give a detailed proof here.

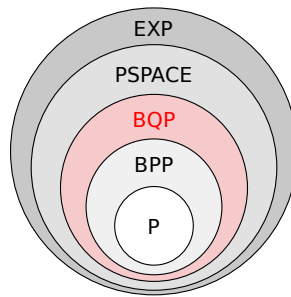


Figure 4: BQP inclusion diagram

We can draw a crucial consequence from this diagram, the first major contribution that complexity theory makes to quantum computing. Namely: in our present state of knowledge, there's little hope of proving unconditionally that quantum computers are more powerful than classical ones, since any proof of $P \neq BQP$ would also imply $P \neq PSPACE$.

5 Next Time: Quantum Algorithms

Next class we'll see some examples of quantum algorithms that actually outperform their classical counterparts:

- The Deutsch-Jozsa Algorithm
- Simon's Algorithm
- Shor's Algorithm

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.

Lecture 24

*Lecturer: Scott Aaronson**Scribe: Chris Granade*

1 Quantum Algorithms

Of course the real question is: can quantum computers actually do something more efficiently than classical computers? In this lecture, we'll see why the modern consensus is that they can.

1.1 Computing the XOR of Two Bits

We'll first see an algorithm due to Deutsch and Jozsa. Even though this algorithm is trivial by modern standards, it gave the first example where a quantum algorithm could provably solve a problem using fewer resources than a classical algorithm.

Suppose we're given access to a Boolean function $f : \{0, 1\} \rightarrow \{0, 1\}$. And suppose we want to compute $f(0) \oplus f(1)$, the XOR of $f(0)$ and $f(1)$. Classically, how many times would we need to evaluate f ? It's clear that the answer is twice: knowing only $f(0)$ or $f(1)$ tells us exactly nothing about their XOR.

So what about in the quantum case? Well, first we need to say what it even *means* to evaluate f . Since this is a quantum algorithm we're talking about, we should be able to evaluate both inputs, $f(0)$ and $f(1)$ in quantum superposition. But we have to do so in a reversible way. For example, we can't map the state $|x, b\rangle$ to $|x, f(x)\rangle$ (overwriting b), since that wouldn't be unitary.

The standard solution is that querying f means applying a unitary transformation that maps $|x, y\rangle \rightarrow |x, y \oplus f(x)\rangle$. Is it reversible? Yeah. Applying it twice gets you back to where you started. I claim we can compute $f(0) \oplus f(1)$ using just a single one of these operations. How?

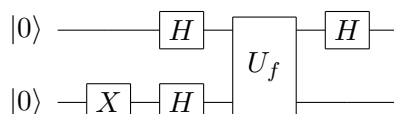


Figure 1: Finding $f(0) \oplus f(1)$ in one query.

In the circuit above, the effect of the gates before U_f is to prepare an initial state $|\psi_0\rangle$:

$$|\psi_0\rangle = |+\rangle |-\rangle = \frac{1}{2} [|0\rangle + |1\rangle] [|0\rangle - |1\rangle]$$

If you think of the effect of U_f on the first qubit in this state, it's just to negate the amplitude if $f(0) \neq f(1)$! Thus, U_f produces $|+\rangle |-\rangle$ if $f(0) = f(1)$ and $|-\rangle |-\rangle$ otherwise. The final Hadamard gate transforms the first qubit back into the computational basis, so that we measure 1 if and only if $f(0) \neq f(1)$.

In particular, this means that if you want to compute the XOR of N bits with a quantum computer, you can do so using $N/2$ queries, as follows: first divide the bits into $N/2$ pairs of bits,

then run the above algorithm on each pair, and finally output the XOR of the results. Of course, this is only a constant-factor speedup, but it's a harbinger of much more impressive speedups to come.

1.2 Simon's Algorithm

Say you're given a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$. You're promised there exists a "secret string" s such that $f(x) = f(y)$ if and only if $y = x \oplus s$, where $\oplus$ denotes a sum mod 2. The problem is to find s by querying f as few times as possible.

How many queries would a classical randomized algorithm need to solve this problem? Something very similar was on your problem set! Right, $2^{n/2}$. This is basically just the birthday paradox. Until it happens to find an x, y pair such that $f(x) = f(y)$, your algorithm is basically just "shooting in the dark"; it has essentially no information about s . And after T queries, the probability of having found an x, y pair such that $f(x) = f(y)$ is at most $T^2/(2^n - 1)$ (why?).

On the other hand, in 1993 Daniel Simon gave a quantum algorithm that solves this problem in polynomial time, in fact using only $O(n)$ queries. This was the first example of a problem that a quantum computer can solve exponentially faster than a classical one. Admittedly, it's a contrived example (and probably for that reason, Simon's paper was originally rejected!). But it's good to see for two reasons: first, it led directly to Shor's factoring algorithm. And second, the easiest way to *understand* Shor's algorithm is to understand Simon's algorithm, and then see Shor's algorithm as the same thing with a different underlying group!

Before proceeding further, though, there's one thing I want to clear up. I said that Simon's problem was the first known example where quantum computers provably give an exponential speedup over classical computers. How is that consistent with what I said before, that we can't prove $P \neq BQP$ unconditionally?

Right, Simon's problem involves the function f as a "black-box." In the black-box setting, we *can* prove unconditionally that quantum computers give an exponential speedup over classical ones.

1.3 RSA

Alright, so let's say you want to break the RSA cryptosystem, in order to rob some banks, read your ex's email, whatever. We all know that breaking RSA reduces to finding the prime factors of a large integer N . Unfortunately, we also know that "trying all possible divisors in parallel," and then instantly picking the right one, isn't going to work. Hundreds of popular magazine articles notwithstanding, trying everything in parallel just isn't the sort of thing that a quantum computer can do. Sure, in some sense you can "try all possible divisors" – but if you then measure the outcome, you'll get a random potential divisor, which almost certainly won't be the one you want.

What this means is that, if we want a fast quantum factoring algorithm, we're going to have to exploit some *structure* in the factoring problem: in other words, some mathematical property of factoring that it *doesn't* share with just a generic problem of finding a needle in a haystack.

Fortunately, the factoring problem has oodles of special properties. What are some examples we discussed in class? Right: if I give you a positive integer, you might not know its prime factorization, but you do know that it has exactly *one* factorization! By contrast, if I gave you (say) a Sudoku puzzle and asked you to solve it, *a priori* you'd have no way of knowing whether it had exactly one solution, 200 million solutions, or no solutions at all. Of course, knowing that there's exactly one needle in a haystack is still not much help in finding the needle! But this uniqueness is a hint that

the factoring problem might have *other* nice mathematical properties lying around for the picking. As it turns out, it does.

The property we'll exploit is the reducibility of factoring to another problem, called period-finding. OK, time for a brief number theory digression. Let's look at the powers of 2 mod 15:

$$2, 4, 8, 1, 2, 4, 8, 1, 2, 4, \dots$$

As you can see, taking the powers of 2 mod 15 gives us a *periodic sequence*, whose period (i.e., how far you have to go before it starts repeating) is 4. For another example, let's look at the powers of 2 mod 21:

$$2, 4, 8, 16, 11, 1, 2, 4, 8, 16, \dots$$

This time we get a periodic sequence whose period is 6.

What's a general rule that governs what the period will be? We discussed this earlier, when we were talking about the RSA cryptosystem! The beautiful pattern, discovered by Euler in the 1760s, is this. Let N be a product of two prime numbers, p and q , and consider the sequence:

$$x \bmod N, x^2 \bmod N, x^3 \bmod N, x^4 \bmod N, \dots$$

Then, provided that x is not divisible by p or q , the above sequence will repeat with some period that divides $(p-1)(q-1)$. So, for example, if $N = 15$, then the prime factors of N are $p = 3$ and $q = 5$, so $(p-1)(q-1) = 8$. And indeed, the period of the sequence is 4, which divides 8. If $N = 21$, then $p = 3$ and $q = 7$, so $(p-1)(q-1) = 12$. And indeed, the period is 6, which divides 12.

Now, I want you to step back and think about what this means. It means that *if* we can find the period of the sequence of powers of $x \bmod N$, *then* we can learn something about the prime factors of N . In particular, we can learn a divisor of $(p-1)(q-1)$. Now, I'll admit that's not as good as learning p and q themselves, but grant me that it's something. Indeed, it's more than something: it turns out that if we could learn several random divisors of $(p-1)(q-1)$ (for example, by trying different random values of x), then with high probability we could put those divisors together to learn $(p-1)(q-1)$ itself. And once we knew $(p-1)(q-1)$, we could then use some more little tricks to recover p and q , the prime factors we wanted. (This is again in your problem set.)

So what's the fly in the ointment? Well, even though the sequence of powers mod N will *eventually* start repeating itself, the number of steps before it repeats could be almost as large as N itself – and N might have hundreds or thousands of digits! This is why finding the period doesn't seem to lead to a fast *classical* factoring algorithm.

Aha, but we have a quantum computer! (Or at least, we're *imagining* that we do.) So maybe there's still hope. In particular, suppose we could create an enormous quantum superposition over all the numbers in our sequence:

$$\sum_r |r\rangle |x^r \bmod N\rangle$$

Then maybe there's some quantum operation we could perform on that superposition that would reveal the period.

The key point is that we're no longer trying to find a needle in an exponentially-large haystack, something we *know* is hard even for a quantum computer. Instead, we're now trying to find the

period of a sequence, which is a *global* property of all the numbers in the sequence taken together. And that makes a big difference.

Look: if you think about quantum computing in terms of “parallel universes” (and whether you do or don’t is up to you), there’s no feasible way to detect a *single* universe that’s different from all the rest. Such a lone voice in the wilderness would be drowned out by the vast number of suburb-dwelling, Dockers-wearing conformist universes. What one can hope to detect, however, is a joint property of *all* the parallel universes together – a property that can only be revealed by a computation to which all the universes contribute ¹.

So, the task before us is not hopeless! But if we want to get this period-finding idea to work, we’ll have to answer two questions:

1. Using a quantum computer, can we quickly create a superposition over $x \bmod N$, $x^2 \bmod N$, $x^3 \bmod N$, ...?
2. Supposing we did create such a superposition, how would we figure out the period?

Let’s tackle the first question first. We can certainly create a superposition over all integers r , from 1 up to N^2 or so. The trouble is, given an r , how do we quickly compute $x^r \bmod N$? We’ve already seen the answer: repeated squaring!

OK, so we can efficiently create a quantum superposition over all pairs of integers of the form $(r, x^r \bmod N)$, where r ranges from 1 up to N or so. But then, given a superposition over all the elements of a periodic sequence, how do we extract the period of the sequence?

Well, we’ve finally come to the heart of the matter – the one part of Shor’s quantum algorithm that actually depends on quantum mechanics. To get the period out, Shor uses something called the *quantum Fourier transform*, or QFT. My challenge is, how can I explain the QFT to you without going through the math? Hmmmm...

OK, let me try this. Like many computer scientists, I keep extremely odd hours. You know that famous experiment where they stick people for weeks in a sealed room without clocks or sunlight, and the people gradually shift from a 24-hour day to a 25- or 26- or 28-hour day? Well, that’s just ordinary life for me. One day I’ll wake up at 9am, the next day at 11am, the day after that at 1pm, etc. Indeed, I’ll happily ‘loop all the way around’ if no classes or appointments intervene.

Now, here’s my question: let’s say I tell you that I woke up at 5pm this afternoon. From that fact alone, what can you conclude about how long my “day” is: whether I’m on a 25-hour schedule, or a 26.3-hour schedule, or whatever?

The answer, of course, is not much! I mean, it’s a pretty safe bet that I’m not on a 24-hour schedule, since otherwise I’d be waking up in the morning, not 5pm. But almost any other schedule – 25 hours, 26 hours, 28 hours, etc. – will necessarily cause me to “loop all around the clock,” so that it’d be no surprise to see me get up at 5pm on some particular afternoon.

Now, though, I want you to imagine that my bedroom wall is covered with analog clocks. These are very strange clocks: one of them makes a full revolution every 17 hours, one of them every 26 hours, one of them every 24.7 hours, and so on for just about every number of hours you can imagine. (For simplicity, each clock has only an hour hand, no minute hand.) I also want you to imagine that beneath each clock is a posterboard with a thumbtack in it. When I first moved into my apartment, each thumbtack was in the middle of its respective board. But now, whenever I

¹For safety reasons, please don’t explain the above to popular writers of the “quantum computing = exponential parallelism” school. They might shrivel up like vampires exposed to sunlight.

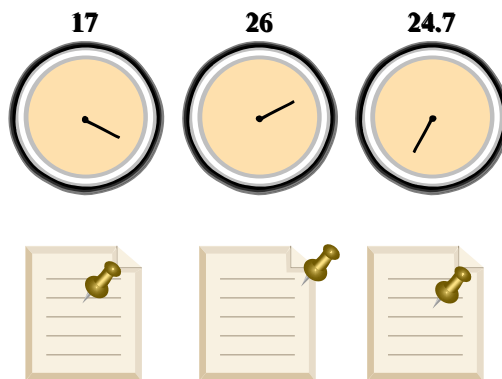


Figure by MIT OpenCourseWare.

Figure 2: A possible configuration of clocks and pegboards.

wake up in the “morning,” the first thing I do is to go around my room, and *move each thumbtack exactly one inch in the direction that the clock hand above it is pointing.*

Now, here’s my new question: *by examining the thumbtacks in my room, is it possible to figure out what sort of schedule I’m keeping?*

I claim that it *is* possible. As an example, suppose I was keeping a 26-hour day. Then what would happen to the thumbtack below the 24-hour clock? It’s not hard to see that it would undergo periodic motion: sure, it would drift around a bit, but after every 12 days it would return to the middle of the board where it had started. One morning I’d move the thumbtack an inch in this direction, another morning an inch in that, but eventually all these movements in different directions would cancel each other out.

On the other hand – again supposing I was keeping a 26-hour day – what would happen to the thumbtack below the 26-hour clock? Here the answer is different. For as far as the 26-hour clock is concerned, I’ve been waking up at exactly the same time each “morning”! Every time I wake up, the 26-hour clock is pointing the same direction as it was the last time I woke up. So I’ll keep moving the thumbtack one more inch in the same direction, until it’s not even on the posterboard at all!

It follows, then, that just by seeing which thumbtack traveled the farthest from its starting point, you could figure out what sort of schedule I was on. In other words, you could infer the “period” of the periodic sequence that is my life.

And that, basically, is the quantum Fourier transform. Well, a little more precisely, the QFT is a *linear transformation* (indeed a unitary transformation) that maps one vector of complex numbers to another vector of complex numbers. The input vector has a nonzero entry corresponding to every time when I wake up, and zero entries everywhere else. The output vector records the positions of the thumbtacks on the posterboards (which one can think of as points on the complex plane). So what we get, in the end, is a linear transformation that maps a quantum state encoding a periodic sequence, to a quantum state encoding the *period* of that sequence.

Another way to think about this is in terms of *interference*. I mean, the key point about quantum mechanics – the thing that makes it different from classical probability theory – is that, whereas probabilities are always non-negative, *amplitudes* in quantum mechanics can be positive, negative, or even complex. And because of this, the amplitudes corresponding to different ways of getting a particular answer can “interfere destructively” and cancel each other out.

And that’s exactly what’s going on in Shor’s algorithm. Every “parallel universe” corresponding to an element of the sequence contributes *some* amplitude to every “parallel universe” corresponding to a possible period of the sequence. The catch is that, for all periods other than the “true” one, these contributions point in different directions and therefore cancel each other out. Only for the “true” period do the contributions from different universes all point in the *same* direction. And

that's why, when we measure at the end, we'll find the true period with high probability.

Questions for next time:

1. Can QCs be built?
2. What are the limits of QCs?
3. Anything beyond QCs?

MIT OpenCourseWare
<http://ocw.mit.edu>

6.045J / 18.400J Automata, Computability, and Complexity
Spring 2011

For information about citing these materials or our Terms of Use, visit: <http://ocw.mit.edu/terms>.